LLD Handbook — Vol 1: OOP, UML, SOLID, Design Principles, Clean Code, Refactoring

OOP · UML · SOLID · Clean Code · Refactoring · Design Patterns · Concurrency · 45+ case studies in Java

6 parts · one place to study LLD: DesignGurus courses + classic books + GitHub problem sets, combined

what it is | problem | how it works | trade-offs | real world | interview | self-check | TL;DR

Personal study notes. Course diagrams belong to DesignGurus. Not for redistribution.

Contents

1. **Chapter 2 — OOP Fundamentals**
   - 2.1 What object-oriented programming is
   - 2.2 Classes, objects, constructors, `this`, `static` and `final`
   - 2.3 Encapsulation
   - 2.4 Abstraction
   - 2.5 Inheritance
   - 2.6 Polymorphism (and the full bank example)
   - 2.7 Compile-time vs runtime polymorphism: overloading vs overriding
   - 2.8 Interfaces (default, static, private methods, functional interfaces)
   - 2.9 Abstract classes vs interfaces
   - 2.10 Enums with fields and behaviour
   - 2.11 Composition over inheritance
   - 2.12 Object identity vs equality: the `equals` / `hashCode` contract
   - 2.13 Immutability and records
   - 2.14 Access modifiers and packages
   - 2.15 Generics basics for LLD
   - 2.16 Coupling and cohesion (the yardsticks)
   - 2.17 Object-oriented analysis and design (OOA / OOD / OOP)
   - Chapter summary: in the interview
   - Self-check (whole chapter)
   - TL;DR
2. **Chapter 3 — Class Relationships & UML**
3. **Part A — Class Relationships**
   - 3.1 The big picture: what a "relationship" really claims
   - 3.2 Association (uni-directional, bi-directional, multiplicity)
   - 3.3 Aggregation
   - 3.4 Composition
   - 3.5 Dependency
   - 3.6 Generalization (inheritance)
   - 3.7 Realization (interface implementation)
   - 3.8 Decision guide: choosing the relationship between two classes
4. **Part B — UML**
   - 3.9 What UML is (structural vs behavioural diagrams)
   - 3.10 Use-case diagrams
   - 3.11 Class diagrams
   - 3.12 Sequence diagrams
   - 3.13 Activity diagrams
   - 3.14 State machine diagrams
5. **Part C — UML in the interview**
   - 3.15 How much UML to draw in an interview
   - 3.16 Chapter self-check
   - 3.17 TL;DR
6. **Chapter 4 — SOLID Principles**
   - 4.1 What SOLID is and why you should learn it
   - 4.2 Single Responsibility Principle (SRP)
   - 4.3 Cohesion
   - 4.4 Coupling
   - 4.5 Separation of Concerns (SoC)
   - 4.6 SRP vs cohesion vs coupling vs SoC — one table
   - 4.7 Open/Closed Principle (OCP)
   - 4.8 Liskov Substitution Principle (LSP)
   - 4.9 Interface Segregation Principle (ISP)
   - 4.10 Dependency Inversion Principle (DIP)
   - 4.11 Refactoring one class through all five principles
   - 4.12 SOLID and design patterns
   - 4.13 SOLID in the design interview
   - 4.14 Common questions about SOLID (with short answers)
   - 4.15 Over-engineering: the warnings in one place
   - Chapter self-check
   - TL;DR
7. **Chapter 5 — More Design Principles**
   - 1. DRY — Don't Repeat Yourself
   - 2. KISS — Keep It Simple, Stupid
   - 3. YAGNI — You Aren't Gonna Need It
   - 4. Law of Demeter (Principle of Least Knowledge)
   - 5. Tell, Don't Ask
   - 6. Composition over Inheritance
   - 7. Program to an Interface, Not an Implementation
   - 8. Encapsulate What Varies
   - 9. Strive for Loosely Coupled Designs
   - 10. The Hollywood Principle (Inversion of Control)
   - 11. Separation of Concerns (SoC)
   - 12. Principle of Least Astonishment (POLA)
   - 13. Fail Fast
   - 14. Immutability
   - 15. Design by Contract (DbC)
   - 16. GRASP — General Responsibility Assignment Software Patterns
   - 16.1 Information Expert
   - 16.2 Creator
   - 16.3 Controller
   - 16.4 Low Coupling
   - 16.5 High Cohesion
   - 16.6 Polymorphism
   - 16.7 Pure Fabrication
   - 16.8 Indirection
   - 16.9 Protected Variations

- 17. Domain-Driven Design (DDD) Essentials

8. **Chapter 6 — Clean Code**
   - What is clean code, and why bother?
   - 1. Meaningful Names
   - 2. Functions
   - 3. Comments
   - 4. Formatting
   - 5. Objects and Data Structures
   - 6. Error Handling
   - 7. Boundaries
   - 8. Unit Tests
   - 9. Classes
   - 10. Systems
   - Chapter summary — how the principles fit together
   - 11. Emergence — Kent Beck's Four Rules of Simple Design
   - 12. Concurrency
   - 13. Successive Refinement and the Case Studies (Chapters 14–16: how clean code is actually written)
   - 14. Smells and Heuristics (the checklist)
   - 15. Code Review Checklist (best practices)
   - 16. Error-Handling Strategy (best practices for real services)
   - 17. Testing Strategies (unit vs integration, test doubles, mocks)
   - Chapter summary

9. **Chapter 7 — Refactoring**
   - 7.1 Refactoring fundamentals
   - 7.2 Code smells
   - 7.3 The refactoring catalogue
   - 7.4 Smell → refactorings lookup table
   - 7.5 Worked example — from messy to clean in six compiling steps
   - 7.6 Refactoring toward patterns
   - 7.7 Putting it together
   - 7.8 Full source code and output

Chapter 2 — OOP Fundamentals

What you will learn in this chapter

- What object-oriented programming (OOP) is and why the LLD round is really a test of it.
- Classes and objects, constructors and constructor chaining, `this`, `static` vs instance members, and `final` (2.2).
- The four pillars — **encapsulation, abstraction, inheritance, polymorphism** — on one running bank example (2.3–2.6); `super` and overriding rules; overloading vs overriding and dynamic dispatch (2.7).
- Interfaces (default, static, private methods, functional interfaces) and abstract classes, and when to pick which (2.8–2.9); enums with fields and behaviour (2.10).
- Composition over inheritance (2.11), identity vs equality (`equals` / `hashCode`, 2.12), immutability and records (2.13), access modifiers and packages (2.14), the generics you need in LLD (2.15), coupling and cohesion (2.16).
- Object-oriented analysis vs design vs programming (OOA / OOD / OOP), with a four-step process (2.17).

Every concept has a Java 21 example that compiles and runs; listings below show the essential lines, the full files are in `code/lld/ch02/`, and the real output is shown under each program.

How to run all examples: `cd chapters/02-oop-fundamentals/code && javac --release 21 -d /tmp/lld_02 $(find . -name '*.java') && java -cp /tmp/lld_02 lld.ch02.<package>.Main`

(`javac` compiles `.java` source into `.class` files; `java` runs them on the **JVM**, the Java Virtual Machine that executes compiled Java code; `-cp` says where the compiled classes live.)

2.1 What object-oriented programming is

What it is

Object-oriented programming (OOP) bundles data with the code allowed to modify it into one unit, an **object**. **Procedural programming** separates them: data sits in one place and free-standing functions anywhere can change it. OOP **fences the data in**: only the object's own methods reach it. Analogy: a bank vault with a teller (the methods) who checks every request, vs a pile of cash on a table.

The problem it solves

When any code can change any data, a rule like "a balance never goes negative" must be copied into every caller, and one forgotten copy is a bug. OOP puts each rule in **one place**, next to the data it protects, and lets a design grow by **adding classes** rather than editing old ones.

Why this matters for the interview

They watch you **break a problem into objects, give each responsibilities and data, and justify it**; every idea here serves that.

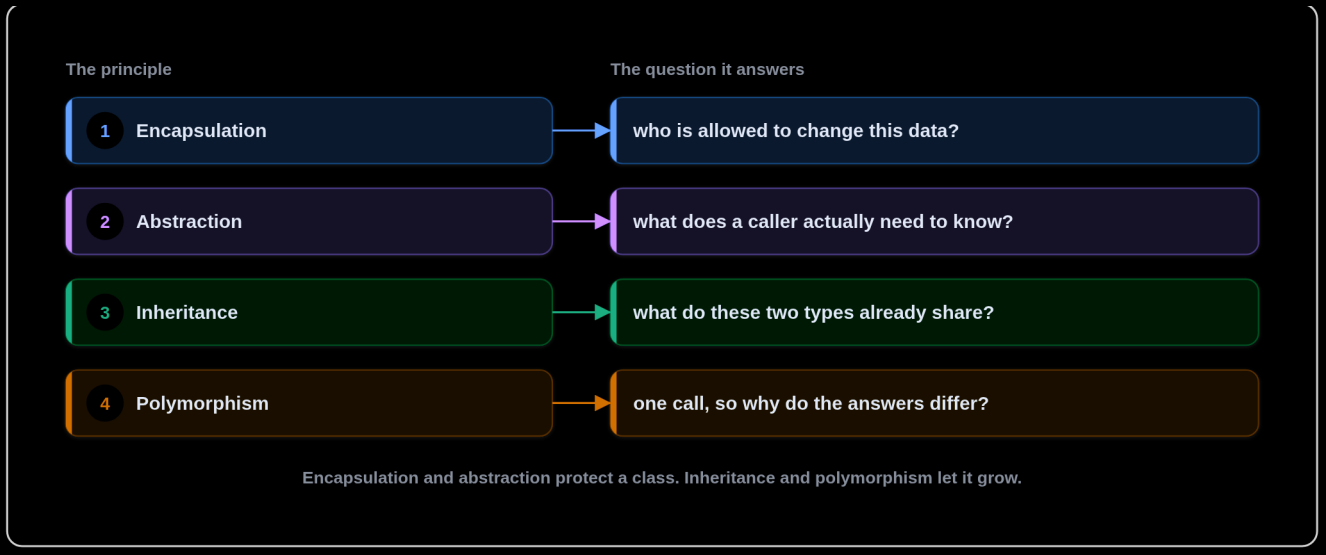


Figure — The four OOP principles and the question each one answers (course diagram)

Pillar	Question it answers
Encapsulation	Who is allowed to change this data?
Abstraction	What does a caller actually need to know?
Inheritance	What do these two types already share?
Polymorphism	One call, so why do the answers differ?

The first two guard a single class; the last two let the design expand with new classes.

2.2 Classes, objects, constructors, `this`, `static` and `final`

What it is

- A **class** is the **blueprint**: the fields a thing has and the operations it supports, written once.
- An **object** (instance) is **one concrete thing built from it**, with actual field values; create as many as needed with `new`. (`Account` is a class; account `S-1`, owner Ada, 50,000 cents, is an object. `Customer` is a class; the shopper behind order 4192 is an object.)
- A **field** is a variable belonging to a class or object; a **method** is a function belonging to one.
- A **constructor** runs when an object is created; it has the class's name and **no return type**, and its job is to put the object into a **valid** state.
- `this` refers to **the current object** inside its instance methods and constructors.
- `static` = belongs to the **class**, shared by all objects; otherwise a member is an **instance** member (one copy per object).
- `final` = cannot change: a `final` variable is assigned once, a `final` method cannot be overridden (replaced by a child class's version), a `final` class cannot be extended (used as a parent; 2.5).

How it works

1. `new Account("S-1", "Ada", 50_000)` allocates the fields, sets defaults (0, `null` meaning "no object", `false`), then runs the constructor.
2. **Initialisation order:** static initialisers (`static` field assignments and `static { ... }` blocks) run **once**, when the class is initialised (just before first real use: first `new` or first static method call). For each `new`: the parent constructor runs first (`super(...)`, implicit if omitted), then instance field initialisers and instance initialiser blocks (plain `{ ... }`) in source order, then the rest of the constructor body.
3. With **no constructor written**, the compiler adds a public no-argument **default constructor**; with any constructor written, it does not.

4. **Overloaded constructors** (same name, different parameters) should **chain** with `this(...)` to one canonical constructor (the "main" one all others call), so validation lives in one place. In Java 21, `this(...)` / `super(...)` must be the **first statement**. (Java 25 relaxes this with "flexible constructor bodies", JEP 513 — JEPs, JDK Enhancement Proposals, are the numbered documents describing new Java features: statements not using the object under construction may come first. This book targets Java 21, so keep the call first.)
5. `this`: disambiguates field from parameter (`this.id = id`), passes the current object on, chains constructors, enables **fluent chaining** (`return this`).
6. `static` methods have **no** `this` (no direct instance-field access): use them for utilities (`Math.max`) and factories (methods that create and return objects).
7. Class-type variables hold **references** (like an address): `Account alias = a;` makes both names point to the **same** object.

Java example

```java
// File: code/lld/ch02/classes/Account.java (essentials)
public class Account {
    private static int openedCount = 0;               // static: ONE counter for the class
    public static final long MIN_OPENING_CENTS = 0;
    private final String id;                          // instance fields: one copy per object
    private final String owner;
    private long balanceCents;

    public Account(String id, String owner, long openingCents) {   // canonical: validates once
        if (openingCents < MIN_OPENING_CENTS) throw new IllegalArgumentException("negative opening balance");
        this.id = id;                // 'this.id' is the field, 'id' is the parameter
        this.owner = owner;
        this.balanceCents = openingCents;
        openedCount++;
    }
    public Account(String id, String owner) { this(id, owner, 0); }   // chains; must be first statement

    public Account deposit(long cents) {              // returns 'this' so calls can be chained
        if (cents <= 0) throw new IllegalArgumentException("deposit must be positive");
        this.balanceCents += cents;
        return this;
    }
    public static int openedCount() { return openedCount; }  // static method: no 'this' here
    @Override public String toString() { return id + "(" + owner + ", " + balanceCents + "c)"; }
}

// File: code/lld/ch02/classes/Main.java
Account a = new Account("S-1", "Ada", 50_000);          // 3-arg constructor
Account b = new Account("C-1", "Kai");                  // 2-arg chains to 3-arg
b.deposit(1_000).deposit(500);                          // chaining via 'return this'
System.out.println("a = " + a);
System.out.println("b = " + b);
System.out.println("accounts opened (static, shared): " + Account.openedCount());
Account alias = a;                                      // copies the REFERENCE, not the object
alias.deposit(1);
System.out.println("after alias.deposit(1), a = " + a + ", same object? " + (alias == a));
try { new Account("X", "Bad", -5); }
catch (IllegalArgumentException e) { System.out.println("constructor refused: " + e.getMessage()); }
System.out.println("accounts opened: " + Account.openedCount() + " (failed construction did not count)");
```

Output:

```
a = S-1(Ada, 50000c)
b = C-1(Kai, 1500c)
accounts opened (static, shared): 2
after alias.deposit(1), a = S-1(Ada, 50001c), same object? true
constructor refused: negative opening balance
accounts opened: 2 (failed construction did not count)
```

`alias == a` is `true`: both hold the same reference. The failing constructor threw **before** incrementing the counter, so no half-built object was counted: **validate first, then change state**.

Trade-offs

Choice	Benefit	Cost
Static members	Shared, no object needed	Global state (reachable from anywhere): hard to test, dangerous with threads (parts of a program running at once)
Many constructor overloads	Convenient	Confusing beyond 3–4; use a **Builder** (Chapter 8: collects settings step by step, then creates the object) or static factory methods
`final` fields	Safer, thread-friendlier	Must be set in every constructor

Common mistakes

- Per-object data in a `static` field (every object shares it).
- Heavy work or overridable calls in constructors (a subclass override may run before the subclass's fields are set).
- Forgetting `this.` so the parameter is assigned to itself (`id = id;`).
- Thinking `=` copies an object; it copies a reference.

Real-world: `Math.max`, `Integer.parseInt`, `List.of` are static; `StringBuilder.append` returns `this`; `String` is a `final` class.

In the interview

> **30-second answer:** "A class is the blueprint, an object is an instance with its own state. Constructors put an object into a valid state; I chain overloads with `this(...)` so validation lives once. Static members belong to the class and are shared, so I keep them for constants, utilities and factories and avoid mutable (changeable) static state."

- *"Constructor with arguments only?"* — No default no-arg constructor is generated; `new X()` stops compiling.
- *"Can a static method use `this`?"* — No; there is no current object.
- *"Why avoid mutable static fields?"* — Hidden global state shared by all threads and tests.

Self-check

- **Q:** Copies of a `static` field for 1,000 objects? **A:** One.
- **Q:** First: parent constructor or child field initialisers? **A:** Parent constructor.
- **Q:** What does `alias == a` compare? **A:** References (same object or not).

TL;DR

Class = blueprint, object = instance. Constructors create valid objects; chain them with `this(...)`. `static` = one per class; `final` = cannot change. Variables hold references.

2.3 Encapsulation

What it is

Encapsulation: an object hides its data (**private**) and lets the outside change it **only through methods it defines and guards**. Analogy: a child-proof medicine cap — reachable, but only through a mechanism that enforces the rules.

The problem it solves

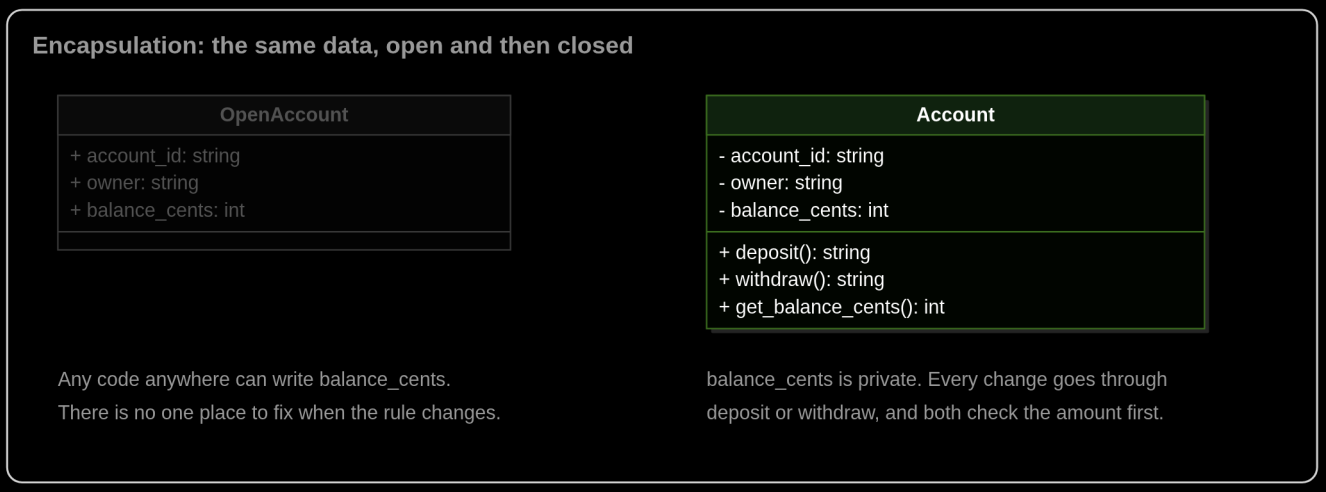


Figure — The same account data, first open (public fields anyone can write) and then closed (private fields changed only through deposit and withdraw) (course diagram)

Public fields (left; a contrast only) let **any file** write a negative `balance`, and a rule change has **nowhere central** to go. Private balance + validating `deposit`/`withdraw` (right): **every route to the balance goes through a method that can say no**. (`Account`'s default `monthlyUpdate` does nothing until 2.6.)

How it works

1. Fields `private` (and `final` where possible).
2. Expose **behaviour** (`deposit`, `withdraw`), not raw setters (`setBalance`, which just overwrite a field).
3. Validate inside those methods: the rule lives **once**.
4. Return copies or unmodifiable views (read-only wrappers that throw on change) of internal collections, never the live list.
5. For every field, settle **which code may modify it**; when the answer is "only this class", say so in the interview — it makes a field list a visible design decision.

The bank `Account` in 2.6 is the example: private `balanceCents`; `deposit` rejects non-positive amounts, `withdraw` amounts above the balance; both `final` so a subclass cannot weaken the rule.

Trade-offs

Benefit	Cost
Invariants (rules that must always hold) enforced in one place	A little more code than public fields
Internals change without breaking callers	Over-encapsulating trivial data carriers is noise (use records)

When not (strictly): plain data carriers with no rules — DTOs (Data Transfer Objects that only carry data between parts of a system), events (records of something that happened) — need no hand-written getters or validation; a **record** gives private final fields and accessors free.

Common mistakes

- **Getter/setter for every field** (getter = method returning a field; "anemic" objects): technically private, but anyone can set anything. Not encapsulation.
- Returning a mutable internal list (`return items;`), letting callers change your state.

Real-world: `String` hides its byte array; `ArrayList` hides its backing array (the plain array storing elements) and `size`; `LocalDate` exposes only validated factories (`LocalDate.of(2026, 2, 30)` throws).

In the interview

> **30-second answer:** "Encapsulation hides state behind methods, and those methods are the single home of every rule about that state. For each field I ask who may change it; if only this class, it is private and changed through intention-revealing methods (named after the business action) like `withdraw`, not a setter."

- *"Are getters and setters encapsulation?"* — Only partly; a rule-free setter is a public field in disguise. Prefer behaviour methods.

Self-check — Q: Where should "no negative balance" live? **A:** Inside `Account.withdraw`, once.

TL;DR

Private data + rule-enforcing methods = encapsulation. Expose behaviour, not setters.

2.4 Abstraction

What it is

Abstraction gives a caller **just the operations it requires** and conceals **how** they are done. Encapsulation conceals **state**; abstraction conceals **design choices**, so the short list of operations survives a full rewrite behind it. Analogy: steering wheel and pedals — you drive without knowing about fuel injection, and an electric motor can replace the engine without changing how you drive.

The problem it solves

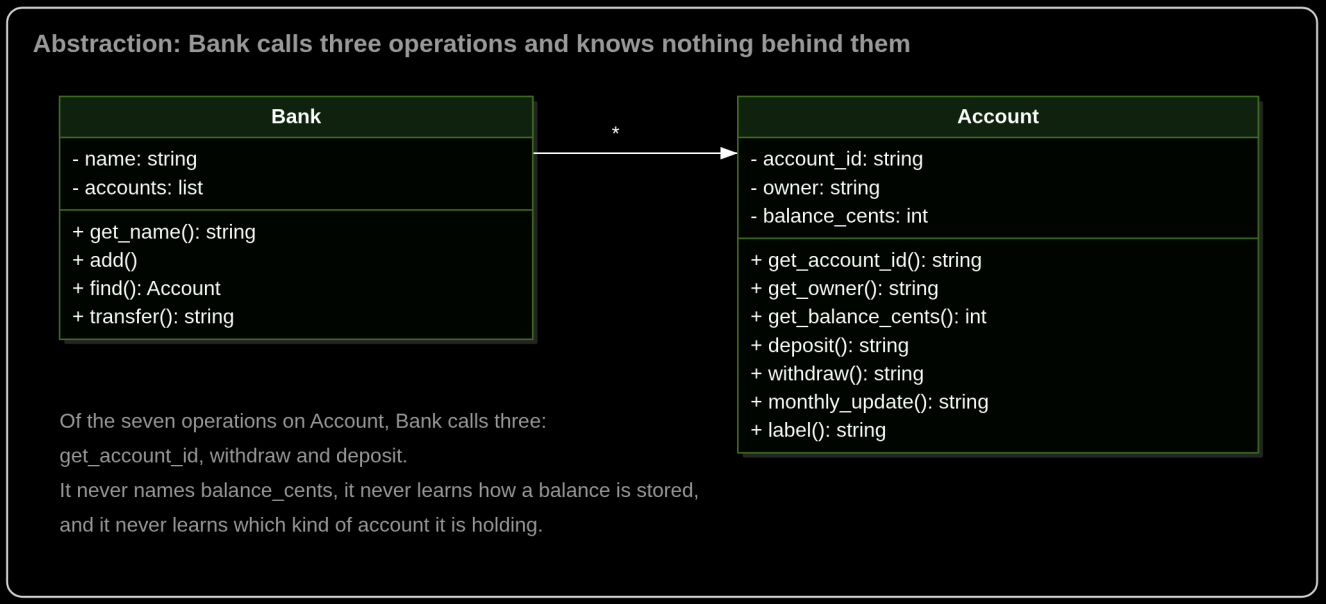


Figure — Of Account's seven public operations, Bank relies on three and sees nothing of what lies behind them (course diagram)

(The diagram's seven include a `label()` helper; our Java `Account` has six: `id`, `owner`, `balanceCents`, `deposit`, `withdraw`, `monthlyUpdate`.) `Bank.transfer` uses `id()`, `withdraw()`, `deposit()` and knows nothing of whole cents, checks or subclasses — the balance could move into a database table and `transfer` would not change.

How it works

1. Decide the **smallest set of operations** a caller needs.
2. Name them for **what** they do (`transfer`), not how.

3. Put them on an interface (a named list of methods with no code that classes promise to provide; 2.8) or a class's public surface; keep the rest private.
4. Callers depend only on that surface.

Trade-offs

Benefit	Cost
Simpler callers that survive internal rewrites	A wrong abstraction is expensive to change
Enables polymorphism (many implementations behind one surface)	Too many layers are hard to follow

Common mistakes

- **Leaky abstraction:** the interface exposes implementation details (`getHashMapOfSeats()`).
- Abstracting before two concrete cases exist.

Real-world: `java.util.List` (`ArrayList` or `LinkedList`, callers do not care); JDBC's `Connection` / `Statement` (Java Database Connectivity: same code for different SQL databases); `java.io.InputStream`.

In the interview

> **30-second answer:** "Abstraction is exposing only what the caller needs. My test is the sentence 'the caller does not need to know that': whenever I can honestly say it, something moves behind the interface, the design shrinks, and I can change the implementation freely."

- *"Encapsulation vs abstraction?"* — Encapsulation hides and protects **data** with access control (keywords such as `private` deciding who sees a member, 2.14); abstraction hides **complexity and decisions** behind a small set of operations. They work together.

TL;DR

Show the minimum surface; hide the how. "The caller does not need to know that."

2.5 Inheritance

What it is

Inheritance lets a **child** (subclass) start with everything its **parent** (superclass) has, then **add** fields/methods or **replace** (override) methods: `class SavingsAccount extends Account`. A savings account **is a** bank account with one extra rule.

The problem it solves

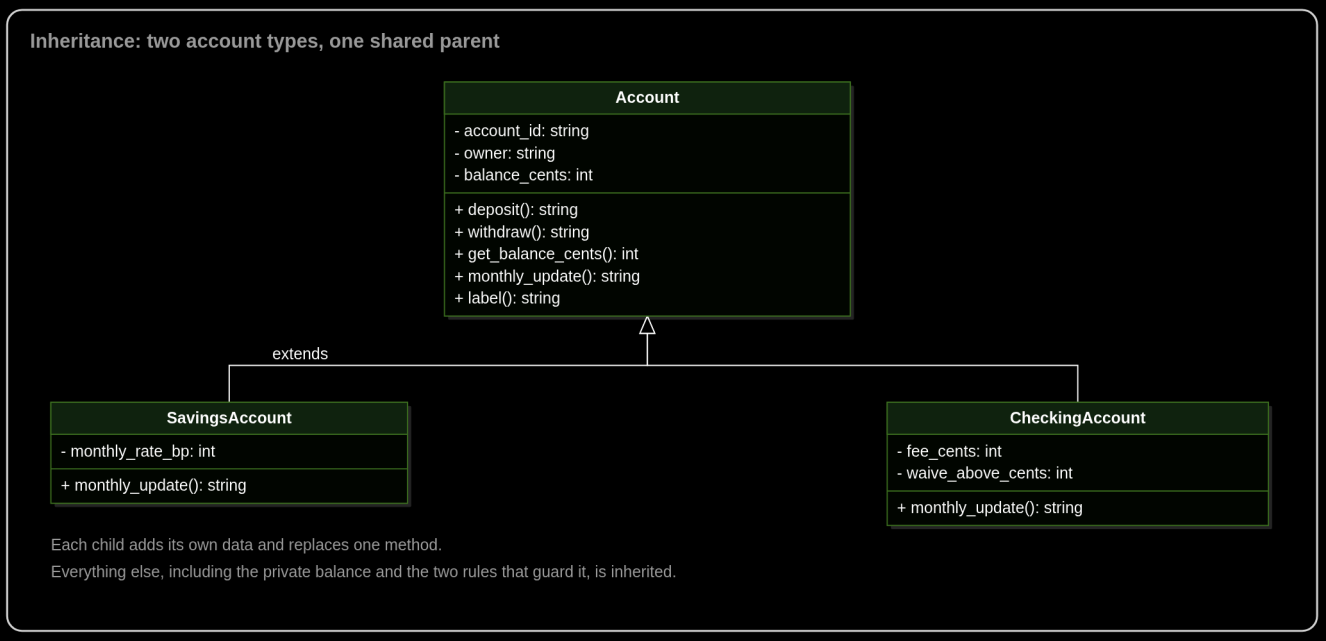


Figure — SavingsAccount and CheckingAccount extend Account; each adds its own data and replaces one method (course diagram)

Savings earns interest; checking charges a fee unless the balance clears a threshold; balance and its checks come from the parent. Savings adds one field, checking two, **each overrides one method**. Were the is-a sentence false, the difference would be a **field**.

How it works

1. `class Child extends Parent`: **one** parent class (single inheritance of classes), many interfaces.
2. The child inherits all **accessible** (non-private) members; private fields exist in the object but are reached only via parent methods. **Constructors are not inherited.**
3. The child constructor must call `super(...)` first (implicitly `super()`, which fails to compile if the parent lacks a no-arg constructor).
4. `super.method()` calls the parent's version inside an override (reuse, then extend).
5. **Overriding rules:**

Rule	Detail
Same signature	Same name and parameter types
Return type	Same, or a **subtype** (a type that extends or implements it) for reference types (*covariant return*); primitives (`int`, `long`, `boolean`...) must match exactly
Access	Same or **wider** (protected — visible to subclasses — → public is fine; public → protected is not)
Checked exceptions	Fewer or narrower, never new or broader; unchecked are free. (A **checked exception**, e.g. `IOException`, must be caught or declared; an **unchecked** one, e.g. `IllegalArgumentException`, extends `RuntimeException` and needs no declaration)
Cannot override	`final` methods, `static` methods (**hidden**, not overridden), `private` methods (not inherited)
Annotation	Always write `@Override` (an annotation is an `@` tag giving the compiler extra information): it catches typos and wrong signatures

6. Every class ultimately extends `java.lang.Object` — hence `toString`, `equals`, `hashCode`, `getClass` everywhere.
7. **The is-a test:** "a SavingsAccount **is an** Account" must be true in every situation. A car is not an engine, so `Car` **holds** an `Engine` (2.11).

Examples: `SavingsAccount` / `CheckingAccount` in 2.6 (each calls `super(...)` and overrides `monthlyUpdate()`); `Circle` in 2.7 (covariant return, `super.who()`, static-method hiding, field hiding).

Trade-offs

Benefit	Cost
Reuse of shared state and behaviour	**Tight coupling** (a change in one class forces changes in another): the child depends on parent internals (fragile base class)
Enables polymorphism	Hierarchy fixed at compile time (before the code runs); single parent only
Natural for true "is-a" families	Deep hierarchies (more than 2–3 levels) are hard to reason about

When not: only to **reuse code** when is-a is false (use composition); when the child would **disable** a parent method (throw `UnsupportedOperationException` or do nothing) — that breaks the **Liskov Substitution Principle** (Chapter 4): the child can no longer stand in for the parent; for variations differing **only in data** (field or enum).

Common mistakes

- `Square extends Rectangle` with independent `setWidth` / `setHeight`: code expecting a rectangle breaks.
- Forgetting `@Override` and silently **overloading** (`equals(Point p)` instead of `equals(Object o)`).
- Calling overridable methods from a parent constructor.

Real-world: good — `FileInputStream extends InputStream`; Spring's (a popular Java application framework) `AbstractController`-style base classes. Known mistakes in the JDK (Java Development Kit: compiler + standard library) itself: `java.util.Stack extends Vector` (a stack, which should only touch its top, exposes random-access `insertElementAt`), and `java.util.Properties extends Hashtable` (allows non-string entries).

In the interview

> **30-second answer:** "Inheritance lets a subclass reuse all of its parent's members, and it is only right when the sentence 'the child is a kind of the parent' holds in every situation. I use it for genuine type families, override with `@Override`, and if a child would have to disable a parent method I switch to composition, because that is a Liskov violation."

- *"Can you override a static method?"* — No. A same-signature static in the child **hides** the parent's; the compile-time type decides which runs (see 2.7 output).
- *"Can an override return a different type?"* — Only a subtype (covariant return), e.g. `Circle copy()` overriding `Shape copy()`.

Self-check

- **Q:** Is `private` inherited? **A:** The field exists in the child object but the child cannot access it directly; private methods are not overridden.
- **Q:** Can an override narrow `public` to `protected`? **A:** No.

TL;DR

`extends` for true is-a families only. `super(...)` first, `@Override` always, never narrow access or disable parent behaviour.

2.6 Polymorphism (and the full bank example)

What it is

Polymorphism ("many forms"): **one method call does different things depending on the object it lands on** — code holds a **parent-type** variable, the concrete object decides. Analogy: one "start" button on a car, a washing machine, a laptop.

The problem it solves

Without it, code asks "what type are you?" everywhere (`if (a instanceof SavingsAccount) ... else if ...`) and every new type edits every branch.

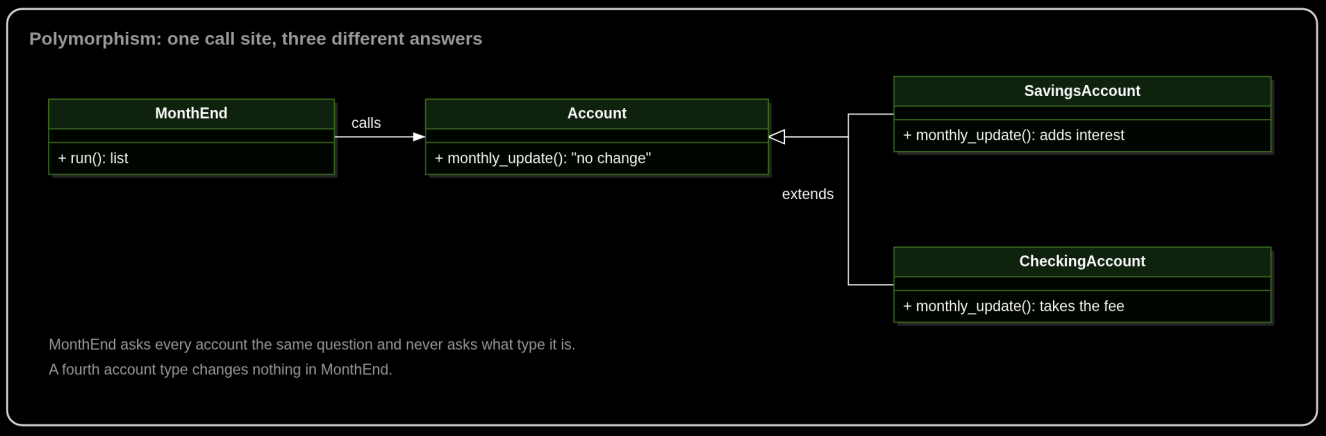


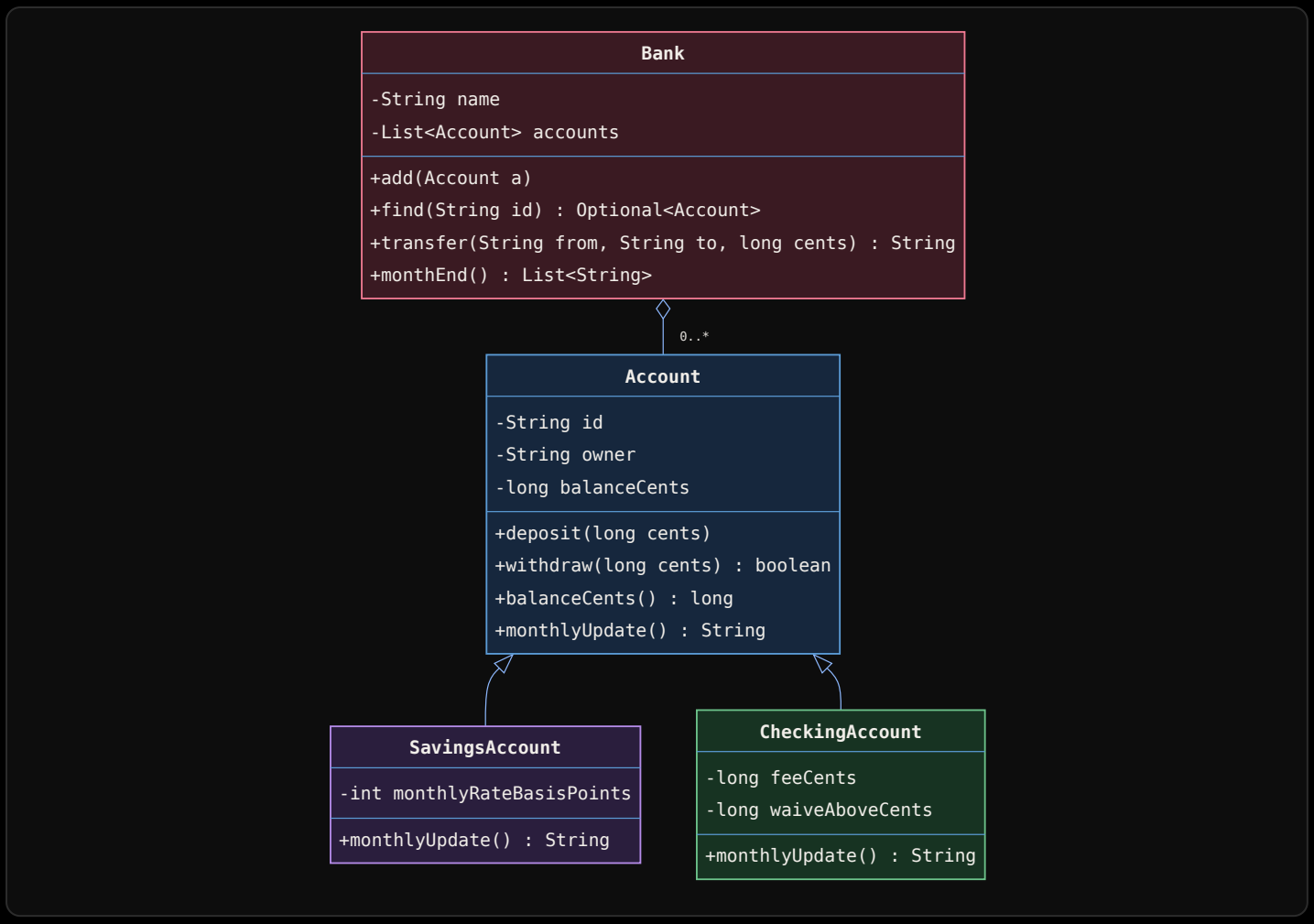
Figure — MonthEnd calls monthlyUpdate on every account and never asks which type it holds (course diagram)

No `if` on type in the loop, so a fourth account type needs no edit.

How it works

1. Declare the method on the parent type (or interface); 2. each child overrides it; 3. callers hold the **parent type** (`Account a`) and call it; 4. at run time the JVM looks at the **actual class of the object** and runs that version: **dynamic dispatch** (late binding).

Java example: the bank (all four pillars)



```java
// File: code/lld/ch02/bank/Account.java — ENCAPSULATION (constructor rejects a negative opening balance)
private long balanceCents;                      // hidden: nobody outside can set it to -5
public final void deposit(long cents) {         // final: children may not weaken the rule
    if (cents <= 0) throw new IllegalArgumentException("deposit must be positive");
    balanceCents += cents;
}
public final boolean withdraw(long cents) {
    if (cents <= 0 || cents > balanceCents) return false;   // the one place the rule lives
    balanceCents -= cents;
    return true;
}
public String monthlyUpdate() { return "no change"; }      // default; subclasses OVERRIDE it
// final accessors id(), owner(), balanceCents()

// File: code/lld/ch02/bank/SavingsAccount.java — INHERITANCE
public SavingsAccount(String id, String owner, long openingCents, int monthlyRateBasisPoints) {
    super(id, owner, openingCents);             // parent constructor runs first
    this.monthlyRateBasisPoints = monthlyRateBasisPoints;   // 25 bp = 0.25 %
}
@Override public String monthlyUpdate() {
    long interest = balanceCents() * monthlyRateBasisPoints / 10_000;
    if (interest > 0) deposit(interest);
    return "interest " + Money.format(interest);
}

// File: code/lld/ch02/bank/CheckingAccount.java — fee unless balance above a threshold
@Override public String monthlyUpdate() {
    if (balanceCents() >= waiveAboveCents) return "fee waived";
    return withdraw(feeCents) ? "fee " + Money.format(feeCents) : "fee could not be taken";
}

// File: code/lld/ch02/bank/Bank.java — ABSTRACTION + POLYMORPHISM
public String transfer(String fromId, String toId, long cents) {
    Account from = find(fromId).orElseThrow();
    Account to = find(toId).orElseThrow();
    if (!from.withdraw(cents)) return fromId + " rejected, not enough money";
    to.deposit(cents);
    return "moved " + Money.format(cents) + " from " + fromId + " to " + toId;
}
public List<String> monthEnd() {               // one call, a different result per object, no 'if' on type
    List<String> lines = new ArrayList<>();
    for (Account a : accounts) {
        String what = a.monthlyUpdate();       // dynamic dispatch picks the override
        lines.add(a.id() + " (" + a.owner() + "): " + what + ", balance " + Money.format(a.balanceCents()));
    }
    return lines;
}
// Money.format(cents) -> String.format("$%d.%02d", cents / 100, cents % 100) (utility, private constructor)

// File: code/lld/ch02/bank/Main.java
Bank bank = new Bank("Gurus Bank");
bank.add(new SavingsAccount("S-1", "Ada", 50_000, 25));
bank.add(new CheckingAccount("C-1", "Kai", 8_000, 250, 50_000));
bank.add(new Account("P-1", "Ravi", 3_000));
// prints bank name + each opening balance
System.out.println(bank.transfer("S-1", "C-1", 10_000));
System.out.println(bank.transfer("S-1", "C-1", 90_000));
System.out.println("month end:");
bank.monthEnd().forEach(line -> System.out.println("  " + line));
```

Output:

```
Gurus Bank, opening balances:
  S-1 (Ada): $500.00
  C-1 (Kai): $80.00
  P-1 (Ravi): $30.00
moved $100.00 from S-1 to C-1
S-1 rejected, not enough money
month end:
  S-1 (Ada): interest $1.00, balance $401.00
  C-1 (Kai): fee $2.50, balance $177.50
  P-1 (Ravi): no change, balance $30.00
```

Explaining each part:

- **Encapsulation:** the $900 transfer from an account holding $400 is refused inside `withdraw`.
- **Abstraction:** `Bank.transfer` uses only `id()`, `withdraw()`, `deposit()`.
- **Inheritance:** each child calls `super(...)` and adds one or two fields.
- **Polymorphism:** three accounts, one method name, three results — 0.25% of $400 = $1.00; checking's $180 is below the $500 waiver so $2.50 is charged; the plain account uses the parent's "no change". **No `if` asks which is which.**
- **Why `long` cents, not `BigDecimal`:** never `double` for money (it cannot store 0.10 exactly; sums drift). Whole **cents** in a `long` are exact but hold no fraction of a cent: the interest line is integer division and **truncates** (toward zero) — 40,000 cents at 25 **basis points** (1 bp = 0.01%) gives exactly 100, but 40,001 drops a 0.0025-cent remainder. For fractions or explicit rounding (interest, tax, currency conversion) use `BigDecimal` (exact decimal type) with a stated `RoundingMode`, like the `Money` value object (a small object defined only by its values, 2.12) in 2.13.

Together: encapsulation and abstraction guard a class; inheritance and polymorphism make **a new requirement an added class**, not surgery. Typical shape: hidden state, a small public API (application programming interface: the methods other code may call), a parent for common parts, subclasses each overriding one method.

Trade-offs

Benefit	Cost
Removes type switches; open to new types	Behaviour spread over classes; you must know the runtime type to know what runs
Enables Strategy, State, Command and most patterns (named reusable designs; Chapters 8–11)	A virtual call (resolved at run time) is marginally slower than a direct call (almost never matters)

Common mistakes

- Long `if/else` or `switch` over a type: almost always where polymorphism belongs. **Pointing this out earns more credit than actually refactoring (restructuring without changing behaviour) it.**
- `instanceof` chains as the main dispatch mechanism (fine for occasional data inspection, e.g. Java 21 pattern matching, which tests a type and binds a variable in one step).

Real-world: `List.add` in `ArrayList` vs `LinkedList`; `Runnable.run()` under any `Executor` (runs tasks, often on background threads); Spring injecting (handing in) any implementation of an interface; `Comparator.compare` in `Collections.sort`.

In the interview

> **30-second answer:** "Polymorphism lets one call behave differently depending on the object receiving it. I code against the parent type or an interface, each subtype overrides the method, and the JVM picks the right one at run time. It removes type switches, so a new type is a new class, not an edit."

- *"Where in a parking lot?"* — Pricing (`PricingStrategy.feeCents`), payments (`Payment.pay`), spot selection.

Self-check — Q: Which `monthlyUpdate` runs for `Account a = new SavingsAccount(...)`? **A:** `SavingsAccount`'s, chosen at run time.

TL;DR

Hold the parent type, call the method, let the object decide. Type switches signal missing polymorphism.

2.7 Compile-time vs runtime polymorphism: overloading vs overriding

What it is

	Overloading (compile-time / static polymorphism)	**Overriding (runtime / dynamic polymorphism)**
What	Same name, **different parameter lists**, usually same class	Same signature re-implemented in a **subclass**
Decided by	The **compiler**, from the **declared (static) types** of the arguments	The **JVM at run time**, from the **actual class** of the receiver
Return type	May differ (but cannot be the only difference)	Same or covariant
Needs inheritance?	No	Yes (class or interface)
Example	`add(int,int)`, `add(double,double)`	`Circle.area()` overriding `Shape.area()`

How it works: overload resolution

Three phases; the first phase with a match wins:

1. Exact match or **widening** (automatic conversion to a bigger/more general type: `int` → `long` → `double`, `Circle` → `Shape`); no boxing, no varargs.
2. Allow **boxing/unboxing** (`int` ↔ `Integer`: wrapping a primitive in its object form and back).
3. Allow **varargs** (`int...`: any number of arguments).

Several candidates in a phase → the **most specific** wins; none most specific → compile error (ambiguous call). **Not polymorphic:** fields (resolved by static type; a same-named child field **hides** the parent's), `static` methods (hidden), `private` methods.

Java example

```java
// File: code/lld/ch02/polymorphism/Calculator.java — OVERLOADS chosen by the compiler
public String add(int a, int b)       { return "add(int,int) = " + (a + b); }
public String add(long a, long b)     { return "add(long,long) = " + (a + b); }
public String add(double a, double b) { return "add(double,double) = " + (a + b); }
public String add(int... xs)          { int s = 0; for (int x : xs) s += x; return "add(int...) = " + s; }
public String greet(Shape s)  { return "greet(Shape)"; }
public String greet(Circle c) { return "greet(Circle)"; }

// File: code/lld/ch02/polymorphism/Shape.java
public class Shape {
    public String kind = "shape";                                  // fields are NOT polymorphic
    public double area() { return 0; }
    public Shape copy() { return new Shape(); }
    public static String describe() { return "static Shape.describe"; }   // static: hidden, not overridden
    public String who() { return "I am a " + kind + " with area " + String.format("%.2f", area()); }
}

// File: code/lld/ch02/polymorphism/Circle.java (constructor: super(); this.r = r;)
public class Circle extends Shape {
    public String kind = "circle";                                 // HIDES Shape.kind
    private final double r;
    @Override public double area() { return Math.PI * r * r; }  // OVERRIDE
    @Override public Circle copy() { return new Circle(r); }    // covariant return
    public static String describe() { return "static Circle.describe"; }
    @Override public String who() { return super.who() + " (radius " + r + ")"; }
}

// File: code/lld/ch02/polymorphism/Main.java
Calculator calc = new Calculator();
System.out.println(calc.add(2, 3));           // exact match int,int
System.out.println(calc.add(2L, 3));          // widening to long,long
System.out.println(calc.add(2.5, 3));         // widening to double,double
System.out.println(calc.add(1, 2, 3));        // varargs, chosen last
Shape s = new Circle(1.0);                    // static type Shape, runtime type Circle
System.out.println("s.area() -> " + String.format("%.2f", s.area()) + "  (runtime override: dynamic dispatch)");
System.out.println("calc.greet(s) -> " + calc.greet(s) + "  (overload picked at compile time from static type)");
System.out.println("s.kind -> " + s.kind + "  (fields use the static type)");
System.out.println("((Circle) s).kind -> " + ((Circle) s).kind);
System.out.println("Shape.describe() -> " + Shape.describe() + ", Circle.describe() -> " + Circle.describe());
System.out.println("s.copy() class -> " + s.copy().getClass().getSimpleName());
System.out.println(s.who());
double total = 0;
for (Shape sh : List.of(new Shape(), new Circle(2), new Circle(0.5))) total += sh.area();  // one call site
System.out.println("total area = " + String.format("%.2f", total));
if (s instanceof Circle c) System.out.println("pattern-matched a circle: " + c.who());   // Java 16+
```

Output:

```
add(int,int) = 5
add(long,long) = 5
add(double,double) = 5.5
add(int...) = 6
s.area() -> 3.14  (runtime override: dynamic dispatch)
calc.greet(s) -> greet(Shape)  (overload picked at compile time from static type)
s.kind -> shape  (fields use the static type)
((Circle) s).kind -> circle
Shape.describe() -> static Shape.describe, Circle.describe() -> static Circle.describe
s.copy() class -> Circle
I am a shape with area 3.14 (radius 1.0)
total area = 13.35
pattern-matched a circle: I am a shape with area 3.14 (radius 1.0)
```

Explaining each part:

- `add(2L, 3)`: no `(long,int)` overload, so `3` widens to `long` (phase 1); `s.area()` runs `Circle.area()` though `s` is declared `Shape` (**dynamic dispatch**).

- `calc.greet(s)` prints `greet(Shape)`: overloads are chosen **at compile time from the declared type** — a favourite interview surprise.
- `s.kind` is `"shape"`: fields are not polymorphic; inside `Shape.who()`, `kind` is `Shape.kind`, hence "I am a shape". Lesson: **never hide fields**; use private fields plus overridable methods.
- `Circle copy()` overrides with a **covariant return type**; `super.who()` reuses the parent's text.
- `s instanceof Circle c` is **pattern matching for instanceof** (Java 16+): test and cast (treat as the more specific type) in one step.

Common mistakes

- Expecting overloads picked by runtime type (that needs double dispatch — choosing a method by the runtime types of two objects; Visitor pattern, Chapter 11).
- `public boolean equals(Point p)`: an **overload**, not an override of `equals(Object)`, so collections (`List`, `Set`, `Map`) never call it; `@Override` would have caught it.

In the interview

> **30-second answer:** "Overloading is compile-time: same name, different parameters, chosen by declared types. Overriding is runtime: a subclass re-implements the same signature and the JVM dispatches on the actual object. Fields and static methods are not polymorphic."

- *"Why does `greet(s)` call `greet(Shape)`?"* — Overload resolution uses the static type of `s`, `Shape`.

Self-check

- **Q:** Can two methods differ only in return type? **A:** No; compile error.
- **Q:** Which phase allows varargs? **A:** The third (last).

TL;DR

Overloading = compiler picks by declared types. Overriding = JVM picks by actual object. Fields and statics never dispatch dynamically.

2.8 Interfaces (default, static, private methods, functional interfaces)

What it is

An **interface** is a **contract**: methods a class promises to provide, without (necessarily) saying how (`class EmailNotifier implements Notifier`). Analogy: a wall-socket standard — any plug fits any socket, whoever made them.

- Abstract methods are implicitly `public abstract`; fields implicitly `public static final` (constants).
- A class can **implement many interfaces** (but extend one class).
- **Default methods** (Java 8+): have a body implementers inherit and may override; added so interfaces can evolve without breaking implementers (`Collection.stream()`, `Iterable.forEach`).
- **Static methods** (Java 8+): belong to the interface (`Notifier.console()`), **not inherited** by implementers.
- **Private methods** (Java 9+): helpers for default methods, invisible to implementers.
- A **functional interface** has **exactly one abstract method** (methods from `Object` like `equals`, and default/static methods, do not count), so a **lambda** (a short inline unnamed function, `(to, msg) -> ...`) or **method reference** (a pointer to an existing method, `String::isBlank`) can implement it. `@FunctionalInterface` makes the compiler enforce the rule.

The problem it solves

Callers depend on **what**, not a concrete class: new implementations need no caller edits (open/closed: open to extension, closed to modification) and tests pass fakes (dependency inversion: depend on interfaces; both Chapter 4). Also safe multiple inheritance (several parents) — of **type**, not state.

How it works

Smallest method set a caller needs; implement in classes (or a lambda); type variables and parameters by the interface. If two interfaces supply the same default method, the class **must** override it and may pick one with `A.super.method()`; if a superclass declares the method, **the class wins** over interface defaults.

Java example

```java
// File: code/lld/ch02/interfaces/Notifier.java
public interface Notifier {
    String send(String to, String message);        // abstract: every implementer provides it
    default String sendAll(java.util.List<String> recipients, String message) {   // default (Java 8)
        StringBuilder sb = new StringBuilder();
        for (String r : recipients) sb.append(send(r, clean(message))).append('\n');
        return sb.toString().stripTrailing();
    }
    static Notifier console() { return (to, msg) -> "[console] " + to + ": " + msg; }  // static
    private String clean(String message) { return message.strip(); }                 // private (Java 9+)
}
// EmailNotifier.send returns "[email] to " + to + ": " + message

// File: code/lld/ch02/interfaces/DiscountRule.java
@FunctionalInterface
public interface DiscountRule {
    long apply(long priceCents);
    default DiscountRule then(DiscountRule next) { return p -> next.apply(apply(p)); }
}

// File: code/lld/ch02/interfaces/Report.java — ABSTRACT class: state + constructor + mixed methods
public abstract class Report {
    private final String title;                    // state: interfaces cannot have instance fields
    protected Report(String title) { this.title = title; }
    public final String render() {                 // final skeleton (Template Method)
        return header() + "\n" + body() + "\n-- end of " + title + " --";
    }
    protected String header() { return "== " + title + " =="; }   // concrete, overridable
    protected abstract String body();                              // must be supplied
}
// SalesReport extends Report: super("Sales"); body() returns "total: " + totalCents + " cents"

// File: code/lld/ch02/interfaces/Main.java
Notifier email = new EmailNotifier();
Notifier sms = (to, msg) -> "[sms] " + to + ": " + msg;     // Notifier is functional too
for (Notifier n : List.of(email, sms, Notifier.console())) System.out.println(n.send("ada", "your order
shipped"));
System.out.println(email.sendAll(List.of("kai", "ravi"), "  sale starts today  "));
DiscountRule tenPercent = p -> p * 90 / 100;
DiscountRule minusFiveDollars = p -> Math.max(0, p - 500);
System.out.println("10% then -$5 on 10000c = " + tenPercent.then(minusFiveDollars).apply(10_000) + "c");
Predicate<String> blank = String::isBlank;                  // JDK functional interface + method reference
System.out.println("'  ' blank? " + blank.test("  "));
System.out.println(new SalesReport(123_45).render());
```

Output:

```
[email] to ada: your order shipped
[sms] ada: your order shipped
[console] ada: your order shipped
[email] to kai: sale starts today
[email] to ravi: sale starts today
10% then -$5 on 10000c = 8500c
'  ' blank? true
== Sales ==
total: 12345 cents
-- end of Sales --
```

`sms` is a lambda implementing `Notifier`; `sendAll` (default) uses the private helper `clean`; `Notifier.console()` is a static factory. `DiscountRule` is an explicit `@FunctionalInterface` with a default combinator (a method joining two functions into one) `then`, like `Function.andThen`. `Predicate<String>` is one of the JDK's built-in functional interfaces in `java.util.function` (`Function`, `Supplier`, `Consumer`, `Predicate`, `BiFunction`, `UnaryOperator`...). `Report`/`SalesReport` show an **abstract class** (2.9).

Trade-offs

Benefit	Cost
Loose coupling (classes know little about each other), easy testing, multiple implementations	One more type to read; pointless if only one implementation will ever exist
Multiple inheritance of type	Cannot hold instance state
Default methods allow evolution	Overused defaults turn interfaces into mini base classes

When not: one implementation, no realistic second, and no test double (fake version for tests) needed = indirection (an extra layer) without benefit; add the interface when the second implementation appears.

Common mistakes

- **Fat interfaces** forcing empty methods (Interface Segregation, Chapter 4).
- "Constant interfaces" holding only constants (use a final class or enum).

Real-world: `Comparable`, `Comparator`, `Runnable`, `Callable`, `Iterable`, `AutoCloseable` (enables try-with-resources, a `try (...)` block that closes files/connections automatically), `List`/`Map`; Spring's `Repository` interfaces; SLF4J's (a logging library) `Logger`.

In the interview

> **30-second answer:** "An interface is a contract of methods. I make services depend on interfaces such as `PaymentGateway` or `PricingStrategy`, so I can add implementations without editing callers and test with fakes. Since Java 8 interfaces can have default and static methods, and an interface with one abstract method is functional, so a lambda can implement it."

- *"Why were default methods added?"* — To add methods to widely implemented interfaces (like `Collection.stream`) without breaking implementations.
- *"Two interfaces, same default method?"* — Compile error until the class overrides it; it can delegate with `X.super.m()`.

Self-check — Q: Is `Comparator` functional despite many methods? **A:** Yes; only `compare` is abstract (`equals` comes from `Object`, the rest are default/static).

TL;DR

Interfaces are contracts; depend on them. Defaults evolve them, statics provide factories/helpers, one abstract method = lambda-friendly.

2.9 Abstract classes vs interfaces

What it is

An **abstract class** is marked `abstract` and **cannot be instantiated**; it may hold fields, constructors, finished methods and **abstract methods** (no body) that subclasses must implement. Analogy: a form with some fields pre-printed and some left blank per department.

How it works

`Report` (2.8) holds state (`title`), has a constructor, a `final` skeleton `render()`, an overridable `header()` and an abstract `body()`; `SalesReport` supplies only `body()`. "Fixed skeleton, variable steps" is the **Template Method** pattern (Chapter 11).

	Abstract class	Interface
Instantiable?	No	No
Instance fields (state)	Yes	No (only `public static final` constants)
Constructors	Yes	No
Method bodies	Any	`default`, `static`, `private` only
Access modifiers on members	Any	Abstract/default/static are public; helpers can be private
How many can a class extend/implement?	**One**	**Many**
Relationship expressed	"is a" (shared identity and state)	"can do" (capability / role)
Evolution	Add concrete methods freely	Add `default` methods
Typical use	Shared state + partial implementation for a close family	Contracts, roles, strategies, test seams (where a test swaps in a fake)

Rule of thumb: start with an **interface** for the contract; add an **abstract class** underneath when implementations share state or a fixed algorithm (JDK: `List` + `AbstractList` skeleton + `ArrayList`). Callers still depend on the interface.

> **Common mistakes:** an abstract class just to share one helper (use composition or a static helper); making everything abstract "for flexibility".

In the interview

> **30-second answer:** "Interfaces define capabilities and allow multiple implementation; abstract classes share state and partial implementation but allow only one parent. I default to interfaces and add an abstract base class when implementations share real state or a template algorithm."

Self-check

- **Q:** Can an abstract class have no abstract methods? **A:** Yes; it just cannot be instantiated.
- **Q:** Can an interface have a constructor? **A:** No.

TL;DR

Interface = contract, many allowed, no state. Abstract class = partial implementation with state, one allowed. Interface first; abstract base class when sharing real code.

2.10 Enums with fields and behaviour

What it is

An **enum** (enumerated type) has a **fixed set of named instances** (`Coin.PENNY`, `OrderStatus.SHIPPED`). In Java it is a full class: **fields, constructors, methods, interfaces**, even a method body per constant. Analogy: coin denominations — no 7-cent coin.

The problem it solves

String/int constants (`"PAID"`, `2`) allow typos and invalid values and carry no behaviour. Enums are **type-safe** (only valid values compile) and hold each value's data and rules, removing scattered `switch`es — the LLD home for "**different data, not different behaviour**" (spot, vehicle, coin types, statuses).

How it works

1. Implicitly extends `java.lang.Enum` (cannot extend anything else); can implement interfaces. Constructors are always private; constants are created once at initialisation.
2. Built-ins: `values()`, `valueOf(String)` (throws `IllegalArgumentException` for unknown names), `name()`, `ordinal()` (0-based position; **never persist** it — save to a database or file — since reordering changes it), `compareTo` (by ordinal).
3. Compare with `==` (one instance per constant). 5. Constant-specific bodies (`PAID { ... }`) override an abstract method: a tiny state machine (fixed states + allowed moves) or strategy.
4. `EnumMap` / `EnumSet`: compact, fast enum-keyed collections.
5. A `switch` expression covering every constant is **exhaustive**: no `default`, and a new constant makes the compiler flag missing cases.
6. A single-element enum is the simplest thread-safe, serialization-safe (serialization turns an object into bytes to store or send) **Singleton** (exactly one instance; Chapter 8).

Java example

```java
// File: code/lld/ch02/enums/Coin.java — FIELD + private constructor + method
public enum Coin {
    PENNY(1), NICKEL(5), DIME(10), QUARTER(25);
    private final int cents;
    Coin(int cents) { this.cents = cents; }
    public int cents() { return cents; }
}

// File: code/lld/ch02/enums/OrderStatus.java — BEHAVIOUR per constant (tiny state machine)
public enum OrderStatus {
    PLACED    { @Override Set<OrderStatus> next() { return Set.of(PAID, CANCELLED); } },
    PAID      { @Override Set<OrderStatus> next() { return Set.of(SHIPPED, CANCELLED); } },
    SHIPPED   { @Override Set<OrderStatus> next() { return Set.of(DELIVERED); } },
    DELIVERED { @Override Set<OrderStatus> next() { return Set.of(); } },
    CANCELLED { @Override Set<OrderStatus> next() { return Set.of(); } };
    abstract Set<OrderStatus> next();
    public boolean canMoveTo(OrderStatus target) { return next().contains(target); }
    public boolean isFinal() { return next().isEmpty(); }
}

// File: code/lld/ch02/enums/Main.java
List<Coin> inserted = List.of(Coin.QUARTER, Coin.QUARTER, Coin.DIME, Coin.PENNY);
int total = inserted.stream().mapToInt(Coin::cents).sum();          // printed with the list
Map<Coin, Integer> counts = new EnumMap<>(Coin.class);             // ordered by declaration
for (Coin c : inserted) counts.merge(c, 1, Integer::sum);          // printed as "counts: ..."
// prints Coin.valueOf("DIME") and Coin.DIME.ordinal()
OrderStatus s = OrderStatus.PLACED;
for (OrderStatus target : List.of(OrderStatus.SHIPPED, OrderStatus.PAID, OrderStatus.SHIPPED,
OrderStatus.DELIVERED)) {
    if (s.canMoveTo(target)) { s = target; System.out.println("moved to " + s); }
    else System.out.println("refused " + s + " -> " + target);
}
System.out.println(s + " is final? " + s.isFinal());
String label = switch (s) {                          // exhaustive: no default needed
    case PLACED, PAID -> "in progress";
    case SHIPPED -> "on the way";
    case DELIVERED -> "done";
    case CANCELLED -> "stopped";
};
System.out.println("label: " + label);
```

Output:

```
inserted [QUARTER, QUARTER, DIME, PENNY] = 61 cents
counts: {PENNY=1, DIME=1, QUARTER=2}
valueOf("DIME") = DIME, ordinal 2
refused PLACED -> SHIPPED
moved to PAID
moved to SHIPPED
moved to DELIVERED
DELIVERED is final? true
label: done
```

Summing is `Coin::cents`, no switch; `EnumMap` prints in declaration order; illegal transitions (`PLACED → SHIPPED`) are refused in one method — the "enum plus a guarded method" alternative to the full State pattern that Chapter 1 recommends for small lifecycles.

Trade-offs

Benefit	Cost
Type-safe, self-documenting, holds data/behaviour	Closed set: a new value means recompiling
Great for small state machines and lookup tables	Large per-constant behaviour gets hard to read; move to State/Strategy classes

When not: values **open-ended or user-defined at run time** (admin-created product categories), or each needing substantial independently evolving behaviour (classes).

> **Common mistakes:** persisting `ordinal()` (store `name()` or an explicit code field); giant `switch`es over an enum in many places instead of a method on it.

Real-world: `java.time.DayOfWeek`, `Month` (with `plus`), `java.util.concurrent.TimeUnit` (`TimeUnit.SECONDS.toMillis(5)`), `java.nio.file.StandardOpenOption`, `Thread.State`.

In the interview

> **30-second answer:** "I use enums for fixed sets like vehicle types, spot types and statuses. Java enums can carry fields and methods, so data like 'which spots fit this vehicle' or 'which statuses can follow this one' lives on the enum rather than in switches around the code."

- *"Enum or class hierarchy for vehicle types?"* — Enum if they differ only in data; classes if they behave differently.

Self-check — Q: `Coin.valueOf("dime")`? **A:** Throws `IllegalArgumentException` (names are case-sensitive).

TL;DR

Enums are type-safe fixed sets carrying data and behaviour: "different data, same behaviour"; compare with `==`; never persist ordinals.

2.11 Composition over inheritance

What it is

Composition builds a class by **holding references to other objects** and delegating ("has-a") instead of inheriting ("is-a"). **"Favour composition over inheritance"** comes from the Gang of Four book and is core to *Head First Design Patterns*. A phone **has** an upgradable camera module; it is not a kind of camera.

The problem it solves

Inheritance couples a child to the parent's **internal implementation** (the **fragile base class problem**: a parent change or hidden detail silently breaks the child) and is compile-time and single-parent. Composition keeps classes independent, **swaps parts at run time**, combines behaviours freely.

How it works

Identify the behaviour to reuse or vary → put it behind an interface (`Engine`, `Set`) → hold it in a private field received via the constructor (replaceable or fakeable) → **forward** (delegate) calls to it, adding your logic around them.

Java example

```
// File: code/lld/ch02/composition/InheritingCountingSet.java — BROKEN on purpose
public class InheritingCountingSet<E> extends HashSet<E> {
    @Serial private static final long serialVersionUID = 1L;   // HashSet is Serializable
    private int attempts = 0;
    @Override public boolean add(E e) { attempts++; return super.add(e); }
    @Override public boolean addAll(Collection<? extends E> c) { attempts += c.size(); return super.addAll(c); }
    public int attempts() { return attempts; }
}

// File: code/lld/ch02/composition/CountingSet.java — FIXED: HAS-A Set, works with ANY Set
public class CountingSet<E> {
    private final Set<E> inner;
    private int attempts = 0;
    public CountingSet(Set<E> inner) { this.inner = inner; }
    public boolean add(E e) { attempts++; return inner.add(e); }
    public boolean addAll(Collection<? extends E> c) {
        attempts += c.size();
        boolean changed = false;
        for (E e : c) changed |= inner.add(e);     // talk to the inner set, not to our own add()
        return changed;
    }
    // size() -> inner.size(); attempts()
}

// File: code/lld/ch02/composition/Car.java — a car HAS an Engine (interface: String start())
public class Car {
    private final String model;
    private Engine engine;
    public Car(String model, Engine engine) { this.model = model; this.engine = engine; }
    public void replaceEngine(Engine e) { this.engine = e; }
    public String drive() { return model + ": " + engine.start(); }
}

// File: code/lld/ch02/composition/Main.java
bad.addAll(List.of("a", "b", "c"));                          // bad = new InheritingCountingSet<>()
good.addAll(List.of("a", "b", "c")); good.add("a");          // good = new CountingSet<>(new TreeSet<>())
// prints bad.attempts(), then good.attempts() and good.size()
Car car = new Car("City", () -> "petrol engine vroom");
System.out.println(car.drive());
car.replaceEngine(() -> "electric motor hum");
System.out.println(car.drive());
```

Output:

```
inheritance: added 3, attempts counted = 6
composition: 4 attempts counted = 4, size = 3
City: petrol engine vroom
City: electric motor hum
```

Why 6: `HashSet` inherits `addAll` from `AbstractCollection`, which calls `add` per element, and the overridden `add` counts again — the subclass depended on **how the parent uses its own methods internally** ("self-use"), documented only in an `AbstractCollection` implementation note that `HashSet` does not

promise to keep, so a future version could silently fix or break the count. `CountingSet` forwards to any `Set` (here a `TreeSet`); `Car`'s engine swaps at run time, which inheritance cannot do.

Trade-offs

	Inheritance	Composition
Relationship	is-a	has-a
Coupling	Tight (to parent internals)	Loose (to an interface)
Change at run time	No	Yes
Boilerplate (repetitive filler code)	Low	Forwarding methods
Polymorphism	Built in	Via the interface the parts implement

When not: a **genuine is-a relationship** designed for extension (e.g. an abstract template class meant to be subclassed): inheritance is fine and simpler.

> **Common mistakes:** extending a concrete library class (`HashMap` , `ArrayList`) to add a feature; composing but exposing the inner object (`getInner()`), losing control again.

Real-world: `java.io.BufferedReader` wraps any `Reader` (Decorator — a wrapper adding behaviour around an object — is composition); `Collections.unmodifiableList` wraps a list; Spring beans (objects the framework creates) composed via dependency injection (the framework hands each object its collaborators); `Comparator` strategies passed into `TreeMap`.

In the interview

> **30-second answer:** "I prefer composition: a class holds an interface-typed collaborator and delegates to it. It avoids the fragile base class problem, lets me swap behaviour at run time, and most patterns such as Strategy, Decorator and State are built on it. I use inheritance only for true is-a relationships."

- *"Show where inheritance breaks."* — The counting set: overriding `add` and `addAll` double counts because the parent's `addAll` calls `add`.

Self-check — Q: Why does `CountingSet.addAll` call `inner.add`, not `this.add`? **A:** To avoid double counting and depend only on the inner set's public contract.

TL;DR

Has-a + delegate beats is-a for reuse. Inheritance only for real, designed-for-extension type families.

2.12 Object identity vs equality: the `equals` / `hashCode` contract

What it is

- **Identity:** do two references point at the **same object**? `==`.
- **Equality:** do two objects **represent the same value**? `equals`. `Object.equals` defaults to identity; value classes override it (`String` , `Integer` , `LocalDate` , records).

Two printed copies of a book are **equal**, not **identical**.

The problem it solves

Hash collections (`HashMap` , `HashSet`) store entries in an array of slots called **buckets**: they compute the object's **hash code** (an `int` derived from its fields), jump straight to one bucket, then compare the few objects there with `equals`. That makes lookups fast — and means equal objects must have equal hash codes, or the search starts in the wrong bucket and objects "disappear" from sets and maps.

How it works: the contracts

`equals` **must be:** 1. **Reflexive** (`x.equals(x)`); 2. **Symmetric** (`x.equals(y)` ⇔ `y.equals(x)`); 3. **Transitive** (x=y and y=z ⇒ x=z); 4. **Consistent** (same answer while compared fields are unchanged); 5. **Non-null** (`x.equals(null)` is false).

`hashCode` **must:** 1. return the same value on repeated calls in one run if `equals` fields are unchanged; 2. give **equal hash codes for equal objects**; 3. unequal objects **may** collide (allowed, just slower).

So: override `equals` ⇒ override `hashCode` on the same fields. Records and IDE generators ("generate equals/hashCode" in a code editor) do this for you.

Java example

```java
// File: code/lld/ch02/equality/Point.java — equals and hashCode TOGETHER, same fields
@Override public boolean equals(Object o) {
    if (this == o) return true;                          // reflexive fast path
    if (!(o instanceof Point p)) return false;           // also handles null
    return x == p.x && y == p.y;
}
@Override public int hashCode() { return Objects.hash(x, y); }

// File: code/lld/ch02/equality/BrokenPoint.java — WRONG on purpose: equals without hashCode
@SuppressWarnings("overrides")    // javac -Xlint warns [overrides]; silenced ONLY for this demo
public final class BrokenPoint {
    private final int x, y;
    public BrokenPoint(int x, int y) { this.x = x; this.y = y; }
    @Override public boolean equals(Object o) { return o instanceof BrokenPoint p && x == p.x && y == p.y; }
    // hashCode() inherited from Object: usually different for two distinct objects
}

// File: code/lld/ch02/equality/Main.java (abridged)
Point a = new Point(1, 2), b = new Point(1, 2);
// prints a == b, a.equals(b), a.hashCode() == b.hashCode()
Set<Point> good = new HashSet<>(); good.add(a); good.add(b);     // prints size and contains(new Point(1, 2))
BrokenPoint x = new BrokenPoint(1, 2), y = new BrokenPoint(1, 2);
Set<BrokenPoint> bad = new HashSet<>(); bad.add(x);              // prints x.equals(y) and bad.contains(y)
String s1 = "hi", s2 = new String("hi");                         // prints s1 == s2 and s1.equals(s2)
Integer i1 = 1000, i2 = 1000;                                    // prints i1.equals(i2)
```

Output:

```
a == b (identity): false
a.equals(b) (equality): true
hash codes equal: true
HashSet<Point> size after adding a and b: 1, contains (1,2)? true
BrokenPoint equals: true, but set.contains(equal one): false
"hi" == new String("hi"): false, equals: true
Integer 1000 equals: true  (never compare boxed numbers with ==)
```

`BrokenPoint`'s inherited identity-based `hashCode` sends an equal object to a different bucket, so `contains` fails; `javac -Xlint` (extra warnings on) flags it (`[overrides] ... overrides equals, but neither it nor any superclass overrides hashCode`) — treat that warning as an error. Compare strings and boxed numbers with `equals`: `Integer` caches (reuses ready-made objects for) only small values (at least −128 to 127), so `==` works for small values and fails for large ones.

Design choice

Kind of class	Equality
Value object (Money, Point, Coordinate)	Value equality on all fields; ideally immutable
Entity (User, Order, ParkingSpot)	Identity, or equality by a stable **ID** only

Common mistakes

- `equals` without `hashCode` (above); `equals(Point p)` instead of `equals(Object o)` (an overload collections never call).
- **Mutable fields** in `hashCode`, then mutating an object already used as a `HashMap` key (it becomes unfindable).
- Mixing `getClass() !=` and `instanceof` across a hierarchy, breaking symmetry.

Real-world: `String`, `Integer`, `BigDecimal` (`new BigDecimal("1.0").equals(new BigDecimal("1.00"))` is false because scale — the number of decimal places — is part of equality; use `compareTo` for numeric comparison), `LocalDate`, `List.equals` (element-wise).

In the interview

> **30-second answer:** "`==` compares identity, `equals` compares logical value. If I override `equals` I must override `hashCode` on the same fields, because hash collections use the hash to find the bucket and `equals` inside it. For value objects I use records; for entities I compare by ID."

- *"Only `equals` overridden?"* — Equal objects can land in different buckets; `HashSet.contains` and `HashMap.get` fail.

Self-check

- **Q:** Can unequal objects share a hash code? **A:** Yes; a collision, allowed.
- **Q:** Can equal objects have different hash codes? **A:** No; that breaks the contract.

TL;DR

`==` = same object; `equals` = same value. Override `equals` and `hashCode` together, on the same (preferably immutable) fields.

2.13 Immutability and records

What it is

An **immutable** object cannot change after construction; "changes" produce **new** objects. A **record** (Java 16+) concisely declares an immutable **data carrier**: `record Order(String id, List<String> items, Money total) {}`. Analogy: a printed receipt — to correct it, print a new one.

The problem it solves

Shared mutable objects break when one part changes what another relies on, and need locks across threads. Immutable ones (all fields `final`, `this` not leaked during construction) are **thread-safe without locks**, safe as `HashMap` keys/`HashSet` elements, easy to cache and reason about.

How it works

Hand-written immutable class (checklist):

1. Class `final` (or private constructors + static factories) so no subclass adds mutability.
2. All fields `private final`.
3. No setters; "modifying" methods return a new object (`plus`, `withItem`).
4. **Defensive copies:** copy mutable inputs in the constructor; never hand out internal mutable objects (copies or unmodifiable views).
5. Do not let `this` escape during construction.

Records generate `private final` fields per component, a canonical constructor, accessors named after components (`id()`, not `getId()`), and `equals`/`hashCode`/`toString` over all components. They are implicitly `final`, extend `java.lang.Record`, cannot declare extra **instance** fields (static fields and extra methods are fine), and can implement interfaces. A **compact constructor** (`public Order { ... }`)

validates or normalises parameters before assignment. **Records are only shallowly immutable:** a mutable `List` component can still change unless copied (`List.copyOf`).

Java example

```
// File: code/lld/ch02/immutability/Money.java — hand-written immutable VALUE OBJECT
public final class Money {
    private final BigDecimal amount;
    private final String currency;
    private Money(BigDecimal amount, String currency) {
        this.currency = Objects.requireNonNull(currency);
        // UNNECESSARY = refuse (ArithmeticException) anything finer than a cent instead of silently rounding
        this.amount = amount.setScale(2, RoundingMode.UNNECESSARY);
    }
    public static Money of(String amount, String currency) { return new Money(new BigDecimal(amount), currency); }
    public Money plus(Money other) {
        if (!currency.equals(other.currency)) throw new IllegalArgumentException("currency mismatch");
        return new Money(amount.add(other.amount), currency);   // new object; 'this' untouched
    }
    // equals/hashCode on amount + currency (safe: scale is always 2); toString = amount.toPlainString() + " " +
currency
}

// File: code/lld/ch02/immutability/Order.java — a RECORD with a compact constructor
public record Order(String id, List<String> items, Money total) {
    public Order {
        if (id == null || id.isBlank()) throw new IllegalArgumentException("id required");
        items = List.copyOf(items);                // DEFENSIVE COPY: unmodifiable, caller's list cannot leak in
    }
    public Order withItem(String item, Money price) {  // "wither": returns a modified copy
        var more = new java.util.ArrayList<>(items);
        more.add(item);
        return new Order(id, more, total.plus(price));
    }
}

// File: code/lld/ch02/immutability/Main.java (abridged)
Money five = Money.of("5.00", "USD");
Money seven = five.plus(Money.of("2.00", "USD"));     // prints both; then Money.of("5", "USD").equals(five)
List<String> callerList = new ArrayList<>(List.of("pen"));
Order o1 = new Order("O-1", callerList, five);
callerList.add("SNEAKY");                              // does not reach the record; prints o1
Order o2 = o1.withItem("ink", Money.of("1.50", "USD"));   // prints o2
// prints o1.items() and o1.equals(new Order("O-1", List.of("pen"), Money.of("5.00", "USD")))
try { o1.items().add("hack"); }
catch (UnsupportedOperationException e) { System.out.println("items() is unmodifiable: " +
e.getClass().getSimpleName()); }
```

Output:

```
five = 5.00 USD, seven = 7.00 USD  (five unchanged)
Money.of("5") equals Money.of("5.00")? true  (scale normalised)
o1 = Order[id=O-1, items=[pen], total=5.00 USD]
o2 = Order[id=O-1, items=[pen, ink], total=6.50 USD]
o1 still = [pen], equals copy? true
items() is unmodifiable: UnsupportedOperationException
```

Explaining each part: `Money.plus` returns a new object; `five` stays 5.00. `BigDecimal` is exact (`double` cannot represent 0.10). **Scale** (decimal places) is fixed at 2 because `BigDecimal.equals` compares scale (2.12); `RoundingMode.UNNECESSARY` throws rather than silently rounding away a fraction of a cent. Private constructor + `Money.of(...)` is the checklist's **static factory** option. `List.copyOf` keeps `"SNEAKY"` out and makes `items()` unmodifiable; records with equal components are `equals`.

Trade-offs

Benefit	Cost
Thread-safe without locks; safe hash keys; no defensive reasoning	New object per change (usually cheap; can matter in hot loops, code run millions of times)
Records remove boilerplate	Records cannot hide their components or extend classes

When not: entities whose identity persists while state changes (a `ParkingSpot` becoming occupied, an `Account` balance) are naturally **mutable**, with encapsulated changes — unless you deliberately adopt event sourcing (state rebuilt from a stored list of past events) or copy-on-write (every change makes a new copy).

> **Common mistakes:** forgetting defensive copies (a "final" list whose contents anyone can change); assuming `final` makes the referenced object immutable (it only fixes the reference); records for entities needing controlled mutation.

Real-world: `String`, boxed primitives (`Integer`, `Long`), `BigDecimal`, `BigInteger`, all of `java.time` (`LocalDate`, `Instant`, `Duration`), `List.of`/`Map.of`; Kafka (a message-streaming system) messages and event-sourcing events are immutable facts.

In the interview

> **30-second answer:** "For value objects like Money, Coordinate or Ticket I use immutable records: final fields, no setters, validation in the compact constructor, and defensive copies of collections. They are thread-safe and safe as map keys. Entities whose state changes stay mutable but encapsulated."

- *"Is a record fully immutable?"* — Shallowly; copy mutable components.

Self-check — Q: What does `final List<String> items` guarantee? **A:** The reference never changes; the contents still can.

TL;DR

Immutable = final class, private final fields, no setters, defensive copies. Records give this (shallowly) in one line. Value objects immutable; entities mutable but encapsulated.

2.14 Access modifiers and packages

What it is

A **package** is a named group of related classes (a namespace and a folder), e.g. `lld.ch02.access.core`. **Access modifiers** control who sees a class or member.

Modifier	Same class	Same package	Subclass in another package	Everyone
`private`	✔	✘	✘	✘
(none) package-private	✔	✔	✘	✘
`protected`	✔	✔	✔ (through inheritance)	✘
`public`	✔	✔	✔	✔

Top-level classes are only `public` or package-private. (Desk drawer, team room, shared floor, lobby.) They make encapsulation **compiler-enforced**; packages group classes by feature, hiding implementation classes behind a small public API.

How it works

1. Start everything `private`; widen only when needed.
2. Group by **feature** (`parking`, `payment`), not only by layer.
3. Keep helpers package-private behind a small `public` facade (one simple front class hiding the others).
4. `protected` is for members designed for subclasses — and **also** opens them to the whole package.
5. Since Java 9, **modules** (`module-info.java`) add a level: only exported packages are visible to other modules.

Java example

```
// File: code/lld/ch02/access/core/Wallet.java — all four levels on one class
public class Wallet {
    public final String owner;                // public: everywhere
    protected long bonusCents;                // protected: same package + subclasses anywhere
    long auditCount;                          // package-private: same package only
    private long balanceCents;                // private: this class only
    public void add(long cents) {
        if (cents <= 0) throw new IllegalArgumentException("positive only");
        balanceCents += cents;
        AuditLog.record(this, "add " + cents);   // package-private helper class, invisible outside core
    }
    protected String describe() { return owner + " has " + balanceCents + "c (+" + bonusCents + "c bonus)"; }
    // public balanceCents()
}
// core/AuditLog.java: package-private final class; record() does w.auditCount++ and keeps "owner: what" lines
// core/Audits.java: PUBLIC facade — all() returns the lines, countFor(w) returns w.auditCount

// File: code/lld/ch02/access/client/PremiumWallet.java — subclass in ANOTHER package
public class PremiumWallet extends Wallet {
    public void grantBonus(long cents) {
        bonusCents += cents;                  // OK: protected, through inheritance
        // auditCount++;                      // would NOT compile: package-private
        // balanceCents += cents;             // would NOT compile: private
    }
    @Override public String describe() { return "[premium] " + super.describe(); }  // widens protected -> public
}

// File: code/lld/ch02/access/client/Main.java (abridged)
// Ada's Wallet: add(1_000). Kai's PremiumWallet: add(2_000), add(500), grantBonus(300).
// prints basic.owner + basic.balanceCents(), premium.describe(), Audits.all(), Audits.countFor(premium)
// new lld.ch02.access.core.AuditLog();  // would NOT compile: class is package-private
```

Run with `java -cp /tmp/lld_02 lld.ch02.access.client.Main`. Output:

```
public field owner: Ada, public method balance: 1000
[premium] Kai has 2500c (+300c bonus)
audit lines: [Ada: add 1000, Kai: add 2000, Kai: add 500]
audits for Kai: 2
```

Only `core` touches `AuditLog` and `auditCount`; outsiders read audits via the public `Audits` facade. `PremiumWallet` may update `protected bonusCents`, not the package-private or private fields, and its override widens `protected` to `public`, which overriding allows.

Common mistakes

- Everything `public` "to be safe".
- `protected` fields as a shortcut instead of protected methods (subclasses then break invariants).
- Every class of a machine-coding solution in one package.

In the interview

> **30-second answer:** "Fields are private by default, collaborators within a feature package can be package-private, protected is only for members designed for subclassing, and public is the

deliberate API. I organise machine-coding solutions into packages such as model, repository, service and strategy."

Self-check — Q: Can a non-subclass in another package read a `protected` field? **A:** No.

TL;DR

private < package-private < protected < public. Default to private; a small public API per package.

2.15 Generics basics for LLD

What it is

Generics let a class, interface or method work with **any type while keeping compile-time type safety**: `List<String>` holds only strings; `Repository<String, User>` only users. `T`, `ID`, `E`, `K`, `V` in angle brackets are **type parameters**. (A "SHOES ONLY" box: checked going in, so not coming out.)

The problem it solves

Without them you store `Object` (root of every class) and cast on every read, failing at run time (`ClassCastException`); with them mistakes are **compile errors** and one `Repository<ID, T>` serves every entity — handy in machine coding.

How it works

1. **Generic class/interface:** `class Repository<ID, T extends Identifiable<ID>>`.
2. **Bounded type parameter:** `T extends X` — T is X or a subtype, so X's methods can be called on T.
3. **Generic method:** `static <T extends Comparable<? super T>> T max(List<? extends T> items)`.
4. **Wildcards:** `? extends T` (read-only producer), `? super T` (write-capable consumer), `?` (unknown). **PECS**: *Producer Extends, Consumer Super* (from *Effective Java*, Joshua Bloch's Java best-practice book).
5. **Diamond operator:** `new Repository<>()` infers type arguments.
6. **Type erasure:** type arguments exist only at compile time; at run time `List<String>` and `List<Integer>` are one class. Hence no `new T()`, no `instanceof List<String>`, no primitive type arguments (`Integer`, not `int`).
7. **Avoid raw types** (`List` without `<...>`): they switch the checks off.

Java example

```java
// File: code/lld/ch02/generics/Repository.java — generic in-memory repository; T BOUNDED
public class Repository<ID, T extends Identifiable<ID>> {    // Identifiable<ID> { ID id(); }
    private final Map<ID, T> store = new LinkedHashMap<>();
    public void save(T item)           { store.put(item.id(), item); }
    public Optional<T> findById(ID id)  { return Optional.ofNullable(store.get(id)); }
    public List<T> findAll()           { return List.copyOf(store.values()); }
}
// record User(String id, String name) implements Identifiable<String>
// record Product(Integer id, String title, long priceCents) implements Identifiable<Integer>, Comparable<Product>
(by price)

// File: code/lld/ch02/generics/Main.java
static <T extends Comparable<? super T>> T max(List<? extends T> items) {   // generic METHOD, bounded
    T best = items.get(0);
    for (T t : items) if (t.compareTo(best) > 0) best = t;
    return best;
}
static <T> void copy(List<? extends T> src, List<? super T> dst) { dst.addAll(src); }   // PECS

Repository<String, User> users = new Repository<>();                // saves u1 Ada, u2 Kai
System.out.println("user u2: " + users.findById("u2").map(User::name).orElse("none"));
System.out.println("user u9: " + users.findById("u9").map(User::name).orElse("none"));
Repository<Integer, Product> products = new Repository<>();          // pen 150, lamp 2_500, mug 800
System.out.println("most expensive: " + max(products.findAll()).title());
List<Integer> ints = List.of(3, 1, 2);
List<Number> numbers = new ArrayList<>();
copy(ints, numbers);                            // Integer list into a Number list
System.out.println("copied into List<Number>: " + numbers + ", max int = " + max(ints));
// users.save(new Product(4, "x", 1));          // would NOT compile
List<String> a = new ArrayList<>();
List<Integer> b = new ArrayList<>();
System.out.println("same runtime class (type erasure)? " + (a.getClass() == b.getClass()));
```

Output:

```
user u2: Kai
user u9: none
most expensive: lamp
copied into List<Number>: [3, 1, 2], max int = 3
same runtime class (type erasure)? true
```

One `Repository` stores `User`s by `String` and `Product`s by `Integer`; `findById` returns `Optional` (a box holding a value or empty, instead of `null`) for a normal "not found". `max` works for any comparable type; `copy` takes `List<Integer>` as producer and `List<Number>` as consumer (PECS); the last line shows erasure.

Common mistakes

- Raw types and unchecked casts.
- Thinking `List<Integer>` is a subtype of `List<Number>` (it is not; use `List<? extends Number>`).
- Over-generic designs with five unreadable type parameters.

Real-world: the Collections framework (`List<E>`, `Map<K,V>`), `Optional<T>`, `CompletableFuture<T>` (a result arriving later from background work), `Comparator<T>`, Spring Data's `CrudRepository<T, ID>` (same idea as our `Repository`).

In the interview

> **30-second answer:** "I use generics for reusable infrastructure like an in-memory `Repository<ID, T>` or an `EventBus<E>`, so type errors are caught at compile time. I know bounded types, wildcards with PECS, and that erasure removes type arguments at run time."

Self-check — Q: Why no `new T()`? **A:** Type erasure: the runtime does not know T; pass a `Supplier<T>` instead.

TL;DR

Generics = type-safe reuse. Bounds let you call methods on T; PECS for wildcards; erasure means no runtime type info.

2.16 Coupling and cohesion (the yardsticks)

- **Cohesion** — how closely the responsibilities **inside** one class belong together. **High cohesion is good**: every method works on the same data toward one purpose (Chapter 1's "one sentence, no 'and'" test).
- **Coupling** — how much one class **depends on** others' details. **Low coupling is good**: classes talk through small interfaces, so a change does not ripple.

Everything in this chapter pushes toward **high cohesion, low coupling**: encapsulation and access modifiers cut coupling; single-responsibility classes raise cohesion; interfaces and composition decouple callers from implementations. Chapters 4–5 turn these into named principles (SOLID, Law of Demeter, GRASP).

2.17 Object-oriented analysis and design (OOA / OOD / OOP)

What it is

Object-oriented analysis and design (OOAD) is the long-established method for turning a written description into objects and links between them; most LLD vocabulary comes from it. It has two activities, then implementation:

Activity	Question it answers	What exists at the end	Talks about
Analysis (OOA)	What does the system consist of, and what must it do?	The things in the problem, in **business (domain) terms**	books, members, loans
Design (OOD)	Which objects, with which responsibilities, provide that behaviour?	Classes, responsibilities, relationships	`Book`, `Member`, `Loan` and who holds whom
Implementation (OOP)	How do we express it in a language?	Running code	`ArrayList`, `HashMap`, `synchronized`

A `HashMap` two minutes in means you **skipped the two activities the interviewer wanted to watch**; and operations chosen before the objects settle land on the wrong class, expensive to move.

How it works: the four steps

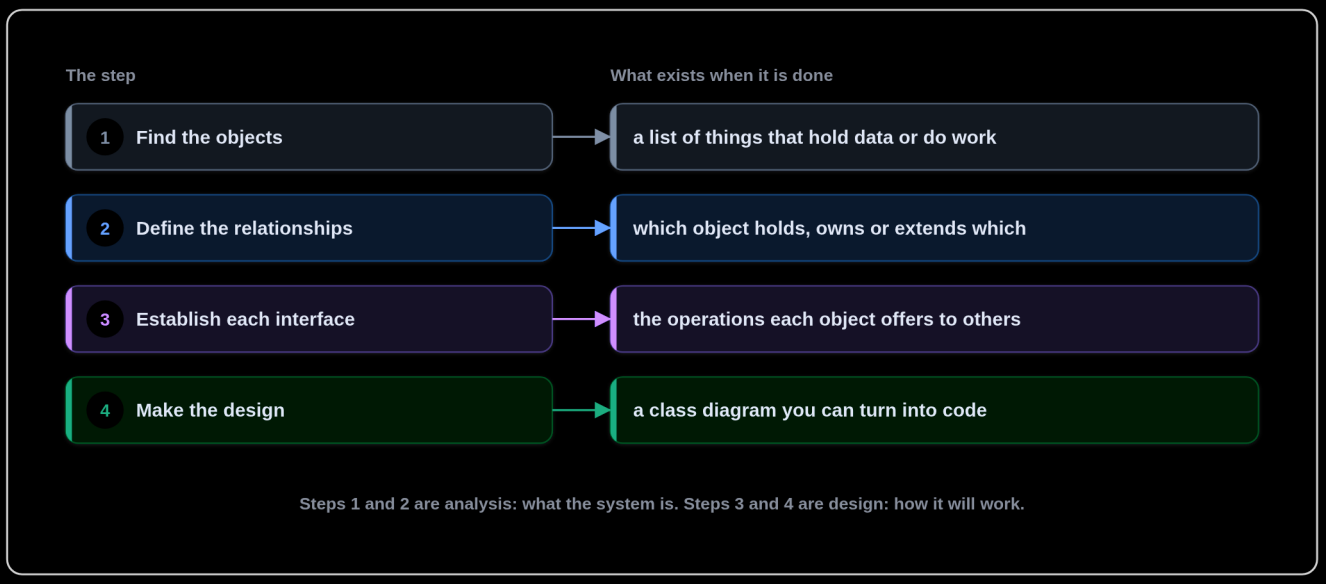


Figure — The four OOAD steps and what exists after each; steps 1–2 are analysis and steps 3–4 are design (course diagram)

1. **Find the objects** — everything that **stores information or performs work** (mostly nouns). Something carrying a single value (a colour, a date) is a **field**, not an object.
2. **Define the relationships** — for each pair mentioned together: who holds a reference to whom, does it **own** it, **how many** of each? (A library holds many books; a loan points at one book and one member.)
3. **Establish each interface** — here *interface* means the operations an object offers **to other objects**. Everything left off is **private**, so this is where encapsulation is really decided.
4. **Make the design** — classes, fields, operations and relationships in one drawing (a UML class diagram), checked against the original description. The code derives from it.

Steps 1–2 are analysis (what the system is), 3–4 design (how it works). **The order is fixed: each step consumes the previous one's output.**

Worked example: "library members borrow books; each book on loan must be returned by a due date"

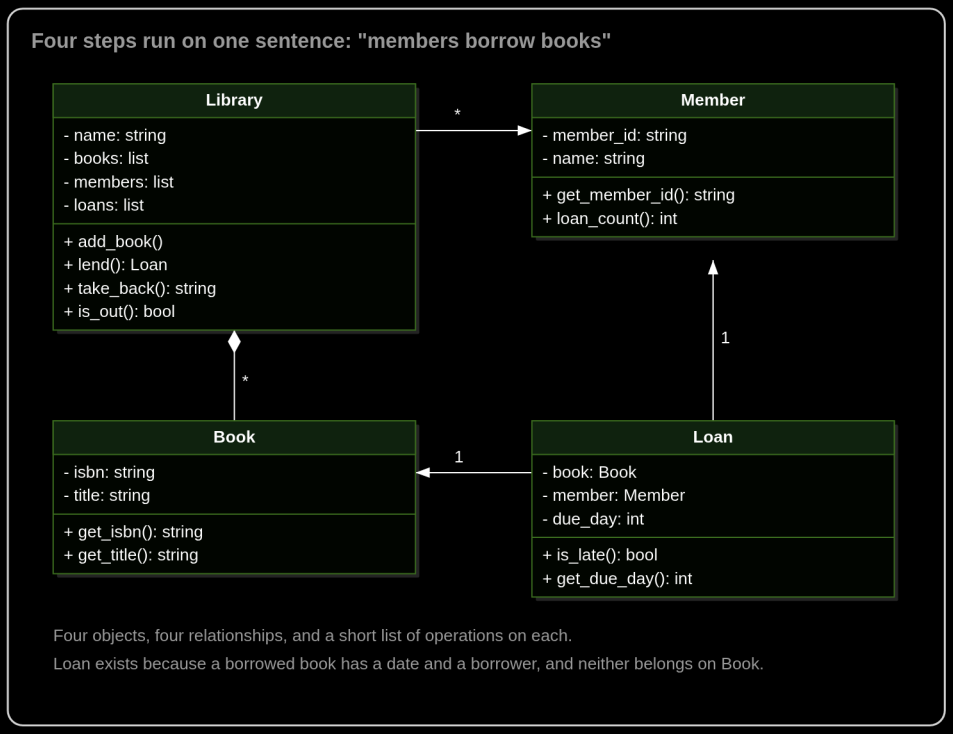


Figure — The four steps applied to one sentence: Library, Member, Book and the connection object Loan (course diagram)

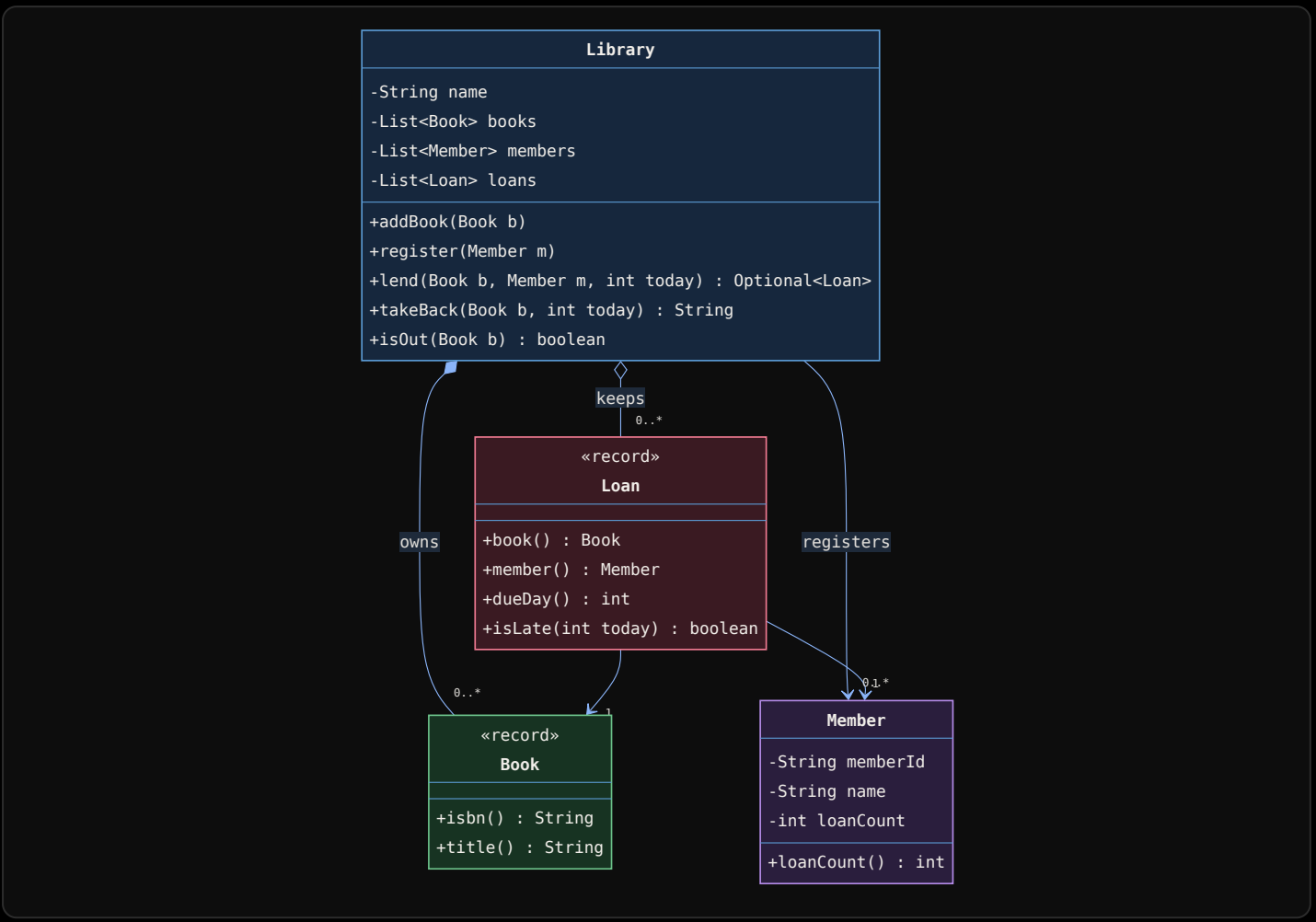
Step 1, objects. Candidates: member, library, book, date, borrowed book. *Date* is one value → field. That leaves `Library`, `Member`, `Book`, and the borrowed book, better named **`Loan`** — the one most often missed, since the sentence never uses it as a noun.

> **General rule:** if two objects are linked and **the link has data of its own**, model the link as an object. A lent book has a borrower and a due date; neither fits on `Book`, because a shelved book has neither. Also: `Reservation` (hotel), `Ticket` (parking lot), `Enrollment` (student–course).

Step 2, relationships. Library **owns** its book records (they disappear with it): **composition**, filled diamond. It **registers** members without owning them (people outlive it): **association** arrow. Each loan **points to** one book and one member, owning neither.

Step 3, interfaces. Library: lend, accept a return, report whether a copy is out. Book: title and ISBN (International Standard Book Number). Member: number of loans. Loan: is it overdue. Everything else, all fields included, private. Note **who answers "is this book out?"**: not `Book` (it knows nothing of loans) but `Library`, which keeps the loans.

Step 4, design. The figure answers the sentence with no code. Our Java version (adding `register` so members can be checked):



```java
// File: code/lld/ch02/ooad/Loan.java — the CONNECTION OBJECT (data belonging on neither Book nor Member)
public record Loan(Book book, Member member, int dueDay) {
    public boolean isLate(int today) { return today > dueDay; }
}
// Book: record(isbn, title). Member: memberId, name, loanCount; package-private loanStarted()/loanEnded()
// so only Library changes the count.

// File: code/lld/ch02/ooad/Library.java (essentials; addBook/register just add to the lists)
public static final int LOAN_DAYS = 14;
public static final int MAX_LOANS_PER_MEMBER = 5;
public boolean isOut(Book b) { return loans.stream().anyMatch(l -> l.book().equals(b)); }  // only Library knows
public Optional<Loan> lend(Book b, Member m, int today) {
    if (!books.contains(b) || !members.contains(m)) return Optional.empty();
    if (isOut(b) || m.loanCount() >= MAX_LOANS_PER_MEMBER) return Optional.empty();
    Loan loan = new Loan(b, m, today + LOAN_DAYS);
    loans.add(loan);
    m.loanStarted();
    return Optional.of(loan);
}
public String takeBack(Book b, int today) {
    for (Loan l : loans) {
        if (l.book().equals(b)) {
            loans.remove(l);
            l.member().loanEnded();
            return b.title() + " returned " + (l.isLate(today) ? "LATE" : "on time");
        }
    }
    return b.title() + " was not on loan";
}

// File: code/lld/ch02/ooad/Main.java — books ddd "Domain-Driven Design", gof "Design Patterns";
// members ada (M1), kai (M2); all added/registered first
System.out.println("Ada borrows DDD on day 1: " + lib.lend(ddd, ada, 1).map(l -> "due day " +
l.dueDay()).orElse("refused"));
System.out.println("Kai borrows DDD on day 2: " + lib.lend(ddd, kai, 2).map(l -> "due day " +
l.dueDay()).orElse("refused (already out)"));
System.out.println("Kai borrows GoF on day 2: " + lib.lend(gof, kai, 2).map(l -> "due day " +
l.dueDay()).orElse("refused"));
System.out.println("is DDD out? " + lib.isOut(ddd) + ", Ada's loans: " + ada.loanCount());
System.out.println(lib.takeBack(ddd, 10));
System.out.println(lib.takeBack(gof, 20));
System.out.println(lib.takeBack(gof, 21));
System.out.println("is DDD out? " + lib.isOut(ddd));
```

Output:

```
Ada borrows DDD on day 1: due day 15
Kai borrows DDD on day 2: refused (already out)
Kai borrows GoF on day 2: due day 16
is DDD out? true, Ada's loans: 1
Domain-Driven Design returned on time
Design Patterns returned LATE
Design Patterns was not on loan
is DDD out? false
```

Limits became constants; `isOut` lives on `Library`, the only object with the information.

Why UML

UML (Unified Modeling Language) is the standard notation that lets another engineer read a design; it displaced rival notations decades ago because it expresses exactly these OOAD ideas: **filled diamond** = ownership (composition), **plain arrow** = one object referring to another, **+ / −** = public / private. Each symbol records a decision — hence concepts before notation. Chapter 3 teaches UML in full.

Three ways this goes wrong

1. **Classes named after tables** (`UserRow`, `OrderTable`, `BookData`): a database schema (layout of tables and columns) in OO clothes. A class with only fields and getters should prompt: what does it **do**?

2. **Missing the connection object** (`Loan`, `Reservation`, `Ticket`): `borrower` and `dueDate` drifting onto `Book` is the signal.
3. **Listing every imaginable operation.** Step 3 is what an object provides **to other objects**; long method lists everywhere show no decisions were made.

In the interview

> **30-second answer:** "I separate analysis from design. First I find the objects from the nouns, dropping single values into fields, and define relationships with ownership and multiplicity. Then I decide each object's public operations, which also decides what is private, and draw the class diagram. When the link between two objects has data of its own, like a loan's due date, that link becomes a class."

- *"Where does `isOut(book)` belong?"* — On `Library`, which owns the loans; a `Book` does not know about loans.
- *"Why is `date` not a class?"* — A single value with no behaviour of its own here; a field.

Self-check

- **Q:** Which OOAD steps are analysis? **A:** Find the objects, define the relationships.
- **Q:** The connection object in a parking lot? **A:** The parking ticket (vehicle + spot + entry time).
- **Q:** When is encapsulation actually decided? **A:** Step 3, listing each object's public operations; everything unlisted is private.

TL;DR

Analysis = what the system is (objects, relationships). Design = how objects deliver it (interfaces, class diagram). Implementation = code. Connections with data become objects. No `HashMap` in minute two.

Chapter summary: in the interview

> **30-second answer** ("Explain the pillars of OOP"): "Encapsulation hides data and gathers the rules about it into the class that owns it. Abstraction exposes only what callers need. Inheritance lets a child reuse a parent when 'child is a parent' is truly correct. Polymorphism lets one call behave differently per object, which removes type switches. In design I prefer interfaces and composition over deep inheritance, use enums for fixed sets, records for immutable values, and I keep equals and hashCode consistent."

Common follow-ups (answered in each section's *In the interview*): abstract class or interface (2.9), overloading vs overriding (2.7), why composition over inheritance (2.11), `hashCode` without `equals` (2.12), how to make a class immutable (2.13).

Self-check (whole chapter)

- **Q:** Class vs object? **A:** Blueprint vs one instance with real values.
- **Q:** What does `this(...)` do? **A:** Calls another constructor of the same class (constructor chaining).
- **Q:** What is dynamic dispatch? **A:** The JVM choosing an overridden method's implementation from the object's actual class at run time.
- **Q:** Can an interface method have a body? **A:** Yes: default, static and private methods.
- **Q:** Where should "which spot types fit a truck" live? **A:** As data on the `VehicleType` enum.
- **Q:** The hashCode rule in one line? **A:** Equal objects must have equal hash codes.
- **Q:** The OOAD steps? **A:** Find objects, define relationships, establish interfaces, make the design.

TL;DR

- OOP = data bundled with the only code allowed to modify it.
- Class = blueprint; object = instance. Constructors create valid objects; `this` = current object; `static` = one per class.
- Four pillars: encapsulation (who may change data), abstraction (what callers need), inheritance (true is-a only), polymorphism (one call, many behaviours; kills type switches).
- Overloading is compile-time; overriding is run-time; fields and statics are not polymorphic.
- Interfaces = contracts (default/static/private methods, functional interfaces for lambdas); abstract classes = shared state and templates.
- Enums hold data and behaviour for fixed sets; composition beats inheritance for reuse.
- `==` identity vs `equals` value; override `hashCode` with `equals`.
- Immutable value objects (records + defensive copies); mutable but encapsulated entities.
- Default to `private`; group by feature packages; generics for type-safe reusable infrastructure.
- OOA (what) → OOD (how, with objects) → OOP (code). Connections that carry data become objects.

Chapter 3 — Class Relationships & UML

> **What you will learn.** How two classes can be connected (association, aggregation, composition, dependency, generalization, realization), how each looks in Java, how to pick the right one in ten seconds, and how to *draw* a design with UML (Unified Modeling Language, the standard picture notation for software designs; 3.9): use-case, class, sequence, activity and state machine diagrams — plus how much UML to actually draw in a 45-minute interview.

Part	Topics	Java package
A. Relationships	3.1 Big picture · 3.2 Association (uni/bi, multiplicity) · 3.3 Aggregation · 3.4 Composition · 3.5 Dependency · 3.6 Generalization · 3.7 Realization · 3.8 Decision guide	`lld.ch03.association`, `aggregation`, `composition`, `dependency`, `generalization`, `realization`
B. UML	3.9 What UML is · 3.10 Use-case · 3.11 Class · 3.12 Sequence · 3.13 Activity · 3.14 State machine	`lld.ch03.library` (use case → class → sequence → activity), `lld.ch03.statemachine`
C. Interview	3.15 How much UML to draw · 3.16 Chapter self-check · 3.17 TL;DR	—

Every package has a runnable `Main`; listings below show the essential lines and the full files are in `code/lld/ch03/` (`javac` compiles `.java` into `.class` files, `java` runs them, `-cp` says where the classes are):

```
cd chapters/03-relationships-and-uml/code
javac --release 21 -d /tmp/lld_03 $(find . -name '*.java')
java -cp /tmp/lld_03 lld.ch03.association.Main     # or aggregation, composition, dependency,
                                                   # generalization, realization, statemachine, library
```

Part A — Class Relationships

3.1 The big picture: what a "relationship" really claims

What it is

A **relationship** is a line between two classes on a class diagram: objects of one are connected to objects of the other. The line makes two precise claims:

1. **Lifetime.** If object A is destroyed, is B destroyed too? (Lifetime = from creation until the object is no longer used.)
2. **Substitution.** Can a B be used anywhere an A is expected? (Handing a B to code that asked for an A, and everything still works.)

Analogy: **body** and **heart** share a lifetime (composition); a **car** and its **driver** do not — the driver walks away (association); a **sports car** *is a* car and goes anywhere a car can (generalization).

Six relationships: three "has-a", one "uses-a", two "is-a":

Family	Relationship	One-line meaning	Coupling (rough order)
uses-a	**Dependency**	A uses B briefly (parameter, local variable, return type)	weakest
has-a	**Association**	A holds a reference to B and can call it; nobody owns anybody	↓
has-a	**Aggregation**	A is a whole holding parts B; the parts can outlive A	↓
has-a	**Composition**	A is a whole *owning* parts B; the parts die with A	↓
is-a	**Realization**	class A implements interface B	↓
is-a	**Generalization**	class A extends class B (inheritance)	strongest coupling

Coupling = how much one class must change when the other changes. Inheritance couples a subclass to every parent detail, hence last. The order is a discussion rule of thumb, not part of the UML standard.

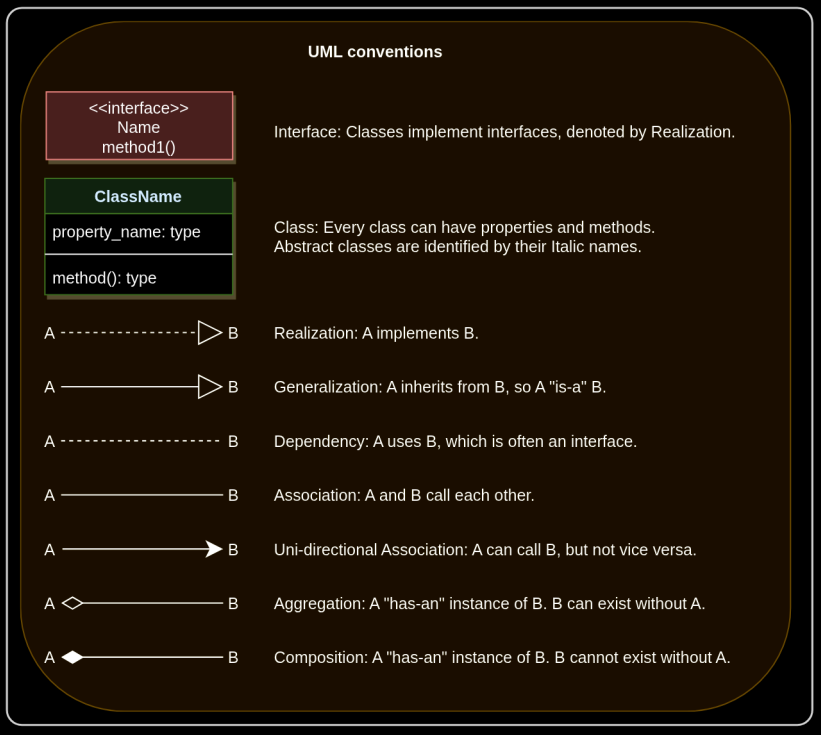


Figure — The UML conventions legend used throughout this book: class and interface boxes, and the line style for realization, generalization, dependency, association, uni-directional association, aggregation and composition (course diagram)

Look at the *ends* of each line: the end shape (triangle, diamond, arrowhead) and solid vs dashed is the whole vocabulary. Two differences from strict UML: the legend draws **uni-directional association** with a *filled* arrowhead and **dependency** as a dashed line with *no* head; standard UML (and this book's Mermaid diagrams — Mermaid is a text-based diagram language) uses an **open** arrowhead (`→`) for both, solid for association, dashed for dependency. Either is understood in an interview if you say what you mean.

The problem it solves

"Order has a Customer" and "ParkingFloor has ParkingSpots" sound alike but differ in code: one points at an object living elsewhere, the other creates, owns and deletes its parts. Mixing them up causes bugs — deleting data you did not own, or leaking objects nobody is responsible for — and in an interview the mistake is visible on the whiteboard before any code.

TL;DR

A relationship line is a claim about **who owns whom (lifetime)** and **who can stand in for whom (substitution)**. Learn the six lines and the one question that separates each.

3.2 Association (uni-directional, bi-directional, multiplicity)

What it is

An **association**: one class keeps a *reference* (a field) to another and can call it over a long period — longer than one method call. Neither owns the other; destroy one and the other is unaffected. Analogy: a **ticket** refers to a **parking spot**; throw the ticket away and the spot remains.

- **Direction (navigability** — "can be reached from"**). Uni-directional**: only one side has a field pointing at the other. **Bi-directional**: both do.
- **Multiplicity:** how many objects take part at each end: `1`, `0..1` (optional), `*` or `0..*` (any number), `1..*` (at least one), `2` (exactly two), `2..4` (a range).
- **Role name / association name:** an optional line label such as `assigned to` or `borrower`.

The problem it solves

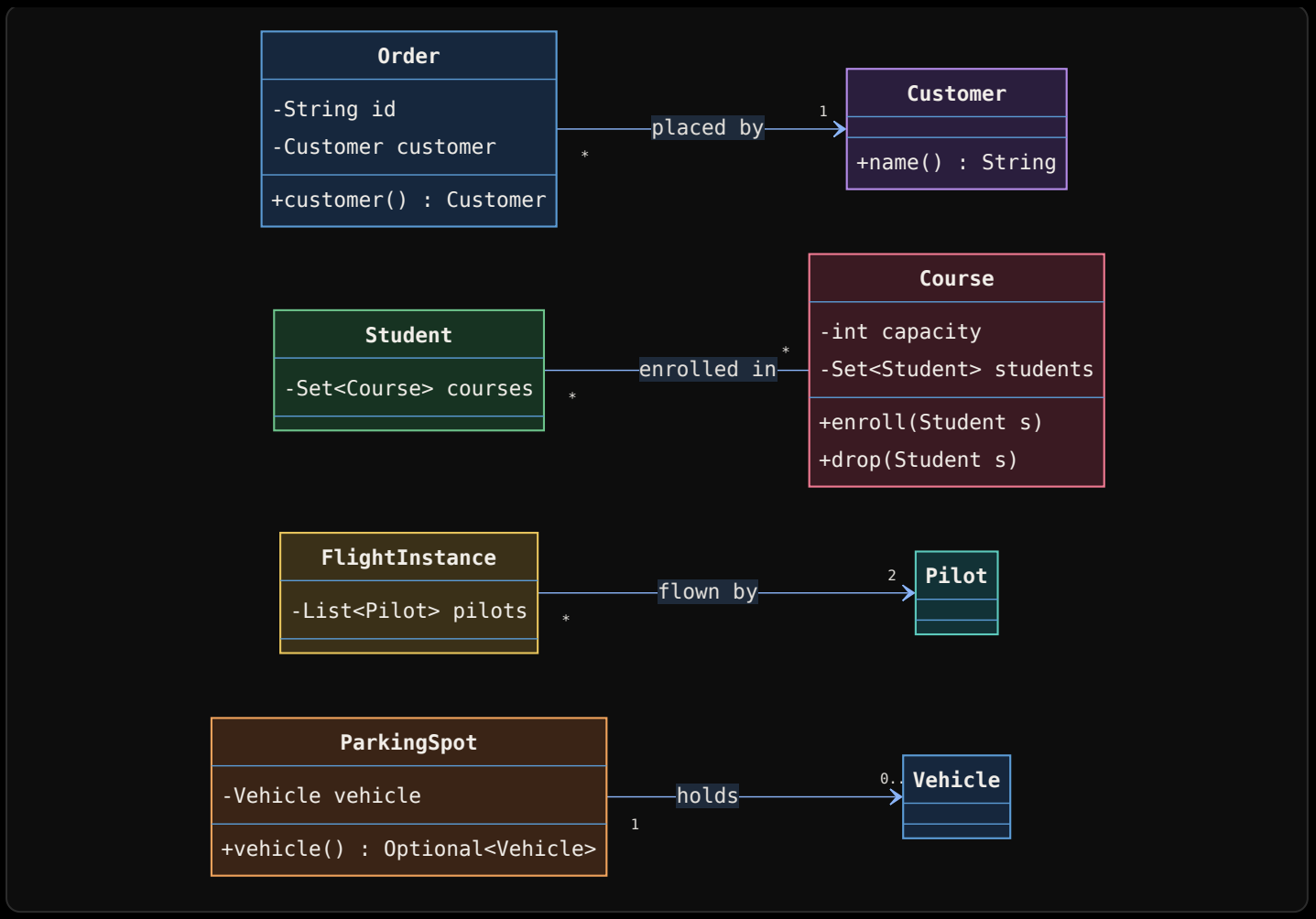
Objects collaborate over time — an `Order` must always know its `Customer`. Association records "A knows B" without claiming ownership. It is the most common relationship in a real design, correctly so.

How it works

1. Decide who calls whom. If only `Order` asks for its customer, make it **uni-directional** (a `customer` field in `Order` only).
2. Go **bi-directional** only if real use cases need both directions ("a student's courses" *and* "a course's students").
3. For a bi-directional link, **one side owns the update** and sets both fields in one method, so the ends never disagree.
4. Write **multiplicity** at each end and **enforce it in code**: `1` → required field, `Objects.requireNonNull` (throws if `null`, i.e. no object) in the constructor; `0..1` → expose as `Optional<T>` (holds one value or is empty, so callers must check); `*` → a collection, `1..*` → a never-empty collection; exact numbers like `2` → size check in the constructor.

UML notation

- Plain solid line = association (UML: "navigability unspecified"; interviews read it as "both know each other").
- Solid line + **open arrowhead** = uni-directional; the tail knows the head. An **×** at an end (rare) = explicitly not navigable that way.
- Numbers at each end = multiplicity, written next to the class they count: `Order "*" --> "1" Customer` = "many orders, each with exactly one customer".


Order
-String id
-Customer customer
+customer() : Customer
placed by
*
1
Customer
+name() : String
Student
-Set<Course> courses
enrolled in
*
*
Course
-int capacity
-Set<Student> students
+enroll(Student s)
+drop(Student s)
FlightInstance
-List<Pilot> pilots
flown by
*
2
Pilot
ParkingSpot
-Vehicle vehicle
+vehicle() : Optional<Vehicle>
holds
1
0..
Vehicle

Java example

```
// File: code/lld/ch03/association/Order.java — uni-directional, "many orders to exactly 1 customer"
public final class Order {
    private final String id;
    private final Customer customer;   // "1" end: never null. record Customer(String name) has no field back
    public Order(String id, Customer customer) {
        this.id = id;
        this.customer = Objects.requireNonNull(customer, "an order needs exactly one customer");
    }
}

// File: code/lld/ch03/association/Course.java — owning side of bi-directional Student * -- * Course
public final class Course {
    private final String code;
    private final int capacity;                              // multiplicity 0..capacity
    private final Set<Student> students = new LinkedHashSet<>();
    public void enroll(Student s) {
        if (students.contains(s)) return;
        if (students.size() == capacity) throw new IllegalStateException(code + " is full (max " + capacity +
")");
        students.add(s);
        s.linkCourse(this);                                  // updates BOTH ends in one place
    }
    public void drop(Student s) { if (students.remove(s)) s.unlinkCourse(this); }
    public Set<Student> students() { return Collections.unmodifiableSet(students); }
}
// Student: Set<Course> courses; package-private linkCourse/unlinkCourse — only Course may touch this side

// File: code/lld/ch03/association/FlightInstance.java — multiplicity "exactly 2" enforced
public FlightInstance(String flightNo, List<Pilot> pilots) {
    if (pilots.size() != 2) throw new IllegalArgumentException("a flight needs exactly 2 pilots, got " +
pilots.size());
    this.flightNo = flightNo;
    this.pilots = List.copyOf(pilots);
}

// File: code/lld/ch03/association/ParkingSpot.java — multiplicity "0..1" exposed as Optional
private Vehicle vehicle;   // null means "0" of 0..1
public void park(Vehicle v) {
    if (vehicle != null) throw new IllegalStateException("spot " + number + " is taken");
    vehicle = v;
}
public Optional<Vehicle> vehicle() { return Optional.ofNullable(vehicle); }

// File: code/lld/ch03/association/Main.java (abridged)
// Orders O-1, O-2 for Customer Asha; Student Ravi, Meera; Course JAVA101 (capacity 2), UML201 (30)
java.enroll(ravi); java.enroll(meera); uml.enroll(ravi);
try { java.enroll(new Student("Kiran")); }
catch (IllegalStateException e) { System.out.println("Multiplicity 0..2 enforced: " + e.getMessage()); }
java.drop(ravi);
// FlightInstance AI-202 with two pilots; AI-303 with one (rejected); ParkingSpot 7 before/after park(KA-01-1234)
```

Output:

```
== Uni-directional: Order -> Customer ==
O-1 belongs to Asha
O-2 belongs to Asha
Customer has no way to list its orders (one-way link).

== Bi-directional many-to-many: Student <-> Course ==
JAVA101 students: [Ravi, Meera]
Ravi's courses:   [JAVA101, UML201]
Multiplicity 0..2 enforced: JAVA101 is full (max 2)
After drop, JAVA101: [Meera], Ravi: [UML201]

== Multiplicity 2 and 0..1 ==
AI-202 pilots: [Pilot[name=Capt. Rao], Pilot[name=F/O Das]]
Rejected: a flight needs exactly 2 pilots, got 1
Spot 7 vehicle: <empty>
Spot 7 vehicle: KA-01-1234
```

`Student` and `Course` each hold a `Set` of the other (**bi-directional many-to-many**); only `Course.enroll/drop` change the link because `Student.linkCourse` is package-private (no access keyword: same-package callers only) — the single "owning side" rule keeps both ends consistent. Every object is created by `Main` and merely *referenced*: dropping a course does not delete the student.

Trade-offs

Choice	Pros	Cons
Uni-directional	Simple; one field; no sync bugs; lower coupling	Reverse question ("which orders does this customer have?") needs a query or repository (a class that stores and looks up objects)
Bi-directional	Fast navigation both ways	Two fields to keep in sync; tighter coupling; risk of infinite loops in `toString` / `equals` /JSON serialisation (turning objects into JSON text)
Strict multiplicity checks	Invalid states cannot be created	A little more constructor code

When not: if you only need the object inside one method, that is a **dependency** (3.5), which couples less; do not default to bi-directional — most associations should be one-way.

Common mistakes

- Every line bi-directional "just in case".
- Updating only one side of a bi-directional link (`student.courses.add(c)` without `course.students.add(s)`).
- No multiplicity on the line, then no null/non-empty check in the code.
- Both sides in `toString()` / `equals()` → stack overflow from infinite recursion (each method calls the other's until Java runs out of room to track calls).

Real-world examples

- **JPA / Hibernate** (Java's standard API for saving objects to SQL tables, and its most popular implementation): `@ManyToOne` on `Order.customer` is uni-directional; a bi-directional `@OneToMany(mappedBy = "customer")` needs an *owning side* (holding the foreign key — the column storing the other row's ID) and helpers setting both ends — the `Course.enroll` rule.
- **Spring** (a popular Java framework): a service with a constructor-injected repository has an association to it; the Spring container (creates and wires objects) creates and destroys the repository.
- **E-commerce:** an order row stores `customer_id`; a customer stores no order-id list.

In the interview

> **30-second answer:** "Association means one class holds a reference to another and can call it, with no ownership. I make it uni-directional unless both directions are needed, and I write multiplicity on every line because it changes the code: `1` is a required field, `0..1` is an Optional, `1..*` is a non-empty collection."

- *Q: How do you keep a bi-directional association consistent?* — A: One owning side with a single method (`enroll`) updating both ends; the other side's mutators (state-changing methods) package-private.
- *Q: What if a parking spot can hold two motorbikes?* — A: `0..1` becomes `0..2`: a small collection with a size check instead of an `Optional`.
- *Q: Why not make everything bi-directional?* — A: It doubles the places that must change together and raises coupling; a repository query usually answers the reverse direction.

Self-check

- *Q:* `Order "*" --> "1" Customer` *— which class has the field?* → A: `Order`; the arrow points at the referenced class.
- *Q: Java type for* `0..1`*?* → A: A nullable field exposed as `Optional<T>`.
- *Q: Is many-to-many always bi-directional?* → A: No; counts and direction are separate choices.

TL;DR

Association = "A knows B", no ownership. Prefer one-way. Multiplicity on every line, enforced in constructors/types.

3.3 Aggregation

What it is

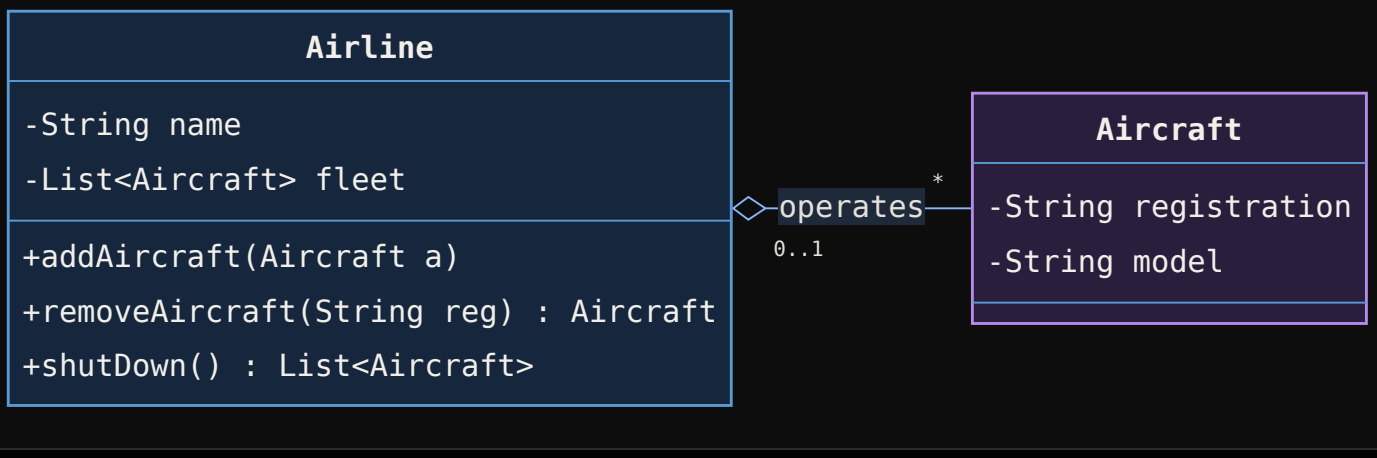
Aggregation is an association meaning "A is a *whole* made of parts B, but the parts have their own life": they exist before the whole, can move to another whole, and survive its destruction. Also called *shared aggregation*, since a part may belong to several wholes. Analogy: an **airline** and its **aircraft** — if the airline shuts down, the planes are sold, not scrapped; a **team** and its **players**.

The problem it solves

It says "these objects form a group" (whole–part, stronger than plain association) while making clear the group does **not** control their lifetime: "do not delete the parts when you delete the whole".

How it works

1. The part is **created outside** the whole (caller, factory or repository).
2. It is **passed in** (`addAircraft(a)`); the whole stores a reference.
3. Removing a part **returns it alive**; destroying the whole **releases** its parts.
4. UML: **hollow diamond at the whole's end**.



Java example

```java
// File: code/lld/ch03/aggregation/Airline.java — Airline 0..1 o-- * Aircraft: holds, never creates or destroys
public final class Airline {
    private final String name;
    private final List<Aircraft> fleet = new ArrayList<>();
    public void addAircraft(Aircraft a) { fleet.add(a); }        // part is passed in
    public Aircraft removeAircraft(String registration) {         // part is handed back, still alive
        for (var it = fleet.iterator(); it.hasNext(); ) {
            Aircraft a = it.next();
            if (a.registration().equals(registration)) { it.remove(); return a; }
        }
        throw new IllegalArgumentException("no aircraft " + registration + " in " + name);
    }
    public List<Aircraft> shutDown() {                            // releases parts instead of destroying them
        List<Aircraft> released = List.copyOf(fleet);
        fleet.clear();
        return released;
    }
}
// Aircraft(registration, model); toString = registration + " (" + model + ")"

// File: code/lld/ch03/aggregation/Main.java (abridged)
// Aircraft a1 VT-AAA A320, a2 VT-BBB B737, a3 VT-CCC ATR72 exist BEFORE any airline;
// Skyways gets a1, a2; BlueJet gets a3; both fleets printed
blueJet.addAircraft(skyways.removeAircraft("VT-BBB"));       // transfer one part between wholes
List<Aircraft> released = skyways.shutDown();
skyways = null;                                              // the Airline object is now unreachable
released.forEach(blueJet::addAircraft);                      // parts survive and join another whole
```

Output:

```
Skyways fleet: [VT-AAA (A320), VT-BBB (B737)]
BlueJet fleet: [VT-CCC (ATR72)]
After transfer, BlueJet fleet: [VT-CCC (ATR72), VT-BBB (B737)]
Skyways shut down. Released aircraft still exist: [VT-AAA (A320)]
BlueJet fleet now: [VT-CCC (ATR72), VT-BBB (B737), VT-AAA (A320)]
```

After `skyways = null` the airline is garbage (nothing points to it, so Java's garbage collector, which frees unreachable objects' memory, reclaims it), but its aircraft are still referenced and join `BlueJet`.

Trade-offs

	Aggregation	Plain association
Extra meaning	"whole–part" grouping	just "knows"
Lifetime claim	parts outlive whole	none
Code difference	usually **none** — both are a field holding a reference	—

Aggregation adds meaning for the reader but almost nothing in code. The UML specification leaves shared aggregation's precise meaning to the modeller, and several UML authors (Martin Fowler's *UML Distilled* among them) advise using it sparingly; interviews still expect it for a clear whole–part grouping.

When not: no "whole made of parts" idea (a ticket and a spot) → plain association; parts created by and dying with the whole → composition.

Common mistakes

- Diamond at the **part's** end — it always sits at the **whole**.
- Choosing aggregation vs composition at random. Test: *imagine the whole is gone — is the part still meaningful by itself?* Yes → aggregation.
- Deleting parts in the whole's "close/delete" method despite drawing aggregation.

Real-world: clearing a `List<Customer>` leaves the customers; archiving a Slack/Teams **channel** keeps **members**' accounts; deleting a Spotify **playlist** keeps the **songs**; a **department** and its **professors**.

In the interview

> **30-second answer:** "Aggregation is a whole–part association where the parts can outlive the whole — hollow diamond at the whole. An airline aggregates aircraft: shut the airline down and the aircraft still exist. In Java the parts are created outside and passed in, and the whole never destroys them."

- *Q: Can an airline exist without aircraft?* — A: A multiplicity question, not an aggregation one: if the business requires at least one, write `1..*`; it is still aggregation because aircraft survive the airline.
- *Q: How does aggregation differ in code from association?* — A: Usually it doesn't; the difference is modelling intent, so say the lifetime test aloud.

Self-check

- *Q: Where does the hollow diamond go?* → A: At the whole (`Airline`).
- *Q: A library closes; do its members disappear?* → A: No — Library ◇— Member is aggregation.

TL;DR

Aggregation = "has-a, parts live on". Parts passed in, never destroyed by the whole. Hollow diamond at the whole.

3.4 Composition

What it is

Composition is the strongest has-a: the whole **owns** its parts. A part belongs to **exactly one** whole at a time, is usually **created by** it, and is **destroyed with** it (UML also allows explicitly removing a part *before* the whole is deleted). Analogy: a **house** and its **rooms** — "a room of a house that does not exist" means nothing. Likewise **parking floor** → **spots**, **parking lot** → **floors**, **flight** → **weekly schedule**.

The problem it solves

Some objects make no sense alone, and letting outsiders create or share them causes bugs (two floors sharing a spot, orphan spots after a floor is removed). Composition gives the owner the parts' full life cycle: one place creates, changes and removes them.

How it works

1. The whole **creates** the parts (constructor or `addX(...)` taking *data*, not a ready-made part).
2. The part's constructor is **not public** (package-private, or a private nested class declared inside the whole's class).

3. The whole **never hands out** its internal collection — only read-only views or copies.
4. **Removing** the whole (or a part) drops the only reference, so the part becomes garbage.
5. UML: **filled (black) diamond at the whole's end**.

Java has no destructors (methods run automatically on destruction, as in C++): "dies with the whole" is *logical* — nobody else holds a reference, so the part becomes unreachable and is collected. For clean-up (files, sockets i.e. open network connections), the whole's `close()` closes its parts — e.g. closing a `BufferedReader` closes the `FileReader` it wraps.



Java example

```java
// File: code/lld/ch03/composition/ParkingSpot.java — package-private constructor: only ParkingFloor can create one
ParkingSpot(String id, SpotType type) { this.id = id; this.type = type; }
void occupy() { free = false; }          // toString = id + ":" + type. SpotType is an enum (a value, not subclasses)

// File: code/lld/ch03/composition/ParkingFloor.java — ParkingFloor 1 *-- 1..* ParkingSpot
ParkingFloor(int level, Map<SpotType, Integer> layout) {
    this.level = level;
    int n = 1;
    for (var e : layout.entrySet())
        for (int i = 0; i < e.getValue(); i++)
            spots.add(new ParkingSpot("L" + level + "-" + n++, e.getKey()));   // the floor CREATES its spots
    if (spots.isEmpty()) throw new IllegalArgumentException("a floor needs 1..* spots");
}
public List<ParkingSpot> spots() { return Collections.unmodifiableList(spots); }
public String park(SpotType type) {      // first free spot of the type; the spot never leaves the floor
    for (ParkingSpot s : spots) if (s.isFree() && s.type() == type) { s.occupy(); return s.id(); }
    return null;
}
// freeSpots() counts isFree()

// File: code/lld/ch03/composition/ParkingLot.java — ParkingLot 1 *-- 1..* ParkingFloor
private final Map<Integer, ParkingFloor> floors = new TreeMap<>();
public ParkingFloor addFloor(int level, Map<SpotType, Integer> layout) {   // caller describes, lot constructs
    ParkingFloor f = new ParkingFloor(level, layout);
    floors.put(level, f);
    return f;
}
public void removeFloor(int level) { floors.remove(level); }   // parts go with it: nobody else holds them
// totalSpots() sums floor sizes; floors() is an unmodifiable view

// File: code/lld/ch03/composition/Main.java (abridged)
ParkingFloor ground = lot.addFloor(0, layout);              // layout: COMPACT 3, LARGE 1 (LinkedHashMap)
lot.addFloor(1, Map.of(SpotType.MOTORBIKE, 4));
// prints ground.spots(), lot.totalSpots(), ground.park(COMPACT), ground.freeSpots()
try { ground.spots().clear(); }
catch (UnsupportedOperationException e) { System.out.println("Outsiders cannot change the parts: spots() is read-only"); }
lot.removeFloor(1);                                         // then prints totalSpots() and floors().keySet()
```

Output:

```
Floor 0 spots: [L0-1:COMPACT, L0-2:COMPACT, L0-3:COMPACT, L0-4:LARGE]
Total spots in lot: 8
Parked compact car at L0-1
Free spots on floor 0: 3
Outsiders cannot change the parts: spots() is read-only
After removing floor 1, total spots: 4 (its 4 spots are gone with it)
Floors left: [0]
```

`addFloor` receives a *layout description* and builds the floor itself; spots cannot be created outside the package; `spots()` is read-only; `removeFloor(1)` drops the floor and its four spots.

Trade-offs

	Composition	Aggregation
Who creates the part	the whole	someone else
Can the part be shared?	no (one owner at a time)	yes
Part outlives whole?	no	yes
Encapsulation	strong — whole controls all changes	weaker
Flexibility	lower — parts cannot be reused elsewhere	higher

When not: the part has an independent identity the business cares about (a *person*, *aircraft*, *song*) → aggregation/association; the part must be shared between wholes.

Common mistakes

- Accepting a ready-made part (`addSpot(ParkingSpot s)`) and calling it composition — the caller may still hold and share it.
- Returning the internal mutable list (`return spots;`), so outsiders add/remove parts behind the owner's back.
- Confusing it with "favour composition over inheritance" (3.6), where "composition" means *holding* an object instead of extending its class, covering association/aggregation too.

Real-world: `HashMap`'s internal `Node` entries (a package-private nested class), created by and gone with the map; **JPA** `@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true)` — delete the `Order`, its `OrderLine` rows go too; a deleted Google Doc takes its comments and revision history; killing an OS process (running program) frees its threads (parallel lines of execution) and memory pages.

In the interview

30-second answer: "Composition means the whole owns the parts: it creates them, they belong to only it, and they die with it — filled diamond at the whole. A parking floor composes its spots: remove the floor and the spots are gone. In Java I build the parts inside the owner, hide their constructor, and only expose read-only views."

- *Q: Java has garbage collection — how can a part 'die'?* — A: Logically: the whole holds the only reference, so removing it makes the part unreachable; resource-holding parts are closed by the whole's `close()`.
- *Q: Is `Flight` → `WeeklySchedule` composition?* — A: Yes; a schedule of a non-existent flight is meaningless.

Self-check

- *Q: The test separating aggregation from composition?* → A: Whole gone — is the part still meaningful alone? No → composition.
- *Q: Why is `ParkingSpot`'s constructor package-private?* → A: So only the owning floor can create spots.

TL;DR

Composition = "owns, parts die with it". Whole creates parts, never shares them, exposes read-only views. Filled diamond at the whole.

3.5 Dependency

What it is

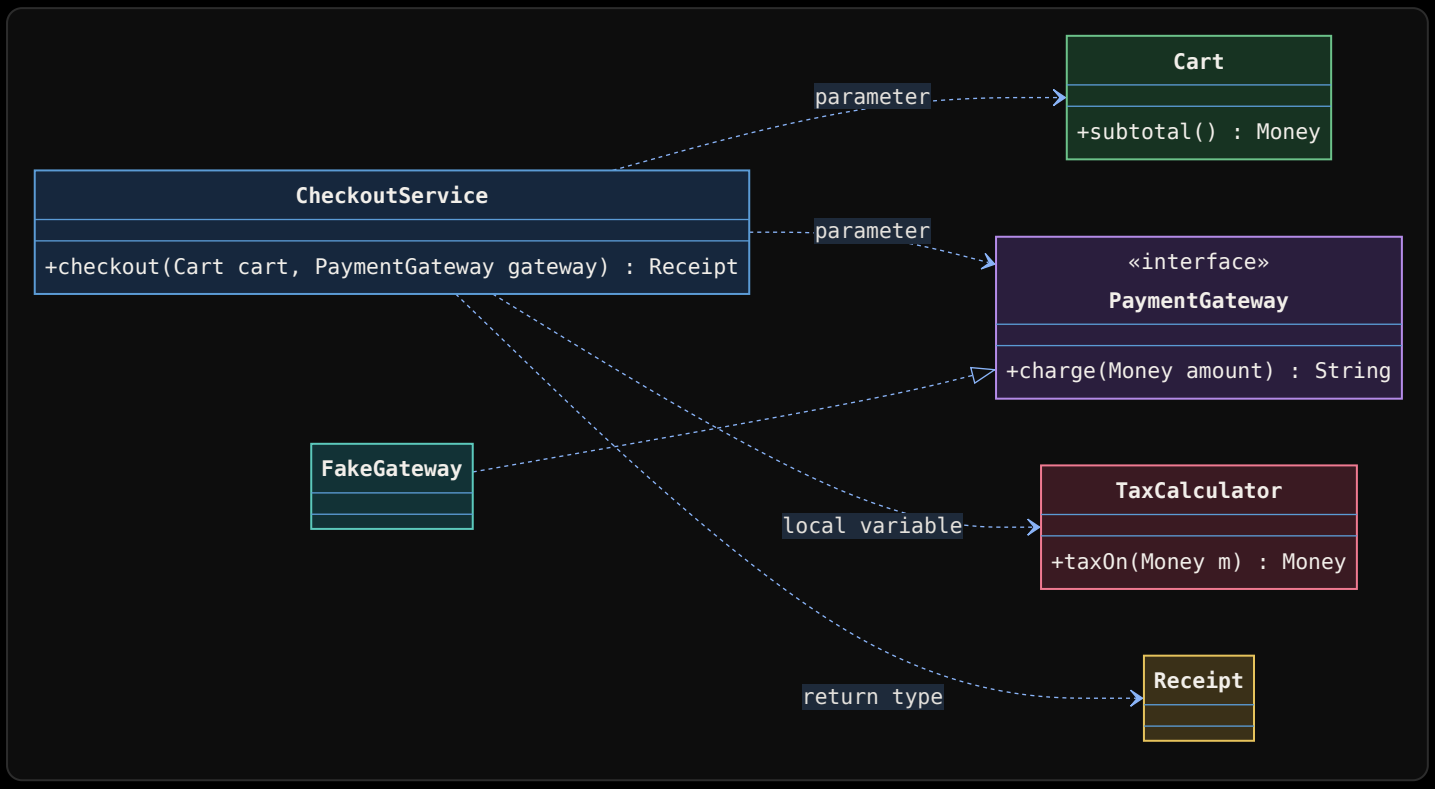
A **dependency**: class A *uses* B briefly — inside one method — without a field pointing at B. B appears as a **method parameter**, **local variable**, **return type**, or **static method call**; when the method returns, the relationship is over. (A **chef** *uses* today's **delivery van**; tomorrow's may differ.)

The problem it solves

It records the weakest link: "if B's interface changes, A may have to change". Recognising it (instead of an association) keeps classes stateless (no data held between calls) and loosely coupled — the caller can pass a different collaborator on every call.

How it works

A method receives B as a parameter (`checkout(Cart cart, PaymentGateway gateway)`), creates it as a local (`new TaxCalculator()`), returns it (`Receipt`), or calls a static method on it — and A has no field of type B. UML: **dashed line with an open arrowhead** from user (A) to used (B), optionally with a keyword like `«use»` or `«create»`.



Java example

```java
// File: code/lld/ch03/dependency/CheckoutService.java — NO fields: every link lasts one call
public final class CheckoutService {
    public Receipt checkout(Cart cart, PaymentGateway gateway) {
        TaxCalculator tax = new TaxCalculator();            // local-variable dependency (GST = 0.18)
        Money subtotal = cart.subtotal();                   // parameter dependency
        Money total = subtotal.plus(tax.taxOn(subtotal));
        String txn = gateway.charge(total);                 // parameter dependency (interface)
        return new Receipt(txn, subtotal, total);           // return-type dependency
    }
    public record Receipt(String txnId, Money subtotal, Money total) { }
}
// Money: record over BigDecimal, scale 2 HALF_UP, toString "Rs " + amount. Cart(List<Money>).subtotal() sums.
// PaymentGateway { String charge(Money amount); }  FakeGateway prints "  [gateway] charging ..." and returns TXN-1, TXN-2...

// File: code/lld/ch03/dependency/Main.java
CheckoutService service = new CheckoutService();   // holds no collaborators at all
Cart cart = new Cart(List.of(Money.of("250"), Money.of("99.50")));
CheckoutService.Receipt r = service.checkout(cart, new FakeGateway());   // caller picks the gateway per call
System.out.println("Receipt: " + r);
PaymentGateway loggingGateway = amount -> {
    System.out.println("  [lambda gateway] charging " + amount);
    return "LMB-42";
};
System.out.println("Receipt: " + service.checkout(cart, loggingGateway));
```

Output:

```
  [gateway] charging Rs 412.41
Receipt: Receipt[txnId=TXN-1, subtotal=Rs 349.50, total=Rs 412.41]
  [lambda gateway] charging Rs 412.41
Receipt: Receipt[txnId=LMB-42, subtotal=Rs 349.50, total=Rs 412.41]
```

`Main` passes a different gateway — even a lambda (a short inline unnamed function) — per call, which a stored field makes harder. `Money` uses `BigDecimal` (exact decimal type), not `double` (binary floating point, cannot hold 0.10 exactly), because currency needs exact arithmetic.

Trade-offs

	Dependency (parameter)	Association (field)
Coupling	lowest	higher
Object state	can be stateless	holds state
Swapping collaborator	per call	per object (or via setter)
Call-site noise	caller passes B every time	caller passes B once

When not: if A needs B across many methods and calls (a service always using the same repository), store it (association via constructor injection, passing it into the constructor once).

Common mistakes

- A solid association line for something that is only a parameter — overstates coupling.
- Drawing *every* dependency (`String`, `List`, `LocalDate`…); draw only those that matter.
- Hiding a dependency as `new ConcreteThing()` deep in a method when the caller should decide (blocks testing); take an interface parameter instead.

Real-world: `Collections.sort(list, comparator)`; a Spring MVC controller method (answers one web request) `handle(HttpServletRequest req)`; `LocalDate.now(Clock clock)`, the JDK's way to make time testable;

Maven/Gradle (build tools that fetch libraries and compile projects) use "dependency" for the same idea at package level.

In the interview

> **30-second answer:** "Dependency is 'uses briefly': B is a parameter, local variable or return type of A's method, and A keeps no field for it. Dashed arrow from A to B. It is the loosest coupling, so when a collaborator is only needed in one method I pass it in rather than store it."

- *Q:* `FlightReservation.pay(Payment p)` *— association or dependency?* — A: Dependency, unless the reservation stores the payment in a field; then association.
- *Q: Relation to Dependency Injection?* — A: DI is about *who supplies* the collaborator (from outside, not `new` inside); it may then be stored (association) or passed per call (dependency). The Dependency Inversion Principle adds: depend on interfaces, not concrete classes (Chapter 4).

Self-check

- *Q: Dashed + open arrow vs dashed + hollow triangle?* → A: Dependency vs realization.
- *Q: A method returns a* `Receipt` *— a relationship?* → A: Yes, a dependency on `Receipt`.

TL;DR

Dependency = "uses for one call". Parameter, local variable, return type or static call; no field. Dashed arrow.

3.6 Generalization (inheritance)

What it is

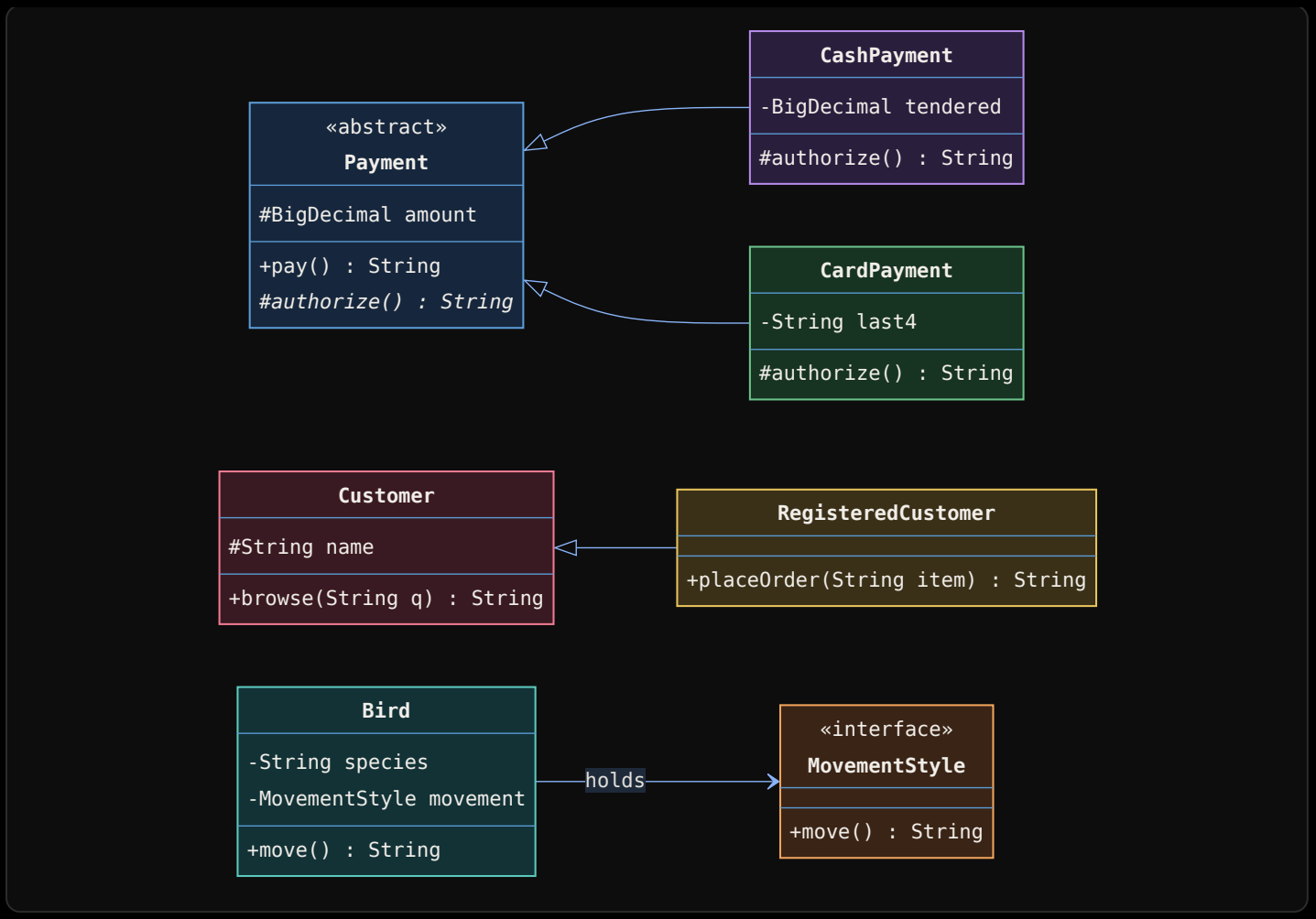
Generalization is UML's word for **inheritance**: class B `extends` A, so B *is a kind of* A. A is the **parent** (superclass, general), B the **child** (subclass, specialised); B inherits A's fields and methods and may override them. Analogy: a **credit-card payment** *is a* **payment** — any code that handles "a payment" handles it.

The problem it solves

When classes truly share a contract **and** some behaviour, inheritance keeps the shared parts in one place and lets callers use the general type (`Payment`) without knowing the specific one (**polymorphism** — one call, different behaviour by actual object).

How it works

1. Run the **substitution test** first: *can B be used anywhere A is expected, with no surprises?* If B would have to refuse or break any method of A, no. (Liskov Substitution Principle — Chapter 4.)
2. If yes, put shared state and workflow in A (often `abstract`: cannot be instantiated, may leave methods for subclasses) and let subclasses override only the varying steps.
3. If no, **hold instead of extend**: give A an interface-typed field and plug in behaviour objects.
4. UML: **solid line with a hollow triangle pointing at the parent**. Abstract class names are *italic* (or `{abstract}` / `«abstract»`); abstract operations too.


CashPayment
-BigDecimal tendered
#authorize() : String
«abstract»
Payment
#BigDecimal amount
+pay() : String
#authorize() : String
CardPayment
-String last4
#authorize() : String
Customer
#String name
+browse(String q) : String
RegisteredCustomer
+placeOrder(String item) : String
Bird
-String species
-MovementStyle movement
+move() : String
holds
«interface»
MovementStyle
+move() : String

Java example

```java
// File: code/lld/ch03/generalization/Payment.java — abstract parent; every subclass really "is a" Payment
public abstract class Payment {
    protected final BigDecimal amount;   // '#' protected; BigDecimal, never double
    protected Payment(BigDecimal amount) {
        if (amount.signum() <= 0) throw new IllegalArgumentException("amount must be positive");
        this.amount = amount;
    }
    public final String pay() {                    // shared, final workflow
        String ref = authorize();
        return getClass().getSimpleName() + " of " + amount + " -> " + ref;
    }
    protected abstract String authorize();          // the varying step (italics in UML)
}

// File: code/lld/ch03/generalization/CashPayment.java (constructor: super(amount); this.tendered = tendered;)
@Override protected String authorize() {
    if (tendered.compareTo(amount) < 0) return "DECLINED (short by " + amount.subtract(tendered) + ")";
    return "OK, change " + tendered.subtract(amount);
}
// CardPayment.authorize() returns "OK, card ****" + last4 + " approved"

// Customer.browse(q) -> name + " browses '" + q + "'" (only what EVERY customer can do);
// RegisteredCustomer extends Customer adds placeOrder(item) -> name + " orders " + item (a guest may not order)
// MovementStyle { String move(); FLY = () -> "flies"; SWIM = () -> "swims"; WALK = () -> "walks"; }

// File: code/lld/ch03/generalization/Bird.java — hold, don't extend
private MovementStyle movement;                    // held, so it can be swapped at run time
public void setMovement(MovementStyle m) { this.movement = m; }
public String move() { return species + " " + movement.move(); }

// File: code/lld/ch03/generalization/Main.java (abridged; prints the three section headers)
for (Payment p : List.of(
        new CashPayment(new BigDecimal("500.00"), new BigDecimal("600.00")),
        new CardPayment(new BigDecimal("1200.00"), "4242"),
        new CashPayment(new BigDecimal("300.00"), new BigDecimal("200.00"))))
    System.out.println(p.pay());                   // caller only knows Payment
List<Customer> visitors = List.of(new Customer("Guest-17"), reg);   // reg = new RegisteredCustomer("Asha")
visitors.forEach(c -> System.out.println(c.browse("headphones")));
System.out.println(reg.placeOrder("headphones"));
System.out.println("guest.placeOrder(...) does not compile: guests never inherit ordering");
// eagle = new Bird("Eagle", FLY), penguin = new Bird("Penguin", SWIM); print both moves
penguin.setMovement(MovementStyle.WALK);             // impossible with a subclass
System.out.println(penguin.move() + " (behaviour swapped at run time)");
```

Output:

```
== Good hierarchy: substitution works ==
CashPayment of 500.00 -> OK, change 100.00
CardPayment of 1200.00 -> OK, card ****4242 approved
CashPayment of 300.00 -> DECLINED (short by 100.00)

== Guest vs registered customer ==
Guest-17 browses 'headphones'
Asha browses 'headphones'
Asha orders headphones
guest.placeOrder(...) does not compile: guests never inherit ordering

== Holding instead of extending ==
Eagle flies
Penguin swims
Penguin walks (behaviour swapped at run time)
```

What each part shows:

- `Payment`'s `pay()` is `final` (shared workflow) and `authorize()` abstract (the varying step); both subclasses honour every promise, so `Main` loops over `List<Payment>` without caring which. Amounts are `BigDecimal` compared with `compareTo` (`double` cannot store `0.10` exactly; money must not drift).

- **Wrong-inheritance fix #1 — the subclass that must refuse something.** If `Guest extends Customer` and `Customer` can order, a guest could order, which is forbidden. Fix: `Customer` holds only what *every* customer can do (browse); ordering moves down to `RegisteredCustomer`.
- **Wrong-inheritance fix #2 — the subclass that only carries a constant.** `CompactSpot`, `LargeSpot`, `HandicappedSpot` whose constructors only pass a different value up add no behaviour: that is a field (the `SpotType` enum in 3.4); a fourth type is one constant.
- **Prefer holding over extending.** `FlyingBird` / `Penguin` subclasses break when a penguin arrives (penguins don't fly); `Bird` *holds* a `MovementStyle`, swappable at run time — impossible with `extends`.

Trade-offs

	Inheritance (extends)	Holding an object (composition/association)
Reuse	automatic	explicit delegation
Change at run time	no — fixed at compile time	yes — swap the held object
Coupling	tight: subclass sees parent internals ("fragile base class")	loose: only the interface
Number of parents in Java	one class	many interfaces / many fields
Best for	true is-a with different behaviour	varying behaviour, features that combine

When not: only to share a few fields or a helper ("code reuse" alone is no reason); when a subclass would throw `UnsupportedOperationException` (Java's standard "not supported" error) for an inherited method; when the variation is just data (field or enum); when behaviours mix (fly + swim + walk) — the class count explodes.

Common mistakes

- Deep hierarchies (more than 2–3 levels).
- `Stack extends Vector`-style: the child inherits methods breaking its own rules (a JDK `Stack` lets you insert in the middle).
- Arrow direction: the triangle points **at the parent**.
- Inheritance between classes where one can do what the other does only *sometimes*: generalization needs "always"; for "sometimes", hold a shared interface or a role object.

Real-world: `java.util.ArrayList extends AbstractList` (reuses the skeleton); `java.io.FileInputStream extends InputStream` (accepted by any `InputStream` code); `IOException extends Exception extends Throwable` — why `catch (Exception e)` catches an `IOException`. **Counter-examples in the JDK:** `java.util.Stack extends Vector` and `java.util.Properties extends Hashtable`, widely cited as inheritance that should have been composition.

In the interview

> **30-second answer:** "Generalization is inheritance — solid line with a hollow triangle at the parent. I use it only when the child passes the substitution test: it can go anywhere the parent goes and never refuses a parent method. A subclass that only passes a constant up becomes an enum field; varying behaviour I hold as an interface instead of extending."

- *Q: Why not `CompactSpot extends ParkingSpot`?* — A: No added behaviour; a `SpotType` field does it and new types are one enum constant.
- *Q: When is inheritance right?* — A: When subclasses behave differently but all keep the parent's promises, like `Payment` → cash/card.
- *Q: Can a Java class have two parents?* — A: It extends one class but implements many interfaces.

Self-check

- *Q: Which way does the hollow triangle point?* → A: At the parent (general class).
- *Q: Guest cannot order, but `Guest extends Customer` and `Customer.placeOrder()` exists — what breaks, and the fix?* → A: Substitution (LSP); move `placeOrder` to `RegisteredCustomer`.

TL;DR

Generalization = "is-a", `extends`, only after the substitution test. Constants → fields; varying behaviour → hold an interface. Triangle points at the parent.

3.7 Realization (interface implementation)

What it is

Realization: a class *implements* an **interface** — a named list of operations (a contract) with no instance state — promising to provide them (`class EmailNotifier implements Notifier`). Analogy: a **wall socket standard**: any plug that "realizes" it works in any socket.

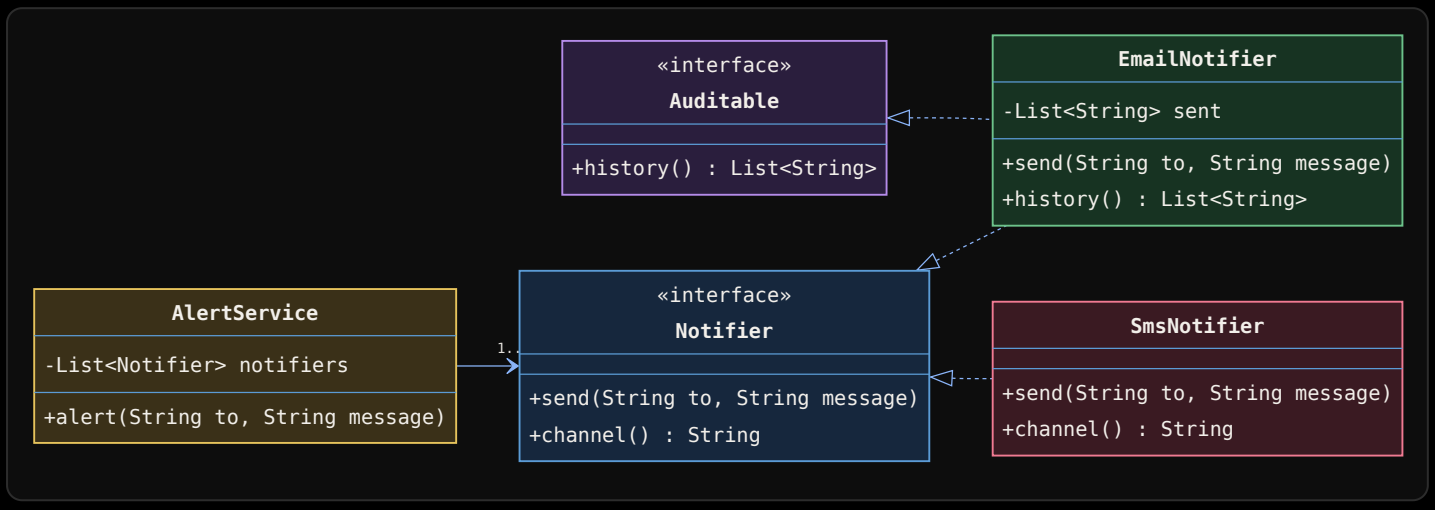
The problem it solves

Callers depend on the **contract**, not the class: new implementations (SMS, push, WhatsApp) need no caller changes, tests use fakes, and one class can play several roles (many interfaces, one parent class).

How it works

1. Define an interface with the operations callers need (`send(to, message)`).
2. Each concrete class `implements` it with method bodies.
3. Callers hold / receive the **interface type** (`List<Notifier>`).
4. Since Java 8 interfaces may have `default` (with body; overridable) and `static` methods.
5. UML: **dashed line with a hollow triangle at the interface**, marked `«interface»` (compact alternative: the "lollipop", a small circle on a stick labelled with the interface name).

It deliberately looks like generalization: the same "is-a" idea, but only the contract is inherited.



Java example

```java
// File: code/lld/ch03/realization/Notifier.java — contract, no state
public interface Notifier {
    void send(String to, String message);
    default String channel() {                      // Java 8+ default method
        String n = getClass().getSimpleName();
        return n.contains("$$Lambda") ? "lambda" : n;   // lambdas get generated class names
    }
}
// Auditable { List<String> history(); }

// File: code/lld/ch03/realization/EmailNotifier.java — realizes TWO interfaces
public final class EmailNotifier implements Notifier, Auditable {
    private final List<String> sent = new ArrayList<>();
    @Override public void send(String to, String message) {
        String line = "EMAIL to " + to + ": " + message;
        sent.add(line);
        System.out.println("  " + line);
    }
    @Override public List<String> history() { return List.copyOf(sent); }
}

// File: code/lld/ch03/realization/SmsNotifier.java
@Override public void send(String to, String message) {
    String text = message.length() > 20 ? message.substring(0, 20) + "..." : message;
    System.out.println("  SMS to " + to + ": " + text);
}
@Override public String channel() { return "SMS"; }   // overrides the default

// File: code/lld/ch03/realization/AlertService.java — associated with the INTERFACE only
public void alert(String to, String message) {       // notifiers = List.copyOf(constructor argument)
    for (Notifier n : notifiers) {
        System.out.println("via " + n.channel());
        n.send(to, message);
    }
}

// File: code/lld/ch03/realization/Main.java
EmailNotifier email = new EmailNotifier();
Notifier sms = new SmsNotifier();
Notifier console = (to, msg) -> System.out.println("  CONSOLE " + to + ": " + msg); // a lambda realizes it too
AlertService alerts = new AlertService(List.of(email, sms, console));
alerts.alert("ops@shop.example", "Disk usage above 90 percent on db-1");
Auditable audit = email;   // same object, seen through its other interface
System.out.println("Email audit trail: " + audit.history());
```

Output:

```
via EmailNotifier
  EMAIL to ops@shop.example: Disk usage above 90 percent on db-1
via SMS
  SMS to ops@shop.example: Disk usage above 90 ...
via lambda
  CONSOLE ops@shop.example: Disk usage above 90 percent on db-1
Email audit trail: [EMAIL to ops@shop.example: Disk usage above 90 percent on db-1]
```

A lambda realizes `Notifier` because it has exactly one abstract method (a *functional interface*). `AlertService` holds `List<Notifier>`, so a new channel (e.g. `PushNotifier`) needs no change there; the same `email` object is a `Notifier` to `AlertService` and an `Auditable` to `Main`.

Trade-offs

	Interface + realization	Abstract class + generalization
Multiple per class	yes	no (single inheritance)
Instance fields	no	yes
Shared code	`default` methods only	any methods, constructors
Best for	roles/capabilities, plug-ins, test seams	a family sharing state and a workflow

When not: one implementation that will never vary and no testing need is ceremony — add the interface when a second implementation or a test double (fake stand-in for tests) appears (YAGNI: "you aren't gonna need it"). "Marker" interfaces with no methods are rarely needed; annotations (`@` tags like `@Override` attaching information to code) usually serve better.

Common mistakes

- Solid line instead of dashed (that means class inheritance).
- Fat interfaces forcing implementers to stub (write empty placeholders for) unneeded methods (breaks Interface Segregation, Chapter 4).
- Declaring variables with the concrete type (`EmailNotifier n = ...`), losing the benefit.

Real-world: `List` realized by `ArrayList`/`LinkedList`; `Comparable`, `Runnable`, `AutoCloseable`; **JDBC** (Java Database Connectivity): each vendor's driver realizes `java.sql.Connection`, your code never names the vendor class; **Spring** injects a `PaymentGateway` bean (Spring-created object) that may be Stripe, Razorpay (payment providers) or a mock (fake); **SLF4J** (a logging facade, one front for several libraries): code calls `Logger`, Logback or Log4j realizes it.

In the interview

> **30-second answer:** "Realization is a class implementing an interface — dashed line, hollow triangle at the interface. I put an interface wherever behaviour can vary or I need a test double (payment gateway, notifier, pricing strategy), and the rest of the design depends on the interface only."

- *Q: Interface or abstract class?* — A: Interface for a capability many unrelated classes can have, or when a class needs several; abstract class when subclasses share state and a fixed workflow. They combine: interface on top, abstract skeleton below (`List` → `AbstractList` → `ArrayList`).
- *Q: Is a lambda a realization?* — A: Yes; it is an instance of a functional interface.

Self-check

- *Q: Generalization vs realization on paper?* → A: Same hollow triangle; solid line vs dashed line.
- *Q: How many interfaces can a Java class implement?* → A: Any number.

TL;DR

Realization = "implements". Dashed line + hollow triangle at the `«interface»`. Depend on the interface, plug in implementations.

3.8 Decision guide: choosing the relationship between two classes

Ask **in order**, stop at the first "yes":

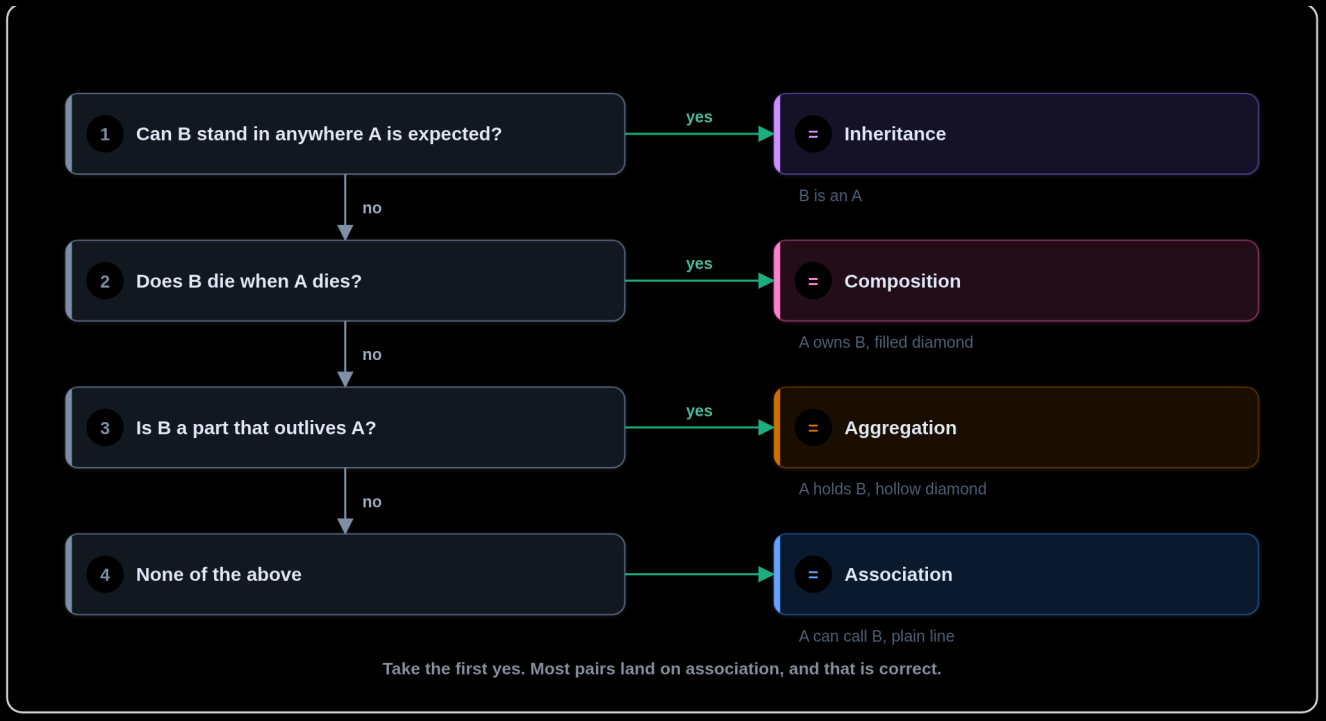
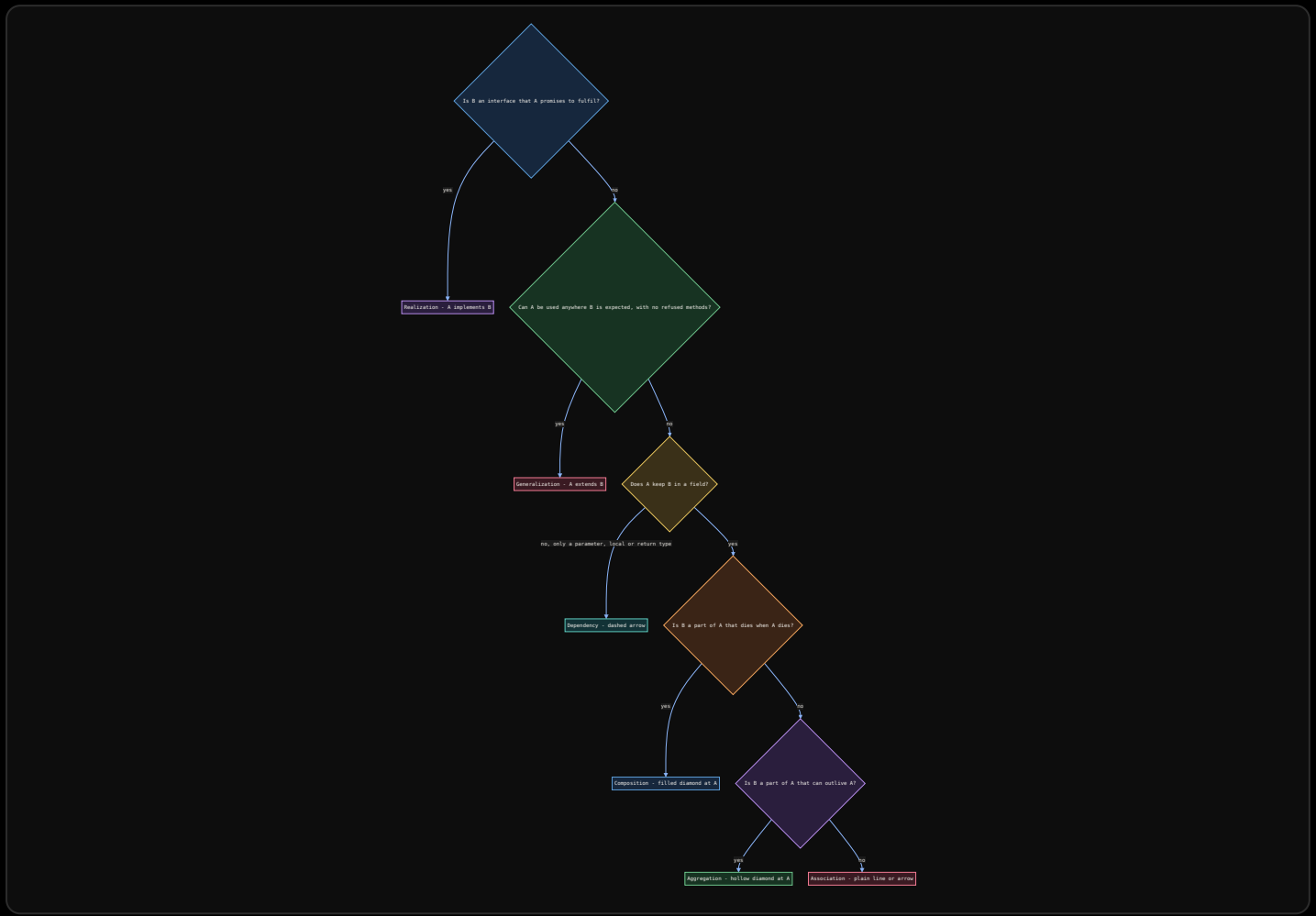


Figure — The four-question decision: inheritance, then composition, then aggregation, else association (course diagram)

Inheritance comes first only because it is easiest to *rule out* (a strict yes to the substitution test, which most pairs fail). Most pairs end as plain association — healthy; if nearly every box extends another, look again. Full guide, with the two "light" relationships:



Summary table (keep this one)

Relationship	Phrase	UML line	Java shape	Lifetime	Example
Dependency	A *uses* B	dashed, open arrow → B	B as parameter / local / return / static call	none	`CheckoutService` uses `PaymentGateway`
Association	A *knows* B	solid line (arrow for one-way)	field of type B, B created elsewhere	independent	`Order` → `Customer`
Aggregation	A *has* B (parts live on)	solid, hollow ◇ at A	field/collection, parts passed in	B outlives A	`Airline` ◇— `Aircraft`
Composition	A *owns* B	solid, filled ◆ at A	A creates B, hides it, B not shared	B dies with A	`ParkingFloor` ◆— `ParkingSpot`
Realization	A *implements* B	dashed, hollow △ at B	`implements`	—	`EmailNotifier` ⇢ `Notifier`
Generalization	A *is a* B	solid, hollow △ at B	`extends`	—	`CardPayment` → `Payment`

Mermaid cheat-sheet for these lines (used in all later chapters):

Mermaid	Meaning
`A ..> B`	A depends on B
`A --> B`	A associated with B, navigable A to B
`A -- B`	association (both ways / unspecified)
`A o-- B`	aggregation, A is the whole
`A *-- B`	composition, A is the whole
`B <|.. A`	A realizes interface B
`B <|-- A`	A extends B
`A "1" *-- "1..*" B`	with multiplicities

Common mistakes (all relationships)

- Diamond at the wrong end (goes at the **whole**); triangle at the wrong end (goes at the **parent / interface**).
- Lines without multiplicity: a line with no type or count means the design question is still open. Candidates who write numbers on every line from the first box report the interviewer simply stops asking — cheap, and it removes a whole class of follow-ups.
- Inheritance used to share data.

In the interview

Say the test **out loud** for each line: *"When a floor is removed its spots go too, so this is composition"*; *"Cash and card can both be asked to take money, so inheriting from Payment is honest."* Spot a constant-only subclass and collapse it into a field yourself. Expect "why not inheritance here?" (substitution test) and "what if a spot can hold two motorbikes?" (change the multiplicity, then say what it does to the code).

Self-check

- *Q:* `Library` *and* `Member`*?* → A: Aggregation (or plain association): members exist either way.
- *Q:* `Flight` *and* `WeeklySchedule`*?* → A: Composition.
- *Q:* `ParkingTicket` *and* `ParkingSpot`*?* → A: Association: the ticket refers to the spot but is not a whole made of spots.
- *Q:* `Report.render(Formatter f)`*?* → A: Dependency.

TL;DR

Realization/generalization for "is-a" (only after the substitution test), composition when parts die with the whole, aggregation when they live on, association when one just knows the other, dependency when only used inside a method.

Part B — UML

3.9 What UML is (structural vs behavioural diagrams)

What it is

UML (Unified Modeling Language) is a standard set of symbols for software designs, so one engineer's diagram reads without a legend. Maintained by the OMG (Object Management Group); current version UML 2.5.1. It is **not** a programming language — it does not run, needs no tool (a whiteboard will do); it is an agreement about what, say, "a line with a filled diamond" means.

In the 1990s three competing OO methods — Grady Booch's, James Rumbaugh's OMT, Ivar Jacobson's OOSE — merged their notations into UML; serving three user groups made it large, so every team uses a subset.

The problem it solves

1. **Makes a design visible:** twelve boxes with lines read in seconds; twelve classes in sentences do not.
2. **Makes decisions explicit:** a sentence can be vague about ownership; a diagram forces a filled or hollow diamond.
3. **Makes disagreement cheap:** the interviewer points at one line — "why composition here?" — which is what a good interview looks like.

How it works — the 14 diagram types

UML 2.5 defines **14 diagram types**: **structural** (what the system *is made of* — static parts existing at any moment) and **behavioural** (what it *does* over time), with **interaction diagrams** a behavioural subgroup focused on messages between objects.

Family	Diagram	Shows	Interview use
Structural	**Class**	classes, attributes, operations, relationships	**always**
Structural	Object	a snapshot of real objects and their links	rarely (to explain an example)
Structural	Package	how classes group into packages/modules	rarely
Structural	Component	deployable parts and their interfaces	system design rounds
Structural	Composite structure	internal parts of one class	rarely
Structural	Deployment	software on hardware nodes (machines)	system design rounds
Structural	Profile	extensions to UML itself	never
Behavioural	**Use case**	actors and what they can achieve	early, often spoken
Behavioural	**Activity**	steps of one flow with branches	one per key flow
Behavioural	**State machine**	states of one object and allowed transitions	when an object has a lifecycle
Behavioural → interaction	**Sequence**	who calls whom, in time order	on request
Behavioural → interaction	Communication	same messages, drawn on a network of objects	rarely
Behavioural → interaction	Interaction overview	an activity diagram whose nodes are interactions	rarely
Behavioural → interaction	Timing	state changes against a time axis	embedded (software inside devices) / hardware

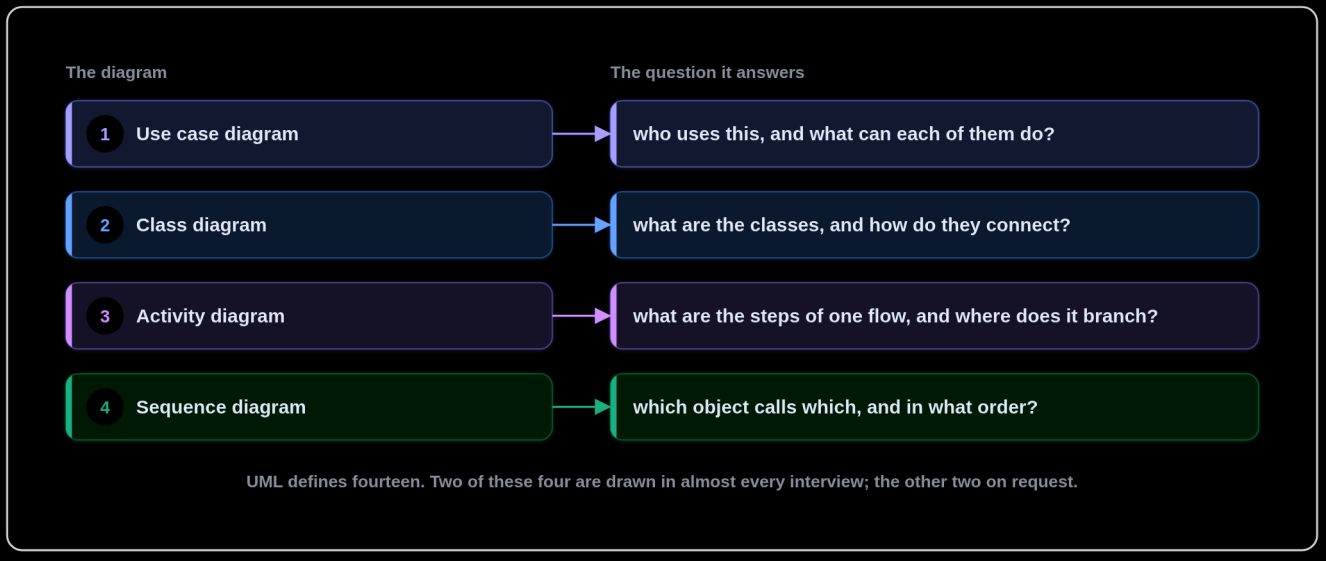


Figure — The four diagrams an OOD interview uses and the question each answers (course diagram)

Four diagrams carry an OOD interview — **use case** (who uses it and what each can do; drawn first, before any class), **class** (*the* answer, the only one mapping to code), **activity** (one flow's steps and branches), **sequence** (who calls whom, in order) — plus a fifth this book adds: **state machine** (one object's states and what moves it between them: parking spot, order, elevator, vending machine…).

Trade-offs

Benefit	Cost
Shared vocabulary, no legend needed	Large spec; easy to over-draw
Forces explicit decisions	Takes whiteboard time from code/discussion
Tool support (PlantUML, Mermaid, draw.io, StarUML: diagram-drawing tools)	Diagrams go stale if not kept with code

When not: as unmaintained documentation, or in full formal detail on a whiteboard. Teams today use lightweight "sketch" UML, the C4 model for architecture (a system drawn at four zoom levels), or text-based Mermaid/PlantUML next to the code. For an interview, UML-lite is right.

Common mistakes

- Thinking notation is graded: what is graded is whether **diagram and words agree** — forget the realization arrow, say "this implements that"; draw a hollow diamond meaning filled and say so; that is discussion, not a lost point. A line you cannot justify is worse than a missing one.

Real-world: Mermaid renders natively in GitHub and GitLab Markdown (a plain-text formatting language), so design docs (RFCs, written design proposals; ADRs, Architecture Decision Records) embed diagrams as text; PlantUML is common in enterprise wikis (Confluence via plug-ins) for service-call sequence diagrams.

In the interview

> **30-second answer:** "UML is a standard notation for designs. It has 14 diagrams: structural ones (class, object, component…) describe what the system is made of; behavioural ones (use case, activity, state machine, and the interaction diagrams such as sequence) describe what it does. In an OOD round I use a quick use-case list, one class diagram, and an activity or sequence diagram for the key flow."

- *Q: Is UML outdated?* — A: Full formal UML is rare, but its core notations (class, sequence, state) remain the shared language of design and live on in Mermaid/PlantUML in modern repos (code repositories). You need only the subset.
- *Q: Which diagram maps to code?* — A: The class diagram.

Self-check

- *Q: Sequence diagram — structural or behavioural?* → A: Behavioural (interaction).
- *Q: How many UML diagram types?* → A: 14 (7 structural, 7 behavioural).

TL;DR

UML = standard symbols, not a language that runs. 14 diagrams: structural (what it is) vs behavioural (what it does). Interviews use class (always), use case, activity, sequence, and state machine when an object has a lifecycle.

3.10 Use-case diagrams

What it is

A **use-case diagram** shows **who** uses the system and **what** each can achieve — no classes, methods, data or ordering.

- An **actor** is a *role* interacting with the system, drawn as a stick figure outside it. Roles, not people: one person can be Member and Librarian; one actor can stand for thousands of users. Actors need not be human — a payment gateway, email service or timer ("System") is an actor.

- A **use case** is one goal producing a result the actor cares about, an oval with a verb phrase: "Borrow a book". "Click the search button" is a step inside one, not a use case.
- The **system boundary** is a rectangle around all use cases, labelled with the system name: inside = what you build.

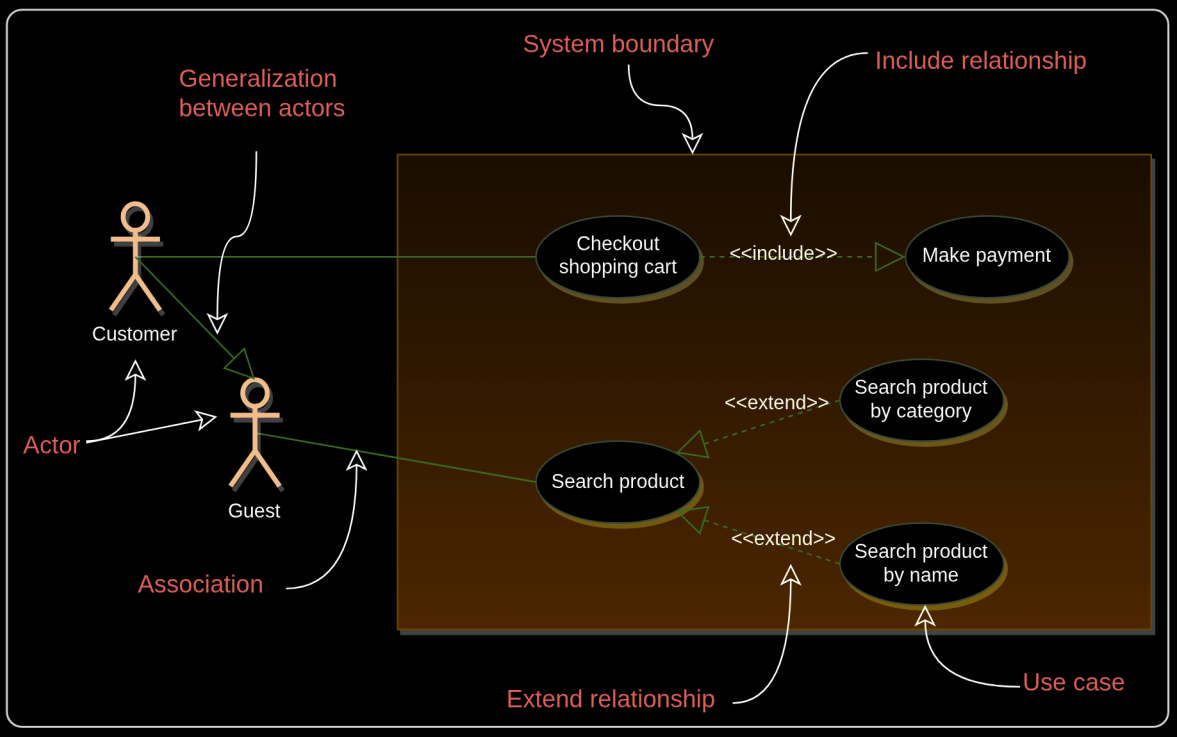


Figure — The parts of a use-case diagram for online shopping: actors (Customer, Guest), use cases, system boundary, generalization between actors, and «include» / «extend» relationships (course diagram)

The problem it solves

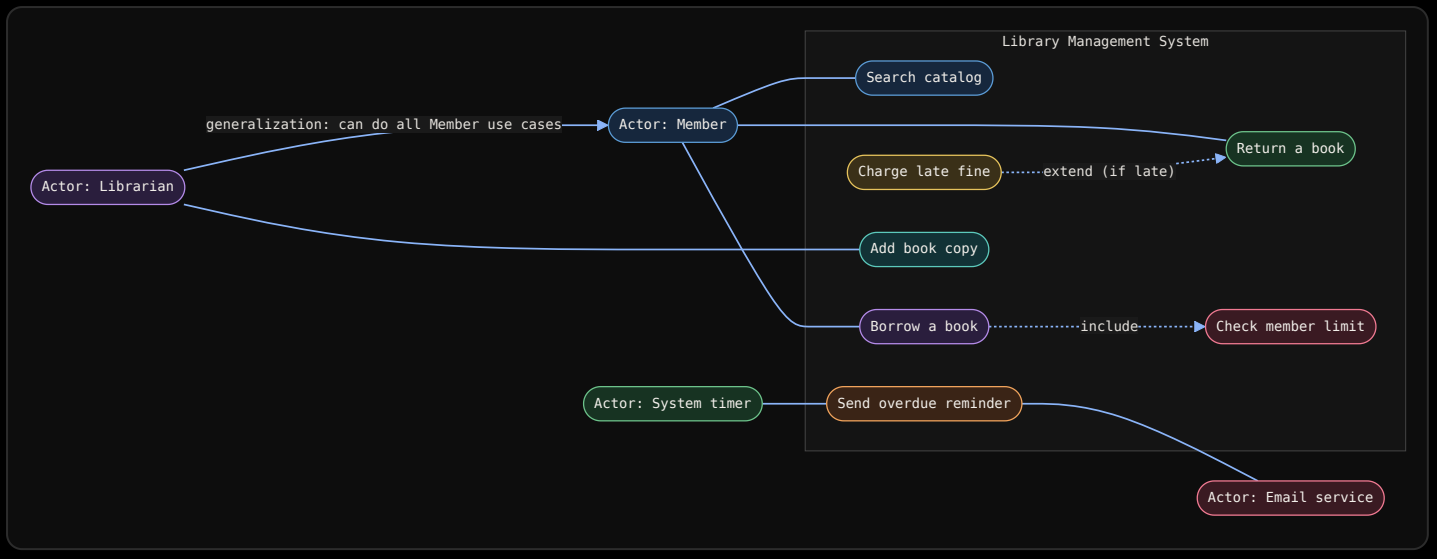
It turns "design a library" into an **agreed scope** ("these nine in; refunds, seasonal pricing, reporting out"), agreed by the interviewer who judges completeness, and yields first classes: **actors usually become classes, use cases methods**.

How it works

1. **Find the actors** with four questions: *Who starts something?* (customer, member, player). *Who administers it?* (adds books, sets rates, blocks accounts — the most-forgotten actor, bringing a whole column of use cases). *Who gets told?* (notification recipients). *What acts with nobody present?* (timers, background jobs, external services — draw as "System").
2. **Primary actors** (who start use cases) on the left, **secondary/supporting actors** (whom the system calls, e.g. payment gateway) on the right.
3. Draw the **system boundary**; put each **use case** (verb phrase) inside.
4. Connect actors to their use cases with **plain lines (associations)**.
5. Add use-case relationships only where they help:
   - **«include»** — *always*: the base always performs the included one. Dashed arrow **from base to included**. *Borrow a book* «include» *Check member's limit*. In code: one method always calls another.
   - **«extend»** — *sometimes*: optional behaviour inserted into the base under a condition, at a named **extension point**. Dashed arrow **from the extending case back to the base**. *Charge fine* «extend» *Return a book* (only if late). Test: can the base complete without it? Yes → extend; no → include.
   - **Generalization** — solid line, hollow triangle: between actors (a Librarian can do everything a Member can) or use cases (*Pay by card* is a kind of *Make payment*).
6. Keywords are singular: «include», «extend». Older material uses «includes», «extends» or «uses» («uses» was replaced by «include» long ago).

In the course figure the Customer→Guest triangle is at **Guest**: a Customer can do everything a Guest can, plus check out. Its «include»/«extend» arrows have hollow triangles; strict UML draws them dashed with an **open** arrowhead (triangles mean generalization). Direction and label are what matter: «include» *Checkout shopping cart → Make payment*; both «extend» arrows run from the search variants back to *Search product*.

Mermaid has no use-case diagram, so this book uses flowcharts (stadium shapes = use cases, a subgraph = the boundary):



From use cases to code (see `lld.ch03.library`)

Use-case element	Becomes in Java
Actor *Member*	class `Member`
Actor *Librarian* (generalizes Member)	`class Librarian extends Member`
Borrow a book	`Member.checkout(BookItem, LocalDate)`
«include» *Check member limit*	private `checkLimit()` called on **every** checkout
Return a book	`Member.returnBook(BookItem, LocalDate)`
«extend» *Charge late fine*	`chargeFine(...)` called **only if** the book is late
Add book copy	`Librarian.addCopy(...)`

A starting point, not a rule: a use-case method sometimes belongs on a service class instead.

Trade-offs

Pro	Con
Fast scope agreement; finds forgotten actors	No ordering, no data — it cannot show *how*
Maps to classes and methods	Easy to bloat with UI (user interface) steps

When not: if the interviewer already gave precise requirements, a spoken list of actors and use cases suffices.

Common mistakes

- **Use cases that are UI steps.** Typing a password, pressing submit, seeing a confirmation = one use case, "Log in". Dozens of ovals usually means a list of screens, not goals.
- **One actor per person.** A librarian who borrows is *two roles*: two actors, linked by generalization if one can do everything the other can.

- Reversing «extend» (it points from optional case to base); «include» for something that happens only sometimes; leaving off the boundary — the most useful part.

Real-world: an Amazon-style checkout: *Place order* always «include»s *Make payment*; *Apply gift card* «extend»s it only if the customer has one. **BookMyShow:** actors are Customer, Theatre admin, Payment gateway and a timer releasing held seats.

In the interview

> **30-second answer:** "Before classes, I list actors and what each can do, and draw a boundary to agree scope. Actors are roles — including admins, external services and timers. «include» is always, «extend» is sometimes. Actors become my first classes and use cases become methods."

- *Q: 'Make payment' — include or extend of 'checkout'?* — A: «include» if every checkout requires payment; "apply coupon", which checkout completes without, is «extend».
- *Q: How long should this take?* — A: Three to five minutes, early.

Self-check

- *Q: Direction of the «include» arrow?* → A: From the base use case to the included one.
- *Q: Is a payment gateway an actor?* → A: Yes — an external system (a secondary actor).
- *Q: "Search product by name/by category" vs "Search product"?* → A: As in the course diagram, optional variants drawn as «extend» (use-case generalization is also defensible if explained).

TL;DR

Actors (roles, outside) + use cases (goals, inside) + boundary (scope). «include» = always, arrow base→included; «extend» = sometimes, arrow extension→base. Actors → classes, use cases → methods.

3.11 Class diagrams

What it is

A **class diagram** shows the **static structure**: which classes exist, what each holds (attributes) and can do (operations), and how they connect (relationships with multiplicity). It is the answer to an OOD question and the only UML diagram mapping directly to code; every other diagram either gets you to it (use case) or checks/explains it (activity, sequence, state), and it puts the whole design on one reviewable page.

How it works — the class box

A rectangle with three compartments: **name** / **attributes** / **operations**.

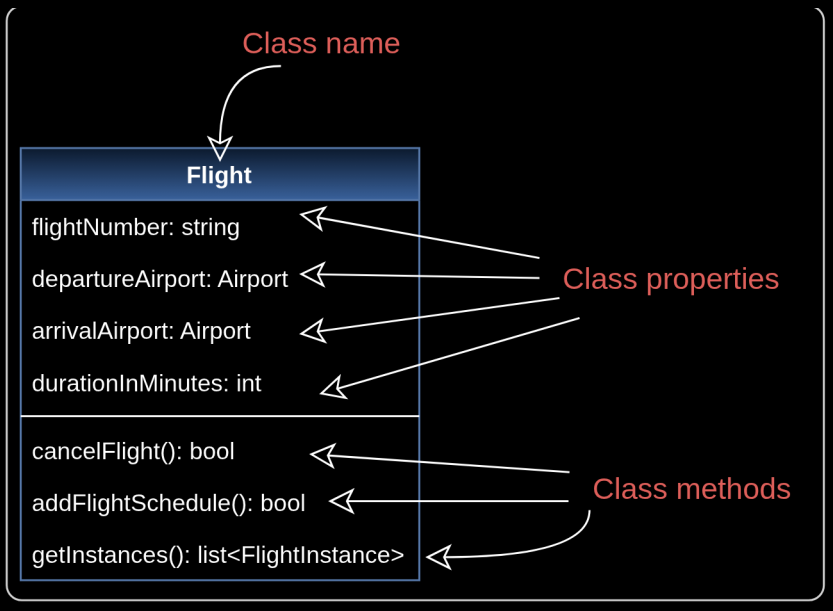


Figure — A class box: name on top, properties (attributes) in the middle, methods (operations) at the bottom, using a Flight class (course diagram)

Attributes are `name: Type`, operations `name(params): ReturnType`. Full syntax:

```
visibility name : Type [multiplicity] = default      e.g.  - spots : ParkingSpot [1..*]
visibility name(param : Type, ...) : ReturnType      e.g.  + park(v : Vehicle) : Ticket
```

UML	Meaning	Java
`+`	public — anyone	`public`
`-`	private — only this class	`private`
`#`	protected — this class and subclasses	`protected` (Java also allows the same package)
`~`	package — same package only	no modifier (package-private)

Notation	Meaning	Java
Italic name, or `{abstract}` / `«abstract»`	abstract class (never instantiated directly) / abstract operation (no body)	`abstract`
`«interface»` above the name	interface	`interface`
`«enumeration»` + literals	enum	`enum`
underlined member	static (class-level)	`static`
`{readOnly}`	cannot change after creation	`final` field
`List<T>` / `T[*]`	collection attribute	`List<T>`

You need **not** fill all three compartments: early on a name-only box is a good class. In Mermaid, visibility uses the same symbols; `*` after a method = abstract, `$` = static; `<<interface>>`, `<<abstract>>`, `<<enumeration>>` are annotations; generics are `List~T~`.

How it works — the relationships (all arrow types)

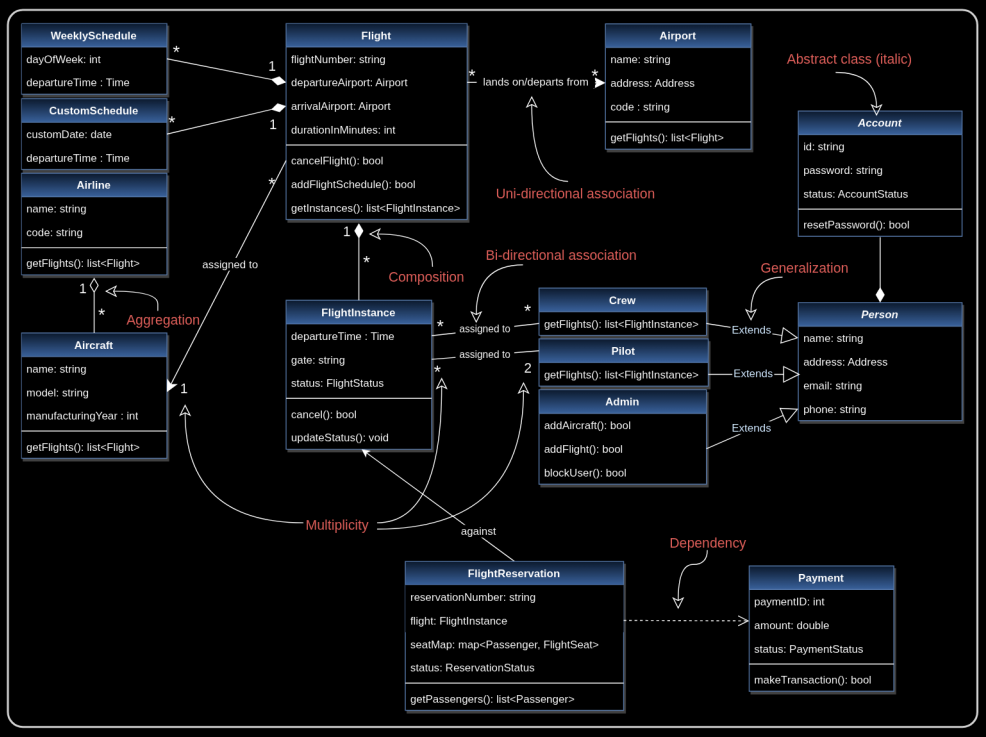


Figure — An airline-system class diagram using every relationship: associations (e.g. Flight to Aircraft, uni-directional), multiplicities (FlightInstance has 2 Pilots), aggregation (Airline–Aircraft), composition (Flight–WeeklySchedule), generalization (Person to Crew/Pilot/Admin) and dependency (course diagram)

The figure uses the complete vocabulary — Part A's six relationships (association two-way and one-way) plus multiplicity:

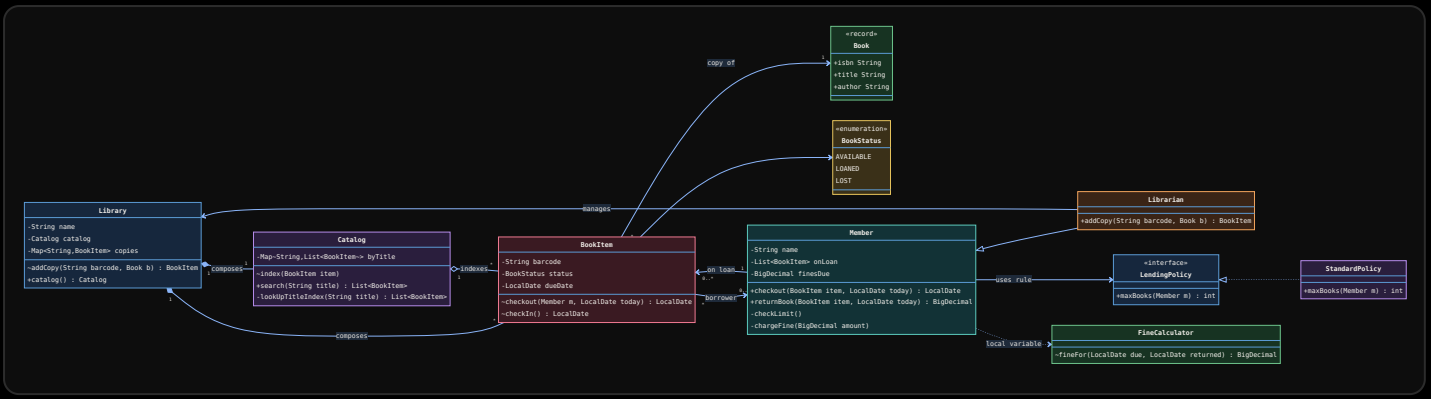
Line	Drawn as	Read as	Example from the figure
Association (bi-directional)	solid line	"A and B know each other"	FlightInstance — Pilot, FlightInstance — Crew ("assigned to")
Uni-directional association	solid line, arrowhead at B	"A can call B, not vice versa"	Flight → Airport ("lands on/departs from"), Flight → Aircraft ("assigned to")
Multiplicity	numbers at line ends: `1`, `*`, `0..1`, `2..4` (a range)	"how many"	FlightInstance `*` — `2` Pilot: each instance has exactly 2 pilots, each pilot flies many
Aggregation	hollow diamond at the whole	"has-a, part lives on"	Airline `1` ◇— `*` Aircraft
Composition	filled diamond at the whole	"owns, part dies with it"	Flight ◆— WeeklySchedule, Flight ◆— CustomSchedule, Flight ◆— FlightInstance
Generalization	solid line, hollow triangle at parent	"is-a"	Pilot, Crew, Admin → Person
Realization	dashed line, hollow triangle at interface	"implements"	(any `«interface»`)
Dependency	dashed line, open arrowhead	"uses"	FlightReservation ⇢ Payment

(The figure's one-way arrowheads are filled; strict UML uses open — see 3.1.) Some figure classes carry getters for the "other" direction (`Aircraft.getFlights()`) though the line is one-way; strictly that makes it navigable both ways — read the arrow as "primary direction", and keep arrows and fields consistent in your own diagrams.

Two extra notations to recognise:

- **Association class** — a class hanging off an association by a dashed line, holding data about the *link itself* (`Enrollment{grade}` on Student–Course).
- **Qualified association** — a small box on one end with a key (`Library [isbn] → Book`): "look up by this key", in Java a `Map<Isbn, Book>`.

Worked example — the library design (code in `lld.ch03.library`)



Every line type appears — note aggregation Catalog ◇ BookItem (the index points at copies it does not own) and dependency Member ⇢ FineCalculator. `~` marks package-private methods like `BookItem.checkout`, callable only by `Member` (same package). The Java essentials are in 3.12.

How much detail to draw

Every class a named box (10–15 is a full answer), **attributes only where they matter** (the two or three carrying the design), **operations only those in your flows**, and **every line typed with a multiplicity** — the part not to skip. Short of time, **drop attributes before relationships**: name-only boxes with precise lines describe a design; rich boxes with vague lines do not (budget in 3.15).

Trade-offs

Detailed diagram	Sketch diagram
Near code-ready; fewer surprises later	Faster; easier to change during discussion
Takes 10+ minutes	Leaves details for the coding phase

When not: it shows no *order* or *time* — use sequence/activity/state diagrams for behaviour.

Common mistakes

- Unlabelled lines; diamonds/triangles at the wrong end; every getter/setter and `String` field listed; associations on the diagram that are not fields in the code (or vice versa).

Real-world: IntelliJ IDEA (Ultimate), a Java IDE (code editor with build/run tools), generates class diagrams from code; JPA entity diagrams (ER-style: entity-relationship pictures of tables) are class diagrams with multiplicities (`@OneToMany` = `1..*`).

In the interview

> **30-second answer:** "I start with named boxes from the nouns and actors, then draw every line with a type and multiplicity, saying the lifetime/substitution test aloud. I add only the attributes and methods that drive the key flows. `+ - # ~` for visibility, italics or «abstract» for abstract, «interface» for interfaces."

- *Q: `#` and its Java nuance?* — A: Protected; in Java it is also visible within the same package, which UML's `#` does not imply.
- *Q: How do you show a static method?* — A: Underline it (Mermaid: `$` suffix).

Self-check

- *Q:* `~` *means?* → A: Package visibility (Java's default, no modifier).
- *Q: The one test for aggregation vs composition?* → A: Whole gone — is the part still meaningful by itself?

TL;DR

Box = name / attributes / operations with `+ - # ~`. Seven lines: association (± arrow), aggregation ◇, composition ◆, generalization (solid △), realization (dashed △), dependency (dashed arrow), plus multiplicity on every line.

3.12 Sequence diagrams

What it is

A **sequence diagram** shows **which object calls which, and in what order**, for one scenario: objects across the top, **time down the page**.

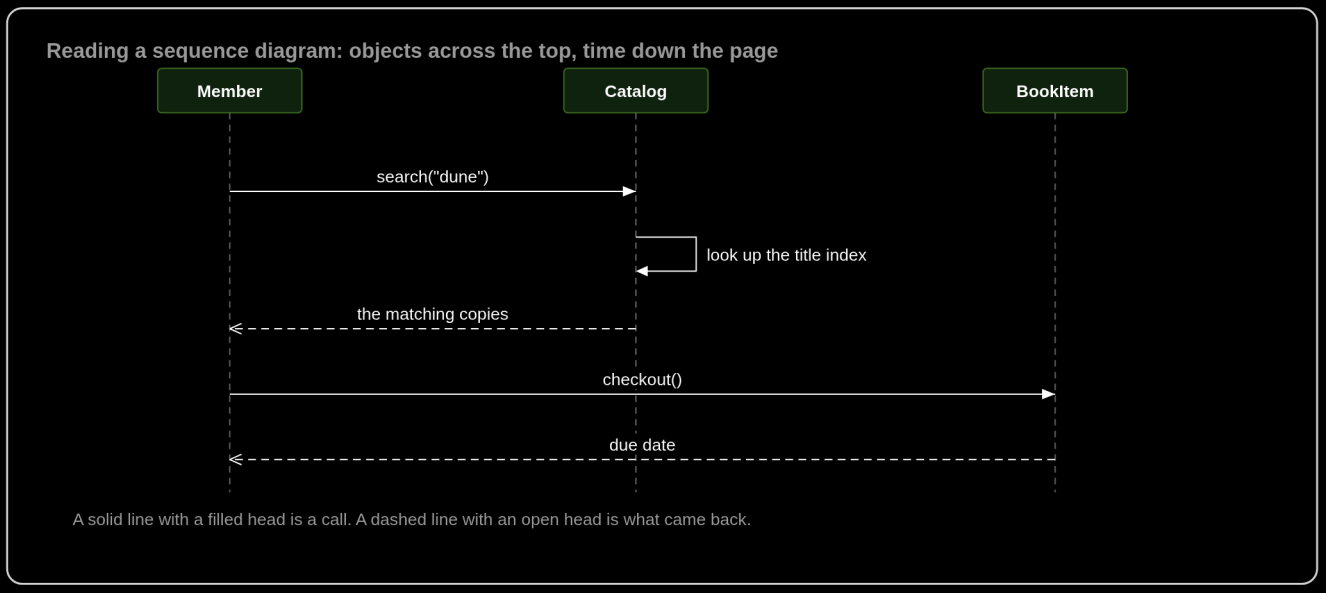


Figure — A member searches the catalog for "dune" and borrows a copy: lifelines, a self-call on Catalog, calls with filled arrowheads and dashed returns (course diagram)

The problem it solves

A class diagram says objects *can* talk; this shows what they *actually say*, in order. Use it when order is easy to get wrong (reserve seat → take payment → confirm; any other order is a bug), when several objects take part (two need no diagram; five usually do), or to explain a drawn class diagram.

How it works — the notation

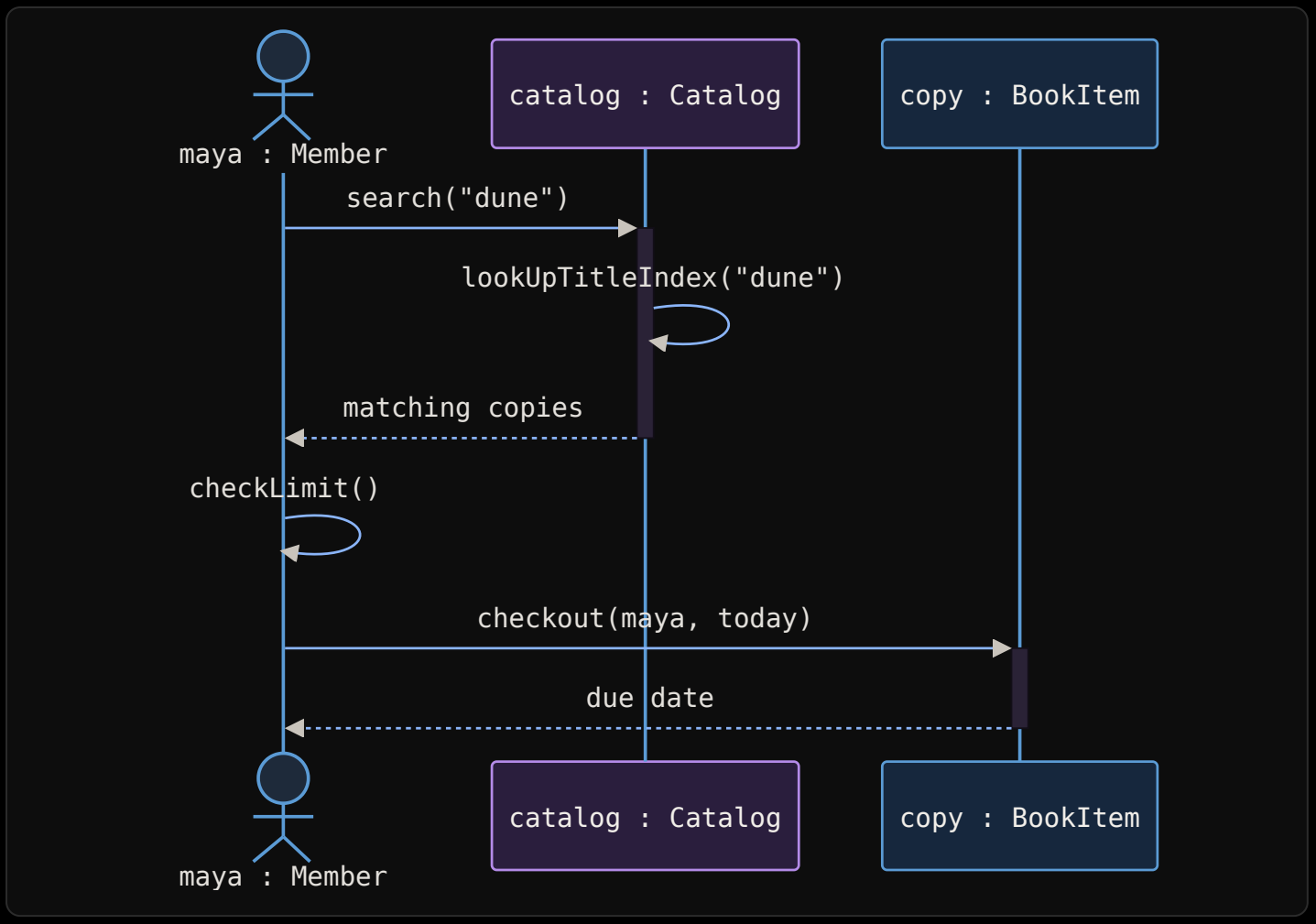
Element	Drawn as	Meaning
Participant / lifeline	box at top + dashed vertical line	one *object* (strictly `catalog : Catalog`; most write the class name)
Actor	stick figure at top	a user or external system starting the flow
Synchronous message	solid line, **filled** arrowhead	a call; the caller waits for it to finish
Asynchronous message	solid line, **open** arrowhead	fire-and-forget; caller continues (events, queues: lists of messages waiting to be processed)
Return (reply)	dashed line, open arrowhead	the value coming back — draw only when it matters
Self-message	arrow looping back to the same lifeline	the object calls its own method
Activation bar (execution specification)	thin rectangle on a lifeline	the object is busy executing
Create message	dashed arrow to the new object's box, often «create»	the object is created at that moment
Destruction	**X** at the end of a lifeline	the object is destroyed
Combined fragment	labelled box around messages	control logic: see below

Combined fragments (top-left label = *operator*; conditions, *guards*, in `[ ]`):

Operator	Meaning	Java equivalent
`alt`	alternatives, one runs (sections split by dashed lines, each guarded, last can be `[else]`)	`if / else if / else`
`opt`	optional, runs only if the guard is true	`if` without `else`
`loop`	repeats while the guard holds (`loop [for each item]`)	`for` / `while`
`par`	sections run in parallel	threads (independent lines of execution) / `CompletableFuture` (a result arriving later from background work)
`break`	if guard true, run this and leave the enclosing fragment	early `return` / `break`
`critical`	atomic region (one indivisible step), no interleaving	`synchronized` (one thread at a time) / lock
`ref`	reference to another diagram	calling a helper flow

(UML also has `neg`, `strict`, `seq`, `assert`, `ignore`, `consider` — not needed in interviews.) On a whiteboard activation bars and fragments are optional and usually skipped: lifelines, calls, the returns that matter and self-calls suffice.

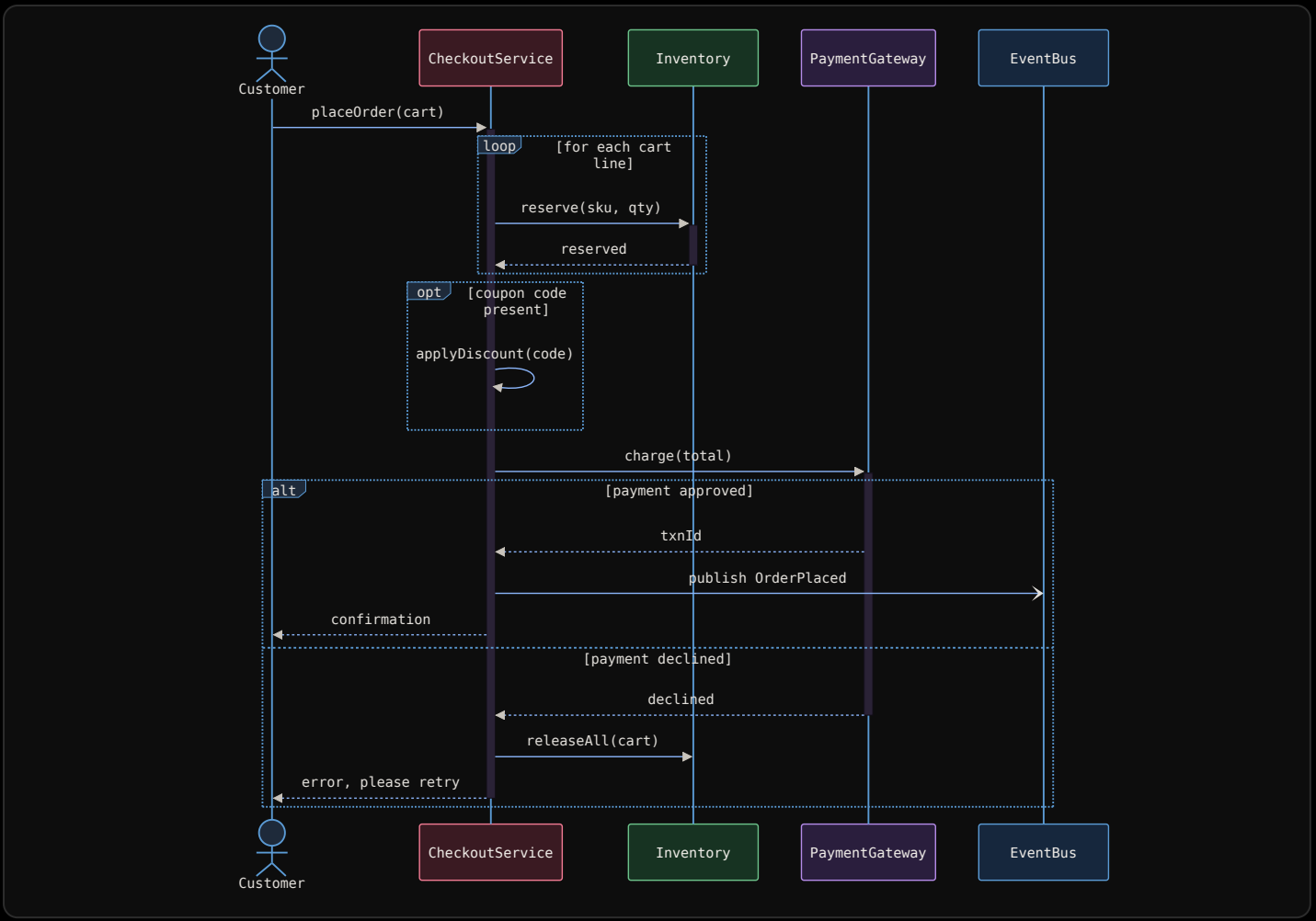
Example 1 — the library checkout, matching the Java code



Four things to notice (as in the course figure):

1. **The first message starts the flow**, always from outside the objects shown: the member calls `search`.
2. **The catalog calls itself** — the title-index lookup needs no other object.
3. **Only two returns are drawn** — copies and due date; other returns are not worth naming.
4. **The member talks to the copy directly**, not back through `Catalog` — a choice no other diagram shows, which an interviewer can question by tapping that arrow.

Example 2 — all fragments and async messages (checkout with payment)



`->>` is a synchronous call, `-->>` a reply, `-)` an **asynchronous** message (publishing does not wait for subscribers). (An `EventBus` delivers each published event to all subscribers; `sku` = stock-keeping unit, a product ID; queues and async messaging: System Design Fundamentals.) The order is the point: reserve stock *before* charging, release it if the charge fails.

Java code for Example 1 (`lld.ch03.library`)

Essentials below; full files in `code/lld/ch03/library/` (`Book` record (isbn, title, author); `BookStatus` enum AVAILABLE/LOANED/LOST; `StandardPolicy.maxBooks` = 10 for a `Librarian`, else 2; `FineCalculator.fineFor` = 5.00 per late day (`ChronoUnit.DAYS`), else 0.00 — used as a local variable, a dependency; `Librarian extends Member` and its `addCopy` delegates to `Library.addCopy`, which creates the `BookItem`, stores it and `catalog.index`es it; `Library` composes its `Catalog`).

```
// File: code/lld/ch03/library/BookItem.java — one physical copy: BookItem * --> 1 Book, 0..1 borrower
static final int LOAN_DAYS = 14;
LocalDate checkout(Member m, LocalDate today) {      // package-private; returns the due date
    if (status != BookStatus.AVAILABLE) throw new IllegalStateException(barcode + " is " + status);
    status = BookStatus.LOANED;
    borrower = m;
    dueDate = today.plusDays(LOAN_DAYS);
    return dueDate;
}
// checkIn(): requires LOANED, resets to AVAILABLE, returns the old due date; toString = barcode[title, STATUS]

// File: code/lld/ch03/library/Catalog.java — aggregates BookItems: points at them, does not own them
public List<BookItem> search(String title) {
    List<BookItem> hits = lookUpTitleIndex(title);   // self-call in the sequence diagram
    return List.copyOf(hits);
}
private List<BookItem> lookUpTitleIndex(String title) { return byTitle.getOrDefault(key(title), List.of()); }
// byTitle: Map<lower-cased title, List<BookItem>>, filled by index(item)

// File: code/lld/ch03/library/Member.java — actor -> class, use cases -> methods
public LocalDate checkout(BookItem item, LocalDate today) {
    checkLimit();                                    // <<include>>: happens every time
    LocalDate due = item.checkout(this, today);
    onLoan.add(item);
    return due;
}
public BigDecimal returnBook(BookItem item, LocalDate today) {
    if (!onLoan.remove(item)) throw new IllegalStateException(name + " does not hold " + item.barcode());
    LocalDate due = item.checkIn();
    BigDecimal fine = new FineCalculator().fineFor(due, today);
    if (fine.signum() > 0) chargeFine(fine);         // <<extend>>: only when late
    return fine;
}
private void checkLimit() {
    int max = policy.maxBooks(this);                 // policy: LendingPolicy interface (association)
    if (onLoan.size() >= max) throw new IllegalStateException(name + " already has " + max + " books (limit)");
}
private void chargeFine(BigDecimal amount) { finesDue = finesDue.add(amount); }   // BigDecimal, not double

// File: code/lld/ch03/library/Main.java (abridged) — Librarian Lee adds B-1, B-2 (Dune), B-3, B-4 (The Hobbit)
LocalDate day1 = LocalDate.of(2026, 3, 1);
List<BookItem> hits = lib.catalog().search("dune");                    // printed
LocalDate due = maya.checkout(hits.get(0), day1);                       // printed
maya.checkout(lib.catalog().search("the hobbit").get(0), day1);         // then prints maya.onLoan()
maya.checkout(hits.get(1), day1);                        // <<include>> limit fires -> "Blocked: ..."
new Member("Omar", policy).checkout(hits.get(0), day1); // copy already LOANED -> "Blocked: ..."
maya.returnBook(hits.get(0), day1.plusDays(10));         // on time
maya.returnBook(hobbitCopy, day1.plusDays(17));          // 3 days late: <<extend>> fine applies
// finally prints policy.maxBooks(lee) and policy.maxBooks(maya)
```

Output:

```
search("dune") -> [B-1[Dune, AVAILABLE], B-2[Dune, AVAILABLE]]
Maya borrowed B-1, due 2026-03-15
Maya on loan: [B-1[Dune, LOANED], B-3[The Hobbit, LOANED]]
Blocked: Maya already has 2 books (limit)
Blocked: B-1 is LOANED
Returned B-1 on time, fine = 0.00
Returned B-3 3 days late, fine = 15.00, Maya owes 15.00
Librarian limit: 10, member limit: 2
```

Check diagram against code: every arrow is a real method — `Catalog.search`, `Catalog.lookUpTitleIndex`, `Member.checkLimit`, `BookItem.checkout`. **A message with no method means your class diagram is missing one** — the activity diagram's check, from another direction.

Trade-offs

Good at	Poor at
Order of calls, who talks to whom	Branch-heavy flows (nested `alt` becomes unreadable)
Showing sync vs async	Showing the whole design
Spotting missing methods	Being quick to draw — each message is a line

When not: flows with three or more decisions — use an **activity diagram** (a second `alt`/`opt` box is the warning sign); unprompted in most OOD interviews — it costs class-diagram time.

Common mistakes

- A return for every call (twice the height, nothing more said); messages that are not methods on the receiver.
- Mixed-up arrowheads (see the notation table).
- Classes instead of specific objects when two of one class take part (`from : Account`, `to : Account`).

Real-world: **OAuth 2.0 / OpenID Connect** ("log in with another account" protocols) flows (browser → app → authorization server → token endpoint); **payment integrations** (Stripe, Razorpay): "create order → redirect → webhook" (an HTTP call the provider makes to your server), the webhook drawn async; **microservice sagas** (microservices: small separately deployed services; saga: a multi-step transaction with an "undo" per step, e.g. flight + hotel + car) showing compensating "undo" calls when a step fails — all documented as sequence diagrams.

In the interview

> **30-second answer:** "A sequence diagram shows one scenario: objects across the top, time downwards. Solid filled arrows are sync calls, open arrows async, dashed arrows returns. `alt` / `opt` / `loop` fragments show if/else, if, and loops. I keep it to three or four objects and six to eight messages, and check that every arrow is a method on my class diagram."

- *Q: Sync vs async?* — A: Sync: the caller blocks until the callee returns (a normal call). Async: it continues at once (queue message, event, `CompletableFuture`).
- *Q: Sequence or activity diagram?* — A: Sequence when *who calls whom* in what order matters; activity when branching does.

Self-check

- *Q: A thin rectangle on a lifeline?* → A: An activation bar — the object is executing.
- *Q: Fragment for "retry up to 3 times"?* → A: `loop [attempt <= 3]`.
- *Q: `Order` creates a `Payment` mid-flow?* → A: A create message (dashed, «create») to the new lifeline's box, which starts lower on the page.

TL;DR

Objects across, time down. Filled = sync call, open = async, dashed = return. `alt` = if/else, `opt` = if, `loop` = loop, `par` = parallel. Keep it small; each arrow must be a real method.

3.13 Activity diagrams

What it is

An **activity diagram** traces **one flow end to end**, marking each point where it **branches** (and any parallel steps). A class diagram is the design at rest; this follows one route through it. It is the UML diagram closest to a flowchart, so anyone can read it — and a careless one is easy to produce.

The problem it solves

It makes a flow's business logic (every `if`, `unless`, `otherwise`) visible and checkable. Best interview use: a **test of your class diagram** — walk the flow asking *which class does this?*; an action with no home is a missing class, found on the whiteboard, not in code.

How it works — the symbols

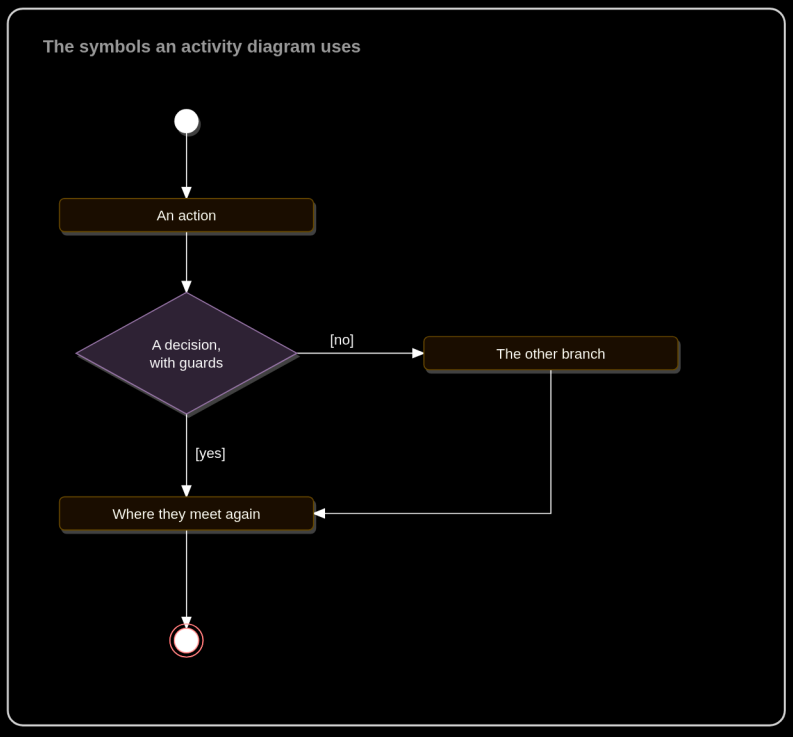


Figure — The core activity-diagram symbols: start, action, a decision with [yes]/[no] guards, the other branch, where the branches meet again, and the end (course diagram)

Symbol	Drawn as	Meaning
Initial node (start)	filled black circle	where the flow begins (draw exactly one; UML allows several, one keeps it readable)
Action	rounded rectangle with a verb phrase	one step: "System calculates the parking fee"
Control flow	arrow	time runs along the arrows
Decision	diamond; each outgoing arrow has a **guard** `[condition]`	a branch; guards must cover every case (use `[else]`)
Merge	diamond (or arrows meeting one node)	branches come back together
Fork	thick bar, one arrow in, many out	following actions run **in parallel**
Join	thick bar, many in, one out	wait until **all** parallel actions finish
Activity final (end)	filled circle in a ring (bullseye)	the whole activity stops; there may be several ("payment failed", "gate opened")
Flow final	circle with an X	only *this* branch stops; others continue
Swimlane (partition)	lanes labelled with an actor/class	*who* performs each action
Object node	rectangle	data passed between actions (e.g. `[Ticket]`)
Send / receive signal	convex / concave pentagon	sending or waiting for an event

Steps: (1) pick **one** flow with an interesting branch — a requirement with "if", "unless" or "otherwise"; (2) start node → actions in the problem's language ("The system calculates the fee", **not** "call

`calculateFee()`"); (3) every decision outcome with a guard; (4) fork/join only for truly simultaneous steps (most interview flows need none); (5) swimlanes if they show who does what; (6) end node(s).

Example 1 — online shopping (course)

Every activity diagram reads as one sentence: *begin, act, ask, follow one of two paths, merge, end.* A bigger example:

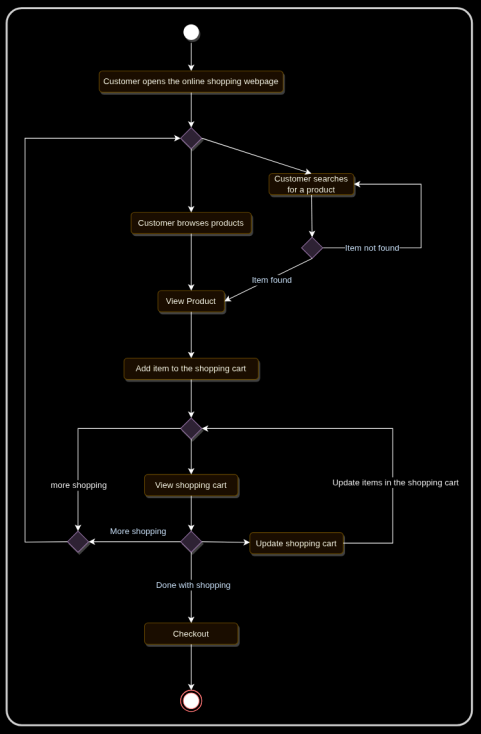
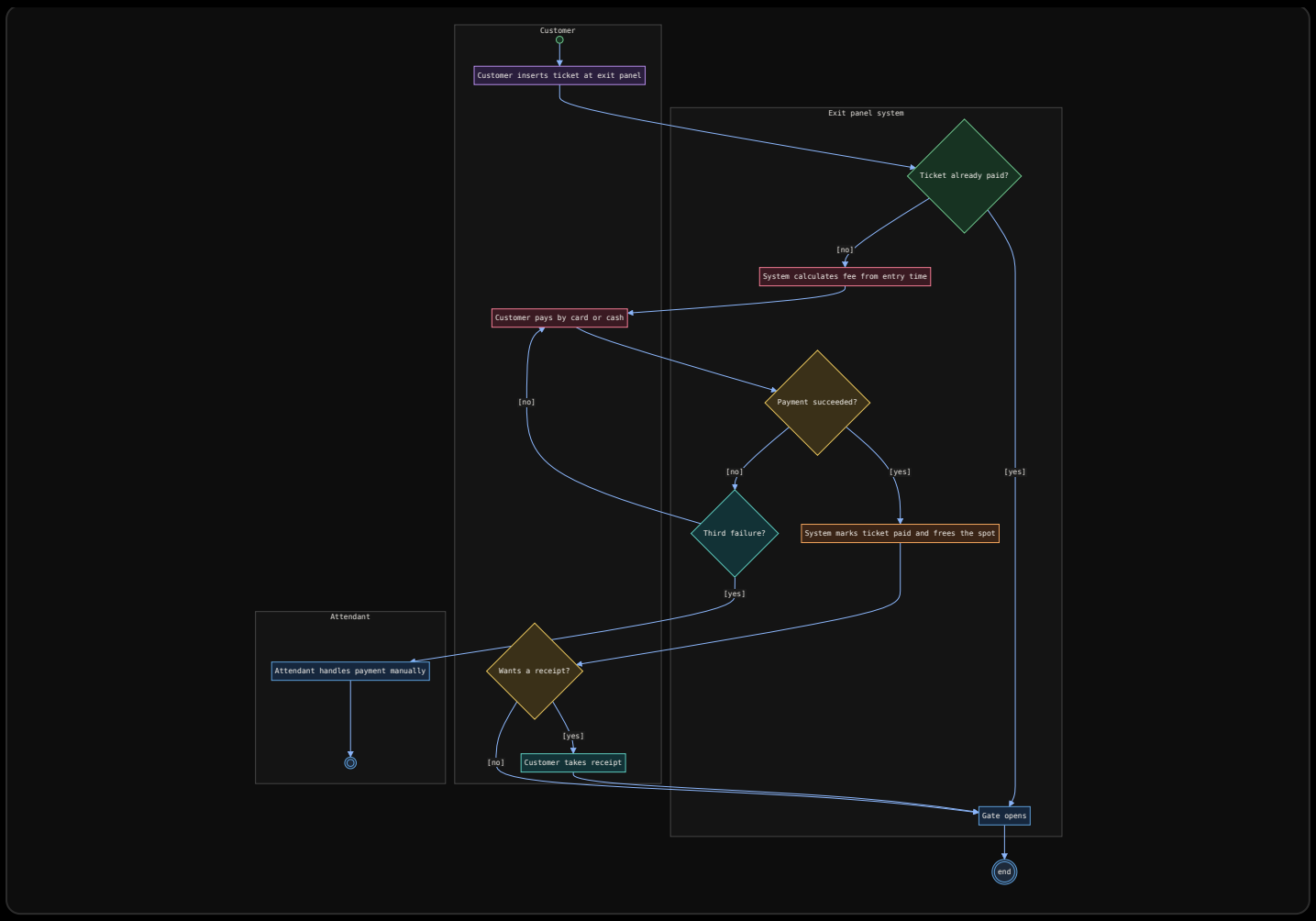


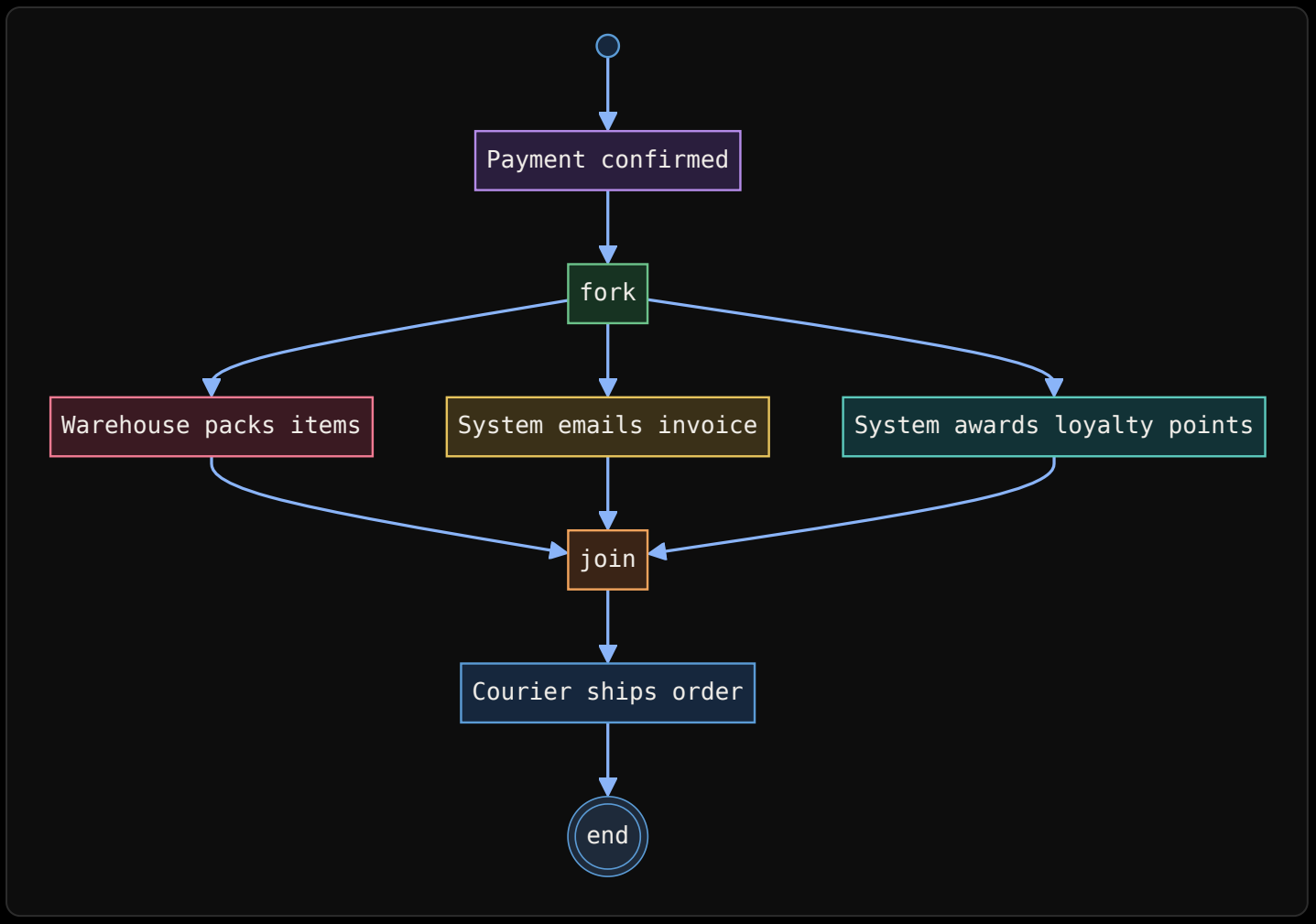
Figure — A customer shopping online: search, item found or not found, add to cart, view/update the cart, more shopping or checkout (course diagram)

Example 2 — pay at the parking exit (three decisions, with swimlanes)

The Parking Lot case study (Chapter 14) has three decisions — *ticket already paid?*, *payment went through?*, *receipt wanted?* — each tracing to a requirement. Drawn with swimlanes (subgraphs), a retry loop, and a hand-off to an attendant after three failures:


Customer
Customer inserts ticket at exit panel
Exit panel system
Ticket already paid?
[no]
System calculates fee from entry time
Customer pays by card or cash
[no]
Payment succeeded?
[no]
[yes]
[yes]
Third failure?
System marks ticket paid and frees the spot
[yes]
Attendant
Attendant handles payment manually
Wants a receipt?
[yes]
[no]
Customer takes receipt
Gate opens
end

Example 3 — fork and join (order fulfilment)



After the **fork** the three actions run in parallel; the **join** waits for all three before shipping.

Activity vs sequence diagram

	Activity diagram	Sequence diagram
Shows	a process as a series of steps	objects sending messages to each other
Branches	natural: decisions with guards	awkward: nested `alt` fragments
Names	actions, in business language	objects and method calls
Use when	the flow has interesting branches	the order of calls is the interesting part
Answers	"what happens?"	"who calls whom?"

Rule of thumb: **three branches** → activity; **four objects** with easily confused call order → sequence.

Trade-offs

Pro	Con
Anyone (including non-engineers) can read it	Does not show which object does what unless you add swimlanes
Great at branches and parallelism	Big flows sprawl quickly

When not: straight-line flows ("customer pays and leaves") — nothing to decide; object-to-object call order — use a sequence diagram.

Common mistakes

- **A diamond with one exit** — nothing is decided: draw the missing branch or drop the diamond.
- **Actions written as code** ("call `calculateFee()`") — keep the problem's language so it checks against requirements.
- Guards not covering every case (`[amount > 100]` and `[amount < 100]` — exactly 100?); fork/join for merely sequential steps.

Real-world: **BPMN** (Business Process Model and Notation), used by workflow engines (run business processes step by step) like Camunda, is a close cousin with lanes and gateways; **CI/CD pipelines** (Continuous Integration / Delivery: build, test, deploy on every change; GitHub Actions, Jenkins) — parallel jobs (fork), a job that `needs:` others (join), conditional steps (guards); food delivery (Swiggy, Zomato, DoorDash) prepares food while assigning a rider — parallel branches joining at "order picked up".

In the interview

> **30-second answer:** "An activity diagram walks one flow: start, actions, decisions with guards covering every case, merges, optional fork/join for parallel steps, swimlanes for who does what, and one or more ends. I draw the flow with the interesting branch, then check that every action has a class to live in."

- *Q: Activity final vs flow final?* — A: Activity final ends the whole activity; flow final ends only that path while parallel paths continue.
- *Q: Which flow do you draw?* — A: The one whose requirement has "if/unless/otherwise" — e.g. payment failure and retry.

Self-check

- *Q: What must every arrow leaving a decision have?* → A: A guard in square brackets; together they cover all cases.
- *Q: Two steps at the same time?* → A: A fork bar before them, a join bar after.

TL;DR

One flow: start → actions → guarded decisions → merge → end. Fork/join for parallel, swimlanes for who. Draw the branching flow and use it to test your class diagram.

3.14 State machine diagrams

What it is

A **state machine diagram** (*statechart*, *state diagram*) shows the **states one object can be in** during its life and the **transitions** moving it between them in response to **events**.

- **State** — a condition in which the object behaves a certain way or waits (rounded rectangle): `CREATED`, `PAID`, `SHIPPED`.
- **Initial pseudostate** — filled circle, arrow to the first state. **Final state** — bullseye: the object's life (in this machine) is over.
- **Transition** — arrow labelled `event [guard] / action`: **event (trigger)** — what happens (`pay(amount)`); **guard** — boolean that must be true to fire (`[amount == total]`); **action (effect)** — what the object does while transitioning (`/ refund()`).
- **Internal activities:** `entry / ...` (on entering), `exit / ...` (on leaving), `do / ...` (while in the state).
- **Self-transition** — arrow back to the same state (exit and re-entry actions run).
- **Choice pseudostate** — a small diamond picking the next state by guards.
- **Composite state** — contains sub-states (`Active` containing `Idle` and `Moving`); **history** `(H)` remembers the last sub-state.

- **Orthogonal regions** — a state split by dashed lines into simultaneously active parts.

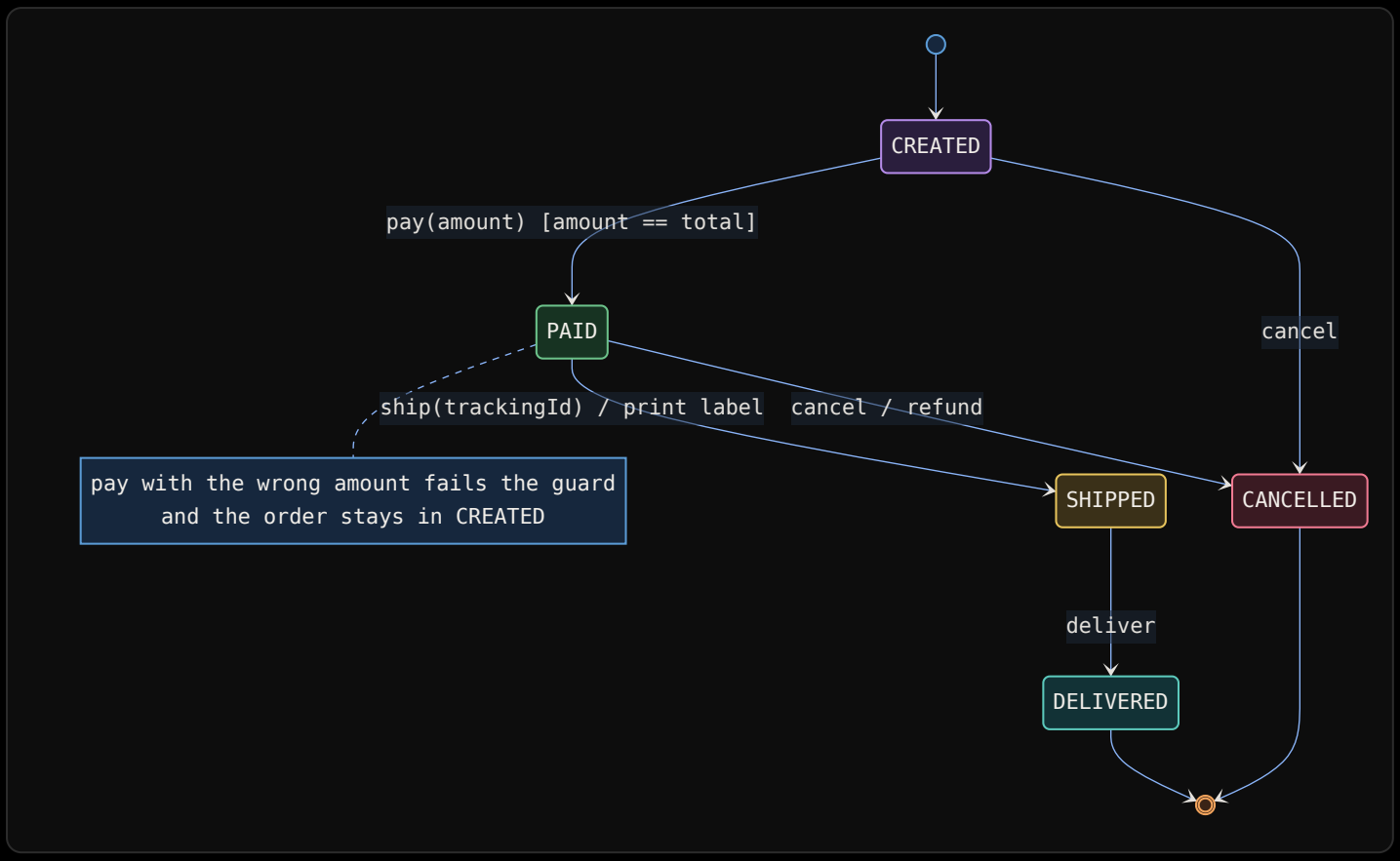
Analogy: a **traffic light** — always one state, a timer moves it on, order fixed (never green straight to red).

The problem it solves

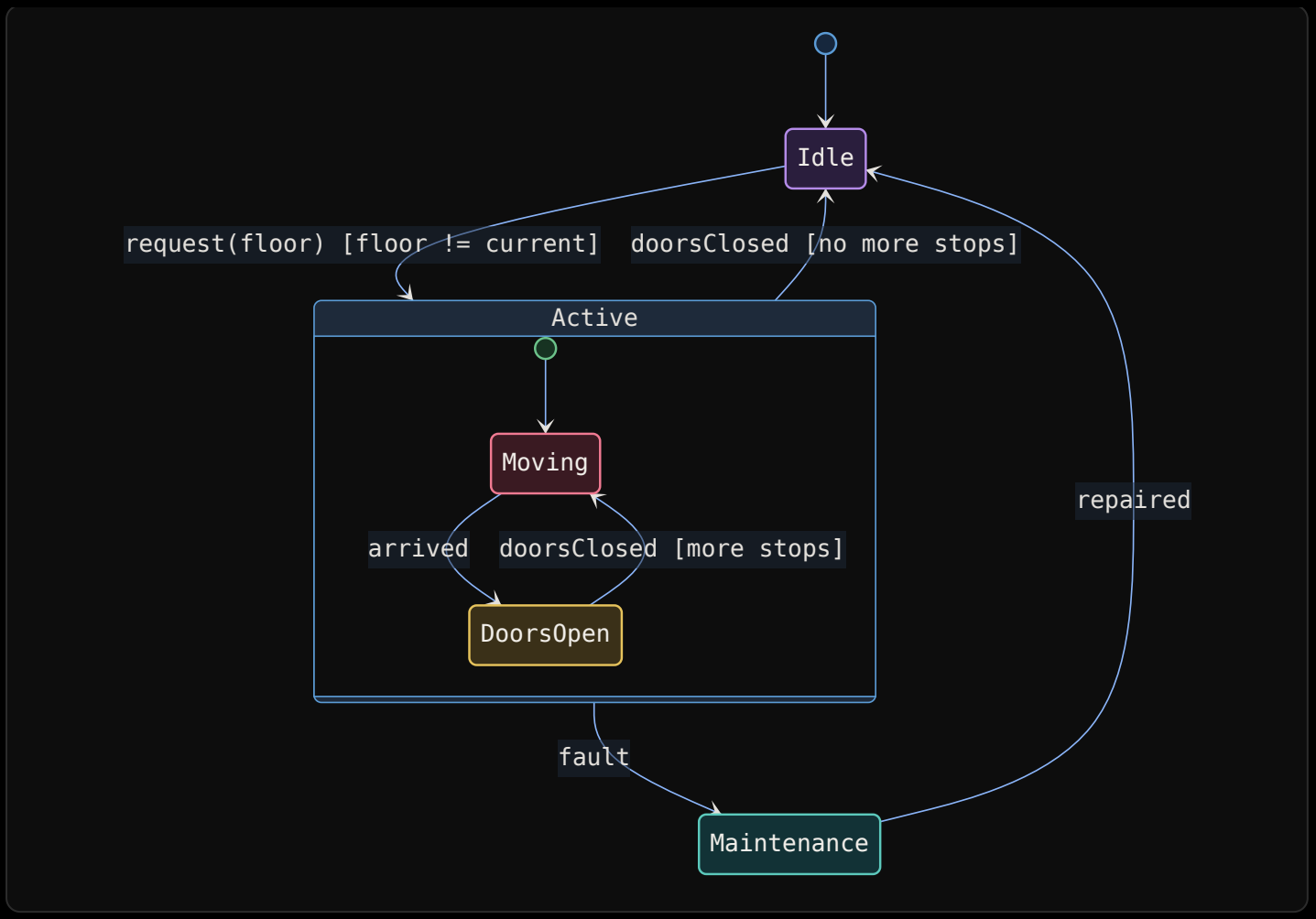
Lifecycle objects (orders, tickets, elevators, vending machines, parking spots, TCP connections — TCP: the reliable-connection network protocol, see System Design Fundamentals) breed bugs: shipping a cancelled order, paying twice, refunding the undelivered. The diagram lists **every legal transition** so all else is rejected, and maps to code (an `enum` + a transition method, or the **State pattern**, Chapter 11).

How it works

1. Pick **one** class with a lifecycle; 2. list its states (usually an `enum`); 3. per state, the accepted events, guard, target and action; 4. everything unlisted is **illegal** — the code rejects it; 5. mark initial and final states.



A composite-state example (an elevator car) for recognition:


Idle
request(floor) [floor != current]
doorsClosed [no more stops]
Active
Moving
arrived
doorsClosed [more stops]
DoorsOpen
repaired
fault
Maintenance

Java example (`lld.ch03.statemachine`)

```java
// File: code/lld/ch03/statemachine/OrderState.java
public enum OrderState { CREATED, PAID, SHIPPED, DELIVERED, CANCELLED;
    public boolean isFinal() { return this == DELIVERED || this == CANCELLED; }
}

// File: code/lld/ch03/statemachine/OrderEvent.java — sealed, so the switch is checked for completeness
public sealed interface OrderEvent {
    record Pay(BigDecimal amount) implements OrderEvent { }
    record Ship(String trackingId) implements OrderEvent { }
    record Deliver() implements OrderEvent { }
    record Cancel(String reason) implements OrderEvent { }
}

// File: code/lld/ch03/statemachine/Order.java — each case = one arrow: event [guard] / action
// fields: id, BigDecimal total, state = CREATED (constructor logs "[*] -> CREATED"), List<String> log
public void handle(OrderEvent event) {
    OrderState from = state;
    OrderState to = switch (event) {
        case OrderEvent.Pay p when state == OrderState.CREATED && p.amount().compareTo(total) == 0 ->
OrderState.PAID;
        case OrderEvent.Pay p when state == OrderState.CREATED ->
                reject(event, "guard failed: paid " + p.amount() + " but total is " + total);
        case OrderEvent.Ship s when state == OrderState.PAID -> {
            log.add("  action: print label " + s.trackingId());
            yield OrderState.SHIPPED;
        }
        case OrderEvent.Deliver d when state == OrderState.SHIPPED -> OrderState.DELIVERED;
        case OrderEvent.Cancel c when state == OrderState.CREATED -> OrderState.CANCELLED;
        case OrderEvent.Cancel c when state == OrderState.PAID -> {
            log.add("  action: refund " + total);
            yield OrderState.CANCELLED;
        }
        default -> reject(event, "no transition from " + state);
    };
    state = to;
    log.add(from + " --" + event.getClass().getSimpleName() + "--> " + to);
}
private OrderState reject(OrderEvent e, String why) {
    throw new IllegalStateException(id + ": " + e.getClass().getSimpleName() + " rejected, " + why);
}

// File: code/lld/ch03/statemachine/Main.java (abridged; tryEvent prints "Rejected: " + message)
// ORD-1 (total 499.00): Pay 499.00, Ship TRK-77, Deliver; prints its log and state().isFinal()
// ORD-2 (total 120.00), each via tryEvent:
tryEvent(refund, new OrderEvent.Pay(new BigDecimal("100.00")));   // guard fails
tryEvent(refund, new OrderEvent.Pay(new BigDecimal("120.00")));
tryEvent(refund, new OrderEvent.Cancel("changed mind"));
tryEvent(refund, new OrderEvent.Ship("TRK-99"));                  // no transition from CANCELLED
// then prints ORD-2's log
```

Output:

```
ORD-1 path:
  [*] -> CREATED
  CREATED --Pay--> PAID
    action: print label TRK-77
  PAID --Ship--> SHIPPED
  SHIPPED --Deliver--> DELIVERED
  final? true
Rejected: ORD-2: Pay rejected, guard failed: paid 100.00 but total is 120.00
Rejected: ORD-2: Ship rejected, no transition from CANCELLED
ORD-2 path:
  [*] -> CREATED
  CREATED --Pay--> PAID
    action: refund 120.00
  PAID --Cancel--> CANCELLED
```

Amounts are `BigDecimal`, so `[amount == total]` is `compareTo(total) == 0` — `BigDecimal.equals` would treat `120.0` and `120.00` as different (it compares scale too). `OrderEvent` is a **sealed** interface (only the listed types may implement it), so the `switch` uses Java 21 **pattern matching with `when` guards**; the `default` branch rejects every transition not on the diagram.

Trade-offs

enum + switch (this example)	State pattern (class per state, Ch. 11)
All transitions visible in one place	Each state's behaviour in its own class
Best for few states, simple actions	Best when states have lots of different behaviour
Adding a state edits the switch	Adding a state adds a class (Open/Closed)

When not: objects with a flag or two and no real lifecycle (`active/inactive`); flows spanning many objects — use an activity diagram (a state machine describes **one** object).

Common mistakes

- States confused with actions: "Paying" (action) vs "PAID" (state).
- Missing transitions (forgetting a paid order can be cancelled with a refund) or allowing illegal ones by default.
- Overlapping guards, so two transitions could fire for one event.
- A state machine for the *system* rather than one object.

Real-world: `java.lang.Thread.State` (`NEW`, `RUNNABLE`, `BLOCKED`, `WAITING`, `TIMED_WAITING`, `TERMINATED`); **TCP** connection states (`LISTEN`, `SYN_SENT`, `ESTABLISHED`, `FIN_WAIT_1`, …), the classic protocol state machine; **Amazon / Flipkart order tracking** (ordered → packed → shipped → out for delivery → delivered; cancel only before shipping); **Uber trip** (requested → driver assigned → driver arrived → in progress → completed / cancelled); **Spring Statemachine** and AWS Step Functions (Amazon's cloud workflow service) are built around state machines.

In the interview

> **30-second answer:** "When an object has a lifecycle — order, ticket, elevator, vending machine — I draw its states with transitions labelled `event [guard] / action`. Anything not on the diagram is illegal and the code rejects it. In Java it is an enum of states with one transition method, or the State pattern if each state has a lot of behaviour."

- *Q: What is a guard?* — A: A boolean in `[ ]` that must be true for the transition to fire, e.g. `[amount == total]`.
- *Q: State machine vs activity diagram?* — A: One object's states over its lifetime vs the steps of a process, possibly across many objects.
- *Q: Thread-safe transitions?* — A: Make the transition method `synchronized` (or use a lock, or compare-and-set on an `AtomicReference<State>` — changes a value only if it still equals what you last read, in one atomic step), so two events cannot both read the old state and both move on.

Self-check

- *Q: Read `PAID --> CANCELLED : cancel / refund`.* → A: In PAID, event `cancel` moves to CANCELLED, performing `refund`.
- *Q: An event with no transition from the current state?* → A: Ignored or rejected — here, rejected with an exception.

TL;DR

One object, its states, `event [guard] / action` arrows. Initial dot, final bullseye. Everything not drawn is illegal. Code: enum + guarded switch, or the State pattern.

Part C — UML in the interview

3.15 How much UML to draw in an interview

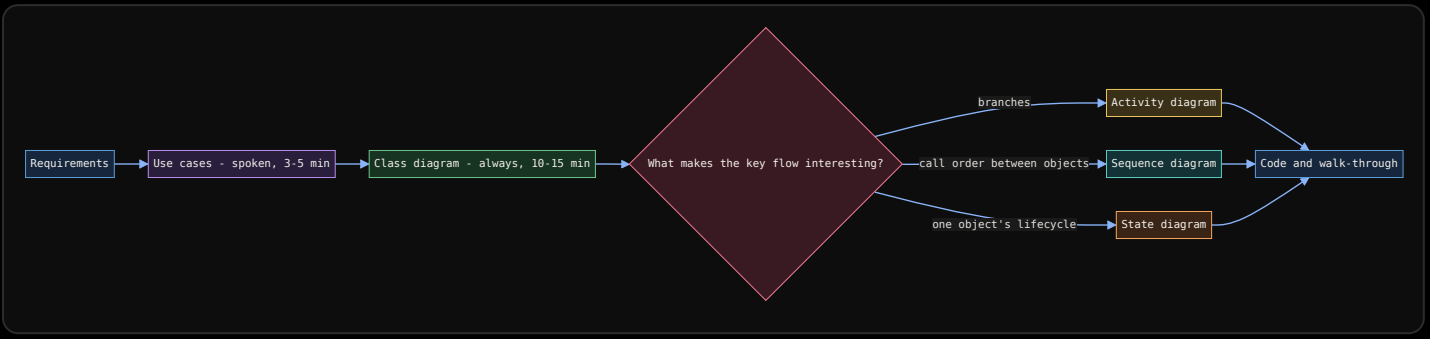
Arrowheads are not marked — **diagram and words agreeing** is (3.9).

Suggested budget for a 45-minute OOD round:

Minute	Diagram	Depth
0–5	**Use case**	Usually *spoken* or a quick list: actors + what each does + what is out of scope (the boundary). Draw only if asked.
5–20	**Class diagram**	*Always drawn.* 10–15 boxes, every line typed with multiplicity; key attributes and methods only.
20–25	**Activity** *or* **sequence** *or* **state**	One diagram for the flow carrying the design: activity if it branches, sequence if call order matters, state if one object has a lifecycle.
25–45	Code + discussion	Code the core classes; walk the flow against your diagrams.

Rules of thumb

1. **Only one picture? The class diagram** — it is the answer.
2. **Drop attributes before relationships**; **never leave a line unlabelled** (type + multiplicity); **say the test aloud** for every diamond and triangle.
3. **Use the other diagrams as tests:** every activity action and sequence arrow must be a method on a class in the class diagram.
4. **Small sequence diagrams:** 3–4 objects, 6–8 messages, no branches.
5. **Machine-coding rounds** (90–120 min, working code expected): a 2-minute class sketch, then code.
6. Forgot a symbol? **Say what you mean** ("this implements that") and move on.



In the interview

30-second answer: "I list use cases verbally to fix scope, spend most of my diagram time on one class diagram with typed, multiplicity-labelled lines, then draw one behaviour diagram for the key flow and use it to check that every step has a class and method."

3.16 Chapter self-check

1. *Q: The six relationships, weakest to strongest coupling?* → A: Dependency, association, aggregation, composition, realization, generalization.
2. *Q: Code smell (a surface sign of a likely design problem):* `CompactSpot`, `LargeSpot` *subclasses only passing a constant to* `super`*. Fix?* → A: One `ParkingSpot` with a `SpotType` enum field.
3. *Q: Java type for* `0..1` *and* `1..*`*?* → A: `Optional<T>` (nullable field); a non-empty collection checked in the constructor.

4. *Q: «include» vs «extend»?* → A: Include = always, base → included; extend = conditional, extension → base.
5. *Q: Sequence-diagram arrowheads?* → A: Solid + filled = sync call; solid + open = async; dashed + open = return.
6. *Q: `alt` vs `opt`?* → A: `alt` = if/else with several guarded sections; `opt` = one guarded section (if without else).
7. *Q: Fork/join?* → A: Fork splits a flow into parallel flows; join waits for all.
8. *Q: The 14 UML diagrams split how?* → A: 7 structural (class, object, package, component, composite structure, deployment, profile), 7 behavioural (use case, activity, state machine, sequence, communication, interaction overview, timing).

3.17 TL;DR

- **Relationships are claims about lifetime and substitution.** Dependency (uses, dashed arrow) · Association (knows, line/arrow) · Aggregation (has, parts live on, ◇ at whole) · Composition (owns, parts die, ◆ at whole) · Realization (implements, dashed △) · Generalization (extends, solid △).
- **Choose in order:** substitution test → dies with whole? → part that outlives? → else association; parameter-only → dependency. Inheritance last.
- **Multiplicity changes code:** `1` required, `0..1` Optional, `1..*` non-empty collection, exact numbers checked in constructors.
- **UML:** 14 diagrams, structural vs behavioural. Interviews: use case (scope), class (the answer), activity (branches), sequence (call order), state machine (lifecycle).
- **Draw little, label everything, say the test out loud,** and use behaviour diagrams to test the class diagram.

Chapter 4 — SOLID Principles

> **What you will learn.** The five SOLID principles, one at a time and then together: for each, a plain definition, an everyday analogy, a broken and a refactored Java example, the rules behind it, its cost, and how to talk about it in an interview — plus the ideas SOLID is built on: **cohesion**, **coupling** and **separation of concerns**.
>
> **How to use this chapter.** Read 4.1, then the principles in order (each sets up the next). 4.11 runs one class through all five — the best single thing to rehearse. 4.13–4.14 are about saying it out loud.

Run the code. Under `chapters/04-solid-principles/code/`, one package per topic; listings below show the essential lines (`javac` compiles, `java` runs, `-cp` says where the classes are):

```
javac --release 21 -d /tmp/lld_04 $(find . -name '*.java')
java -cp /tmp/lld_04 lld.ch04.srp.Main        # also: cohesion, coupling, soc, ocp, lsp, isp, dip, checkout
```

Package	What it shows
`lld.ch04.srp`	SRP: an `Employee` class split by actor
`lld.ch04.cohesion`	Low cohesion vs high cohesion
`lld.ch04.coupling`	Tight coupling vs loose coupling, in three steps
`lld.ch04.soc`	Separation of concerns: rules vs storage
`lld.ch04.ocp`	OCP: invoice formats behind an interface
`lld.ch04.lsp`	LSP: cars, the four contract rules, the history rule, birds and composition
`lld.ch04.isp`	ISP: fat printer interface, over-splitting, role interfaces
`lld.ch04.dip`	DIP: notifications, dependency injection, service locator, plug-ins
`lld.ch04.checkout`	One checkout class refactored through all five principles

A note on money. Every amount (pay, invoice totals, prices, balances, order totals) is a `java.math.BigDecimal` (Java's exact decimal type) built from a string (`new BigDecimal("19.99")`); any result that could gain decimals (multiplying by a rate) is rounded with `setScale(2, RoundingMode.HALF_EVEN)` — two places, "banker's rounding": an exact half such as 2.345 goes to the even neighbour, 2.34, so errors do not all lean one way. Adding/subtracting two-decimal amounts stays at two decimals. Never `double`: binary floating point cannot store 0.1 exactly (`0.1 + 0.2` prints `0.30000000000000004`) and the errors add up. One `BigDecimal` trap: `equals` compares scale (decimal places) too (`2.0` ≠ `2.00`), so compare with `compareTo` or `signum()`, as the code does.

4.1 What SOLID is and why you should learn it

What it is

SOLID is five design guidelines for object-oriented code:

Letter	Principle	The one-line rule
S	Single Responsibility Principle (SRP)	A class should have one reason to change.
O	Open/Closed Principle (OCP)	Add new behaviour by adding code, not by editing working code.
L	Liskov Substitution Principle (LSP)	A subtype must be usable anywhere its parent type is expected.
I	Interface Segregation Principle (ISP)	No client should be forced to depend on methods it does not use.
D	Dependency Inversion Principle (DIP)	Depend on abstractions, not on concrete details.

Terms used throughout:

- A **class** is a blueprint for objects; a **module** is a bigger unit (package, library, service).
- A **client** (**caller**) is any code using another piece of code.
- An **interface** is a list of methods a class promises, with no bodies (Java's `interface`); in design talk also "the methods a caller can see".
- An **abstraction** is an interface or abstract class (marked `abstract`: cannot be created directly, may leave methods to subclasses) — it says *what*, not *how*. A **concrete class** is one you can `new`.
- A **subtype** extends a class or implements an interface; what it extends/implements is its **parent** (base) type.

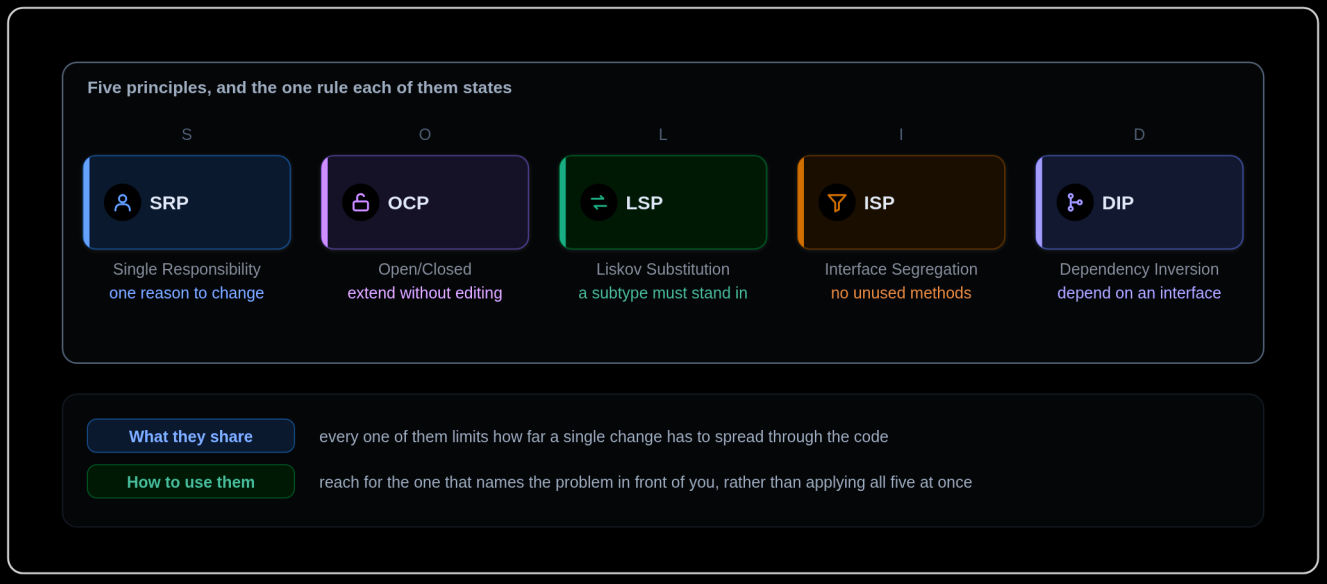


Figure — the five principles and the single rule each one states (course diagram)

The common thread: every principle **limits how far one change spreads**. SRP gives each class one owner; OCP lets you add instead of edit; LSP keeps a new subclass from breaking old callers; ISP stops a change in one method reaching callers that never use it; DIP stops a change in a low-level detail (a technical part such as a database driver) reaching business logic (the rules the application exists for, such as pricing).

History. Robert C. Martin ("Uncle Bob") collected the principles around 2000, most fully in the paper *Design Principles and Design Patterns* and the book *Agile Software Development: Principles, Patterns, and Practices* (2002); Michael Feathers noticed the initials spelled SOLID. Two ideas are older: Open/Closed from Bertrand Meyer (1988), Substitution from Barbara Liskov (1987).

The problem it solves

Working code must still change; without guidelines classes grow until one edit breaks three unrelated features. SOLID gives **cleaner code** (small focused classes), **flexibility** (features plug in without breaking old ones), **fewer bugs** (less touched per change), **reuse** (one-job classes fit unforeseen places), **easier change** (a shift lands in one place) and — the forgotten benefit — **a shared vocabulary**: "this class breaks ISP" is understood by all, so design disagreements are settled by argument, not taste.

How to use SOLID (the honest version)

1. Treat each principle as a **name for a problem**, not a rule to obey everywhere.
2. When code is painful to change, find the principle that names the pain.
3. Apply the fix **when a real change asks for it**, not in advance.
4. Always say the cost (more classes, interfaces, wiring — code that creates objects and connects them).

Applied without judgement, SOLID yields a maze of single-implementation interfaces; this chapter states each recommendation's cost with its benefit.

TL;DR

SOLID = five guidelines that keep one change from spreading. Use them to **diagnose**, apply them when a real change needs them, always state the cost.

4.2 Single Responsibility Principle (SRP)

What it is

Martin's first wording: **a class should have only one reason to change.** In *Clean Architecture* (2017) he sharpened it: **a module should be responsible to one, and only one, actor** — an **actor** being a person or group who can ask for a change. The everyday "every component should have one job" is a useful start but hard to measure.

> **Analogy: army knife vs kitchen knife.** A Swiss army knife (scissors, bottle opener, screwdriver) does many jobs, none best; a kitchen knife does one, cutting, excellently.

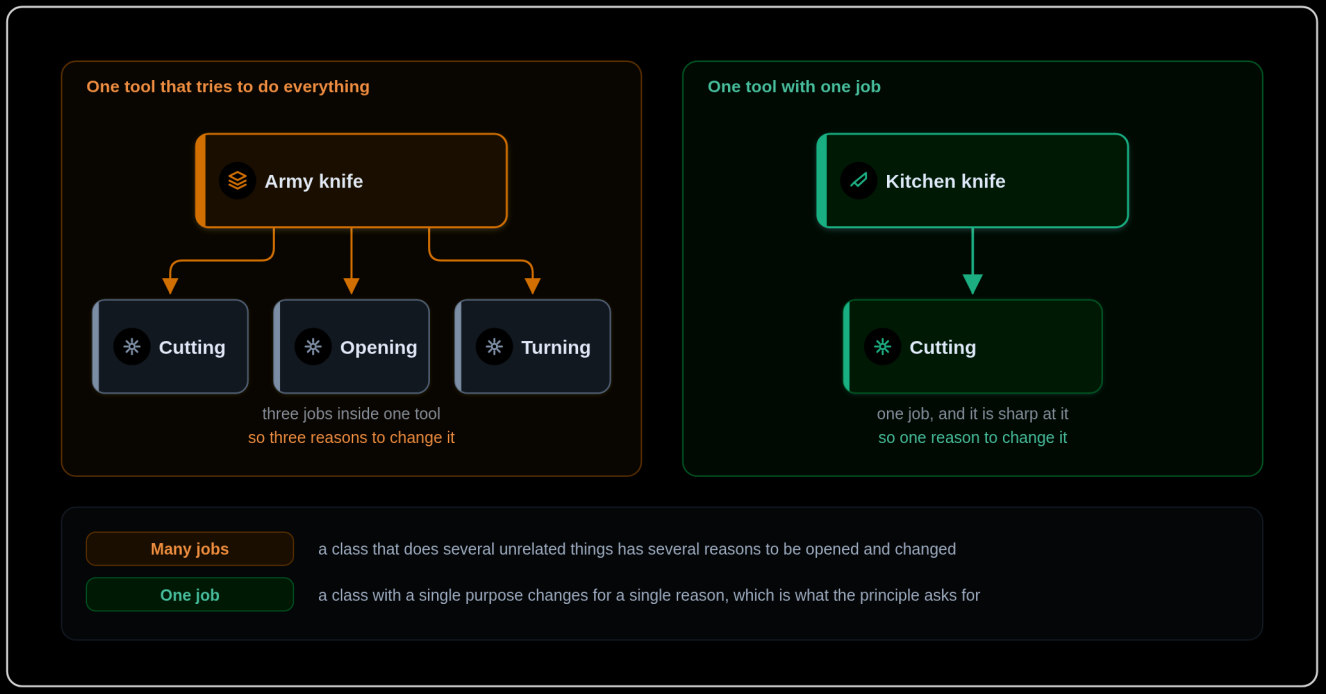


Figure — an army knife with three jobs (three reasons to change) next to a knife with one job (course diagram)

The army knife is the SRP-breaking class: three jobs, three reasons to change. "One job" is applied at every size — a **method** performs one action, a **class** serves one purpose, a **module** covers one area — but be precise: Martin's SRP is about **classes and modules** and their reasons to change; "a function should do one thing" is a separate (good) rule from *Clean Code*. Both push the same way; only the first is SRP.

The problem it solves

When one class serves several groups, every edit for one ships inside code the others rely on — a payroll tweak can break an operations report — and unrelated logic is mixed, so finding the right place is slow. One responsibility per class means you know which class to open.

What counts as one responsibility?

"Do one thing" has no size: is checking the email part of "register a user", or its own job? The **actor** view gives a yardstick: a reason to change comes from someone, so ask **who can send me a change request for this code?** Two groups → two responsibilities, however small the class.

`Employee`: every method is "about employees", so the "one thing" test passes. Now ask who can demand a change:

Method	Who asks for the change	Example request
`calculatePay`	Finance	"Overtime starts after 38 hours, not 40."
`reportHours`	Operations	"Show the week-ending date in the report."
`save`	Database team	"Move from SQL (table database) to a document store (stores JSON-like documents)."

Three owners, three independent reasons. A finance overtime change re-ships the report and storage code; if it breaks the report layout, operations pays for a decision it had no part in.

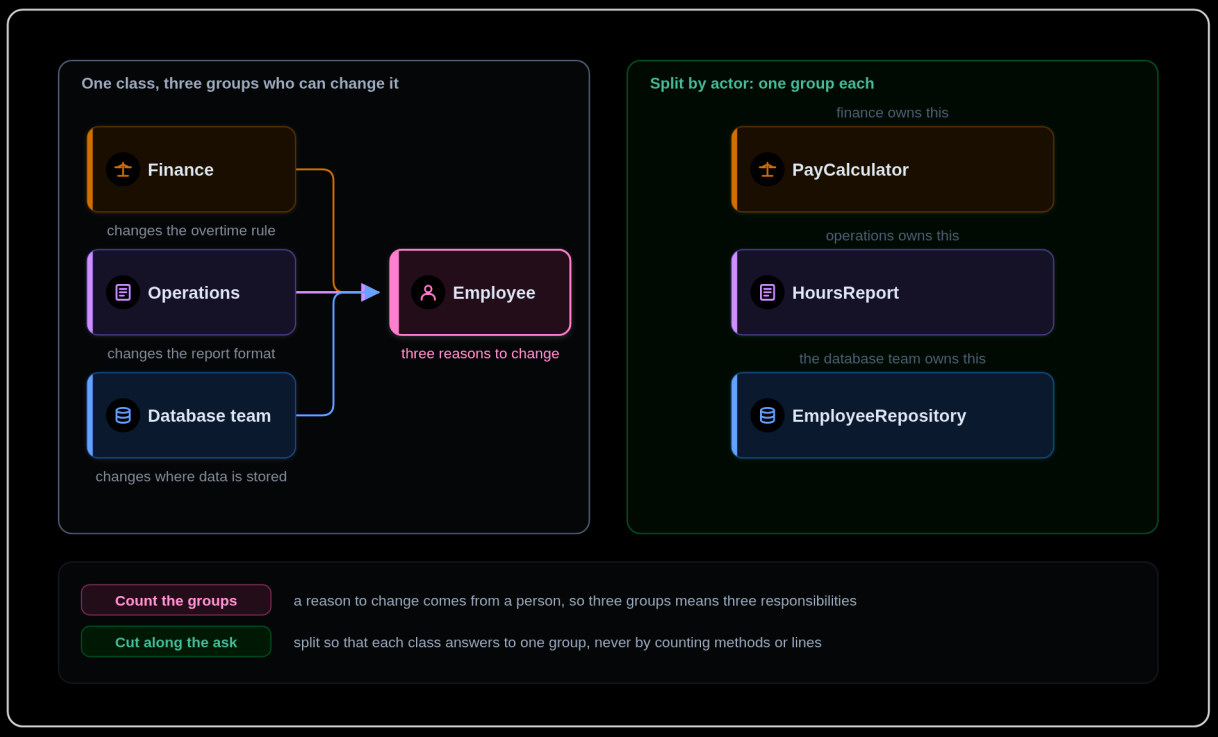


Figure — finance, operations and the database team all reach into one Employee class; splitting by actor gives each group its own class (course diagram)

Martin's sharper version (*Clean Architecture*): `calculatePay` and `reportHours` share a private helper `regularHours()` because the calculations match today. Finance changes it; the operations report silently shows wrong numbers. It looked like good reuse, but the two uses belonged to different actors — **accidental duplication**: code that looks the same but changes for different reasons should not be merged.

How it works

1. List the methods. 2. Name **who** would ask each to change. 3. Group methods by actor. 4. Move each group into its own class named after its job. 5. Keep plain data in a simple data class (a Java `record`: a compact class holding a fixed set of fields). 6. Verify with the three checks below.

Java example: splitting `Employee` by actor

```java
// File: code/lld/ch04/srp/before/Employee.java — VIOLATION: three actors share one class
public class Employee {
    private final String name;
    private final int hoursWorked;
    private final BigDecimal hourlyRate; // money: BigDecimal, never double

    // Actor 1: Finance -- "overtime should start after 38 hours, not 40"
    public BigDecimal calculatePay() {
        int overtime = Math.max(0, hoursWorked - 40);
        int normal = hoursWorked - overtime;
        BigDecimal normalPay = hourlyRate.multiply(BigDecimal.valueOf(normal));
        BigDecimal overtimePay = hourlyRate.multiply(new BigDecimal("1.5"))
                .multiply(BigDecimal.valueOf(overtime));
        return normalPay.add(overtimePay).setScale(2, RoundingMode.HALF_EVEN);
    }
    // Actor 2: Operations -- "show the week-ending date in the report"
    public String reportHours() { return name + " worked " + hoursWorked + " hours"; }
    // Actor 3: Database team -- "move from SQL to a document store"
    public void save() { System.out.println("INSERT INTO employees VALUES ('" + name + "')"); }
}

// File: code/lld/ch04/srp/after/ — data in a record, one class per actor
public record Employee(String name, int hoursWorked, BigDecimal hourlyRate) { }

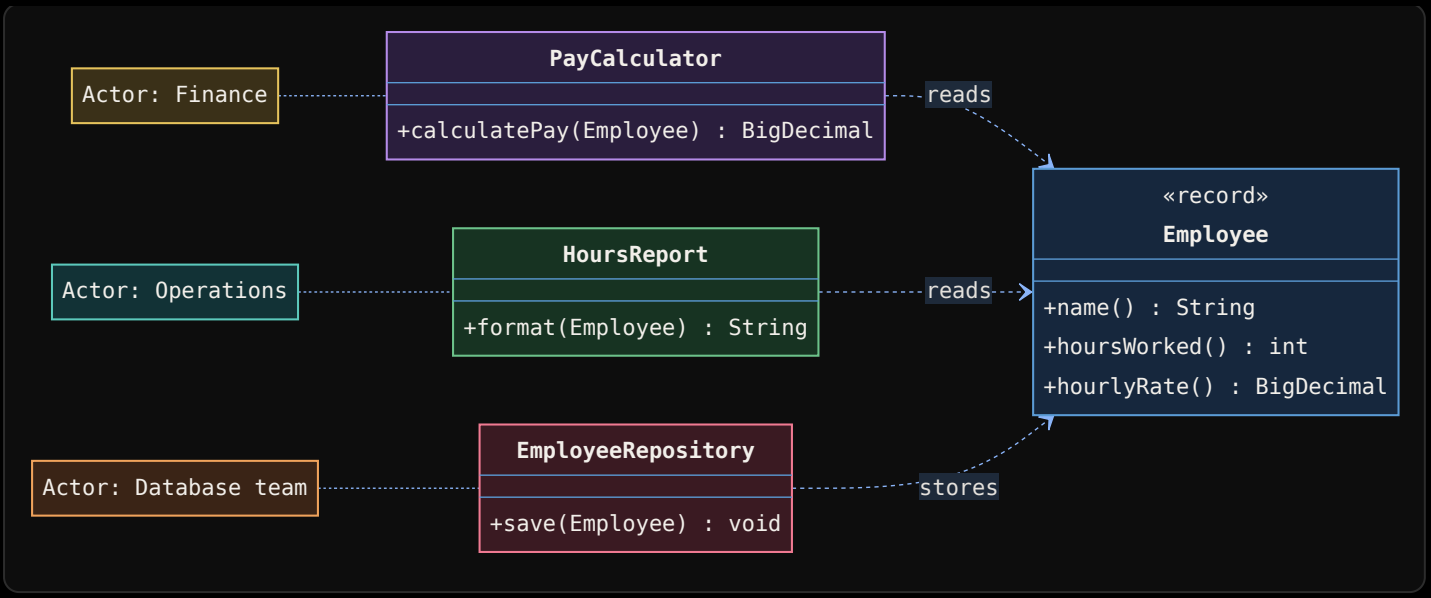
public class PayCalculator {                                // only Finance
    private static final int OVERTIME_THRESHOLD = 40;
    private static final BigDecimal OVERTIME_MULTIPLIER = new BigDecimal("1.5");
    public BigDecimal calculatePay(Employee e) { /* same formula, using the named constants */ }
}
public class HoursReport {                                  // only Operations
    public String format(Employee e) { return e.name() + " worked " + e.hoursWorked() + " hours"; }
}
public class EmployeeRepository {                           // only the database team
    public void save(Employee e) { System.out.println("INSERT INTO employees VALUES ('" + e.name() + "')"); }
}

// File: code/lld/ch04/srp/Main.java — runs both versions for Ada, 45 hours at 20.00, printing pay, report, save
```

Output:

```
--- Before: one class, three actors ---
Pay: 950.00
Ada worked 45 hours
INSERT INTO employees VALUES ('Ada')
--- After: split by actor ---
Pay: 950.00
Ada worked 45 hours
INSERT INTO employees VALUES ('Ada')
```

- `Employee` is a `record` — an immutable (unchangeable after creation) data carrier; no rule lives there, so no actor edits it for a rule change.
- `PayCalculator` belongs to finance; threshold and multiplier are named constants, so the next finance request is a one-line change in one class. `HoursReport` → operations; `EmployeeRepository` → database team.
- Output is **identical** before and after: behaviour did not change, only who must be consulted before an edit.



Where splitting goes wrong

Badly applied SRP is **worse** than none: a big class chopped by verb (one class per method or step) without asking who owns what gives tiny classes that must always be edited together, spreading each change over half a dozen files. Three checks:

1. **Do the pieces change together?** Read the version-control log (history kept by a tool such as Git; each **commit** is one saved change): classes in nearly every commit together were one responsibility cut in two.
2. **Does each new class's name stand on its own?** `EmployeePayCalculator` describes a job; `EmployeeHelper` / `EmployeeUtils` describe nothing.
3. **Could it be used elsewhere?** A pay calculator could serve a payroll export, a contractor quote, or a unit test (a small automated test of one class alone); no imaginable second caller → question the split.

Splitting does not automatically reduce dependencies: the two classes still call each other, and if the extracted one was shaped around its origin's exact needs, changes drag both along. The split is step one; step two is what each may know about the other — **cohesion** (4.3) and **coupling** (4.4). SRP usually helps both but promises neither.

Why SRP is important

Simpler maintenance (one job: easy to understand, debug, change); reuse; scales with the codebase (features add classes instead of bloating old ones); **high cohesion**; and it **opens the door to low coupling without finishing the job** — separation gives a seam (a boundary where one side can be swapped without editing the other), but coupling drops only when the dependency across it points at an abstraction (4.4).

Trade-offs

You gain	You pay
A change requested by one group touches only that group's code	More classes and files to navigate
Smaller classes are easier to test in isolation	The caller now wires several objects together
Clear ownership (who reviews what)	Risk of over-splitting into classes that always change together

When not: when you cannot name two different actors (then you do not know where the seam is); short-lived code like a one-off script; pieces that always change together anyway.

Common mistakes

- **Splitting by counting methods or verbs** instead of by actor; **Helper/Utils classes** that exist only because something had to move somewhere.
- **Merging "duplicate" code belonging to different actors** (the `regularHours()` trap).
- **"SRP means one method"** — ten methods can be one responsibility; **confusing SRP with "a function should do one thing"** (*Clean Code*; SRP is about reasons to change).

Code smells that point to SRP (from *Refactoring*)

A **code smell** is a surface sign of a probable design problem; *Refactoring* is Martin Fowler's book naming common smells and step-by-step fixes (refactorings).

- **Divergent Change** — one class edited for many kinds of reasons: SRP broken.
- **Large Class** — too many fields and methods, often several jobs.
- **Shotgun Surgery** — one change forces small edits in many classes: often **over-splitting**, the opposite mistake.
- Usual fix: **Extract Class** (and **Move Function**/**Move Field**).

Real-world examples

- **Spring layers** (Spring: a popular Java framework): `@Controller` handles HTTP (the web's request/response protocol, see System Design Fundamentals), `@Service` business rules, `@Repository` persistence (saving data permanently, usually in a database) — each answering a different kind of change.
- **JDK:** `java.time.format.DateTimeFormatter` formats/parses; `LocalDate` holds the date — formatting changes never touch the value type.
- **Logging frameworks** (Logback, Log4j 2) split *logger* (what), *appender* (where) and *layout* (how it looks): a new format does not touch destination code.

In the interview

> **30-second answer:** "SRP says a class should have one reason to change. The sharp version is: one actor, one group of people who can request changes. I find the actors by asking who would ask for each method to change. If finance, operations and the DBAs (database administrators) can all ask for edits to `Employee`, I split it into `PayCalculator`, `HoursReport` and `EmployeeRepository`, and keep `Employee` as data. The cost is more classes and some wiring, so I only split when I can name the different groups."

- *"Would you always split?"* — No. If the halves always change together, I've made two files for nothing; I split when two different people can ask for two different changes.
- *"How do you know you split too far?"* — The pieces share commits, their names only make sense together (`XHelper`), and nothing else could use them.
- *"Is SRP about methods?"* — It applies to methods, classes and modules, but the test is reasons to change, not method count.

Self-check

1. **Q:** An "actor" in SRP? **A:** A person or group who can request a change, e.g. finance or the database team.
2. **Q:** A `ReportService` formats a report and emails it — SRP problem? **A:** Yes, if different people own wording and delivery: they change for different reasons.
3. **Q:** The three checks for a good split? **A:** Change together? Names stand alone? Usable elsewhere?
4. **Q:** Does splitting always lower coupling? **A:** No. If the new class names a concrete collaborator the dependency just moved; point it at an abstraction.

TL;DR

One class, one actor. Find the actors, split along them, keep data in a record, check the split, and do not split what always changes together.

4.3 Cohesion

What it is

Cohesion is how closely the contents of one class (module, method) belong together. **High**: every method works toward one purpose. **Low**: unrelated jobs bundled together. Analogy: a **garbage dump** (plastic, paper, food in one pile) vs **sorted bins**, one job each.

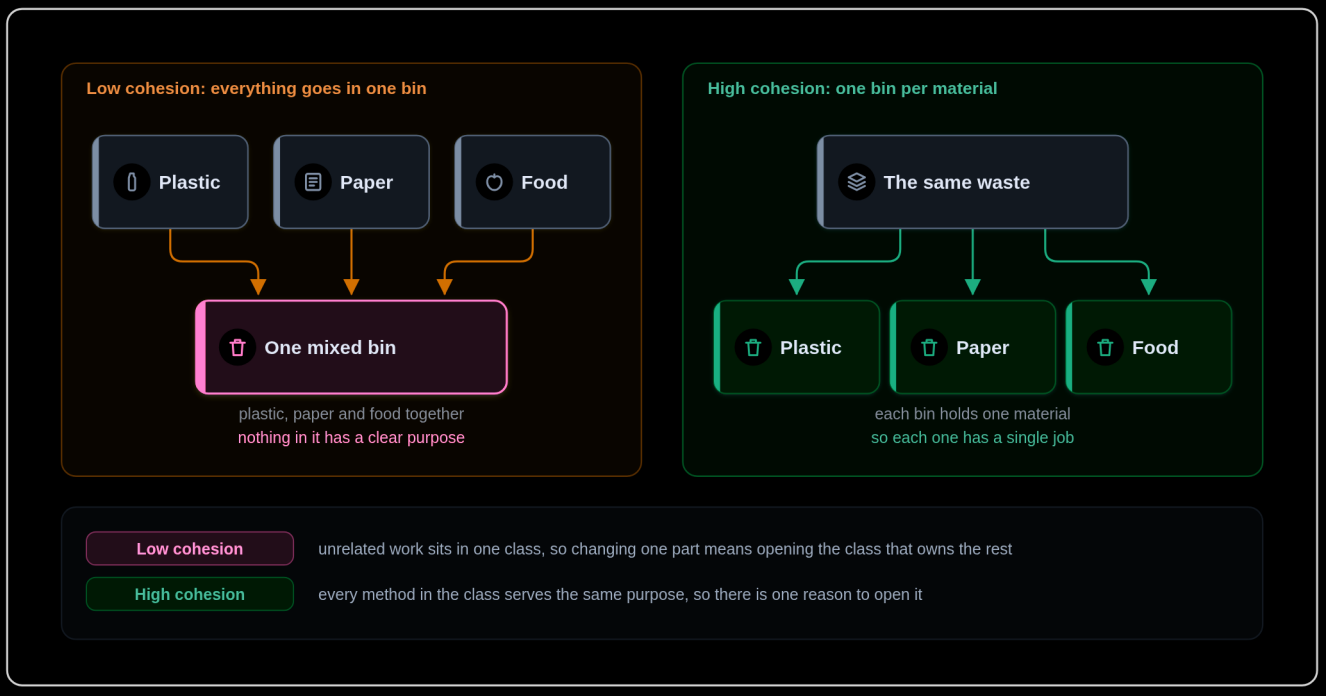


Figure — one mixed bin (low cohesion) next to one bin per material (high cohesion) (course diagram)

The problem it solves

In a low-cohesion class, changing one job means opening, reading and redeploying (releasing to production again) a class owning others. It also works in reverse: code for **one** job **scattered** over classes (`addUser` here, `deleteUser` there) must be hunted down for each change. Both are cohesion problems — mixing unrelated things, or separating related ones.

The classic types of cohesion

Structured-design authors (Larry Constantine, Ed Yourdon, Glenford Myers, 1970s) ranked cohesion worst to best:

Type (worst → best)	What groups the code	Java-flavoured example
Coincidental	Nothing, it just ended up together	`Utils` with `formatDate`, `sendEmail`, `parseCsv`
Logical	Same category, chosen by a flag	`handle(type)` with a big `switch` over unrelated actions
Temporal	Things that run at the same time	`startup()` that opens the DB, warms caches (fills fast temporary copies of data) and prints a banner
Procedural	Steps that follow an order	A method that validates, then logs, then retries
Communicational	Work on the same data	`ReportBuilder` methods that all read the same `Order` list
Sequential	Output of one step is input to the next	`parse → validate → transform` pipeline in one class
Functional (best)	Everything contributes to one well-defined task	`PayCalculator`

No need to memorise: aim for **functional**, recognise **coincidental** (the "Utils" class) as a warning.

How it works

Check which fields each method uses; if methods fall into groups using **different** fields for **different** purposes, cohesion is low — move each group to its own class. If one job is scattered over classes, gather it into one.

Java example

```java
// File: code/lld/ch04/cohesion/before/UserManager.java — LOW cohesion: two disjoint groups
public class UserManager {
    private final List<String> users = new ArrayList<>();       // used by addUser/deleteUser
    private final List<String> auditLines = new ArrayList<>();  // used by writeToFile/readFromFile
    public void addUser(String user)    { users.add(user);    System.out.println("user added: " + user); }
    public void deleteUser(String user) { users.remove(user); System.out.println("user deleted: " + user); }
    // --- file work, unrelated to the two methods above ---
    public void writeToFile(String line) {
        auditLines.add(line);
        System.out.println("wrote a line to the audit file: " + line);
    }
    public List<String> readFromFile() {
        System.out.println("read " + auditLines.size() + " line(s) back from the audit file");
        return List.copyOf(auditLines);
    }
}

// File: code/lld/ch04/cohesion/after/ — HIGH cohesion: the same methods split into
//   UserManager (users; addUser, deleteUser) and AuditFile (lines; writeLine, readAll), same messages.
// Main runs: add "ada", write "created ada", delete "ada", read back — once per version.
```

Output:

```
--- Before: one class, two unrelated jobs ---
user added: ada
wrote a line to the audit file: created ada
user deleted: ada
read 1 line(s) back from the audit file
--- After: two classes, one job each ---
user added: ada
wrote a line to the audit file: created ada
user deleted: ada
read 1 line(s) back from the audit file
```

Identical output; the difference is structural — a new audit-log format edits `AuditFile` alone. **Cost:** the caller creates and holds two objects, worth it once the jobs change for different reasons.

Cohesion and SRP: low cohesion usually means several unrelated jobs (breaks SRP); high cohesion means one job (what SRP asks), so aiming for it tends to give SRP for free.

Trade-offs

High cohesion gains	Costs
Easy to name, read and test each class	More classes to wire
A change lands in one class	Callers may hold several objects

When not: do not split a functionally cohesive class just to make it smaller; five tiny classes can be harder to follow than one clear one.

Common mistakes

- Calling every mixed class a "coupling problem": one class mixing jobs is a **cohesion** problem; coupling needs two classes.
- A `Utils` dumping ground (coincidental cohesion).

Real-world: `java.lang.Math` — static math functions, functional cohesion around "numeric calculations"; `java.util.Collections` is looser (sorting, searching, wrapping, synchronising) — still "operations on collections", but showing how a utility class drifts toward logical cohesion.

In the interview

> **30-second answer:** "Cohesion is how much the things inside one class belong together. I check which fields each method uses. If the methods split into two groups that touch different fields for different reasons, the class has low cohesion and I split it. High cohesion is basically SRP seen from inside the class."

- *"Is a big class always low-cohesion?"* — No; if every method serves one task it is cohesive even if large. Size is a hint, not the measure.

Self-check

1. **Q:** Worst kind of cohesion? **A:** Coincidental: grouped for no reason, like `Utils`.
2. **Q:** "The code for one job is spread over three classes" — cohesion problem? **A:** Yes; related code should live together.

TL;DR

Cohesion = do the parts of this class belong together? One purpose per class; split disjoint groups; gather scattered pieces.

4.4 Coupling

What it is

Coupling is how much one class depends on another. It describes a pair: a lone class cannot be "tightly coupled" (nothing else to depend on).

- **Low (loose)**: change, replace or test one class without changing the other.
- **High (tight)**: a change to one forces a change to the other.

> **Analogy: TV and remote.** A remote for one specific TV, and a TV accepting only that remote, are tightly coupled — replace one, replace both. A universal remote and a TV accepting any remote follow a shared standard (the infrared signal) and swap independently.

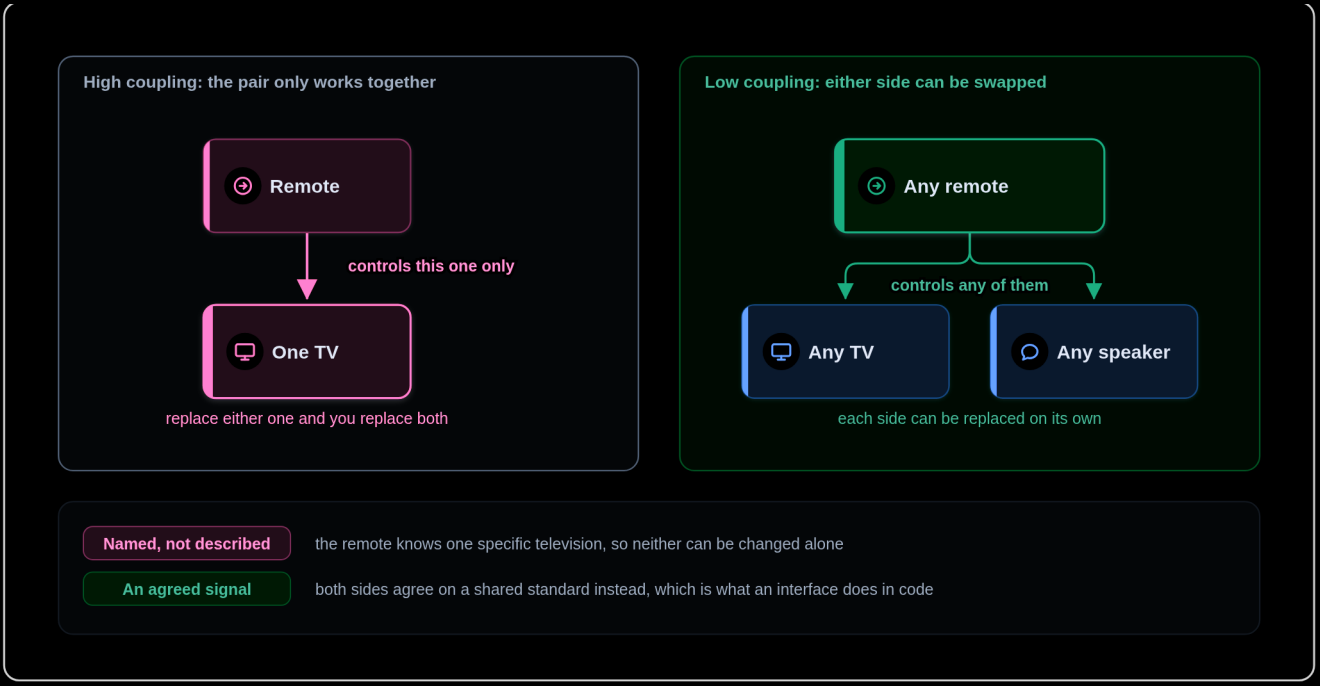


Figure — a remote tied to one TV (high coupling) vs any remote working with any device through an agreed signal (low coupling) (course diagram)

The "agreed signal" is exactly what an **interface** does: both sides agree on a contract instead of knowing each other.

The problem it solves

With tight coupling you cannot swap an implementation (MySQL → Postgres, two popular SQL databases) without editing the caller; cannot unit-test the caller without the real dependency (a running database); and a change in how the dependency is **constructed** ripples into every class that builds it.

The classic types of coupling

From structured design, worst to best:

Type (worst → best)	What A knows about B	Java-flavoured example
Content	A reaches into B's internals	Reflection (Java's ability to inspect and change classes while running) that sets B's private field
Common	A and B share global mutable data	Both read/write a `public static Map`
External	Both depend on an external format or device	Both parse the same hand-rolled file format
Control	A passes a flag telling B *how* to work	`print(doc, boolean asPdf)`
Stamp	A passes a whole structure when B needs one field	`sendInvoice(Customer c)` that only reads `c.email()`
Data	A passes only the values B needs	`sendInvoice(String email, Money total)`
Message (lowest)	A sends a message through an interface	`notifier.send(message)` on an interface

Practical OO goal: **depend on interfaces, pass only what is needed, never share mutable globals.**

How it works

1. Find where A **names** B (a field type, `new B()`, a static call `B.x()`).

2. Ask: does A need *this particular* B, or *something that does the job*?
3. If the latter, describe the job as an interface; let A accept any implementation, usually through its constructor.
4. Let the caller (or the entry point, the `main` method where the program starts) choose the implementation.

Java example: three steps with `Student`

```java
// File: code/lld/ch04/coupling/start/Student.java — ONE class, two jobs: a COHESION problem (nothing to couple to yet)
public void save() {                                  // Student holds the data AND knows how to store it
    System.out.println("Connecting to MySQL...");
    System.out.println("Saving student with ID: " + studentId);
}

// File: code/lld/ch04/coupling/extracted/Student.java — Step 1: cohesion fixed, but TIGHT coupling
private final MySqlDatabase database = new MySqlDatabase(); // hard-wired; MySqlDatabase.saveStudent prints the same lines
public void save() { database.saveStudent(studentId); }

// File: code/lld/ch04/coupling/decoupled/ — Step 2: depend on an interface -> LOOSE coupling
public interface StudentRepository { void save(String studentId); }   // what Student needs, as a contract
// MySqlStudentRepository.save prints "Saving student <id> to MySQL";
// InMemoryStudentRepository keeps ids in a list, prints "Keeping student <id> in memory for a test", saved() returns them
public class Student {
    private final String studentId;
    private final StudentRepository repository;
    public Student(String studentId, StudentRepository repository) {
        this.studentId = studentId;
        this.repository = Objects.requireNonNull(repository);
    }
    public void save() { repository.save(studentId); }
}

// File: code/lld/ch04/coupling/Main.java (step 2 part)
new Student("S-1024", new MySqlStudentRepository()).save();
var fake = new InMemoryStudentRepository();
new Student("S-2048", fake).save();
System.out.println("test double recorded: " + fake.saved());
```

Output:

```
--- Start: one class holds data and SQL ---
Connecting to MySQL...
Saving student with ID: S-1024
--- Step 1: extracted, but tightly coupled to MySqlDatabase ---
Connecting to MySQL...
Saving student with ID: S-1024
--- Step 2: depends on the StudentRepository interface ---
Saving student S-1024 to MySQL
Keeping student S-2048 in memory for a test
test double recorded: [S-2048]
```

After step 1, three costs: storing anywhere else edits `Student`; every test needs a live MySQL server; if `MySqlDatabase` gains constructor arguments, `Student` changes too. The extraction moved the code but `Student` **depends** on it as much as before.

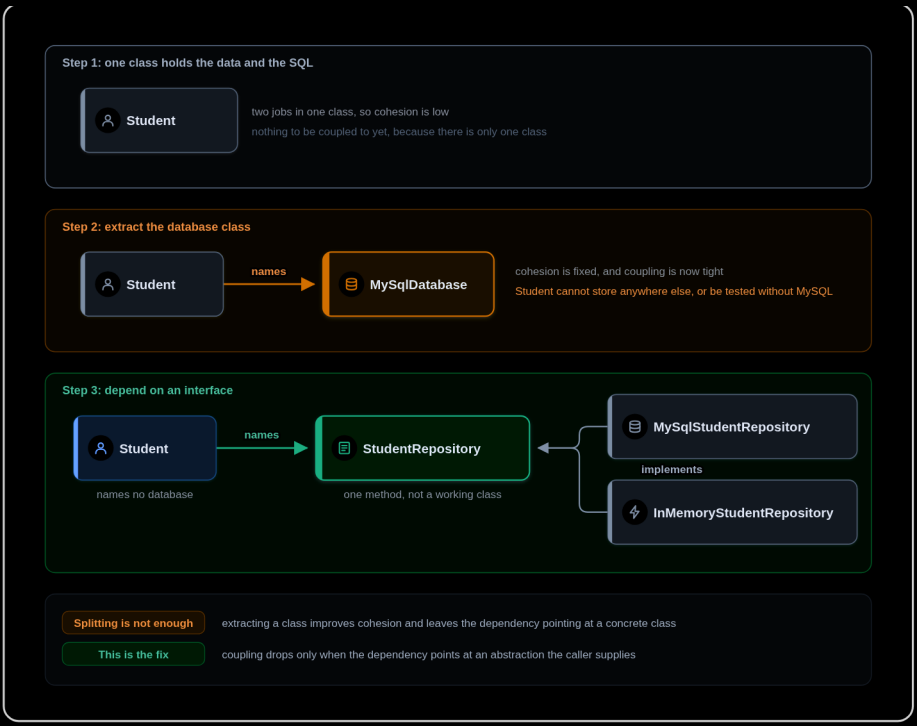


Figure — the three steps: one class with two jobs, an extracted but tightly coupled class, and a class that depends on an interface (course diagram)

In the last panel `Student` names only `StudentRepository`, a one-method contract, and the concrete repositories point *toward* it: Postgres is a new repository class handed in by the caller, and a unit test hands in the in-memory one. This is **depend on abstractions, not concrete classes** — the D in SOLID (4.10), the standard cure for tight coupling.

Coupling and SRP: separating the jobs (SRP) fixed cohesion but left `Student` welded to `MySqlDatabase`; aiming the dependency at an interface fixed coupling, a step SRP alone never requires. So SRP *usually* reduces coupling (a single-job class needs fewer collaborators) without *guaranteeing* it — inspect dependencies after each split.

Trade-offs

Loose coupling gains	Costs
Swap implementations without editing callers	An extra interface and a constructor parameter
Unit tests with in-memory fakes (stand-ins keeping data in a list instead of a database)	Someone must do the wiring (the entry point or a framework)
Changes stop rippling	"Go to definition" lands on an interface, one more hop to the real code

When not: no interface in front of a **stable** class that never varies or needs faking (`String`, `LocalDate`, a pure calculation) — coupling to stable things is harmless; one implementation and no test double (fake stand-in used in tests) is usually just an extra file.

Common mistakes

- "I extracted a class, so coupling is lower" — only depending on an abstraction lowers it.
- `new ConcreteThing()` hidden in a constructor — still hard-wired.
- Passing whole objects when one field is needed (stamp coupling).

Real-world: **JDBC** (Java Database Connectivity): code uses the interfaces `java.sql.Connection`, `Statement`, `ResultSet`; the MySQL or Postgres driver (library speaking one database's protocol) implements them. **SLF4J** (a logging facade — one front for several libraries): code uses

`org.slf4j.Logger`; Logback or Log4j is plugged in at deployment (when installed and started). **USB, HTTP, SQL** are themselves "agreed signals".

In the interview

> **30-second answer:** "Coupling is how much one class depends on another. I check where a class names a concrete collaborator. If it only needs something that does the job, I describe the job as an interface and pass the implementation in. That lets me swap MySQL for Postgres and test with an in-memory fake. I do not add interfaces in front of stable things like `String`."

- *"Can SRP increase coupling?"* — Yes: extract a class built around the old class's exact needs and the two are tightly bound. Check dependencies after every split.

Self-check

1. **Q:** Can a single class be "tightly coupled"? **A:** Not to anything inside itself; coupling is between two things. One class with two jobs has a cohesion problem.
2. **Q:** `Student` does `new MySqlDatabase()` in its constructor — tight or loose? **A:** Tight: it names and creates a concrete class.
3. **Q:** "Control coupling"? **A:** Passing a flag telling the other code how to behave, e.g. `print(doc, asPdf)`.

TL;DR

Coupling = how much A must know about B. Loosen it by depending on an interface and passing the implementation in. Extracting a class alone is not enough.

4.5 Separation of Concerns (SoC)

What it is

Separation of Concerns: split a program into parts that each handle one **concern** — one area of the work (storing data; deciding whether an email is acceptable; drawing the screen). Term from Edsger Dijkstra (1974). Analogy: in a restaurant the kitchen cooks, waiters serve, the cashier takes payment; a new menu does not change payment.

The problem it solves

Mixed concerns force you to read and edit code about other concerns for each change; separated, each change lands in one module. Benefits: **modularity** (each part one area), **maintainability** (changes rarely affect others), **reusability** (well-defined modules reused), **testability** (test one concern with the others replaced by stand-ins).

How it works

1. List the feature's concerns (registration: storage, business rules). 2. Give each its own module. 3. Let modules talk only through small, clear methods. 4. Test it: for each likely requirement change, **count the modules you would edit** — ideally one.

Java example: user registration

Storage keeps users and never judges validity; **business rules** decide what is acceptable and ask storage to save, without knowing how it works.

```java
// File: code/lld/ch04/soc/UserStore.java — storage concern only
public record User(String username, String email) { }
private final List<User> users = new ArrayList<>();
public void addUser(String username, String email) {
    users.add(new User(username, email));
    System.out.println("Stored user: " + username);
}
// count() returns users.size()

// File: code/lld/ch04/soc/UserService.java — business-rules concern only
boolean isValidUsername(String username) { return username.length() >= 3; }
boolean isValidEmail(String email) { return email.contains("@") && email.contains("."); }
public boolean registerUser(String username, String email) {
    if (!isValidUsername(username)) { System.out.println("Rejected: " + username + " is too short"); return false;
}
    if (!isValidEmail(email)) { System.out.println("Rejected: " + email + " is not an email address"); return
false; }
    store.addUser(username, email);
    return true;
}

// File: code/lld/ch04/soc/Main.java
UserStore store = new UserStore();
UserService service = new UserService(store);
service.registerUser("ada", "ada@example.com");
service.registerUser("jo", "jo@example.com");
service.registerUser("grace", "grace-at-example");
System.out.println("Users in store: " + store.count());
```

Output:

```
Stored user: ada
Rejected: jo is too short
Rejected: grace-at-example is not an email address
Users in store: 1
```

Every accept/reject decision was `UserService`'s, which never touched the list; `UserStore` kept whatever it was handed.



Figure — UserService decides what is acceptable and asks UserStore to save it; each change lands in one module (course diagram)

Change	Module you open
Move from an in-memory list to a real database	`UserStore` only
Require usernames of at least eight characters	`UserService` only
Test the rules without a database	Pass `UserService` a stand-in store

(In a bigger system `UserStore` would also sit behind an interface — the 4.4 coupling fix.)

SoC and SRP: the same instinct at different scales — SRP for a class, SoC for modules, layers and subsystems; single-responsibility classes are one route to separated concerns.

Trade-offs

Gains	Costs
Changes stay local	More modules and boundaries to design
Each concern tested alone	Cross-cutting concerns (logging, security, transactions — groups of database changes that succeed or fail together) still touch many modules; they need special tools such as aspects (code woven automatically around many methods) or decorators (wrappers that add behaviour)

When not: a 50-line script needs no layers; separate concerns when they change for different reasons or at different speeds.

Common mistakes

- Validation quietly inside the storage class ("the DAO also checks the email"; a DAO, Data Access Object, reads and writes one kind of data in the database).
- Layers that exist on paper but call each other in every direction.

Real-world: **MVC** (Model–View–Controller: model = data and rules, view = display, controller = input), followed by Spring MVC and most web frameworks; **HTML / CSS / JavaScript** — structure, style, behaviour in three languages; **layered architecture** controller → service → repository; **Spring** `@Transactional` separates transaction handling (cross-cutting) from business code via a proxy (a stand-in wrapping the real object, adding behaviour before and after each call).

In the interview

> **30-second answer:** "Separation of concerns means each module handles one area of the work, like storage vs business rules. My test is: when a requirement changes, how many modules do I open? Ideally one. It is SRP at module size."

Self-check

1. **Q:** A "concern"? **A:** One area of the work: storage, validation, presentation.
2. **Q:** SoC vs SRP? **A:** Mostly size: SRP a class, SoC modules and layers.

TL;DR

One module per area of work; measure it by how many modules a change forces you to open.

4.6 SRP vs cohesion vs coupling vs SoC — one table

Idea	Question it asks	Scope	How it relates to SRP
SRP	How many actors can ask this class to change?	Class / module	—
Cohesion	Do the parts of this class belong together?	Inside one unit	Applying SRP usually raises cohesion, because one job means related parts.
Coupling	How much does A depend on B?	Between two units	SRP tends to lower coupling but does not guarantee it; you still need to depend on abstractions.
SoC	Is each area of work in its own module?	Modules, layers	SRP applied to every class is one way to achieve SoC.

The textbook goal, **"high cohesion, low coupling"**: SRP gets the first half; depending on abstractions (DIP) the second.

4.7 Open/Closed Principle (OCP)

What it is

Software entities (classes, modules, functions) should be open for extension but closed for modification: add new behaviour **by adding code**, without editing working code.

- **Closed for modification**: once written and tested, a class need not be opened for the next variation — every edit risks breaking it.
- **Open for extension**: new behaviour arrives as new parts, via **interfaces**, **polymorphism** (one call, many implementations) or **inheritance**.

Origins. Bertrand Meyer, 1988 (*Object-Oriented Software Construction*): in **Meyer's OCP** a class is *closed* once compiled, published and used (changed only for bug fixes) and *open* because a new class can **inherit** from it and add or redefine features — extension by implementation inheritance. Robert C. Martin's 1996 article gave the **polymorphic OCP**: stable code depends on an **abstract interface**; new behaviour arrives as swappable implementations. The polymorphic version is what people usually mean today and what this chapter uses.

> **Analogy: a game console.** You add a racing wheel, motion controller, VR headset; nobody rewires the console. It is **closed** (internals sealed) and **open** (a standard port for accessories).

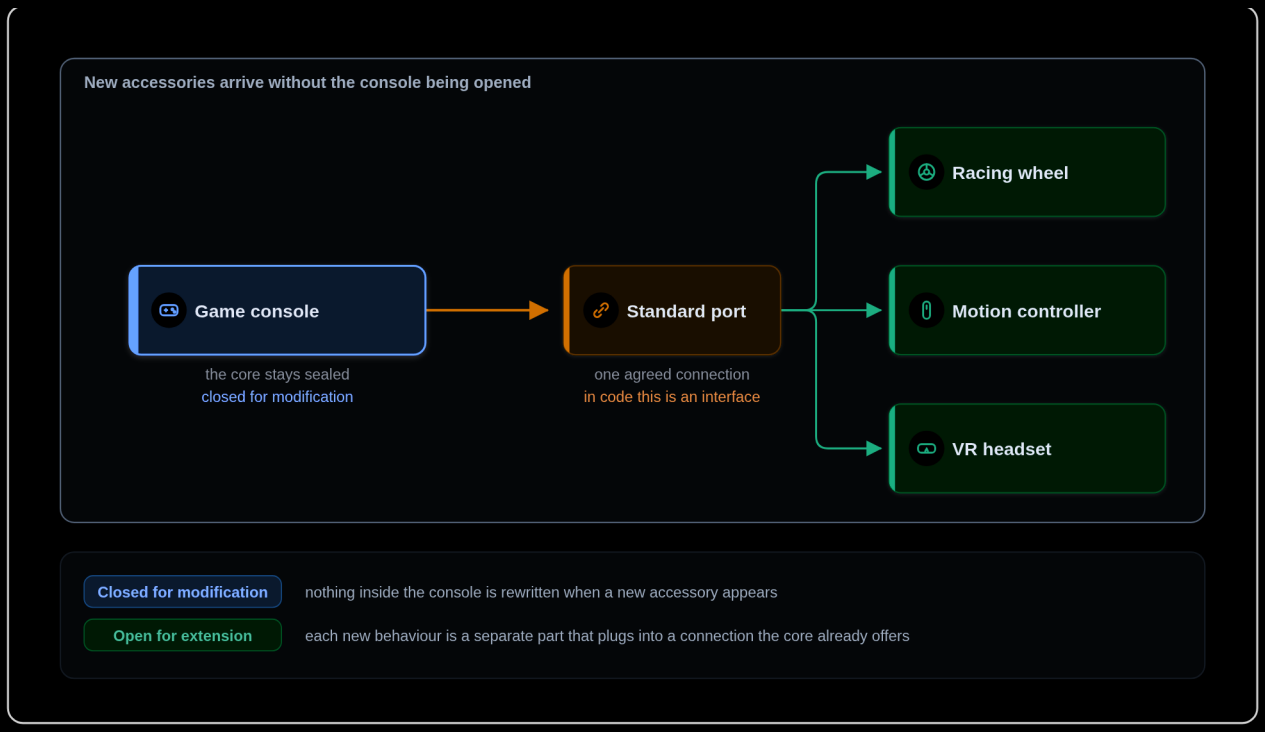


Figure — new accessories plug into a standard port; the console itself is never opened (course diagram)

The **standard port** is the interface; accessories are classes implementing it.

The problem it solves

The textbook symptom is a growing `if/else` or `switch` on a type code: each new case edits a tested method and can break the others, and the same `switch` often appears in several places, one copy missed. **Quick test:** *can I add the next case without opening this file?* If not, the class is not closed against that kind of change.

How it works

1. Wait until the variation has happened (at least a second case). 2. Identify **the part that varies** (the invoice format). 3. Describe it as an interface. 4. Move each variant into its own implementing class. 5. Make the stable class **delegate** (hand the work over) instead of branching. 6. Add the next variant as a new class; nothing existing opened. 7. Choose the variant in one place at the program's edge (entry point, a factory — a method or class whose job is creating objects — a map, or configuration).

Java example: invoice formats

Before. With one format there was nothing to fix (OCP matters only for code receiving variations). Then sales wants **international invoices**, and the quick route is a type parameter and a branch:

```
// File: code/lld/ch04/ocp/before/Invoice.java — VIOLATION: every new type opens this tested method
public void generate(String invoiceType) {
    if (invoiceType.equals("basic")) {
        System.out.println("Generating basic invoice for amount: " + amount);
    } else if (invoiceType.equals("international")) {
        System.out.println("Generating international invoice for amount: " + amount);
    } else {
        throw new IllegalArgumentException("Unknown invoice type: " + invoiceType);
    }
}
public void applyDiscount() { System.out.println("Applying discount on invoice: " + amount); }
```

Doing two jobs would be an SRP complaint; the OCP complaint is that **each new type edits code that already works**.

After. (1) The varying part becomes an interface; (2) each format gets a class — named *generators*, since they *produce* invoices (`BasicInvoice` would falsely suggest a subtype); (3) `Invoice` delegates; (4)

a third format costs one class plus one line where formats are chosen.

```java
// File: code/lld/ch04/ocp/after/InvoiceGenerator.java — Step 1
@FunctionalInterface
public interface InvoiceGenerator { void generate(BigDecimal amount); }

// Step 2: BasicInvoiceGenerator prints "Generating basic invoice for amount: " + amount;
// InternationalInvoiceGenerator prints the international line + "  including customs declaration and currency
conversion"
// File: code/lld/ch04/ocp/after/DetailedInvoiceGenerator.java — Step 4: the ONLY new code for a third format
public class DetailedInvoiceGenerator implements InvoiceGenerator {
    @Override public void generate(BigDecimal amount) {
        System.out.println("Generating detailed invoice for amount: " + amount);
        System.out.println("  line items, tax breakdown and payment terms");
    }
}

// File: code/lld/ch04/ocp/after/Invoice.java — Step 3: delegates; knows nothing of how many formats exist
public class Invoice {
    private final BigDecimal amount;
    private final InvoiceGenerator generator;              // both requireNonNull in the constructor
    public void generate() { generator.generate(amount); } // no branching
    // Still closed? No -- a new discount rule would still edit this class.
    // That is fine until discounts actually start to vary.
    public void applyDiscount() { System.out.println("Applying discount on invoice: " + amount); }
}

// File: code/lld/ch04/ocp/Main.java — amount 120.00; old.generate("basic"/"international") first, then:
List<InvoiceGenerator> generators = List.of(
        new BasicInvoiceGenerator(),
        new InternationalInvoiceGenerator(),
        new DetailedInvoiceGenerator(),
        // A lambda is also an extension: no new file, no edited file.
        a -> System.out.println("Generating pro-forma invoice for amount: " + a));
for (InvoiceGenerator g : generators) new Invoice(amount, g).generate();
```

Output:

```
--- Before: branch per type ---
Generating basic invoice for amount: 120.00
Generating international invoice for amount: 120.00
--- After: one class per format ---
Generating basic invoice for amount: 120.00
Generating international invoice for amount: 120.00
  including customs declaration and currency conversion
Generating detailed invoice for amount: 120.00
  line items, tax breakdown and payment terms
Generating pro-forma invoice for amount: 120.00
```

`Invoice` has **no branch**; format three touched no existing class; and since `InvoiceGenerator` is a `@FunctionalInterface` (exactly one abstract method), a **lambda** (a short inline unnamed function, `a -> ...`) is also an extension — OCP with zero new files.

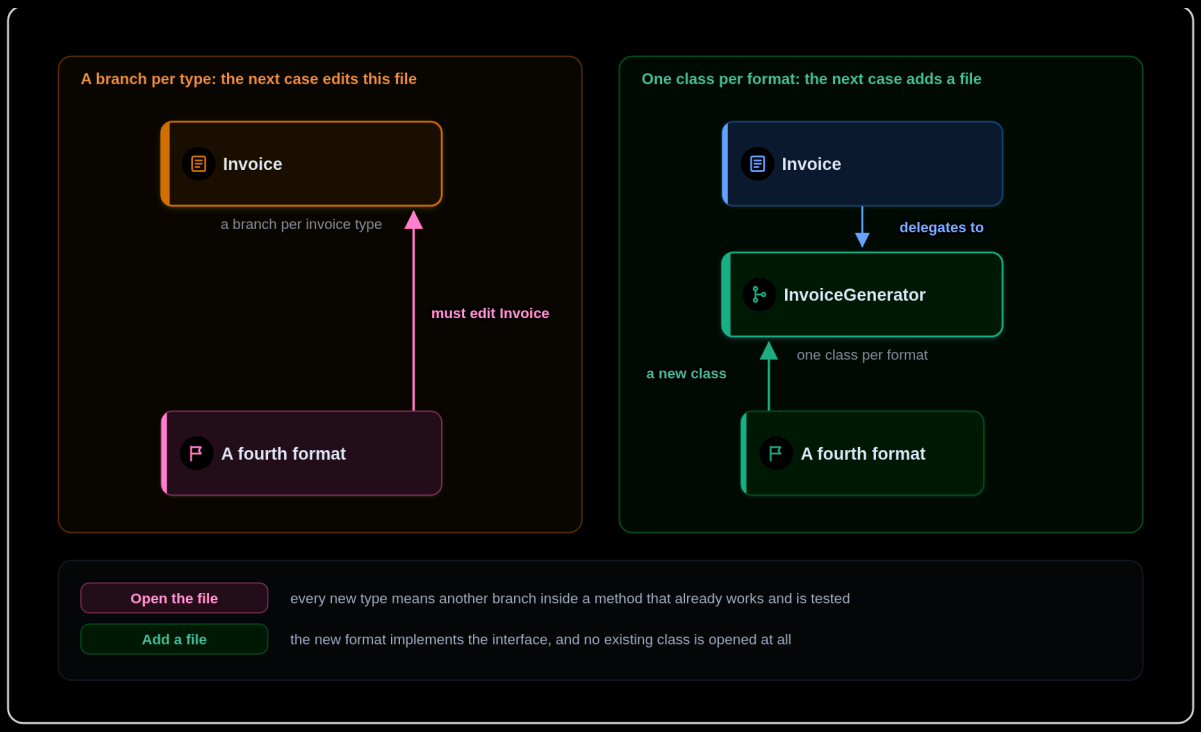
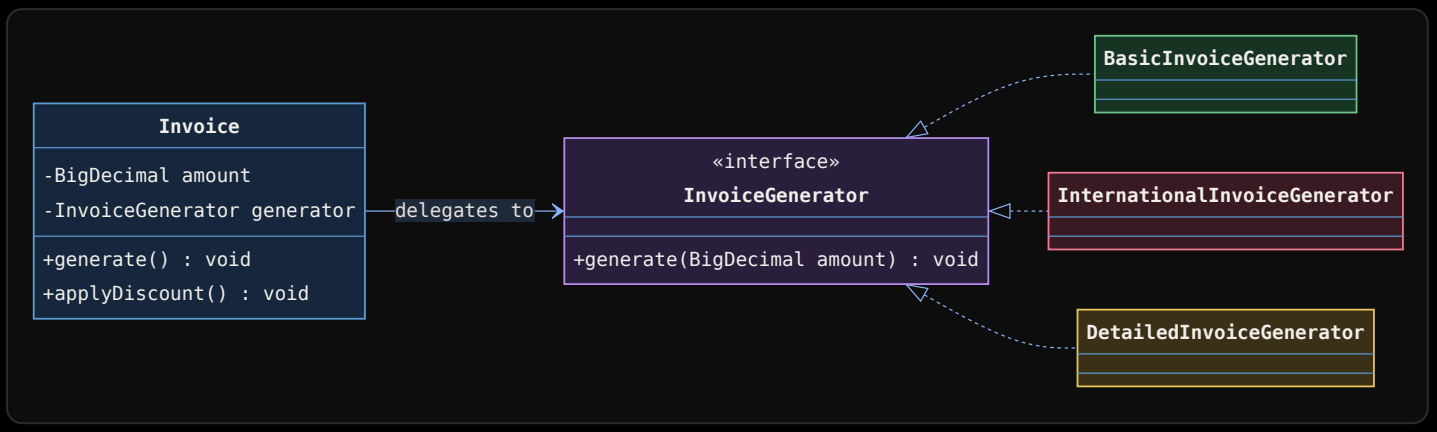


Figure — a branch per type forces an edit to Invoice; one class per format lets a new format be added as a file (course diagram)



This shape — a context object holding an interface-typed field with interchangeable implementations — is the **Strategy pattern** (a named reusable design; Chapters 8–11): OCP applied to an algorithm that varies.

Why not just subclass `Invoice`?

`class InternationalInvoice extends Invoice` overriding `generate()` is also OCP-compliant (Meyer's original style) and fine in a small program. Delegation wins as the system grows:

- **A second dimension of change.** Format **and** currency: subclassing needs a class per pairing (3 formats × 4 currencies = 12); delegation combines two independent objects (3 + 4 = 7).
- **Deciding at run time.** An object's class is fixed at `new`; a generator can be picked once the customer's country is known, even replaced later.

(*Head First Design Patterns* calls this "favour composition over inheritance".)

What is still not closed

Only *generation* was opened up; a new discount kind still edits `Invoice`, and that is fine. **"Closed" is never absolute: a class is closed against a specific kind of change**, normally one already seen. Martin calls this **strategic closure**: no program can be closed against every change, so the designer chooses which, guided by experience.

Approaches to OCP

Approach	How it extends	Pattern / Java feature	Good when
Interface + delegation	New class implements the interface	Strategy, `Comparator`	The algorithm varies, choose at run time
Inheritance + overriding	Subclass overrides a hook method (an overridable step the parent calls)	Template Method, abstract classes	Fixed skeleton, a few steps vary
Wrapping	New class wraps the old and adds behaviour	Decorator (`BufferedInputStream`)	Add features around existing behaviour
Listening	New subscriber registers	Observer, event listeners	New reactions to existing events
Plug-ins / registry	New implementation registers itself in a map or via `ServiceLoader` (Java's built-in way to discover implementations at start-up)	Plug-in architecture	Third parties add features
Functions	Pass a lambda	`java.util.function`, `@FunctionalInterface`	Small, one-method variations
Data / configuration	New entry in a table or config file	Rules engines, feature flags (on/off switches kept in configuration)	Variations that are just different numbers

The `switch` does not always vanish: it moves to **one** place at the edge — a factory or a `Map<String, InvoiceGenerator>` — instead of repeating in business logic.

Why it matters: extensibility (features without touching the tested core); **faster testing** (test the new class; core tests stay valid); **fewer regressions** (old features breaking after a change); **adaptability** (new requirements without rewrites).

Caution: do not over-engineer

- **Start simple:** write the `if` first; introduce the interface when the second or third case actually arrives (the "rule of three").
- **Balance:** too many abstraction layers make code harder to read, defeating the purpose.
- **Real needs only:** extension points for futures that never come are waste — *Refactoring*'s **Speculative Generality** smell.

OCP and SRP reinforce each other: OCP pulls changing parts away from the stable core, each into a single-job class (SRP). In practice SRP comes first — the extension point stays hidden until the class is down to one job.

Trade-offs

You gain	You pay
New variants without editing old code	An interface plus one class per variant
Each variant tested on its own	Behaviour is spread over files; reading the whole flow takes more hops
Variants selectable at run time	Someone must choose the variant (factory, map, DI, i.e. dependency injection, Section 4.10)

When not: nothing has varied yet; the "variants" are just different numbers (use data); the set of cases is fixed, small and will not grow (the two values of a boolean). For a fixed set, Java 21's **sealed interfaces with pattern-matching** `switch` (a sealed interface lists the only types allowed to

implement it, so a `switch` can check all are covered) fit well: the compiler checks every case — the opposite trade (easy to add operations, hard to add types).

Common mistakes

- "OCP means never edit a file" — bug fixes, refactoring and changes to the extension point itself all edit.
- Plug-in systems for one implementation; a `switch` in business logic replaced by a `switch` in five factories.

Smells and refactorings: Repeated Switches (*Switch Statements* in *Refactoring*'s first edition); fixes **Replace Conditional with Polymorphism**, **Replace Type Code with Subclasses**.

Real-world: `list.sort(comparator)` never changes — pass a new `Comparator`; **Java I/O decorators** — `new BufferedInputStream(new FileInputStream(f))` adds buffering without editing `FileInputStream`; **Servlet filters / Spring** `HandlerInterceptor` (code running before and after every web request) — new cross-cutting behaviour is a filter registered in a chain; **JDBC drivers** via `ServiceLoader` — a new database is a driver JAR (zipped bundle of compiled classes), not an edit to `DriverManager`; **IDE plug-ins** (IntelliJ, VS Code) and **browser extensions** — closed core, plug-in API as the port.

In the interview

> **30-second answer:** "OCP says you should be able to add the next variation by adding code, not editing working code. When I see a `switch` over a type that keeps growing, I put the varying behaviour behind an interface, one class per variant, and the stable class delegates to it. That is Strategy. I only do it once the variation has actually appeared, and I say that a class is closed against a specific kind of change, not against everything."

- *"Does OCP mean you never edit a class?"* — No: the next variation of a known kind is an addition; bug fixes and refactoring still edit.
- *"Inheritance or delegation?"* — Both satisfy OCP; delegation handles several axes of variation and run-time choice better.
- *"Interface for pricing up front?"* — Only if pricing has varied or clearly will; otherwise leave the `if` and refactor at the second case.
- *"Trucks are charged a different rate."* — A design that answers "add a class" beats "add a branch in this switch".

Self-check

1. **Q:** "Closed for modification"? **A:** Adding the next variation does not edit tested code.
2. **Q:** After the refactor, is `Invoice` closed against new discount rules? **A:** No, only new formats — fine until discounts vary.
3. **Q:** "OCP for an algorithm"? **A:** Strategy.
4. **Q:** Quick OCP test? **A:** "To add the next case, must I open this file?"

TL;DR

Put the varying part behind an interface, add variants as new classes, choose them at the edge — when variation is real. Closure is always against a specific kind of change.

4.8 Liskov Substitution Principle (LSP)

What it is

Objects of a subtype must be usable wherever objects of the parent type are expected, without making the program incorrect. Barbara Liskov introduced the idea in a 1987 keynote; with Jeannette Wing she defined it precisely (1994) as *behavioural subtyping*. Martin's short form: **subtypes must be**

substitutable for their base types. "Substitutable" means more than "it compiles": code written against the parent must still get **correct results** with a child.

> **Analogy:** a hire-car firm promises "a car that takes petrol"; hand over an electric car and the promise is broken at the petrol station, though an electric car is certainly a car.

The problem it solves

Polymorphism works only if callers can trust the base type. If a subclass secretly behaves differently, every caller must check which one it got (`instanceof`), and the benefit of polymorphism — and of OCP — is gone.

Example 1: `Car` and `ElectricCar`

```
// File: code/lld/ch04/lsp/car/before/ — Car prints "Car is starting/stopping/refueling"
public class Car {
    public void start()  { System.out.println("Car is starting"); }
    public void stop()   { System.out.println("Car is stopping"); }
    /** Names a MECHANISM (liquid fuel) instead of a goal. */
    public void refuel() { System.out.println("Car is refueling"); }
}
/** VIOLATION: inherits a promise (refuel) it cannot keep. */
public class ElectricCar extends Car {
    @Override public void refuel() { throw new UnsupportedOperationException("Electric cars don't need
refueling"); }
    public void recharge() { System.out.println("Electric car is recharging"); }
}
```

Code that *knows* it holds an `ElectricCar` calls `recharge()` and never notices — but `refuel()` is still on the object's public face. LSP is tested by code that does **not** know the subclass: `service(Car car)` in `Main` calls `start`, `refuel`, `stop` on any `Car`, and crashes for an electric one.

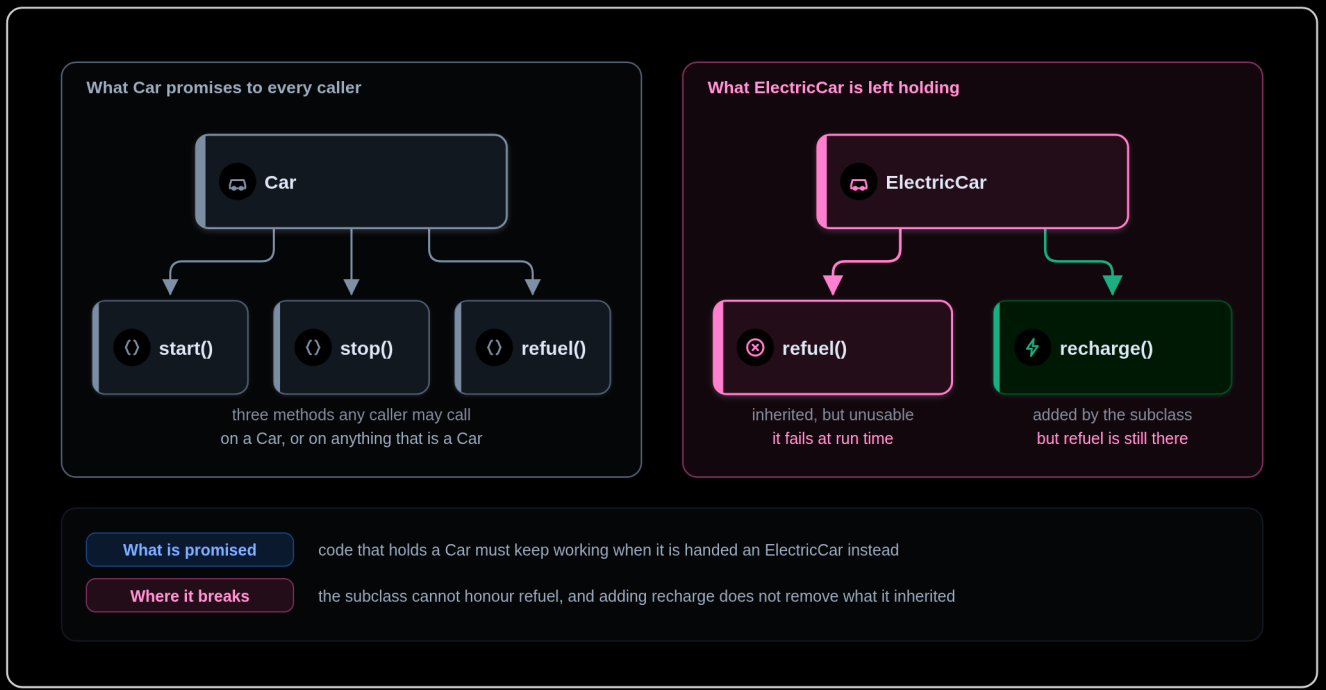


Figure — what Car promises every caller, and the unusable refuel() that ElectricCar is left holding (course diagram)

Name the real mistake. "`ElectricCar` should not extend `Car`" is half the diagnosis. The root cause: `refuel()` describes a **mechanism** (pouring in liquid fuel) only some vehicles use. A base-class method should describe the shared **goal**: every vehicle takes on energy, so `replenishEnergy()`.

The fix: raise a general parent. `Vehicle` declares the goal; `Car` and `ElectricCar` both extend it and **neither extends the other**:

```java
// File: code/lld/ch04/lsp/car/after/ — the base type promises the GOAL, not one mechanism
public abstract class Vehicle {
    public void start() { System.out.println(getClass().getSimpleName() + " is starting"); }
    public void stop()  { System.out.println(getClass().getSimpleName() + " is stopping"); }
    public abstract void replenishEnergy();
}
public class Car extends Vehicle {
    @Override public void replenishEnergy() { System.out.println("Car is pumping gasoline"); }
}
public class ElectricCar extends Vehicle {          // a sibling of Car, not a child
    @Override public void replenishEnergy() { System.out.println("Electric car is charging its battery"); }
}
```

- **The easy workaround is still wrong.** Keeping `refuel()` and having `ElectricCar.refuel()` secretly charge removes the exception, but the name keeps lying about half the fleet; renaming fixes the promise itself.
- **Why not just add `recharge()`?** Code typed as `Vehicle` cannot call it; callers may depend only on what the base type declares.
- **Substitution is child → parent, never between siblings.** `Car` and `ElectricCar` each replace a `Vehicle`; neither replaces the other.

When to break a hierarchy: a subclass **cannot implement** a parent method (throws or does nothing); a base method describes a **mechanism**, not a goal; callers **test the type** (`if (v instanceof ElectricCar)`) — the base promises too little or the wrong thing; subclasses share almost nothing, so the parent invents a relationship. Usual fixes: **raise a general parent** (above) or **move the varying behaviour out of the hierarchy** with composition (next).

Example 2: birds, and composition instead of inheritance

`fly()` on `Bird` makes penguins inherit and reject it. The fault is `Bird`'s promise, not the penguin (*Refactoring*: a subclass not wanting what it inherits is the **Refused Bequest** smell).

- **Inheritance is is-a**: `Sparrow extends Bird` receives **every** `Bird` method, suitable or not.
- **Composition is has-a**: `Bird` holds a `MovementStyle` field and calls it, inheriting nothing from it.

Fix: make "how a bird moves" its own small type, given via the constructor — **no bird subclasses** at all.

```java
// File: code/lld/ch04/lsp/birds/inheritance/ — Bird.fly() prints getClass().getSimpleName() + " is flying";
// Sparrow extends Bird { }  Penguin overrides fly() to throw UnsupportedOperationException("Penguins can't fly")

// File: code/lld/ch04/lsp/birds/composition/
@FunctionalInterface
public interface MovementStyle { void move(String birdName); }      // the behaviour that varies
// Flying -> "<name> is flying through the air"; Swimming -> "... swimming through the water";
// Running -> "... running along the ground"
public final class Bird {                    // a bird HAS-A movement style; nothing inherited, nothing refused
    private final String name;
    private MovementStyle movement;          // requireNonNull in constructor and changeMovement
    public void eat() { System.out.println(name + " is eating"); }
    public void travel() { movement.move(name); }                              // tell, don't ask
    public void changeMovement(MovementStyle movement) { this.movement = Objects.requireNonNull(movement); }  //
e.g. an injured bird
}
```

`Bird` now promises only what every bird can do (`eat()`, `travel()`), so nothing must be refused; flight is an optional object some birds get. The ostrich took one new movement style, not an `Ostrich` subclass with a meaningless `fly()`; and the style can be replaced at run time (`changeMovement`).

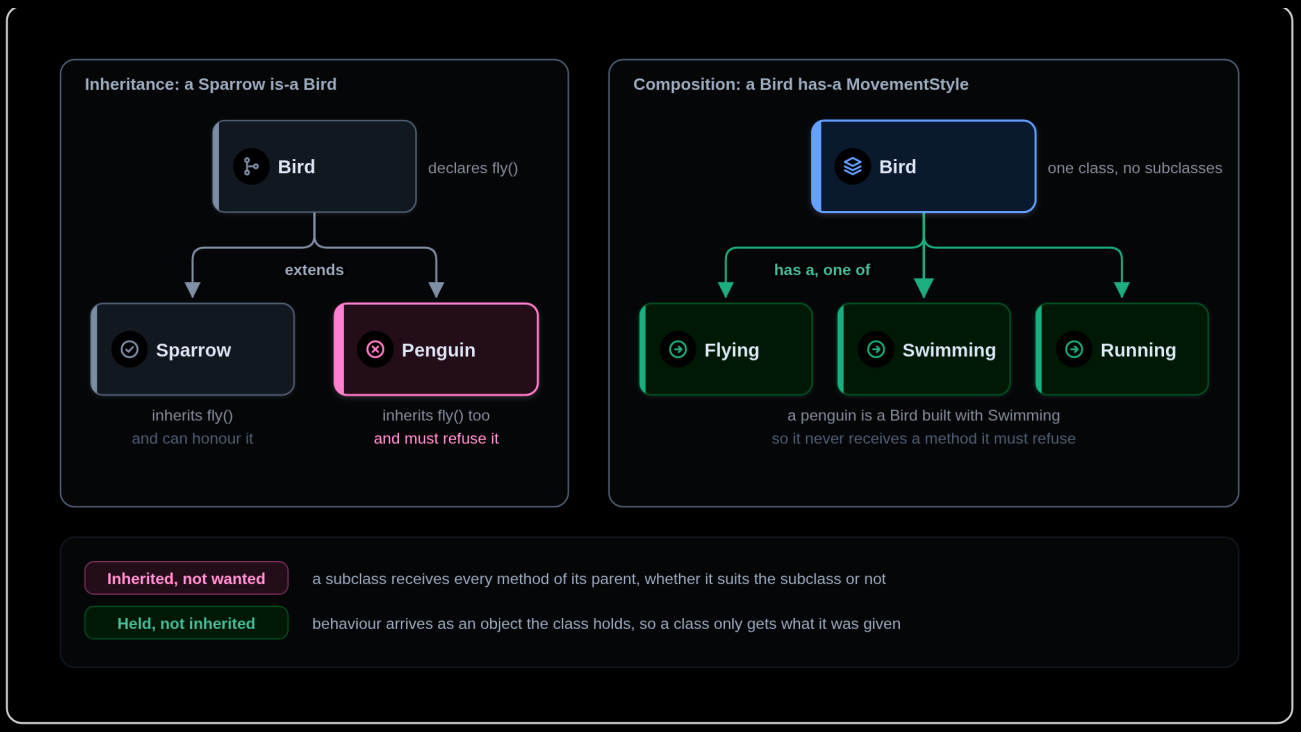
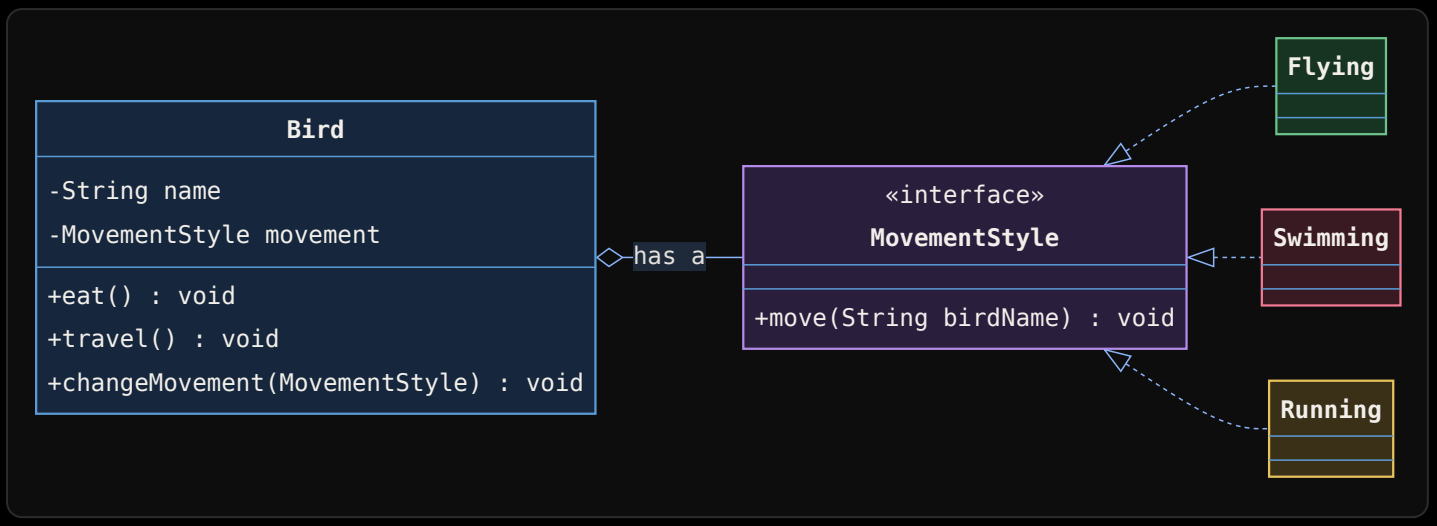


Figure — inheritance (Penguin inherits fly() and refuses it) vs composition (a Bird has a MovementStyle) (course diagram)



	Inheritance	Composition
Kind of link	Penguin **is a** Bird	Bird **has a** MovementStyle
In code	`class Penguin extends Bird`	`Bird` holds a `MovementStyle` field
What it gets	all of the parent's methods	just the object handed to it
To change behaviour	add another subclass	supply another object
Change while running?	no, fixed at creation	yes, replace the field
LSP risk	high: inherited methods may not suit	low: nothing is inherited

When to use which: inheritance when the child genuinely is a kind of the parent *and* keeps every inherited promise; **composition** when what differs is a behaviour, or the moment a subclass rejects something it inherited — an override that throws, silently does nothing, or carries a "never call this on X" comment should become a separate object.

Tell, Don't Ask

The composition design follows a guideline popularised by Andy Hunt and Dave Thomas (*The Pragmatic Programmer*): **tell an object what to do; do not ask about its state and decide for it.**

```
// Asking: the caller inspects the object and makes the decision.
if (bird instanceof Penguin) { swim(bird); } else { fly(bird); }

// Telling: the caller states the goal, the object decides how.
bird.travel();
```

Asking code must know every subtype, breaking OCP and hiding LSP problems behind `instanceof`; telling code works with any substitute. A caller needing `instanceof` on a base type strongly hints the base type promises the wrong thing.

The rules a subclass must follow

Exceptions are easy to spot; the harmful violations **quietly produce a wrong result and keep running**.

`Square extends Rectangle`. Every square is a rectangle in geometry; to stay square, `Square` couples its sides (setting width sets height). Nothing throws, yet a method for `Rectangle` setting width 5 and height 4 expects area 20 and gets 16 — in production, a wrong bill or inventory figure nobody notices for weeks. `Rectangle` implied, never stated, that **width and height change independently**; its two setters invited callers to rely on it. A square is a rectangle mathematically but **not a valid subtype of this `Rectangle` class. Base inheritance on the promises written in the code, not on real-world classification.**

A subclass may do the job *differently*, but may not **require more** from callers, **give them less**, or **break an assumption they depend on**. The vocabulary is Bertrand Meyer's **Design by Contract**: **preconditions** (what the caller must provide), **postconditions** (what the method guarantees afterwards), **invariants** (what is always true of the object).

Rule 1 — Preconditions may not be strengthened. An override may accept *wider* input, never narrower. `Discount` accepts any amount ≥ 0; `SmallOrderDiscount` refuses ≥ 100, so a caller passing 250 (allowed by the parent) fails — and would have to discover which discount it got, the very check substitution should make unnecessary.

Rule 2 — Postconditions may not be weakened. An override may guarantee *more*, never less. `Account.withdraw` guarantees balance ≥ 0; `OverdraftAccount` lets it go negative — a fine banking feature, but a broken guarantee: callers relying on "balance ≥ 0" are silently wrong (a payment job keeps paying suppliers from an overdrawn account). An overdraft account can be a good class, just not a subtype of one promising a non-negative balance.

Rule 3 — Invariants must be preserved. An invariant holds throughout the object's life; `Square` broke "width and height are independent". (Equally a weakened postcondition — `Rectangle.setWidth` implicitly guarantees "height unchanged" and `Square.setWidth` drops it; Martin describes it that way; same conclusion.) Hardest rule, since invariants are rarely written down: a subclass inherits every promise, documented or not.

Rule 4 — No new exceptions callers cannot handle. Only the parent's declared exceptions (or subtypes); a new type reaches callers with no `catch`. `ElectricCar.refuel()` broke it — and compiled, because Java enforces this only for **checked** exceptions and `UnsupportedOperationException` is unchecked. Most visible, least harmful: the failure is immediate and loud.

Rule 5 — The history constraint (Liskov and Wing): **a subtype must not allow state changes the parent does not allow.** `Price` promises its value never changes; `AdjustablePrice` adds `adjust()` — each call looks innocent, but a caller who captured a quote of 100 is charged 120 on "the same" price. That is why immutable classes like `String` and `Integer` are `final`: a mutable subclass could break the immutability callers rely on.

```
// File: code/lld/ch04/lsp/rules/ (essentials)
public class Discount {          // precondition: amount >= 0; postcondition: 90% of amount
    public BigDecimal apply(BigDecimal amount) {
        if (amount.signum() < 0) throw new IllegalArgumentException("amount must be >= 0");
        return amount.multiply(new BigDecimal("0.90")).setScale(2, RoundingMode.HALF_EVEN);
    }
}
public class SmallOrderDiscount extends Discount {   // RULE 1 BROKEN: demands MORE
    @Override public BigDecimal apply(BigDecimal amount) {
        if (amount.compareTo(new BigDecimal("100")) >= 0) throw new IllegalArgumentException("only for orders
under 100");
        return super.apply(amount);
    }
}
public class Account {           // postcondition: balance never negative
    protected BigDecimal balance;
    public BigDecimal withdraw(BigDecimal amount) {
        if (amount.compareTo(balance) <= 0) balance = balance.subtract(amount);
        return balance;
    }
}
public class OverdraftAccount extends Account {      // RULE 2 BROKEN: promises LESS; nothing throws
    @Override public BigDecimal withdraw(BigDecimal amount) { balance = balance.subtract(amount); return balance;
}
}
public class Rectangle {         // unwritten invariant: width and height set independently
    protected int width, height;
    public void setWidth(int width)   { this.width = width; }
    public void setHeight(int height) { this.height = height; }
    public int area() { return width * height; }
}
public class Square extends Rectangle {              // RULE 3 BROKEN: couples the setters, silent wrong answer
    @Override public void setWidth(int width)   { this.width = width;  this.height = width; }
    @Override public void setHeight(int height) { this.width = height; this.height = height; }
}
public class Price {             // history contract: never changes after creation (private final value)
    public BigDecimal value() { return value; }
}
public class AdjustablePrice extends Price {         // HISTORY CONSTRAINT BROKEN: adds a mutator
    private BigDecimal current;
    @Override public BigDecimal value() { return current; }
    public void adjust(BigDecimal newValue) { this.current = newValue; }
}
```

Why it matters so much: code written against a base type cannot protect itself — it cannot see, and should not need to see, which subclass it got. Only the subclass author can keep the contract.

The fix for Rectangle/Square: share no mutable promise one cannot keep. Make both **immutable** behind a common interface promising only what both can do (`area()`); "resizing" returns a new object, so no setter can break an invariant. Java 21 `record`s and a `sealed` interface:

```
// File: code/lld/ch04/lsp/shapes/
public sealed interface Shape permits Rect, Sq { int area(); }
public record Rect(int width, int height) implements Shape {
    public Rect withWidth(int w) { return new Rect(w, height); }   // "resizing" = a new object
    @Override public int area() { return width * height; }
}
public record Sq(int side) implements Shape { @Override public int area() { return side * side; } }
```

The driver runs every example through client methods that know **only the base type**:

```java
// File: code/lld/ch04/lsp/Main.java (client methods; main calls each with parent and child, under === headers)
static void service(lld.ch04.lsp.car.before.Car car) { car.start(); car.refuel(); car.stop(); }
static void takeATrip(Vehicle v) { v.start(); v.replenishEnergy(); v.stop(); }
static void checkout(Discount d, BigDecimal amount) {             // called with 250.00
    try { System.out.println("  Charged: " + d.apply(amount)); }
    catch (IllegalArgumentException e) { System.out.println("  Broke on a valid amount: " + e.getMessage()); }
}
static void paySupplier(Account account, BigDecimal amount, String label) {   // balance 50.00, pay 80.00
    BigDecimal balance = account.withdraw(amount);
    System.out.println("  " + label + " balance after payment: " + balance
            + (balance.signum() < 0 ? "   <-- the caller never expected this" : ""));
}
/** A contract test written against Rectangle -- run it for every subclass. */
static void resizeAndCheck(Rectangle r, String label) {
    r.setWidth(5);
    r.setHeight(4);
    System.out.println("  " + label + " expected area 20, got " + r.area());
}
static void quoteThenCharge(Price p, Runnable somethingElseHappens) {   // sneaky.adjust(120.00) for
AdjustablePrice
    BigDecimal quoted = p.value();
    somethingElseHappens.run();
    System.out.println("  quoted " + quoted + ", charged " + p.value());
}
// 7: prints each of new Rect(5, 4), new Sq(4), new Rect(2, 3).withWidth(5) with "area="
// 8: old Sparrow/Penguin fly() (catching the refusal); then Birds Sparrow(Flying), Penguin(Swimming),
//    Ostrich(Running) travel(); then sparrow.changeMovement(new Running()); sparrow.travel();
```

Output:

```
=== 1. Car / ElectricCar (exception rule) ===
Car is starting
Car is refueling
Car is stopping
Car is starting
  substitution broke: Electric cars don't need refueling
=== 2. Fixed hierarchy: Vehicle -> Car | ElectricCar ===
Car is starting
Car is pumping gasoline
Car is stopping
ElectricCar is starting
Electric car is charging its battery
ElectricCar is stopping
=== 3. Precondition strengthened ===
  Charged: 225.00
  Broke on a valid amount: only for orders under 100
=== 4. Postcondition weakened ===
  Account:          balance after payment: 50.00
  OverdraftAccount: balance after payment: -30.00   <-- the caller never expected this
=== 5. Invariant broken (Rectangle / Square) ===
  Rectangle: expected area 20, got 20
  Square:    expected area 20, got 16
=== 6. History constraint broken ===
  quoted 100.00, charged 100.00
  quoted 100.00, charged 120.00
=== 7. Fix for shapes: no shared mutable promise ===
  Rect[width=5, height=4] area=20
  Sq[side=4] area=16
  Rect[width=5, height=3] area=15
=== 8. Birds: inheritance vs composition ===
Sparrow is flying
  Substitution broke: Penguins can't fly
Sparrow is flying through the air
Penguin is swimming through the water
Ostrich is running along the ground
Sparrow is running along the ground
```

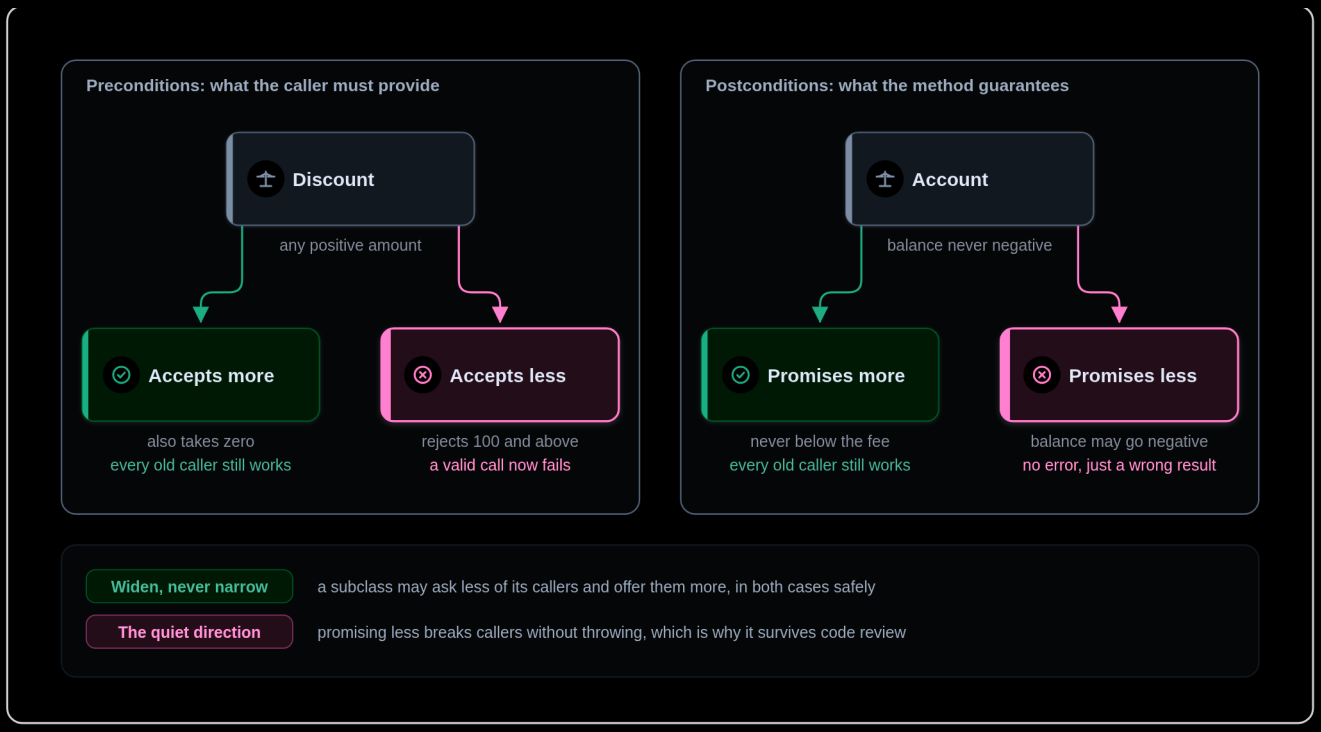


Figure — preconditions and postconditions: accepting more or promising more is safe; accepting less or promising less breaks callers (course diagram)

Widen, never narrow: a subclass may ask less of callers and offer them more. (The figure's parent `Discount` accepts positive amounts and the safe subclass also zero; here the parent already accepts zero — same direction. Its "promises more" account keeps the balance above a fee, stronger than "never negative".)

Rule	The subclass may not…	How it usually shows up
Preconditions	demand more from the caller	valid input is rejected
Postconditions	guarantee less to the caller	a returned value breaks a caller's assumption
Invariants	break a rule that always held	a silently wrong result (the square)
Exceptions	throw something new	a crash in code with no matching `catch`
History	allow state changes the parent forbids	a "fixed" value changes between two reads

Signature rules (compiler-checked). Behavioural subtyping also has type rules: parameters may be *wider* (contravariant), return types *narrower* (covariant), exceptions only *narrower*. Java enforces a version: an override may return a **subtype** (covariant returns); may **not** declare new or broader **checked** exceptions; may **not** reduce visibility; parameter types must match exactly (otherwise it is an **overload** — `@Override` catches this). Java checks none of the **behavioural** rules: unchecked exceptions, weakened postconditions and broken invariants all compile. That part is on you.

How to check a subclass

1. **List the parent's promises first** — promises, not signatures ("the balance never drops below zero", "width and height change independently"); most violations slip in because the promise was never stated.
2. **Write one test against the parent type and run it for every subclass.** `resizeAndCheck(Rectangle)` never mentions `Square`, yet `Square` fails it; any subclass failing a base-type test violates LSP, however reasonable its code looks. (In JUnit, Java's standard testing library, an *abstract contract test* each subclass's test class extends.)

Why LSP matters: prevents fragile code (no surprised callers); keeps polymorphism usable (new subclasses without `instanceof` — what makes OCP work); correctness (right results, not just no crash); safe reuse of base types.

Trade-offs

Following LSP gains	Costs
Callers trust the base type; no type checks	You must think about and document contracts
New subclasses are safe to add	Some "natural" hierarchies (Square/Rectangle) must be abandoned
Composition gives run-time flexibility	More small objects; behaviour is spread out

When not: LSP is not optional where you use polymorphism — what you can skip is inheritance: a subclass that cannot keep the parent's promises should **not be a subclass**.

Common mistakes

- Modelling the real world ("a square is a rectangle", "a penguin is a bird") instead of the code's promises.
- "Fixing" a violation by making the method silently do nothing — a loud failure becomes a quiet wrong result.
- Thinking LSP is only about exceptions; the silent rules (postconditions, invariants, history) are more dangerous.
- Believing siblings must substitute for each other; substitution is only child → parent.

Refactorings: smell **Refused Bequest**; fixes **Replace Subclass with Delegate**, **Replace Superclass with Delegate** (composition), **Extract Superclass** (raise a general parent), **Push Down Method** (move a method only some subclasses support out of the parent).

Real-world examples

- **`java.sql.Date extends java.util.Date`**: time methods like `getHours()` throw `IllegalArgumentException` (an SQL date has no time), breaking code written for `java.util.Date`.
- **`java.sql.Timestamp`**: its Javadoc (official docs generated from code comments) warns its `equals` is not symmetric with `java.util.Date.equals` and calls the inheritance implementation, not type, inheritance — a textbook LSP warning in the JDK.
- **`java.util.Stack extends Vector`**: inherits `insertElementAt` and `get(int)`, letting callers break stack order; the Javadoc now recommends `Deque` (a double-ended queue adding/removing only at its ends).
- **`List.of(...)` / `Collections.unmodifiableList(...)`** throw `UnsupportedOperationException` from `add`. `List` marks `add` an *optional operation*, so it is in the contract — but a known compromise: a caller holding a `List` cannot know whether `add` works.

In the interview

> **30-second answer:** "LSP says a subclass must work anywhere its parent is expected, giving correct results, not just no crash. Concretely: it cannot demand more input, promise less output, break an invariant, throw new exceptions, or allow state changes the parent forbids. Square extending a mutable Rectangle is the classic silent break: set width 5 and height 4 and you get 16. I test it by writing one test against the base type and running it for every subclass. When a subclass must refuse a method, I either raise a better parent or move the behaviour into a composed object."

- *"An LSP violation you've seen?"* — Rectangle/Square, or `java.sql.Date` throwing from `getHours()`; then the rule: demand no more, promise no less, keep the invariants.
- *"Composition over inheritance, always?"* — No; inheritance is fine when the child honours every parent method. A subclass refusing an inherited method is the signal to switch.
- *"Is `ElectricCar` a substitute for `Car` after your fix?"* — No, nor should it be: both substitute for `Vehicle`; siblings do not substitute.

Self-check

1. **Q:** A subclass accepts **fewer** inputs. Which rule? **A:** Preconditions strengthened (rule 1).

2. **Q:** Why is Square/Rectangle more dangerous than `ElectricCar.refuel()` throwing? **A:** A silent wrong answer; the exception fails loudly and is found at once.
3. **Q:** The history constraint? **A:** A subtype must not allow state changes the parent does not, e.g. a mutable subclass of an immutable class.
4. **Q:** What does `@Override` protect you from? **A:** Accidentally overloading (different parameter type) instead of overriding.
5. **Q:** "Tell, Don't Ask"? **A:** Tell an object to do the job (`bird.travel()`) instead of inspecting its type/state and deciding for it.

TL;DR

A subclass keeps every parent promise: accepts at least as much, delivers at least as much, keeps invariants and history, throws nothing new. Inheritance follows promises, not real-world categories. When a subclass must refuse something, raise a better parent or compose.

4.9 Interface Segregation Principle (ISP)

What it is

Clients should not be forced to depend on methods they do not use. Instead of one large general interface, offer smaller ones shaped around what each kind of caller needs.

Origin. Martin met the problem consulting for Xerox: one big `Job` class served every printer-system task (printing, stapling...), so a change for one task forced every other task's code to be rebuilt. Splitting its services into one interface per kind of client fixed it.

> **Analogy:** handing every restaurant guest the 40-page menu (wine list, kids' menu, catering prices) when someone wants only coffee — a coffee card is enough.

A **fat interface** has many methods, usually grown one feature at a time; a **role interface** is a small interface for the role an object plays for one kind of caller.

The problem it solves

- Implementers must write methods they cannot honour, so they throw or do nothing (also breaking LSP).
- Callers depend on methods they never call, so changes to those still reach them (recompile, redeploy, retest).
- Test doubles get long: a fake must implement every method.

Example: a bloated `Printer` interface

```java
// File: code/lld/ch04/isp/fat/Printer.java — FAT: everything any printer might do
public interface Printer {
    void printDocument(String doc);
    void scanDocument(String doc);
    void faxDocument(String doc);
    void stapleDocument(String doc);
}
// File: code/lld/ch04/isp/fat/BasicPrinter.java — VIOLATION: forced to "implement" three methods it can only refuse
@Override public void printDocument(String doc) { System.out.println("BasicPrinter printing " + doc); }
@Override public void scanDocument(String doc)  { throw new UnsupportedOperationException("BasicPrinter cannot scan"); }
// faxDocument and stapleDocument throw likewise ("cannot fax", "cannot staple").
// AdvancedPrinter really does all four ("AdvancedPrinter printing/scanning/faxing/stapling " + doc): the wide interface is honest for it.
```

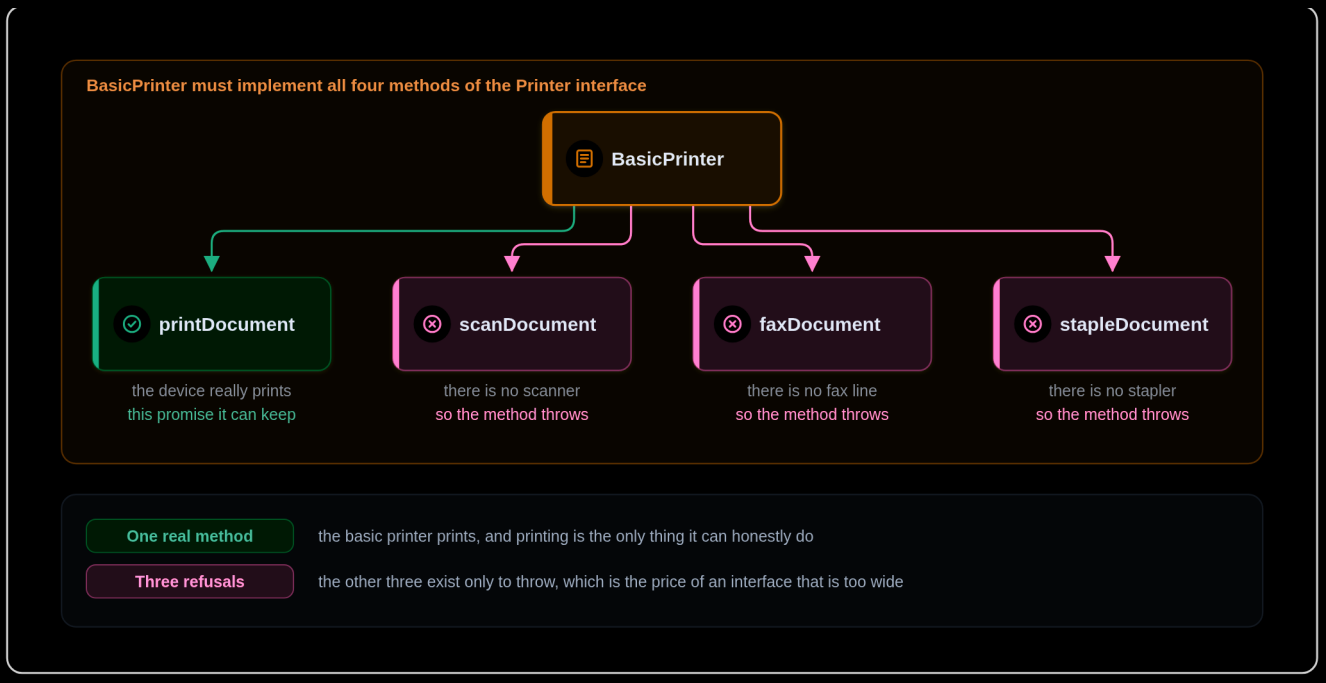


Figure — BasicPrinter keeps one real method and three that exist only to throw (course diagram)

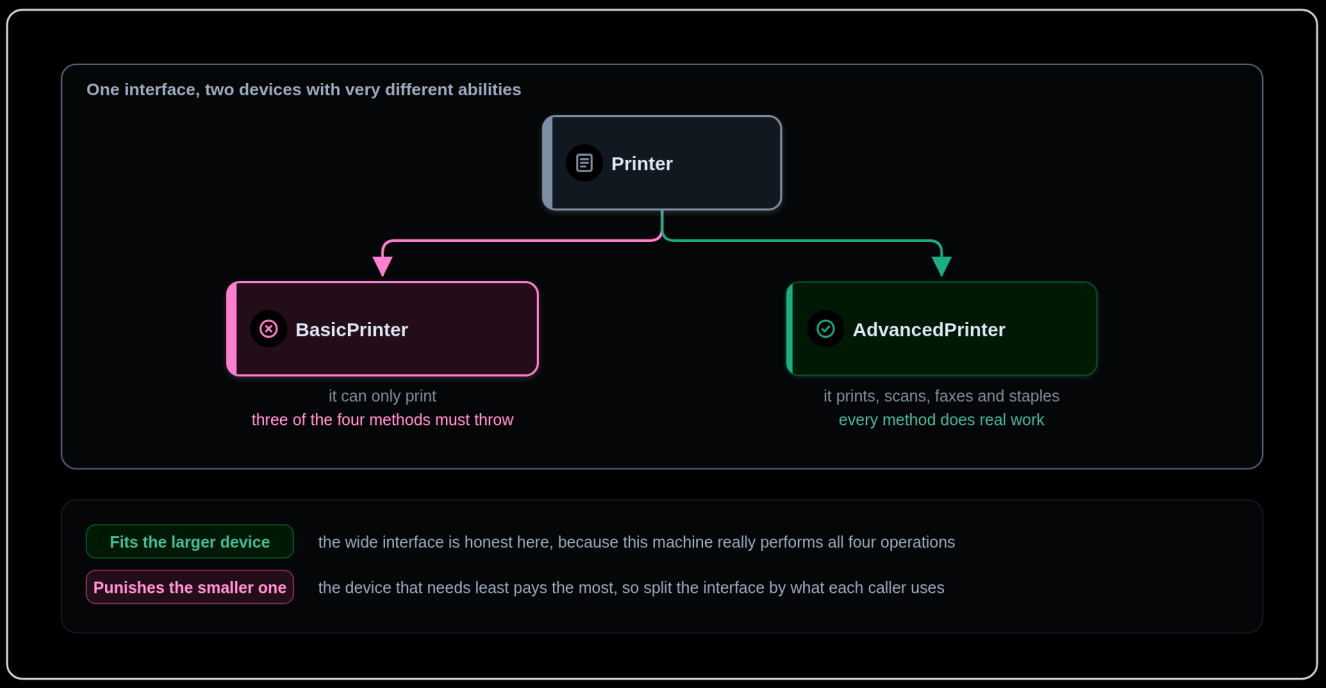


Figure — one interface, two devices: it fits the larger device and punishes the smaller one (course diagram)

ISP in one picture: the simplest device carries the heaviest burden. Split the interface by what each caller uses.

Restructuring: first attempt, one interface per capability

Split into `Printer`, `Scanner`, `Fax`, `Stapler` (one method each); `BasicPrinter implements Printer`; `AdvancedPrinter implements Printer, Scanner, Fax, Stapler`. Nobody implements what it cannot do. This satisfies ISP and shows the mechanics — but **one interface per method is not the goal**: if every scanner also faxes and every caller uses both, separate interfaces add names and remove no dependency.

How small should an interface be?

Group by caller, not by method: an interface carries the methods one kind of caller needs. Right-sized when both answers are "no":

1. **Is any implementer forced to supply a method it cannot honour?** Yes → too wide.
2. **Does any caller depend on methods it never calls?** Yes → too wide for that caller.

One method or six — the count was never the measure.

What over-splitting costs. A caller that scans, prints and staples needs three parameters, and its creator hands over **one device three times**:

```
// File: code/lld/ch04/isp/split/OfficeWorkflow.java — OVER-SPLIT: three dependencies, nothing gained
public OfficeWorkflow(Printer printer, Scanner scanner, Stapler stapler) { ... }
public void run(String doc) {
    scanner.scanDocument(doc);
    printer.printDocument(doc);
    stapler.stapleDocument(doc);
}
```

Technically ISP-compliant, but `OfficeWorkflow` calls all three, so the split removed no dependency — it added two type names and two parameters.

Grouping by role. `OfficeWorkflow` wants one device that scans, prints and staples: one role, `DocumentProcessor`. `PrintQueue` only prints: role `Printer`. Two interfaces are justified because a `BasicPrinter` can play the second role but not the first; `DocumentProcessor extends Printer`, so any processor also fits a print queue.

```
// File: code/lld/ch04/isp/roles/
public interface Printer { void printDocument(String doc); }                // callers that only print
public interface DocumentProcessor extends Printer {                        // callers that scan, print AND
staple
    void scanDocument(String doc);
    void stapleDocument(String doc);
}
// BasicPrinter implements Printer ("Basic printer printing " + doc);
// OfficeDevice implements DocumentProcessor ("Printing " / "Scanning " / "Stapling " + doc)
public class PrintQueue {                        // depends on printing only, so a basic printer is enough
    private final Printer printer;
    public void submit(String doc) { printer.printDocument(doc); }
}
public class OfficeWorkflow {                    // one dependency: one role covers everything it needs
    private final DocumentProcessor processor;
    public void run(String doc) {
        processor.scanDocument(doc);
        processor.printDocument(doc);
        processor.stapleDocument(doc);
    }
}

// File: code/lld/ch04/isp/Main.java (abridged)
// 1: for fat AdvancedPrinter and BasicPrinter: printDocument("report.pdf"), then scanDocument (catching
//    the refusal as "  forced to implement: ...")
// 2: split BasicPrinter prints memo.pdf; AdvancedPrinter adv faxes memo.pdf; then
new lld.ch04.isp.split.OfficeWorkflow(adv, adv, adv).run("contract.pdf");   // same object three times
// 3: role interfaces
new PrintQueue(new BasicPrinter()).submit("receipt.pdf");
new PrintQueue(new OfficeDevice()).submit("receipt.pdf"); // bigger device fits the small role
new OfficeWorkflow(new OfficeDevice()).run("contract.pdf");
```

Output:

```
=== 1. Fat interface ===
AdvancedPrinter printing report.pdf
AdvancedPrinter scanning report.pdf
BasicPrinter printing report.pdf
  forced to implement: BasicPrinter cannot scan
=== 2. One interface per method ===
BasicPrinter printing memo.pdf
AdvancedPrinter faxing memo.pdf
AdvancedPrinter scanning contract.pdf
AdvancedPrinter printing contract.pdf
AdvancedPrinter stapling contract.pdf
=== 3. Role interfaces (grouped by caller) ===
Basic printer printing receipt.pdf
Printing receipt.pdf
Scanning contract.pdf
Printing contract.pdf
Stapling contract.pdf
```

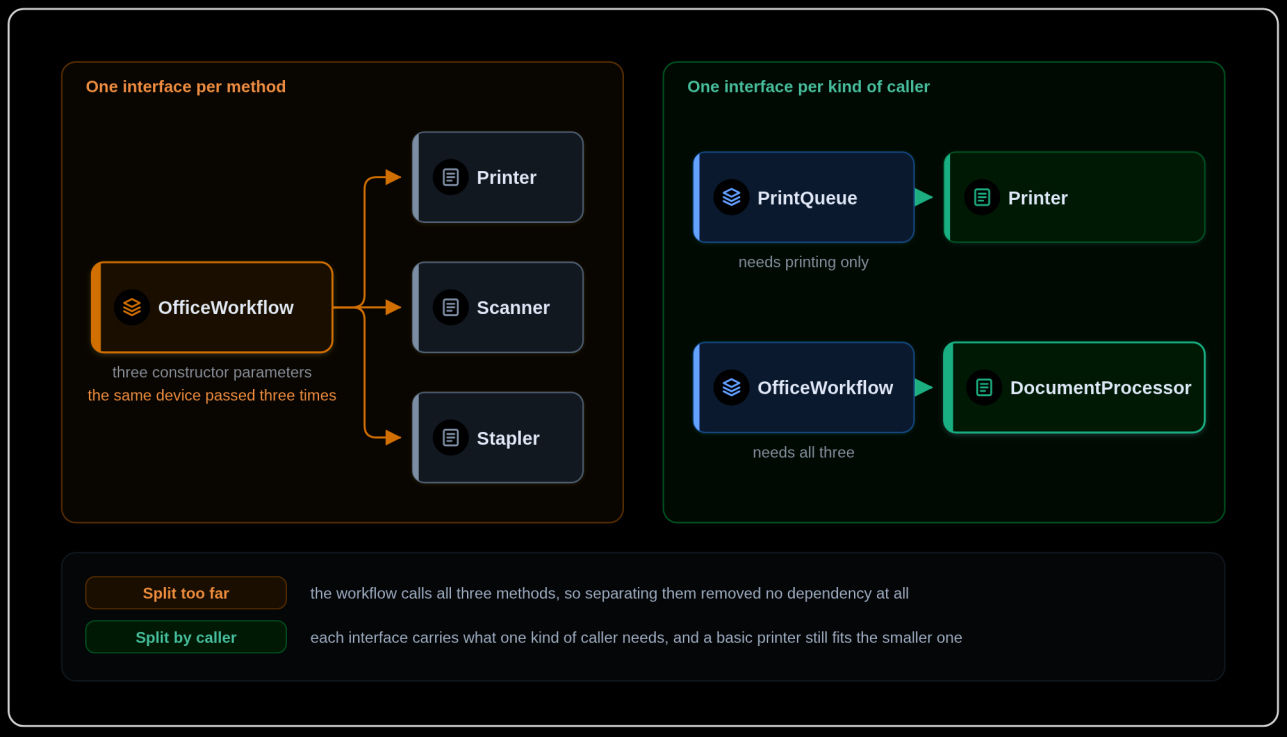
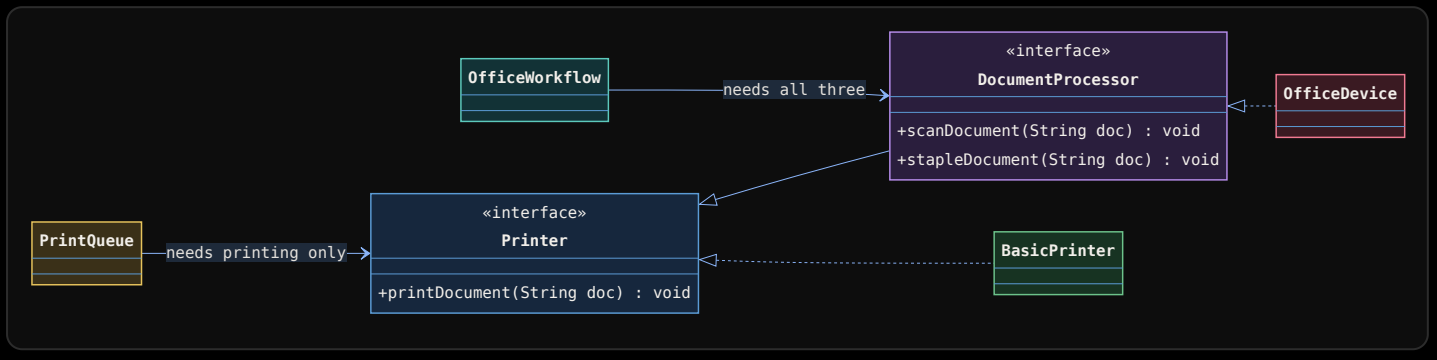


Figure — split too far (three interfaces, the same device passed three times) vs split by caller (one role per kind of caller) (course diagram)



	Four interfaces, one method each	Two role interfaces
Implementer forced to supply what it cannot do?	No	No
Caller depending on unused methods?	No	No
Dependencies for `OfficeWorkflow`	3	1
Names to learn	4	2

Both satisfy ISP; the role-based one is leaner because its interfaces follow callers, not methods.

Rule of thumb: split only when you can **point to the caller** that needs the smaller piece (otherwise it is speculation); conversely, two interfaces always implemented by the same classes and consumed by the same callers should probably merge.

Languages without an `interface` keyword (Python, JavaScript): the interface is invisible but real — **the set of methods a caller actually calls**. Handing a 20-method object to a function that calls one creates the same needless dependency, unflagged by any compiler. Tests hide it further: Java forces a long stub (a fake with every method filled in) for a wide interface, whereas Python lets you pass a one-method object — convenient, but masking the problem. So check deliberately what a function calls on its argument. Python's `abc` module and `typing.Protocol` (its two ways to declare an interface) can write the role down; for a one-place role a comment listing needed methods often suffices.

Five signals of an ISP violation

1. Low cohesion in the interface — unrelated method groups.

```
interface MediaDevice { void playAudio(); void playVideo(); void recordVideo(); }
// A Speaker can only playAudio(); playVideo() and recordVideo() must throw.
```

2. A fat interface — a long list grown feature by feature; mechanism-named methods are a tell that some implementer won't use them.

```
interface PaymentProcessor { void processCreditCard(BigDecimal a); void processPayPal(BigDecimal a); void
processBankTransfer(BigDecimal a); }
// A shop that only takes PayPal must still supply the other two.
// Better: interface PaymentMethod { void pay(BigDecimal amount); } with one class per method.
```

3. Empty or throwing methods — the clearest signal: the interface asks for what the class cannot provide.

```
interface Vehicle { void drive(); void fly(); void sail(); }
// A Car drives; fly() and sail() throw UnsupportedOperationException.
```

4. One interface serving several kinds of client — it describes a category ("everything an employee can do"), so every role implements the union of all roles.

```
interface Employee { void calculateSalary(); void handleCustomerQueries(); void manageTeam(); }
// A support agent handles queries; the other two methods must throw.
```

5. Tests needing long stand-ins — a fake must implement every method even if one is exercised; dozens of empty bodies for one call mean the interface is too wide **for that caller**. It measures the distance between what a caller uses and what it must depend on.

Signal	When it shows up	Typical cause
Low cohesion	Reading the interface	Groups of unrelated methods should be split into their own interfaces
Fat interface	Reading the interface	It grew feature by feature and was never split
Empty / throwing methods	Reading an implementer	The implementer was forced to accept what it cannot do
One interface, many roles	Comparing implementers	It describes a category, not a caller's needs
Long test stand-ins	Writing a test	Callers are tied to much more than they actually call

How it works (step by step)

1. For each caller, list the methods it actually calls. 2. Group callers with the same list: each group is a **role**. 3. One interface per role (interfaces may extend each other, like `DocumentProcessor`

`extends Printer`). 4. Each class implements every role it can honestly fill. 5. Change each caller's parameter type to the narrowest role it needs.

Why it matters: no bloated classes forced to implement unneeded methods; maintainability (a change to one role does not ripple to another's callers); flexibility (small roles are easy to implement anew); simpler tests (fakes implement one or two methods); clearer design (each interface documents one relationship).

Relation to the others: SRP — ISP is SRP for interfaces; **OCP** — new capabilities arrive as new small interfaces, not edits to a big one; **LSP** — small-interface implementers can honour every method, while a fat interface **forces** LSP violations (the throwing `BasicPrinter`); **DIP** — high-level code depends on the smallest abstraction it needs.

Trade-offs

Small role interfaces gain	Costs
No forced, throwing methods	More interface names
Callers insulated from unrelated changes	Over-splitting adds wiring with no benefit
Tiny test doubles; lambdas for one-method roles	Harder to see "everything this device can do" in one place

When not: every implementer supports every method and every caller uses them all (already right-sized); you cannot name the caller wanting the smaller piece.

Common mistakes

- **One interface per method everywhere** — the commonest mistake after learning ISP. A fat interface at least shows its cost; hundreds of tiny ones cost slowly, in names and wiring, and are harder to undo.
- Splitting by the implementer's features instead of the caller's needs.
- Keeping the fat interface and adding an "adapter" base class with empty methods (like AWT's `MouseAdapter`; AWT is Java's original desktop UI toolkit): convenient, but a symptom.

Real-world examples

- **Java's tiny role interfaces**: `Runnable`, `Comparable`, `AutoCloseable`, `Iterable`, `CharSequence`, all of `java.util.function`; `try-with-resources` needs only `AutoCloseable`, not "a file".
- **AWT** `MouseListener` has five methods though most listeners need one, hence `MouseAdapter`'s empty bodies; later APIs (JavaFX event handlers) use one-method handlers.
- **Collections**: `List.of(...)` and unmodifiable views throw on `add` because `Collection` bundles reading and writing; Kotlin (another JVM language) separates `List` (read) and `MutableList` (write).
- **Spring Data** (generates repository code): `CrudRepository`, `PagingAndSortingRepository` and the bare `Repository` marker expose only the operations needed.

In the interview

> **30-second answer:** "ISP says no caller should depend on methods it does not use. The symptom is a class implementing an interface method by throwing, or a test fake full of empty methods. I split interfaces by caller: one role interface per kind of caller, not one per method. For printers, a print queue needs `Printer`, an office workflow needs `DocumentProcessor`. I split only when I can name the caller that wants the smaller piece."

- *"SRP vs ISP?"* — SRP protects the **class** (one reason to change); ISP protects the **caller** (no unused dependencies). SRP splits classes, ISP interfaces; either can break without the other.
- *"How small is too small?"* — When a caller takes several interfaces that are always the same object: always implemented and used together means one interface.

Self-check

1. **Q:** The two right-size questions? **A:** Is any implementer forced to supply what it cannot do? Does any caller depend on methods it never calls?
2. **Q:** Three ISP signals? **A:** Throwing/empty methods, a fat interface with mechanism-named methods, long test stand-ins (also: low cohesion, one interface for several roles).
3. **Q:** Why didn't four interfaces help `OfficeWorkflow`? **A:** It uses all three methods, so no dependency was removed; only names and parameters were added.

TL;DR

One interface per kind of caller. Throwing methods and long fakes mean too wide; the same object passed three times means too thin.

4.10 Dependency Inversion Principle (DIP)

What it is

Martin's two statements, paraphrased:

1. **High-level modules should not depend on low-level modules. Both should depend on abstractions.**
2. **Abstractions should not depend on details. Details should depend on abstractions.**

A **high-level module** holds business policy — *what* the application does ("notify the user when an order ships"); a **low-level module** a technical detail — *how* (SMTP email, SMTP being the mail-sending protocol; an SMS gateway; MySQL); the **abstraction** is the interface between.

Why "inversion"? In traditional layering, high-level code calls low-level code **and** its source depends on it (`import com.mail.SmtpClient`). DIP keeps the call direction (policy still calls the detail at run time) but **reverses the source-code dependency**: the detail imports and implements an interface belonging to the policy.

> **Analogy: a wall socket.** A lamp plugs into a standard socket, not the power station; the building wiring (the detail) is also built to the standard; either side is replaceable.

The problem it solves

Business logic tied to one vendor or technology (email → SMS edits business code); untestable without the real infrastructure (servers and services the program runs on: mail server, database); low-level library changes rippling up into policy.

Example: notifications

Before. High-level `NotificationService` creates and names low-level `EmailService`:

```
// File: code/lld/ch04/dip/before/ — EmailService.sendEmail prints "Sending email: " + message
/** VIOLATION: high-level policy creates and names a low-level detail. */
public class NotificationService {
    private final EmailService emailService = new EmailService();
    public void notifyUser(String message) { emailService.sendEmail(message); }
}
```

Tightly coupled: SMS or push means editing `NotificationService`.

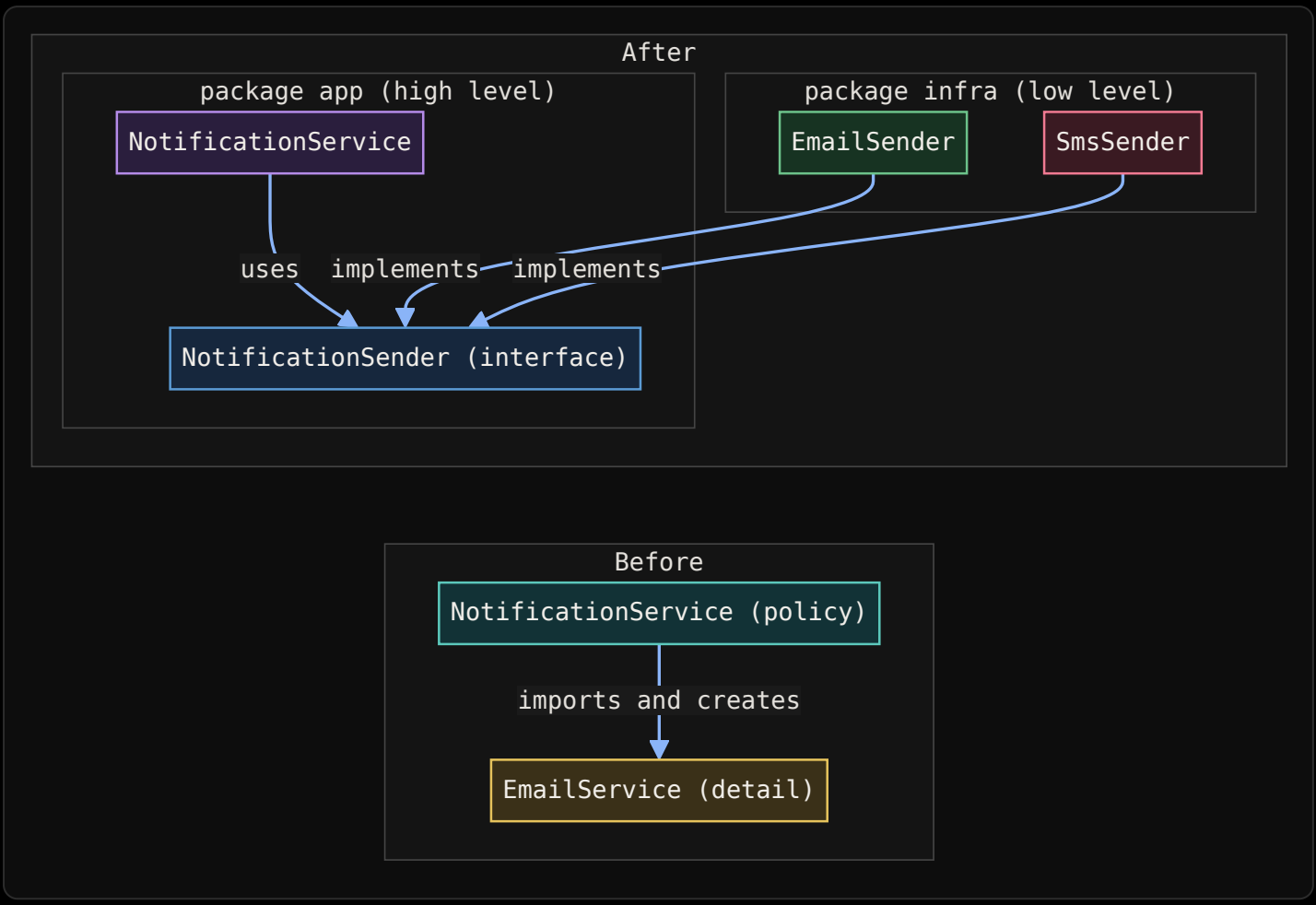
After, in four steps: (1) **introduce an abstraction**, `NotificationSender`; (2) **low-level modules implement it** (`EmailSender`, `SmsSender`) — details depend on the abstraction; (3) **the high-level module depends only on it**, via its constructor; (4) **test** with different senders, including a recording fake.

Note the packages: the interface lives in `app`, **next to the policy using it**, not in `infra` with the implementations. The most-missed part of DIP: **the high-level side owns the abstraction**, shaped by what the policy needs, not by what email offers. `app` imports nothing from `infra`; `infra` imports `app`.

```java
// File: code/lld/ch04/dip/app/NotificationSender.java — the abstraction lives with the policy (app)
public interface NotificationSender { void send(String recipient, String message); }

// File: code/lld/ch04/dip/app/NotificationService.java — imports nothing from infra
public class NotificationService {
    private final NotificationSender sender;
    public NotificationService(NotificationSender sender) { this.sender = Objects.requireNonNull(sender); } // constructor injection
    public void notifyUser(String userId, String message) {
        if (message.isBlank()) throw new IllegalArgumentException("empty notification");
        sender.send(userId, message);
    }
}

// File: code/lld/ch04/dip/infra/EmailSender.java — the detail depends on the abstraction (arrow points UP into app)
public class EmailSender implements NotificationSender {
    @Override public void send(String recipient, String message) { System.out.println("[email -> " + recipient + "] " + message); }
}
// SmsSender prints "[sms -> recipient] message"; RecordingSender (a test double: no mail server needed)
// stores recipient + ":" + message in a list returned by sent()
```



In "After", every inter-package arrow points **from the detail into the policy** — the inversion.

DIP in practice: four real-world techniques

1. Dependency Injection (DI) — give a class its dependencies from outside instead of letting it create them:

Kind	How	Use when
Constructor injection	Pass dependencies to the constructor	Default choice: required dependencies, immutable fields, obvious in the signature
Setter injection	Call `setX(...)` after construction	Optional dependencies with a sensible default
Method (parameter) injection	Pass the dependency to the one method that needs it	It varies per call (e.g. if the format changed on every call, `invoice.generate(generator)` instead of the constructor injection used in 4.7)

Typical web case: a `DataService` receives a `DataRepository` interface in its constructor instead of building a MySQL repository, so another database means passing a different implementation, no edit. The class doing all the `new`-ing is the **composition root** — here each `Main`, the one place knowing the concrete classes.

2. Service Locator — a central registry hands out services by name or type; the high-level class depends on a `Locator` abstraction and asks it at run time.

```
// File: code/lld/ch04/dip/locator/ — PaymentService { void processPayment(BigDecimal amount); }
// PayPalService prints "Processing payment via PayPal: $" + amount
public interface Locator { PaymentService paymentService(String name); }   // high level depends on this
public class ServiceLocator implements Locator {                          // central registry
    private final Map<String, PaymentService> services = new ConcurrentHashMap<>();
    public void register(String name, PaymentService service) { services.put(name, service); }
    @Override public PaymentService paymentService(String name) {
        PaymentService s = services.get(name);
        if (s == null) throw new NoSuchElementException("No payment service named " + name);
        return s;
    }
}
public class CheckoutFlow {          // asks at run time: its real dependency (PaymentService) is hidden inside
    private final Locator locator;
    public void pay(String serviceName, BigDecimal amount) {
locator.paymentService(serviceName).processPayment(amount); }
}
```

This follows DIP: `CheckoutFlow` names no concrete payment class and gets the `Locator` *interface* via its constructor, so even the registry is replaceable (e.g. a test locator). But note the criticism (Martin Fowler compares the approaches; Mark Seemann calls Service Locator an anti-pattern — a common solution causing more problems than it solves): **real** dependencies hide inside method bodies, invisible from the constructor, and a missing registration fails only at run time. Prefer constructor injection; use a locator mainly at framework boundaries.

3. Inversion of Control (IoC) containers — frameworks like **Spring** (Java) and **.NET**'s built-in container (Microsoft's platform) create objects and inject dependencies. In Spring a single-constructor class is injected automatically (`@Autowired`, an annotation marking where Spring injects, is optional for a single constructor since Spring 4.3; a **bean** is a Spring-managed object; the **classpath** is where Java looks for classes and libraries):

```
// Spring version -- NOT part of this chapter's compiled code (needs Spring on the classpath).
@Service
public class OrderService {
    private final PaymentService payments;          // an interface
    public OrderService(PaymentService payments) {  // Spring finds the one PaymentService bean
        this.payments = payments;
    }
    public void placeOrder(BigDecimal amount) { payments.processPayment(amount); }
}
```

Without Spring it is one line in `main`: `new OrderService(new StripePayments())`. **The framework is a wiring convenience, not the principle**: `OrderService` was inverted the moment it accepted an abstraction.

4. Plug-in architecture — the core defines an abstraction; plug-ins implement it and install without changing the core (a media player's `AudioDecoder`; MP3 and FLAC as plug-ins).

```java
// File: code/lld/ch04/dip/plugin/ — contract owned by the core
public interface AudioDecoder { String extension(); void decode(String fileName); }
// Mp3Decoder ("mp3", prints "Decoding MP3 file: ..."), FlacDecoder ("flac", "Decoding FLAC file: ...")
public class MediaPlayer {           // the core: never edited for new formats
    private final Map<String, AudioDecoder> decoders = new HashMap<>();
    public void install(AudioDecoder decoder) { decoders.put(decoder.extension(), decoder); }
    public void play(String fileName) {
        String ext = fileName.substring(fileName.lastIndexOf('.') + 1);
        AudioDecoder decoder = decoders.get(ext);
        if (decoder == null) { System.out.println("No plug-in installed for ." + ext); return; }
        decoder.decode(fileName);
        System.out.println("Playing audio...");
    }
}

// File: code/lld/ch04/dip/Main.java — the COMPOSITION ROOT (abridged)
new lld.ch04.dip.before.NotificationService().notifyUser("Your order shipped");
new NotificationService(new EmailSender()).notifyUser("u-42", "Your order shipped");
new NotificationService(new SmsSender()).notifyUser("u-42", "Your order shipped");
var recorder = new RecordingSender();
new NotificationService(recorder).notifyUser("u-7", "Test message");    // then prints recorder.sent()
locator.register("PayPal", new PayPalService());
new CheckoutFlow(locator).pay("PayPal", new BigDecimal("100.00"));
// player installs Mp3Decoder and FlacDecoder, then plays song.mp3, concert.flac, voice.ogg
```

Output:

```
=== 1. Before: NotificationService builds EmailService itself ===
Sending email: Your order shipped
=== 2. After: inject any NotificationSender ===
[email -> u-42] Your order shipped
[sms -> u-42] Your order shipped
test double captured: [u-7:Test message]
=== 3. Service locator ===
Processing payment via PayPal: $100.00
=== 4. Plug-in architecture ===
Decoding MP3 file: song.mp3
Playing audio...
Decoding FLAC file: concert.flac
Playing audio...
No plug-in installed for .ogg
```

DI vs plug-in architecture: both hand an implementation to a core, at different scales. **DI** is small-scale — how one class gets its collaborators. **Plug-in architecture** is system-level extensibility — whole modules, often from other teams, added independently and frequently discovered at run time (Java's `ServiceLoader`, OSGi — a Java module system built for plug-ins, IDE plug-in APIs).

DIP vs DI vs IoC — do not mix them up

Term	What it is	Kind	Example
Dependency Inversion (DIP)	Depend on abstractions; the policy owns the abstraction	A **principle** (what good structure looks like)	`NotificationService` depends on `NotificationSender`
Dependency Injection (DI)	Pass dependencies in from outside instead of creating them	A **technique** (one way to achieve DIP)	`new NotificationService(new SmsSender())`
Inversion of Control (IoC)	The framework calls your code, not the other way round ("don't call us, we'll call you" — the Hollywood Principle)	A broad **idea**; DI is one form of it	Spring creating beans; a servlet container (a Java web server such as Tomcat) calling `doGet`; JUnit calling your `@Test` methods
IoC container	A framework that performs DI for you	A **tool**	Spring `ApplicationContext`, Google Guice (a lightweight DI library)

They differ: **injection without inversion** — `new OrderService(new MySqlDatabase())` with a concrete `MySqlDatabase` parameter type is injected, yet the policy still depends on a detail; **inversion without a framework** — everything in this chapter: constructor + interface + a wiring `main`.

Stable vs volatile dependencies

DIP targets **volatile** dependencies — things that change often or that you want to fake (databases, networks, clocks, third-party APIs i.e. other companies' services, varying business rules). Depending directly on **stable** concrete classes is fine: `String`, `List`, `LocalDate`, `Math`; nobody wraps `String` in an interface. (Head First's DIP guidelines, like "no variable should hold a reference to a concrete class", are admitted ideals you will break; stable vs volatile is how you decide when.)

Why it matters: flexibility (swap implementations without touching business logic); maintainability (detail changes stay in the detail); testability (pass fakes and mocks — fakes that also record how they were called; no real database or mail server in unit tests).

Relation to the others: SRP — injected collaborators keep each class on its job; creating and configuring dependencies becomes the composition root's job; **OCP** — new implementations extend abstraction-dependent code without edits; **LSP** — every implementation must be a proper substitute, or injection is dangerous; **ISP** — the policy's abstraction should be the **smallest** role it needs.

How it works (step by step)

1. Find a high-level class naming or creating a volatile low-level class. 2. Write what it **needs**, in its own words (`send(recipient, message)`). 3. Make that an interface **in the high-level package**. 4. Make the low-level class implement it. 5. Inject the implementation (constructor by default). 6. Move all `new` calls for concrete classes to the composition root (`main`, a factory, or a DI container).

Trade-offs

You gain	You pay
Policy independent of technology	An interface per volatile dependency
Easy unit tests with fakes	Wiring code (or a framework and its learning curve)
Swappable implementations	Indirection: "go to implementation" takes an extra hop

When not: stable classes (`String`, collections, value objects); an interface with exactly one implementation forever and no test double; tiny scripts.

Common mistakes

- The interface in the **low-level** package (`mysql.MySqlRepository` and `mysql.Repository`): the policy still depends on the detail's package.
- An interface shaped by one vendor's API (`sendSmtpMessage(...)`) rather than the policy's need (`send(...)`), so changing vendors changes the interface.
- Breaking the **second** clause ("abstractions should not depend on details"): methods returning `java.sql.ResultSet` or throwing `javax.mail.MessagingException` drag detail types into the policy; use the policy's own types (`Optional<Order>`, a box that may hold an order; a domain exception, an error type named in business terms).
- "We use Spring, so we follow DIP" — Spring injects concrete classes just as happily.
- `new ConcreteThing()` inside a class "just this once".

Real-world examples

- **JDBC**: code depends on `java.sql.Connection` / `DataSource`; MySQL, PostgreSQL, Oracle drivers implement them; `DriverManager` finds drivers via `ServiceLoader`.
- **SLF4J**: the app depends on the logging facade; Logback or Log4j 2 is the plugged-in detail.
- **Spring** and **Jakarta EE CDI** (enterprise Java's built-in injection): IoC containers doing constructor injection.

- **JPA** (Java's standard object-to-SQL API): code uses `EntityManager`; Hibernate or EclipseLink implements it.
- **Hexagonal architecture (ports and adapters)**: the core defines *ports* (interfaces); *adapters* for HTTP, databases and queues (message queues, see System Design Fundamentals) implement them — DIP applied to a whole service.

In the interview

30-second answer: "DIP says business logic should not depend on technical details; both depend on an abstraction, and the business side owns that abstraction. So `NotificationService` depends on a `NotificationSender` interface in its own package, and email and SMS senders implement it. I inject the implementation through the constructor and wire everything in `main`. That makes the service testable with a fake and lets me swap providers. DI is the technique; DIP is the principle; Spring is just a tool that does the wiring."

- *"Dependency inversion vs dependency injection?"* — Inversion is the principle (depend on abstractions); injection one technique (pass dependencies in). Injecting a concrete class is injection without inversion.
- *"Who owns the interface?"* — The high-level module that uses it.
- *"Is Service Locator OK?"* — It satisfies DIP but hides dependencies and fails late; I prefer constructor injection.

Self-check

1. **Q:** What is "inverted"? **A:** The source-code dependency: the detail implements an abstraction owned by the policy, instead of the policy importing the detail.
2. **Q:** `new OrderService(new MySqlDatabase())` with a `MySqlDatabase` parameter — DIP? **A:** No; injection of a concrete class, the policy still depends on the detail.
3. **Q:** A composition root? **A:** The one place (usually `main` or container config) where concrete classes are created and wired.
4. **Q:** Put `String` behind an interface? **A:** No; it is stable — DIP is for volatile dependencies.

TL;DR

Business code depends on interfaces it owns; details implement them; `main` (or a container) wires them. DI is the how, DIP the why, IoC containers the tool.

4.11 Refactoring one class through all five principles

In real code the principles seldom appear one at a time; they come **in a fixed sequence**, each exposing the need for the next. Here one order-checkout class — it validates, prices, saves and emails, quietly grown over two years — is refactored five times.



Figure — one checkout class, five steps: SRP, OCP, LSP, ISP, DIP, each creating the situation the next addresses (course diagram)

Step 0: the starting class

It runs correctly, yet each principle objects to something.

```
// File: code/lld/ch04/checkout/step0/ — record CustomerOrder(String orderId, String customerType, BigDecimal amount, String email)
public class OrderService {          // STEP 0 -- the class that grew quietly for two years. It works.
    public void process(CustomerOrder order) {
        // 1. Validation  (business-rules owner)
        if (order.amount().signum() <= 0) { System.out.println("Rejected: amount must be positive"); return; }
        // 2. Pricing, one branch per customer type  (finance)
        BigDecimal total = order.amount();
        if (order.customerType().equals("member")) total = total.multiply(new BigDecimal("0.90"));
        else if (order.customerType().equals("staff")) total = total.multiply(new BigDecimal("0.70"));
        total = total.setScale(2, RoundingMode.HALF_EVEN); // money: always 2 decimals
        // 3. Storage  (database team)
        System.out.println("INSERT INTO orders VALUES ('" + order.orderId() + "', " + total + ")");
        // 4. Telling the customer  (marketing)
        System.out.println("Emailing " + order.email() + ": your order total is " + total);
    }
}
```

Step 1: Single Responsibility — split by actor

Section of `process`	Requested by
Validation	The business-rules owner
Pricing	Finance
The `INSERT` statement	The database team
The email wording	Marketing

Four groups, one method: rewording the email redeploys pricing. Split by actor into `OrderValidator` (`isValid`: amount > 0), `PriceCalculator`, `OrderRepository` (`save` prints the `INSERT`) and `Notifier` (`send` prints the email):

```java
// File: code/lld/ch04/checkout/step1/PriceCalculator.java — actor: finance. SRP met, but the branch remains -> OCP next
public BigDecimal total(CustomerOrder order) {
    BigDecimal total = switch (order.customerType()) {
        case "member" -> order.amount().multiply(new BigDecimal("0.90"));
        case "staff" -> order.amount().multiply(new BigDecimal("0.70"));
        default -> order.amount();
    };
    return total.setScale(2, RoundingMode.HALF_EVEN);
}
// File: code/lld/ch04/checkout/step1/Checkout.java — coordinates the four; still news them up itself
private final OrderValidator validator = new OrderValidator();   // likewise calculator, repository, notifier
public void process(CustomerOrder order) {
    if (!validator.isValid(order)) { System.out.println("Rejected: amount must be positive"); return; }
    BigDecimal total = calculator.total(order);
    repository.save(order, total);
    notifier.send(order, total);
}
```

Step 2: Open/Closed — one class per pricing rule

The branch on customer type survived the split, so a "student" rate would edit working code. Each rate gets a class behind `PricingRule`; from here the order carries no type string — the caller picks the rule:

```java
// File: code/lld/ch04/checkout/Order.java
public record Order(String orderId, BigDecimal amount, String email) { }

// File: code/lld/ch04/checkout/pricing/PricingRule.java
/**
 * STEP 2 (OCP): one class per rate; a new rate is a new class.
 * CONTRACT every implementation must keep (STEP 3, LSP):
 *   precondition : amount >= 0
 *   postcondition: result >= 0, rounded to 2 decimals   (a total is never negative)
 */
@FunctionalInterface
public interface PricingRule { BigDecimal apply(BigDecimal amount); }
// RegularPrice: amount.setScale(2, HALF_EVEN); MemberDiscount: x 0.90; StaffDiscount: x 0.70 (each rounded HALF_EVEN)

// File: code/lld/ch04/checkout/step2/PriceCalculator.java — no branch; never learns how many rates exist
public BigDecimal total(Order order, PricingRule rule) { return rule.apply(order.amount()); }
```

A student rate is now one new class, no edits.

Step 3: Liskov Substitution — every rule keeps the promise

Several implementations of one interface raise the question whether each truly substitutes. Finance wants a **fixed** discount of 20; a naive version turns a 10 order into **−10** with no exception — saved, and perhaps later credited to a customer who never paid it. This is 4.8's postcondition/invariant rule: every pricing rule must guarantee **a never-negative total**. Fix: state the promise (in the `PricingRule` Javadoc), honour it in every rule, and verify with one contract test over every implementation:

```java
// File: code/lld/ch04/checkout/pricing/CarelessFixedDiscount.java — LSP VIOLATION kept for the demo
@Override public BigDecimal apply(BigDecimal amount) { return amount.subtract(off).setScale(2,
RoundingMode.HALF_EVEN); }

// File: code/lld/ch04/checkout/pricing/FixedDiscount.java — STEP 3 fix (constructor rejects off < 0)
@Override public BigDecimal apply(BigDecimal amount) {
    return amount.subtract(off).max(BigDecimal.ZERO).setScale(2, RoundingMode.HALF_EVEN);
}

// File: code/lld/ch04/checkout/step3/PricingRuleContract.java — ONE test against the interface, run for EVERY
implementation
private static final List<BigDecimal> SAMPLE_AMOUNTS = List.of(
        new BigDecimal("0.00"), new BigDecimal("10.00"), new BigDecimal("100.00"), new BigDecimal("250.00"));
public static boolean holds(PricingRule rule) {
    for (BigDecimal amount : SAMPLE_AMOUNTS) {
        BigDecimal total = rule.apply(amount);
        if (total.signum() < 0) {
            System.out.println("  " + rule.getClass().getSimpleName() + " broke the contract: apply(" + amount +
") = " + total);
            return false;
        }
    }
    System.out.println("  " + rule.getClass().getSimpleName() + " keeps the contract");
    return true;
}
```

Step 4: Interface Segregation — the checkout needs a save method, not a repository

Repositories accumulate methods (`save`, `findById`, `delete`, `listAll`) as other parts ask, but the checkout only **saves**; typed against the full repository, each new reporting query is one more method it depends on without calling. So describe the **role** it needs — an order saver, one method wide — and let the real repository play that role and others:

```java
// File: code/lld/ch04/checkout/fin/
@FunctionalInterface public interface OrderSaver { void save(Order order, BigDecimal total); }  // owned by the
checkout
public interface OrderFinder { Optional<Order> findById(String orderId); List<Order> listAll(); } // reporting
role; checkout never sees it
@FunctionalInterface public interface OrderNotifier { void send(Order order, BigDecimal total); }
public class SqlOrderRepository implements OrderSaver, OrderFinder {   // grows, but plays two separate roles
    private final List<Order> table = new ArrayList<>();                // stands in for a real DB table
    @Override public void save(Order order, BigDecimal total) {
        table.add(order);
        System.out.println("  INSERT INTO orders VALUES ('" + order.orderId() + "', " + total + ")");
    }
    // findById filters table by id; listAll copies it
}
```

Step 5: Dependency Inversion — the entry point decides

The step-1 `Checkout` constructs its collaborators, so it names concrete classes (SQL repository, email notifier) and cannot run in a test without a database and mail server. Pass the abstractions in; the checkout names no concrete class and the entry point chooses:

```java
// File: code/lld/ch04/checkout/fin/ — InMemoryOrders implements OrderSaver (one method, because the role is
// one method wide): stores orderId + "=" + total, prints "  kept <id> in memory", saved() returns them.
// EmailNotifier prints "  emailing <email>: total <total>"; SilentNotifier prints "  (test run, no email sent)".

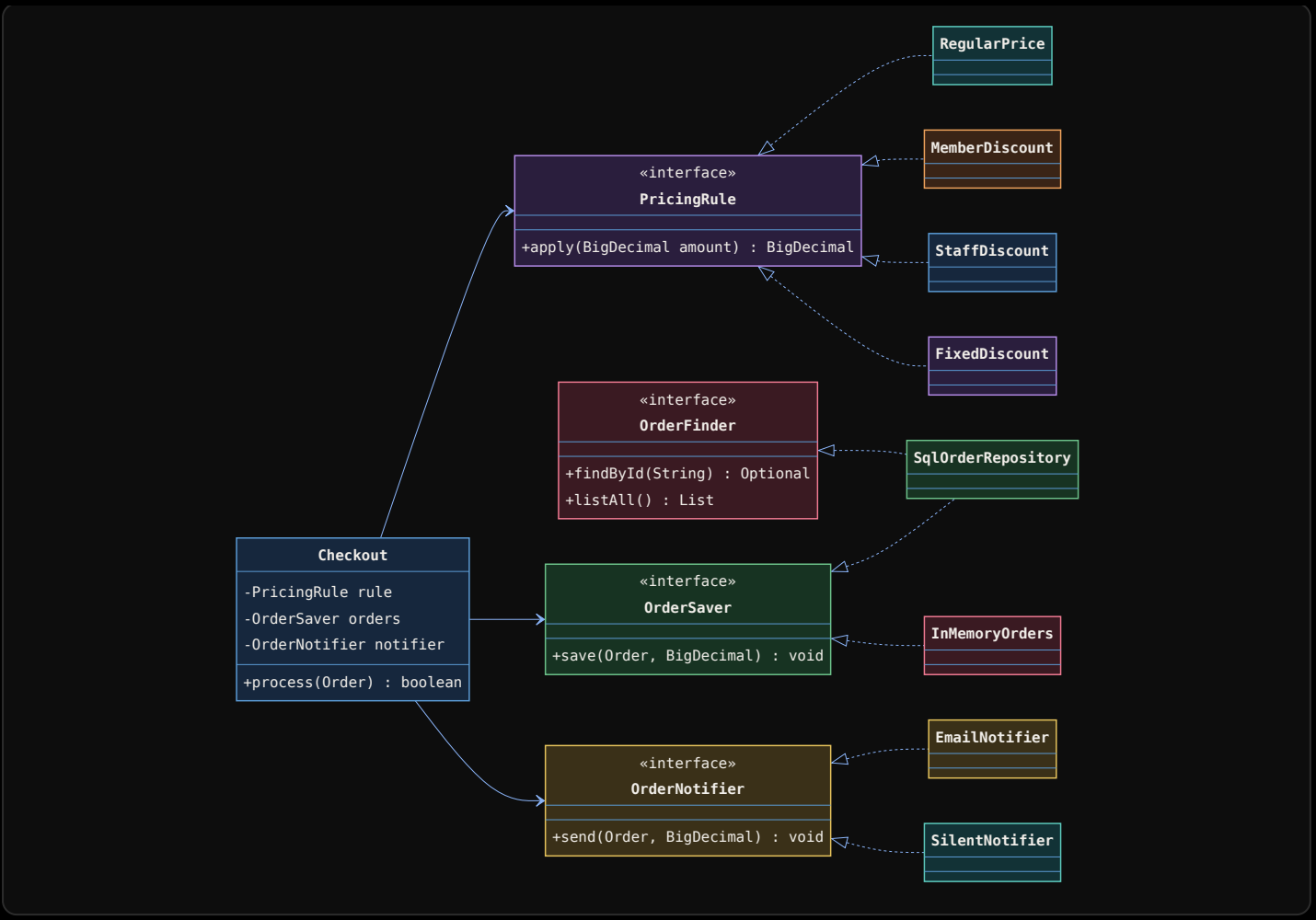
// File: code/lld/ch04/checkout/fin/Checkout.java — STEP 5 (DIP): every volatile collaborator arrives from outside
public class Checkout {
    private final PricingRule rule;
    private final OrderSaver orders;
    private final OrderNotifier notifier;          // all three requireNonNull in the constructor
    public Checkout(PricingRule rule, OrderSaver orders, OrderNotifier notifier) { ... }
    public boolean process(Order order) {
        // Validation has not varied yet, so it stays a plain check (see "What we did not do").
        if (order.amount().signum() <= 0) { System.out.println("  rejected: amount must be positive"); return
false; }
        BigDecimal total = rule.apply(order.amount());
        orders.save(order, total);
        notifier.send(order, total);
        return true;
    }
}

// File: code/lld/ch04/checkout/Main.java (abridged; TWENTY = 20.00)
// Step 0: legacy OrderService on A-1 "regular" 100.00 and A-2 "staff" 100.00; Step 1: step-1 Checkout on A-2
// Step 2: calc.total(A-3 100.00, r) for RegularPrice, MemberDiscount, StaffDiscount
// Step 3: careless and correct FixedDiscount(TWENTY) on A-4 10.00; then PricingRuleContract.holds over
//         Regular, Member, Staff, FixedDiscount(TWENTY), CarelessFixedDiscount(TWENTY), printing the passing
count
var a9 = new Order("A-9", new BigDecimal("200.00"), "ada@example.com");
var db = new SqlOrderRepository();
new Checkout(new MemberDiscount(), db, new EmailNotifier()).process(a9);          // production wiring
System.out.println("  reporting side can still find it: " + db.findById("A-9").isPresent());
var fake = new InMemoryOrders();
new Checkout(new MemberDiscount(), fake, new SilentNotifier()).process(a9);       // test wiring, same class
System.out.println("  test double recorded " + fake.saved());
new Checkout(amount -> amount, (o, t) -> System.out.println("  saved " + o.orderId()),
        (o, t) -> System.out.println("  notified " + o.email())).process(a9);    // lambda wiring
new Checkout(new RegularPrice(), fake, new SilentNotifier())
        .process(new Order("A-10", BigDecimal.ZERO, "x@example.com"));
```

Output:

```
=== Step 0: the original class ===
INSERT INTO orders VALUES ('A-1', 100.00)
Emailing ada@example.com: your order total is 100.00
INSERT INTO orders VALUES ('A-2', 70.00)
Emailing grace@example.com: your order total is 70.00
=== Step 1: SRP -- split by actor (same output) ===
INSERT INTO orders VALUES ('A-2', 70.00)
Emailing grace@example.com: your order total is 70.00
=== Step 2: OCP -- one class per pricing rule ===
  RegularPrice -> 100.00
  MemberDiscount -> 90.00
  StaffDiscount -> 70.00
=== Step 3: LSP -- every rule must keep 'total >= 0' ===
  careless FixedDiscount(20) on 10 -> -10.00
  correct  FixedDiscount(20) on 10 -> 0.00
  RegularPrice keeps the contract
  MemberDiscount keeps the contract
  StaffDiscount keeps the contract
  FixedDiscount keeps the contract
  CarelessFixedDiscount broke the contract: apply(0.00) = -20.00
  4/5 rules are safe substitutes
=== Steps 4+5: ISP + DIP -- same Checkout, two wirings ===
Production wiring:
  INSERT INTO orders VALUES ('A-9', 180.00)
  emailing ada@example.com: total 180.00
  reporting side can still find it: true
Test wiring, same Checkout class:
  kept A-9 in memory
  (test run, no email sent)
  test double recorded [A-9=180.00]
Lambda wiring (roles are functional interfaces):
  saved A-9
  notified ada@example.com
  rejected: amount must be positive
```

One unmodified `Checkout` ran against production collaborators, test doubles and lambdas, **without being edited or subclassed** — what the five steps bought.



Reading the five steps back

Step	The question it asked	What changed
SRP	Who might request changes here?	The single class split into four, one per actor
OCP	What breaks when a new case shows up?	Each pricing branch turned into its own class
LSP	Does each rule work when called through the interface?	All rules guarantee the total never drops below zero
ISP	Is the caller tied to more than it calls?	Checkout depends on one save method rather than a whole repository
DIP	Who decides which concrete class is used?	The entry point decides, not the checkout

The sequence has a reason: SRP first, because the varying point stays hidden until each class has one job; OCP then introduces an interface; LSP matters once it has a second implementation; ISP when new callers (reporting) appear; DIP last, because inversion needs abstractions to point at. Each step sets up the next — why the five are learned as a group.

What we did not do

- **Not every principle on every class:** `Order` stays a plain record without an interface — nothing made it vary.
- **Not closed against every change:** a new validation rule still edits `Checkout`, fine while validation is stable; the third validation request is the day it gets pricing's treatment.
- **Not everything injected:** `Order` and `BigDecimal` arithmetic are stable, used directly.

Pre-emptively applying all five everywhere turns a small clear program into a confusing one. **Each step was triggered by an actual change request, never by a checklist.**

Self-check

1. **Q:** Why SRP before OCP here? **A:** Until pricing had its own class, the varying part (the type branch) was buried in a four-job method.
2. **Q:** Step 3's contract? **A:** Every `PricingRule` returns ≥ 0 for any amount ≥ 0.
3. **Q:** Why is `OrderSaver` one method wide? **A:** The checkout only saves; the full repository would tie it to reporting methods it never calls.
4. **Q:** Proof the design worked? **A:** One `Checkout` ran under three wirings (production, test doubles, lambdas) unedited.

TL;DR

SRP → OCP → LSP → ISP → DIP, each triggered by a real change. Proof: one class, several wirings, zero edits.

4.12 SOLID and design patterns

Most classic patterns (Chapters 8–11) exist because of these principles: **the principles say what good structure looks like; the patterns are named, reusable ways to produce it.**

Pattern	Principle(s) it serves	How
Strategy	OCP, DIP	A varying algorithm behind an interface (`InvoiceGenerator`, `PricingRule`)
Template Method	OCP	A fixed skeleton in a base class; subclasses fill in hook steps
Decorator	OCP, SRP	Adds behaviour by wrapping, without modifying the original class
Adapter	DIP, ISP	Lets a caller keep depending on the interface it already owns while a foreign class is plugged in
Factory Method / Abstract Factory	DIP, OCP	Keeps callers away from concrete constructors; the `switch` lives in one place
Observer	OCP, DIP	New reactions are new subscribers; the subject depends only on a listener interface
Composite	LSP, OCP	Leaves and groups share one interface, so callers treat them the same
Proxy	SRP, OCP	Adds access control, caching or laziness (creating things only when first needed) in a separate class
Command	SRP, OCP	Each request is its own object; new commands are new classes
Visitor	OCP (for operations)	Easy to add operations, hard to add element types: OCP along a different axis
Null Object	LSP	A do-nothing implementation that still keeps the contract, instead of `null` checks

Head First Design Patterns states several as "OO design principles": **encapsulate what varies**, **favour composition over inheritance**, **program to an interface, not an implementation**, **classes should be open for extension but closed for modification** (Decorator chapter), **depend upon abstractions, do not depend upon concrete classes** (Factory chapter's DIP), **a class should have only one reason to change** (Iterator/Composite chapter). *Clean Code* covers SRP and "organising for change" in its classes chapter; "isolating from change" is DIP by another name.

4.13 SOLID in the design interview

Keeping the design honest: five checks over your class diagram

In an OOD round SOLID is most useful as **five quick checks over the diagram you just drew** (about five minutes): an "and" in a class's one-sentence responsibility (S), a `switch` over a type (O), a subclass refusing a parent method — re-read inheritance arrows at the end, they get added late (L), empty method bodies in a wide interface (I), a high-level class naming a low-level one, e.g. `ParkingLot` constructing a `CreditCardTransaction` (D). Calibration: **two findings is a good pass, five means rework, zero means you did not look**; S and O earn most credit, and SOLID is a set of checks, not a score — no single-implementer interfaces, no splitting beyond the one-sentence test. The full checklist, the signal figure and a worked parking-lot pass are in §1.8.

Naming a design problem out loud

Interviewers rarely ask for OCP's definition. They ask for a parking lot, a rate limiter (caps requests per time window) or a file store and judge whether you can **explain the reasons behind your design's shape** — separating memorised acronyms from use. Weighed most important first:

1. **Can you spot and name a problem in the code in front of you** — an ever-growing `if/else` on a type field — in one crisp sentence?
2. **Can you defend a choice?** "SOLID says so" earns nothing; "pricing rules change every quarter and I don't want each change to reopen the checkout" earns a lot.
3. **Do you know when to hold back?** The senior marker: adding abstractions is easy; choosing which to skip is rarer.

A sentence pattern that works — three things, in order: (1) **what you see** — a specific fact about the code; (2) **what it will cost** — the future change it makes painful; (3) **what you would do, and what that costs you** — the part most people omit, and what makes an answer sound experienced rather than memorised.

Example: a `ReportGenerator` that formats a report, writes it to disk and emails it.

- Weak: "It breaks SRP since it does three things; I'd make three classes." Correct, but sounds like last night's reading.
- Strong: "Three different owners can request changes here: marketing controls the wording, the platform team controls where files land, and the email owners control delivery. That means a copy edit redeploys the file-writing code. I'd pull formatting apart from delivery, likely into a formatter and a sink (the destination the output is sent to). It costs me two more classes and a little wiring, which pays off because the wording changes frequently." Actors, pain, fix, price, and a decision.

Observation before principle. "`Reporting` implements nine methods but calls only two of them, which is an interface segregation problem" shows you noticed something; opening "This violates ISP because…" sounds like hunting for a place to use the acronym. Still name the principle at the end — it hands the interviewer a label for the write-up.

"Would you always do that?" checks you know your suggestion's price; rarely a trap, and the answer is almost never "yes":

- *Splitting classes:* "No. If the two parts would always be edited together, I've just created an extra file. I split when two different people would ask for two different changes."
- *Extension points:* "No. I keep the `if/else` until a second or third case actually appears. An extension point built in advance is usually wasted effort and makes the code harder to follow in the meantime."
- *Interfaces:* "Only where behaviour varies or a test needs a substitute. An interface with a single implementation and no fake is just one more file."

Treating a principle as unconditional is the quickest way to show you have never applied it.

Three things that cost marks: reciting definitions untied to the whiteboard code; **applying everything at once** (five interfaces for a forty-line problem looks inexperienced — real systems earn their abstractions); **naming the wrong principle confidently** (mixing up cohesion and coupling reveals second-

hand vocabulary; when unsure, describe in plain words — an accurate unnamed description beats a wrong name).

A one-minute drill for any class you own: (1) Who can ask me to change it, and which parts belong to whom? (2) What is the likely next feature, and would I edit this class for it? (3) Which dependencies could I not replace in a test, and does it matter? Practising on your own code beats rereading definitions; the interview asks it of code you have never seen.

> **In the interview:** describe before you label, and end every recommendation with its cost, for example: "I'd move delivery into its own class; that costs a file and some wiring, and in return a wording change no longer touches file handling."

4.14 Common questions about SOLID (with short answers)

Give the two-line version and stop; expand only if asked — long unprompted definitions eat design time.

1. SRP vs ISP? *Whom* each guards: **SRP the class** doing the work (more than one reason to change?), **ISP the caller** at a boundary (depends on methods it never calls?). `Printer` with `print` / `scan` / `fax` is ISP for a basic printer; a `ReportService` that formats and emails is SRP with no interface in sight. SRP splits classes, ISP splits interfaces; either breaks without the other.

2. When would you not apply these principles? Have a concrete answer — it separates candidates: nothing has varied yet (an extension point for a case that never arrives adds indirection for nothing); small or throwaway code; an arbitrary split (you cannot name the two groups requesting changes); hurting readability (five four-line files vs one twenty-line file). *Framing:* each principle buys **flexibility later** with **simplicity now**; if "later" never comes, the price was wasted.

3. Is SOLID still relevant, given the criticism? Yes, and knowing the criticism helps. The principles came from large OO systems and are often taught as absolutes to people building small ones, yielding layers of single-implementation interfaces; some argue plainer advice (small functions, explicit dependencies) captures most of the benefit. What outlasts it is the **vocabulary and the diagnosis** — a compact way to explain why code will resist change, in any language or paradigm. *Short:* valuable as diagnostics, misleading as rules.

4. Dependency inversion vs dependency injection? Inversion is the **principle** (depend on abstractions); injection one **technique** (supply dependencies from outside). Injecting a concrete class is injection without inversion; a hand-written constructor with no framework is inversion without a container; Spring only automates wiring (4.10).

5. An LSP violation? Prefer one from your work; else Square/Rectangle, explaining the rule, not just the story: `Rectangle` lets callers set width and height separately, `Square` cannot, so `Rectangle` code computes a wrong area silently. A subclass must not require more, guarantee less, or break an assumption; an exception is merely the most obvious break.

6. Does OCP mean never editing a file? No: one more variation *of a supported kind* is added without touching existing code; bug fixes, refactorings and extension-point changes still edit. Closure is relative to a kind of change (4.7's invoice: closed to formats, open to discount rules — the right trade then).

7. Composition over inheritance, always? A guideline: inherit when the child truly is a kind of the parent and keeps every promise; compose when behaviour differs or a subclass would reject an inherited method (an override that throws, does nothing, or says "don't call this on X").

8. How do the principles relate to patterns? Many patterns are the principles made concrete: **Strategy** = OCP for a varying algorithm; **Adapter** keeps a caller on the interface it knows; **Decorator** adds behaviour without editing the wrapped class; **Factory** hides concrete constructors (DIP). See 4.12.

9. Which principle is violated most often? Two defensible answers: **SRP**, because classes grow by small, sensible-looking additions (nobody sets out to write a two-thousand-line service); and **ISP in reverse** (over-splitting) — an interface per method, meeting the rule's wording while adding names and removing no dependency. Citing the second shows you have watched the principles misused, not just ignored.

> **Interview tip:** if invited deeper, expand "when would you not apply this?" and "dependency inversion vs dependency injection" — most candidates stay vague on both. The strongest SOLID answers **admit a limit**: saying where a principle stops paying off is what sounds like real maintenance experience.

4.15 Over-engineering: the warnings in one place

Warning sign	Principle misapplied	What to do instead
Classes named `XHelper`, `XUtils`, `XManager2` after a split	SRP	Split by actor; each new class needs a name that stands alone
Six small classes that always change in the same commit	SRP (over-split) — *Shotgun Surgery*	Merge them back
An interface + factory + registry for a single implementation	OCP — *Speculative Generality*	Write the `if`; refactor at the second or third case (rule of three)
A subclass that "works" by silently doing nothing	LSP	Raise a better parent or use composition
One interface per method; the same object passed three times	ISP (over-split)	Group by caller into role interfaces
An interface in front of `String`-like stable classes	DIP	Invert only volatile dependencies
A DI container for a 200-line program	DIP	Wire by hand in `main`

YAGNI ("You Aren't Gonna Need It", from Extreme Programming — an agile method of small frequent releases and heavy testing) is the counterweight to all of SOLID: build nothing for requirements you do not have. SOLID says **what** a good seam looks like; YAGNI says **when** to cut it: when a real change asks.

Chapter self-check

1. **Q:** Each principle in one line? **A:** SRP: one reason (actor) to change. OCP: add behaviour by adding code, not editing. LSP: subtypes substitutable for their base types. ISP: no caller depends on methods it does not use. DIP: policy and details both depend on abstractions owned by the policy.
2. **Q:** A `switch (vehicleType)` computing parking fees? **A:** OCP: fees behind a `ParkingRate` interface, one class per rate (Strategy), chosen at the edge.
3. **Q:** `ReadOnlyList extends ArrayList` throwing on `add`? **A:** LSP (new exception / refused bequest); prefer composition or a read-only interface.
4. **Q:** `Robot` implements `Worker` with `work()` and `eat()`? **A:** ISP: the robot must implement `eat()`; split into `Workable` and `Eater` roles by caller.
5. **Q:** `OrderService` has `private final MySqlOrderDao dao = new MySqlOrderDao();` — two problems? **A:** Tight coupling / DIP violation (names and creates a volatile detail) and untestable without MySQL; inject an `OrderSaver` interface owned by the service.
6. **Q:** Why is Square/Rectangle worse than `Penguin.fly()` throwing? **A:** It is silent: the program continues with a wrong result.
7. **Q:** Coupling vs cohesion? **A:** Cohesion: how well one class's insides belong together; coupling: how much one class depends on another.
8. **Q:** Usual order during a refactor, and why? **A:** SRP → OCP → LSP → ISP → DIP; each creates the situation the next addresses.
9. **Q:** DIP vs DI vs IoC? **A:** Principle (depend on abstractions); technique (pass dependencies in); broader idea that a framework calls your code — IoC containers perform DI.

10. **Q:** When not to add an interface? **A:** One implementation, no test double, nothing varied; or a stable dependency.

TL;DR

- **SOLID** = five names for the ways code becomes painful to change. Use them to diagnose, not as a checklist.
- **SRP**: one actor per class. Split by *who asks for changes*, not by method count. High cohesion inside, and check coupling after every split.
- **OCP**: put the part that *has* varied behind an interface; add variants as classes (Strategy). A class is closed against a specific kind of change.
- **LSP**: a subclass must accept at least as much, promise at least as much, keep invariants and history, and throw nothing new. Inheritance follows promises, not real-world categories. Prefer composition when a subclass would refuse a method.
- **ISP**: one role interface per kind of caller. Throwing methods mean too wide; the same object passed three times means too thin.
- **DIP**: policy owns the abstraction; details implement it; `main` wires them. DI is the technique, IoC containers are the tool.
- **Order**: SRP → OCP → LSP → ISP → DIP, each triggered by a real change.
- **In the interview**: observation → cost → fix → price of the fix. Always know when *not* to apply a principle.

Chapter 5 — More Design Principles

SOLID (Chapter 4) is only five of the rules good designers use. This chapter covers the rest of the everyday toolkit: rules of thumb (DRY, KISS, YAGNI); rules about how objects talk (Law of Demeter, Tell-Don't-Ask, loose coupling, the Hollywood Principle); the object-oriented principles that *Head First Design Patterns* and the Gang of Four book (*Design Patterns*, 1994, by Gamma, Helm, Johnson and Vlissides — "GoF") build every pattern on (encapsulate what varies, favour composition, program to an interface); correctness rules (fail fast, immutability, Design by Contract); **GRASP** — nine guidelines for deciding *which class should do what*; and the essentials of **Domain-Driven Design**, which decides *which classes exist and where their boundaries are* by starting from the business.

Key terms used throughout (defined once here):

Term	Meaning
Coupling	How much one class depends on (knows about) another. High coupling = a change in one forces changes in the other.
Cohesion	How closely the things inside one class belong together. High cohesion = the class does one focused job.
Abstraction	A simplified view (usually an interface or abstract class) that hides details that callers do not need.
Encapsulation	Keeping an object's data private and letting other code change it only through the object's methods.
Invariant	A fact about an object that must always be true (e.g. "balance is never negative").
Side effect	Anything a method changes besides returning a value: fields, files, the database, the screen.
Responsibility	Something a class *knows* (its data) or *does* (its behaviour).

Chapter map

#	Principle	One-line idea	Comes from
1	DRY	Each piece of knowledge has one authoritative home	Hunt & Thomas, *The Pragmatic Programmer* (1999)
2	KISS	Choose the simplest design that works	US Navy (1960); popularised by Kelly Johnson, Lockheed Skunk Works
3	YAGNI	Don't build features before they are needed	Extreme Programming, late 1990s (usually credited to Ron Jeffries)
4	Law of Demeter	Only talk to your immediate friends	Ian Holland / Karl Lieberherr, Demeter project, Northeastern University (1987)
5	Tell, Don't Ask	Tell objects what to do; don't pull their data out and decide for them	Andy Hunt & Dave Thomas (Pragmatic Programmers), building on Alec Sharp's *Smalltalk by Example* (1997); explained by Martin Fowler
6	Composition over inheritance	Prefer "has-a" to "is-a" for reuse	GoF book (1994), *Effective Java* (Joshua Bloch)
7	Program to an interface	Depend on supertypes, not concrete classes	GoF book
8	Encapsulate what varies	Isolate the part that changes	GoF book, *Head First Design Patterns*
9	Loose coupling	Interacting objects should know little about each other	*Head First Design Patterns*
10	Hollywood Principle / IoC	"Don't call us, we'll call you"	*Head First*; frameworks, IoC containers
11	Separation of Concerns	Each part of the program handles one concern	Edsger Dijkstra (1974)
12	Least Astonishment	Behave the way readers expect	User-interface and programming-language design folklore (no single author)
13	Fail fast	Detect problems early and loudly	Popularised by Jim Shore's "Fail Fast" article, *IEEE Software* (2004)
14	Immutability	Objects that never change after creation	*Effective Java* ("minimize mutability")
15	Design by Contract	Preconditions, postconditions, invariants	Bertrand Meyer, Eiffel language (term coined 1986)
16	GRASP (9 patterns)	Guidelines for assigning responsibilities	Craig Larman, *Applying UML and Patterns* (1st ed. 1997; today's nine from the 2nd ed. 2001)
17	Domain-Driven Design essentials	Model the business: bounded contexts, entities vs value objects, aggregates, domain events, repositories	Eric Evans, *Domain-Driven Design* (2003); Vaughn Vernon, *Implementing Domain-Driven Design* (2013)

How to read each section. What it is → problem → how to apply → Java *before/after* → trade-offs → when not to use → mistakes → real-world examples → interview answer → self-check → TL;DR. Listings show only the essential lines; the full runnable program is `code/lld/ch05/<topic>/Main.java` (compiled with `javac --release 21`, the Java compiler targeting Java 21), and each "Output" block is the real output of that full program.

---

1. DRY — Don't Repeat Yourself

What it is

DRY says: every piece of *knowledge* in a system — a business rule, a formula, a validation, a constant — should have **a single, unambiguous, authoritative representation**. Everything else refers to that place. From Andy Hunt and Dave Thomas's *The Pragmatic Programmer* (1999).

Analogy: a company keeps one master price list; if each shop kept a hand-written copy, a price change would reach some shops and not others.

DRY is about **knowledge**, not lines of text that happen to look the same. Two identical-looking snippets that represent *different* rules (which may change for different reasons) are *not* a DRY violation.

The problem it solves

Copies of a rule eventually **drift**: someone updates one and forgets the others, and the system gives two answers to the same question — a bug that is hard to spot because each copy looks reasonable alone. Duplication also multiplies the effort of every change.

How it works

1. Notice a rule that appears in more than one place (a regex — a text pattern such as "something@something" — a tax rate, a status check, a SQL fragment; SQL is the query language of relational databases).
2. Ask: *"If this rule changes, must all copies change together?"* If yes, it is one piece of knowledge.
3. Give it one home: a constant, a method, a value object, an enum, a config entry or a shared module, and replace every copy with a reference to it.
4. Apply the **Rule of Three** for code-level duplication (credited to Don Roberts, popularised by Martin Fowler's *Refactoring* — refactoring means improving structure without changing behaviour, Chapter 7): write it once; tolerate the second copy; extract on the third.

Java example — before/after

Two services each carry their own copy of "what a valid email is" and "what the tax is", and the copies have drifted. The *after* puts the email rule in an `EmailAddress` value object (a small immutable type — its value never changes after creation — whose constructor validates it) and the tax rule in `TaxPolicy`. Full program: `code/lld/ch05/dry/Main.java`.

```java
// BEFORE: the same rule written twice; the copies have drifted
class SignupService {
    boolean signup(String email) {
        return email != null && email.contains("@");          // copy #1 (someone relaxed it later)
    }
    BigDecimal priceWithTax(BigDecimal price) {
        return price.multiply(new BigDecimal("1.18")).setScale(2, RoundingMode.HALF_UP); // 18% hard-coded
    }
}
class ProfileService {
    boolean changeEmail(String email) {
        return email != null && email.matches("[^@\\s]+@[^@\\s]+\\.[a-z]{2,}");   // copy #2 (the stricter
original)
    }
    // ...priceWithTax with 1.18 hard-coded again
}

// AFTER: each rule has a single, authoritative home
record EmailAddress(String value) {                // the ONLY definition of "valid email"
    private static final Pattern FORMAT = Pattern.compile("[^@\\s]+@[^@\\s]+\\.[a-z]{2,}");
    EmailAddress {
        if (value == null || !FORMAT.matcher(value).matches())
            throw new IllegalArgumentException("invalid email: " + value);
    }
}
static final class TaxPolicy {                     // the ONLY definition of the tax rule
    private static final BigDecimal RATE = new BigDecimal("0.18");
    static BigDecimal withTax(BigDecimal price) {
        return price.add(price.multiply(RATE)).setScale(2, RoundingMode.HALF_UP);
    }
}
// both services now do: new EmailAddress(rawEmail).value()
```

Output:

```
== BEFORE ==
signup accepts 'bob@localhost'? true
profile accepts 'bob@localhost'? false   <-- same rule, two answers
== AFTER ==
signed up bob@example.com
email changed to bob@example.com
signup rejected: invalid email: bob@localhost
profile rejected: invalid email: bob@localhost
price 100.00 with tax = 118.00
```

`EmailAddress` is a Java **record** (a compact, immutable data class); its *compact constructor* (a record constructor without a parameter list, used to check or normalise the values) runs the check, so an invalid email cannot exist as an `EmailAddress`. Changing GST from 18% means editing one line of `TaxPolicy`.

Trade-offs

Benefit	Cost
One change point per rule; no drift	A shared abstraction couples its users together
Less code to read and test	A premature or wrong abstraction is worse than duplication
Rules get a name (self-documenting)	Extra indirection to find the real code

When not to use it

- Snippets alike **by coincidence** that will evolve separately ("max username length 30" and "max tag length 30") — merging creates false coupling.
- Across **microservices** (an application split into small, independently deployed services talking over the network): a shared domain library couples their release cycles; some duplication is the lesser evil.

- In tests, a little repetition often makes each test readable on its own (DAMP — "descriptive and meaningful phrases").

Common mistakes

- Treating DRY as "never two similar lines" and building one monster function with flags for every caller.
- Extracting after the first repetition, before you understand what really varies.
- Forgetting non-code duplication: the same rule in code *and* a database constraint *and* docs.

Real-world examples

- **Spring's** `@Value` **/** `application.yml` (Spring: the dominant Java server framework; `@Value`: an *annotation*, a `@` tag tools read): config lives in one file, injected where needed.
- **Database normalisation** is DRY for data: an address is stored once and referenced by key.
- **Bean Validation (**`jakarta.validation`**)** annotations such as `@Email` define a rule once for REST input (web-API requests), forms and persistence (saving data, usually to a database).
- **OpenAPI / Protobuf schemas** (machine-readable API descriptions — HTTP/JSON and Google's binary format) generate client and server code from one contract.

In the interview

> **30-second answer:** "DRY means each piece of knowledge has one authoritative representation. It is about rules, not identical-looking text. I extract a rule when copies must change together — e.g. validation into a value object — and use the Rule of Three so I don't abstract too early."

Follow-ups:

- *Q: Is duplicated code always a DRY violation?* A: No. If the copies represent different rules that change for different reasons, merging them adds wrong coupling.
- *Q: DRY vs. SRP?* A: They cooperate (SRP, Chapter 4: one reason to change per class). DRY gives each rule one home; SRP makes sure that home has one reason to change.
- *Q: How do you spot duplicated knowledge?* A: "If this business rule changes, how many files do I edit?" More than one is a smell.

Self-check

1. Q: What exactly must not be repeated? → A: Knowledge (rules, formulas, decisions), not merely similar-looking lines.
2. Q: What bug does duplication cause? → A: Drift — copies change independently and start disagreeing.

TL;DR

One rule, one home. Extract when copies must change together; leave coincidental look-alikes alone.

---

2. KISS — Keep It Simple, Stupid

What it is

KISS: most systems work best when kept simple. The phrase was used by the US Navy around 1960 and is associated with Kelly Johnson, lead engineer of Lockheed's Skunk Works, who wanted jets a mechanic could repair in the field with basic tools. Choose the plainest design that fully solves the real problem. *Simple* means easy to understand and change — not necessarily fewest lines.

The problem it solves

Engineers love clever solutions: home-made frameworks, deep inheritance trees, generic engines configured by lambdas (short inline functions such as `x -> x * 2`). Every moving part must be read,

tested, debugged and explained. **Accidental complexity** (complexity our solution adds, as opposed to what the problem truly has) is the biggest source of bugs and slow delivery.

How it works

1. State the actual requirement in one sentence.
2. Write the most direct solution using standard language features and libraries.
3. Ask: "Could a new team member understand this in two minutes?"
4. Add structure (patterns, abstractions) only when a concrete need — a second variant, a test seam, a real performance issue — shows up; delete cleverness that no longer earns its keep.

Java example — before/after

Three fixed customer tiers get three fixed discounts. The *before* builds a priority-sorted rule engine of predicates and functions; the *after* is a `switch` *expression* over an enum (a type with a fixed list of named constants). With no `default`, it must cover every constant, so adding a tier fails to compile until handled (old-style `switch` *statements* are not checked this way). Money is `BigDecimal` (exact decimal), never `double` (binary floating point cannot represent most decimal amounts exactly). Full program: `code/lld/ch05/kiss/Main.java`.

```
// BEFORE: a home-made "rule engine" for three fixed discounts
record Rule(int priority, Predicate<Tier> when, Function<BigDecimal, BigDecimal> apply) {}
static class DiscountRuleEngine {
    private final List<Rule> rules = new ArrayList<>();
    DiscountRuleEngine register(int priority, Predicate<Tier> when, Function<BigDecimal, BigDecimal> apply) {
        rules.add(new Rule(priority, when, apply));
        rules.sort(Comparator.comparingInt(Rule::priority).reversed());
        return this;
    }
    BigDecimal evaluate(Tier tier, BigDecimal amount) {
        return rules.stream().filter(r -> r.when().test(tier)).findFirst()
                .map(r -> r.apply().apply(amount)).orElse(amount);
    }
}

// AFTER: a plain switch expression; the compiler checks every tier is handled
static BigDecimal discounted(Tier tier, BigDecimal amount) {
    return switch (tier) {
        case GOLD -> times(amount, GOLD_FACTOR);       // 0.80
        case SILVER -> times(amount, SILVER_FACTOR);   // 0.90
        case REGULAR -> amount;
    };
}
```

Output:

```
== BEFORE (rule engine) ==
REGULAR -> 100.00
SILVER  -> 90.00
GOLD    -> 80.00
== AFTER (switch) ==
REGULAR -> 100.00
SILVER  -> 90.00
GOLD    -> 80.00
```

Same results, but the *after* has no ordering bugs (a forgotten priority silently changes the engine's behaviour), no mutable list, and reads like the business rule.

Trade-offs

Simple design	"Flexible" complex design
Fast to read, change, test	Handles hypothetical cases
May need refactoring when real variation arrives	Costs you every day, even if variation never arrives
Uses standard, well-known constructs	Custom mechanisms need docs and training

When not to use it

KISS is no excuse to ignore real complexity. A payment system *needs* idempotency (repeating a request — say after a timeout — has the same effect as doing it once, so nobody is charged twice; see System Design Fundamentals), retries and auditing; leaving them out is incompleteness, not simplicity. And when a rule engine really is the requirement (business users edit rules at runtime), build or buy one.

Common mistakes

- Confusing *simple* with *short*: a one-line stream with five nested lambdas can be harder to read than a loop.
- Using KISS to justify skipping tests, error handling or validation.
- Copy-pasting instead of simplifying (violates DRY).

Real-world examples

- **Unix philosophy** — small tools that do one thing and compose through pipes (`|`).
- **Go language design** deliberately left out many features.
- `java.util.Optional` was kept small (no `Serializable`, not meant as a field type) to steer people to its one purpose: return values that might be absent.

In the interview

> **30-second answer:** "KISS means the simplest design that fully meets the requirements. I start with a straightforward model and introduce patterns only when a requirement demands variation or extension — and I say why."

Follow-ups:

- *Q: Isn't a design pattern always better?* A: No. A pattern adds classes and indirection; it pays only if it solves a problem you actually have.
- *Q: KISS vs YAGNI?* A: KISS is about *how* you build (simplest form); YAGNI is about *what* you build (only needed features).

Self-check

1. Q: Why did the `switch` version win? → A: Same behaviour, fewer moving parts, compiler-checked exhaustiveness.

TL;DR

Solve the real problem in the plainest way; add machinery only when a real need appears.

---

3. YAGNI — You Aren't Gonna Need It

What it is

YAGNI: don't build a capability until you actually need it. It comes from Extreme Programming (XP), Kent Beck's agile method (short cycles, frequent feedback) of the late 1990s; the slogan is usually credited to XP co-founder Ron Jeffries. The opposite habit — hooks, options and layers "just in case" —

is **speculative generality**, a **code smell** (a visible sign of a deeper design problem) in Fowler's *Refactoring*.

The problem it solves

Early features cost four times: building, maintaining/testing, the complexity they add around them, and being wrong (the real requirement usually differs from the guess, so the code is rewritten anyway).

How it works

1. Implement the requirements/user stories you have **now** (a *user story* is a short feature request from the user's view: "As a shopper I can …").
2. Keep the code clean and tested so it is cheap to change later — YAGNI depends on easy refactoring.
3. Extend the design when a new requirement actually arrives; delete unused options, parameters and classes you find.

Java example — before/after

Requirement: *export orders as CSV* (comma-separated values). The *before* builds a universal exporter with five formats, encodings and compression, four of which throw `UnsupportedOperationException`. The *after* is one method. Full program: `code/lld/ch05/yagni/Main.java`.

```
// BEFORE: speculative generality
enum Format { CSV, PDF, XML, EXCEL, YAML }
interface ReportExporter {
    String export(List<Order> orders, Format format, String encoding, boolean compress);
}
// UniversalExporter.export(...):
//   if (compress) throw new UnsupportedOperationException("compression not implemented yet");
//   case CSV -> ...join rows...;  case PDF, XML, EXCEL, YAML -> throw new UnsupportedOperationException(format +
" not implemented yet");

// AFTER: exactly what is needed today
static String toCsv(List<Order> orders) {
    return orders.stream()
            .map(o -> String.join(",", o.id(), o.customer(), o.total().toPlainString()))
            .collect(Collectors.joining("\n", "id,customer,total\n", ""));
}
```

Output:

```
== BEFORE ==
id,customer,total
o1,asha,250.00
o2,ravi,99.50
dead option: PDF not implemented yet
== AFTER ==
id,customer,total
o1,asha,250.00
o2,ravi,99.50
```

Every caller of the *before* API (Application Programming Interface — the methods and types other code calls) passes three meaningless arguments, and readers can't tell which formats work.

Trade-offs

YAGNI	Build ahead
Less code, faster delivery	Might save time if the guess is right
Design fits real needs	Guesses are usually wrong in detail
Needs good tests + refactoring discipline	Leaves dead options that confuse readers

When not to use it

- **Hard-to-change decisions** — public API formats, database schemas (table/column structure) holding huge data, security, data-retention laws. Think ahead there.
- Known architectural requirements (e.g. "must scale to 10k requests/s" — a *throughput* target) are requirements, not speculation.

Common mistakes

- Using YAGNI as an excuse for messy code.
- Ignoring a requirement the interviewer *did* mention because "YAGNI".
- `interface` + `Impl` pairs for every class "in case we swap it" — add the interface when a second implementation or a test seam (a place where a test can swap in a fake collaborator) is needed.

Real-world examples

- **Minimum Viable Product (MVP)** thinking is YAGNI at the product level.
- **Feature flags** (config switches that enable a code path without redeploying; e.g. LaunchDarkly, Unleash) let teams ship only what is needed now.
- XP and many agile teams plan per iteration so only near-term work is built.

In the interview

> **30-second answer:** "YAGNI means I build what today's requirements need and keep the design easy to extend. In an LLD (low-level design — classes, interfaces and their interactions) interview I add an extension point like a `PaymentStrategy` interface only if the interviewer hinted at multiple variants; otherwise I mention it verbally."

Follow-ups:

- *Q: Doesn't YAGNI conflict with OCP?* A: Not really. OCP (Chapter 4: open for extension, closed for modification) applies where variation is known or likely; YAGNI stops you inventing variation that isn't there.
- *Q: What makes YAGNI safe?* A: A good test suite and clean code, so adding the feature later is cheap.

Self-check

1. Q: Name the smell YAGNI prevents. → A: Speculative generality.
2. Q: When should you design ahead anyway? → A: For expensive-to-reverse decisions (public APIs, schemas, security).

TL;DR

Build for today's requirements with clean, changeable code; tomorrow's features get built tomorrow.

---

4. Law of Demeter (Principle of Least Knowledge)

What it is

The **Law of Demeter (LoD)**, or **Principle of Least Knowledge**: a method should only talk to its **immediate friends**, not to strangers reached through other objects. A method `m` of object `O` should only call methods on:

1. `O` itself (`this`);
2. objects passed to `m` as parameters;
3. objects `m` creates;
4. `O`'s own fields (its direct components).

(The original formulation by Ian Holland and Karl Lieberherr, Northeastern University, 1987, also allows global objects accessible to `O`.) Slogan: **"use only one dot"** — `a.getB().getC().doIt()` is a **train wreck** (a chain of calls like connected railway cars).

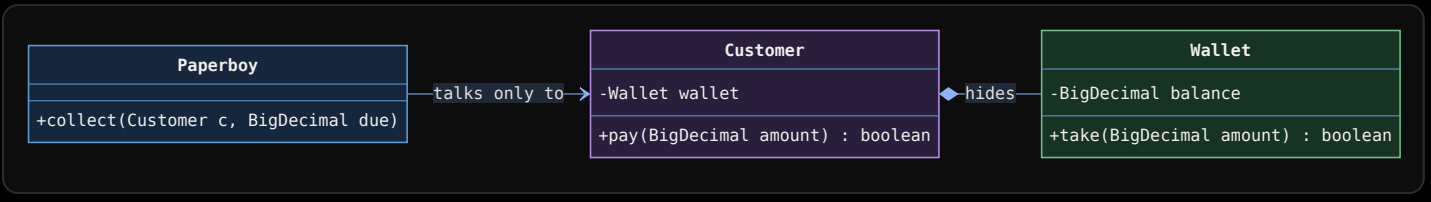
Analogy: a paperboy asks the customer to pay; he doesn't reach into the customer's pocket for the wallet.

The problem it solves

A train wreck couples the caller to the **whole chain's structure**. In `customer.getWallet().getBalance()` the paperboy depends on `Customer` *and* `Wallet`; if the customer starts paying by UPI (India's instant mobile bank payment) the paperboy breaks.

How it works

1. Spot a chain that navigates through other objects' internals.
2. Ask what the caller really wants ("get paid", not "get the wallet").
3. Add a method on the first object that does that job (`customer.pay(amount)`), delegating internally, and keep the internal object private.



Java example — before/after

Full program: `code/lld/ch05/demeter/Main.java`.

```
// BEFORE: paperboy -> customer -> wallet -> balance (a train wreck)
void collect(Customer c, BigDecimal due) {
    // knows Customer has a Wallet, knows Wallet has a balance, changes it directly
    if (c.getWallet().getBalance().compareTo(due) >= 0) {
        c.getWallet().setBalance(c.getWallet().getBalance().subtract(due));
        collected = collected.add(due);
        System.out.println("collected " + due + " from " + c.getName());
    } else System.out.println(c.getName() + " cannot pay");
}

// AFTER: the paperboy asks the customer (a direct friend) to pay
static class Wallet {
    private BigDecimal balance;
    boolean take(BigDecimal amount) {
        if (amount.compareTo(balance) > 0) return false;
        balance = balance.subtract(amount);
        return true;
    }
}
static class Customer {
    private final Wallet wallet;          // private detail -- could become a card or UPI tomorrow
    boolean pay(BigDecimal amount) { return wallet.take(amount); }   // Customer decides HOW to pay
}
void collect(Customer c, BigDecimal due) {
    if (c.pay(due)) { collected = collected.add(due); /* print */ }  // talk only to a parameter: allowed
    else System.out.println(c.name() + " cannot pay");
}

// Chained calls on a fluent API / stream are NOT violations: each call returns the same kind of thing
long longNames = List.of("Meera", "Kabir", "Jo").stream().filter(n -> n.length() > 2).count();
```

Output:

```
== BEFORE ==
collected 4.00 from Meera
Kabir cannot pay
== AFTER ==
collected 4.00 from Meera
Kabir cannot pay
total collected = 4.00
stream chain is fine: 2 long names
```

After, `Paperboy` knows only `Customer.pay`, and `Customer` is free to change how it pays.

Trade-offs

Following LoD	Ignoring LoD
Changes to internals stay local	Structural changes ripple to callers
Easier mocking (mock = fake object used in tests) — one collaborator	Tests must build whole object graphs
May add small **wrapper/delegate methods**; too many bloat the middle class	No extra methods

When not to use it

- **Fluent interfaces and builders** (methods return an object so calls chain; a builder assembles an object step by step — `StringBuilder.append(..).append(..)`, `stream().filter().map()`) return the same object or a sibling abstraction — *not* violations.
- **Data structures / DTOs** (Data Transfer Objects — plain data, no behaviour, Chapter 6) are meant to expose data; `order.shippingAddress().city()` on a record is fine.
- Don't mechanically add dozens of pass-through methods — that is the **Middle Man** smell; rethink responsibilities instead.

Common mistakes

- Counting dots instead of thinking about *knowledge*: splitting a chain into temps (`var w = c.getWallet(); w.getBalance()`) still violates the law.
- Adding getters for everything, which invites train wrecks.

Real-world examples

- *Head First Design Patterns* presents it ("talk only to your immediate friends") in the Facade chapter — a **Facade** (one simple class in front of a group of complex ones) reduces how many classes a client talks to.
- **Spring Data** repositories (objects that load/save domain objects and hide the database): services call `orderRepository.findByCustomerId(id)`, not `entityManager.getSession().getConnection()...`.
- `java.nio.file.Files.readString(path)` hides channels, buffers and decoders behind one call.

In the interview

> **30-second answer:** "The Law of Demeter says a method should only call methods on itself, its parameters, objects it creates and its own fields. Chains like `a.getB().getC()` couple the caller to internal structure; I replace them with a method that expresses intent — `customer.pay()` instead of reaching for the wallet."

Follow-ups:

- *Q: Is `list.stream().map(...).collect(...)` a violation?* A: No — each call returns a stream (a pipeline processing elements step by step) meant for chaining; you aren't navigating internals.
- *Q: Cost of following it strictly?* A: Wrapper methods pile up (Middle Man). Balance by moving behaviour to where the data is (Tell-Don't-Ask).

Self-check

1. Q: List the four "friends" of a method. → A: `this`, parameters, objects it creates, its own fields.
2. Q: Do records/DTOs have to obey it? → A: Not really; they are data structures meant to expose data.

TL;DR

Only talk to your friends. Replace `a.getB().getC().do()` with a method on `a` that says what you want.

---

5. Tell, Don't Ask

What it is

Tell, Don't Ask (TDA): instead of *asking* an object for its data and deciding for it, *tell* the object what you want and let it decide using its own data. Behaviour lives with the data it needs. Popularised by Andy Hunt and Dave Thomas, building on Alec Sharp's observation in *Smalltalk by Example* (1997) that procedural code (functions acting on separate data) fetches data and then decides, while object-oriented code tells objects to act.

Analogy: you tell a taxi driver "take me to the airport"; you don't ask for the steering angle every second and decide the turns yourself.

The problem it solves

With ask-then-act (`if (acc.getBalance() >= x) acc.setBalance(acc.getBalance() - x)`) the business rule is spread over every caller. One caller forgets part of it ("frozen accounts can't withdraw") and the object's **invariant** breaks. The object becomes a passive data bag — the **Anemic Domain Model** anti-pattern.

How it works

1. Look for code that reads an object's fields, decides, then writes back.
2. Move that decision into a method on the object (`withdraw(amount)`) that rejects illegal requests (exceptions or result types).
3. Remove setters (`setBalance(x)`) that are no longer needed.

Java example — before/after

Full program: `code/lld/ch05/telldontask/Main.java`.

```java
// BEFORE: callers ask for state and decide themselves (Account has getBalance/setBalance/isFrozen/setFrozen)
static void atmWithdraw(Account a, long amt) {   // ATM remembered the "frozen" rule...
    if (!a.isFrozen() && a.getBalance() >= amt) a.setBalance(a.getBalance() - amt);
}
static void appWithdraw(Account a, long amt) {   // ...the mobile app forgot it
    if (a.getBalance() >= amt) a.setBalance(a.getBalance() - amt);
}

// AFTER: the rule lives with the data it protects
static final class Account {
    private long balancePaise;
    private boolean frozen;
    void freeze() { frozen = true; }
    void withdraw(long amt) {
        if (amt <= 0) throw new IllegalArgumentException("amount must be positive");
        if (frozen) throw new IllegalStateException("account is frozen");
        if (amt > balancePaise) throw new IllegalStateException("insufficient funds");
        balancePaise -= amt;
    }
    long balance() { return balancePaise; }   // queries are fine for display -- just don't decide outside
}
```

Output:

```
== BEFORE ==
after ATM on frozen account: 1000
after APP on frozen account: 700   <-- rule bypassed
== AFTER ==
withdrew 300, balance 700
rejected: account is frozen
balance still 700
```

`withdraw` is now the only way to take money, so the rule cannot be bypassed.

Trade-offs

Tell	Ask
Invariants protected in one place	Rules duplicated across callers
Richer, more expressive objects	Simple data objects
Risk: object grows too many responsibilities	Fine for pure data (DTOs, reports)

When not to use it

- **Queries for display or reporting** — asking for a balance to show it is fine; the rule is about *deciding* outside.
- When the decision genuinely belongs to another object combining data from several sources (that object is the Information Expert — see GRASP).

Common mistakes

- Adding a getter "just for this one check" — that is where the rule starts leaking.
- Pushing *every* behaviour into the entity, including emails and database calls (breaks SRP; use services for cross-cutting work).

Real-world examples

- `List.removeIf(predicate)` — tell the list what to remove instead of iterating and removing.
- `Map.computeIfAbsent`, `Map.merge` — tell the map the update; it handles the check-then-act.
- `AtomicInteger.incrementAndGet()` — the object does the read-modify-write atomically (as one indivisible step); asking then setting would be a **race condition** (a result that depends on the timing of two *threads* — concurrent paths of execution — touching the same data).

In the interview

> **30-second answer:** "Tell, Don't Ask means sending commands to the object that owns the data, e.g. `account.withdraw(100)`, rather than reading its balance and setting a new one. It keeps invariants in one place and avoids anemic models."

Follow-ups:

- *Q: Relation to the Law of Demeter?* A: Both push behaviour to where the data is; LoD is about *who* you talk to, TDA about *what* you ask for.
- *Q: Concurrency angle?* A: Ask-then-act across two calls is a check-then-act race; a single `withdraw` can be made atomic.

Self-check

1. Q: What anti-pattern does TDA fight? → A: The Anemic Domain Model (data bags with logic outside).
2. Q: Are getters forbidden? → A: No; they're fine for display. Avoid deciding outside the object.

TL;DR

Don't pull data out to make the object's decisions — tell the object what you want.

---

6. Composition over Inheritance

What it is

Favour object composition over class inheritance (GoF). *Inheritance* ("is-a") reuses code by subclassing. *Composition* ("has-a") reuses code by holding a reference to another object and **delegating** (forwarding) calls to it.

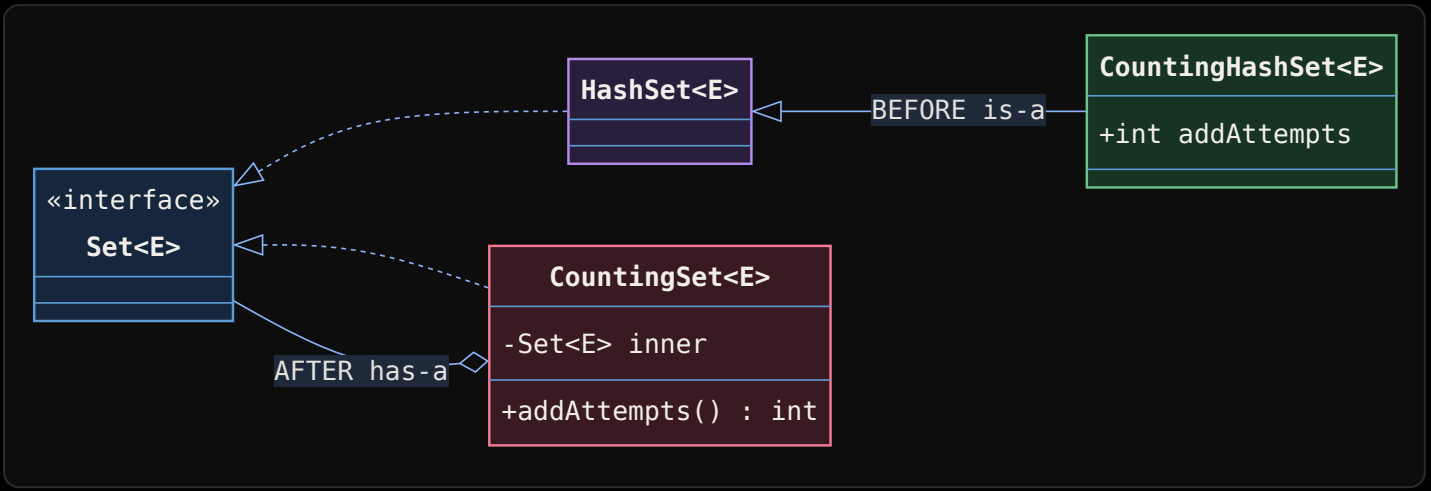
The problem it solves

Inheritance is the tightest coupling in OOP:

- **Fragile base class problem** — a subclass breaks when the parent's *internal* implementation changes, even if its public API is unchanged (the subclass depends on details like "does `addAll` call `add`?").
- **Fixed at compile time** — you cannot change the parent at runtime.
- **Class explosion** — combining features by subclassing (`LoggingCachingBufferedStream`) multiplies classes.
- It exposes all parent methods, even those meaningless for the child.

How it works

1. Instead of `class B extends A`, give `B` a private field of type `A` (preferably an interface type), received through the constructor so any implementation can be plugged in.
2. Implement `B`'s methods by adding your behaviour and **forwarding** to the wrapped object.
3. Keep inheritance for genuine "is-a" relationships where the subclass is substitutable (Liskov, Chapter 4) and the parent was **designed for extension** (documented hooks, often abstract).



Java example — before/after

Count how many elements were ever added to a set — the classic trap from *Effective Java*. Full program: `code/lld/ch05/composition/Main.java`.

```java
// BEFORE: inherit from HashSet to count insert attempts
static class CountingHashSet<E> extends HashSet<E> {
    int addAttempts;
    @Override public boolean add(E e) { addAttempts++; return super.add(e); }
    @Override public boolean addAll(Collection<? extends E> c) {
        addAttempts += c.size();
        return super.addAll(c);   // HashSet's inherited addAll calls add() for each element -> counted twice!
    }
}

// AFTER: wrap ANY Set and forward calls (a decorator)
static final class CountingSet<E> extends AbstractSet<E> {
    private final Set<E> inner;   // "has-a", not "is-a"
    private int addAttempts;
    CountingSet(Set<E> inner) { this.inner = inner; }
    @Override public boolean add(E e) { addAttempts++; return inner.add(e); }
    @Override public boolean addAll(Collection<? extends E> c) { addAttempts += c.size(); return inner.addAll(c);
}
    @Override public boolean contains(Object o) { return inner.contains(o); }   // keep the inner set's O(1)/O(log
n) lookup
    @Override public boolean remove(Object o) { return inner.remove(o); }
    @Override public Iterator<E> iterator() { return inner.iterator(); }
    @Override public int size() { return inner.size(); }
    int addAttempts() { return addAttempts; }
}
// main also wraps a TreeSet chosen at runtime: new CountingSet<>(new TreeSet<>())
```

Output:

```
BEFORE: added 3 items, counter says 6
AFTER : added 3 items, counter says 3
AFTER over a TreeSet: [blue, green, red] attempts=3
```

Why 6: `HashSet` inherits `AbstractCollection.addAll`, which calls *our overridden* `add` per element — we depended on undocumented "self-use", the fragile base class problem. The wrapper's inner calls never reach our counter. It forwards `contains` and `remove` because `AbstractSet` would otherwise scan the iterator (which walks a collection one element at a time) — O(n), time growing with the number of elements *n* (*Big-O notation*) — instead of the inner set's fast lookup.

Trade-offs

Composition	Inheritance
Loose coupling to the public API only	Coupled to parent internals
Swap behaviour at runtime	Fixed at compile time
Combine behaviours freely (decorators)	Class explosion for combinations
More boilerplate (forwarding methods)	Free reuse of all parent methods
No automatic polymorphism (one variable working with many types) — need an interface	Subclass is automatically a subtype

When not to use it

- True "is-a" hierarchies with a base **designed for extension**: extending `AbstractList` (implement `get` and `size`) or a Template Method base (Hollywood Principle, below).
- Sealed hierarchies (a `sealed` type lists exactly which subtypes may exist) modelling a closed set of variants (`sealed interface Shape permits Circle, Square`).
- When you control both classes, in the same package, evolving together.

Common mistakes

- Inheriting just to reuse helpers ("`OrderService extends BaseDao`").
- Composition without an interface — then you can't substitute implementations.

- Exposing the wrapped object with a getter, defeating encapsulation.

Real-world examples

- **JDK mistakes admitted by its authors** (*Effective Java*): `java.util.Stack extends Vector` (a stack that lets you insert in the middle) and `java.util.Properties extends Hashtable` (accepts non-String values).
- **`java.io` streams** — `new BufferedInputStream(new FileInputStream(f))` composes behaviour (Decorator, Chapter 9).
- **`Collections.unmodifiableList(list)` / `synchronizedList(list)`** wrap a list instead of subclassing each list type.
- **Game engines (Unity)** use Entity–Component: a game object *has* components (physics, renderer) instead of deep inheritance.

In the interview

> **30-second answer:** "Inheritance couples a subclass to its parent's implementation, is fixed at compile time and explodes combinatorially. Composition holds a reference to an interface and delegates, so behaviour can be swapped at runtime. I use inheritance only for true is-a relationships with classes designed for extension."

Follow-ups:

- *Q: Which patterns are built on composition?* A: Strategy, Decorator, Composite, State, Bridge, Proxy, Adapter (object form) — most GoF patterns.
- *Q: A concrete inheritance failure?* A: The counting `HashSet` above, or `Stack extends Vector` exposing `insertElementAt`.
- *Q: When is inheritance OK?* A: When substitutable (LSP holds) and the parent documents its self-use or provides abstract hooks.

Self-check

1. Q: What is the fragile base class problem? → A: Subclasses breaking because the parent's internal implementation changed.
2. Q: Why did `CountingHashSet` count 6? → A: The inherited `addAll` calls the overridden `add`.

TL;DR

Reuse by holding and delegating ("has-a"); reserve "is-a" for real substitutable subtypes designed for extension.

---

7. Program to an Interface, Not an Implementation

What it is

Declare variables, fields, parameters and return types using the **most general supertype** that offers what you need (an interface or abstract class), not a concrete class. "Interface" here means *supertype*, not only the Java `interface` keyword.

The problem it solves

Code written against a concrete class (`ArrayList`, `MySqlUserStore`) works only with that class: you can't test it without the real thing (a MySQL server), swap implementations or reuse it, and changing the implementation edits every caller.

How it works

1. Identify the **role** a collaborator plays ("stores users", "a collection of names") and express it as an interface with only the needed methods (`UserRepository.findById`).
2. Type fields/parameters by that interface (`List<String>`, `Collection<String>`, `UserRepository`).

3. Create concrete objects only at the edges — `main`, a factory (an object/method whose job is creating objects) or a DI container (e.g. Spring's, which builds objects and wires their dependencies) — and pass them in (**Dependency Injection**: giving an object its collaborators from outside instead of letting it `new` them).
4. Accept the widest parameter type you can (`Collection` beats `List` beats `ArrayList`); return an interface specific enough to be useful.

Java example — before/after

Full program: `code/lld/ch05/interfaces/Main.java`.

```
// BEFORE: every signature names a concrete class
static class GreetingService {
    private final MySqlUserStore store = new MySqlUserStore();   // hard-wired; needs a real MySQL server
    String greet(String id) { return "Hello, " + store.load(id).name(); }
}
static int countLongNames(ArrayList<String> names) { ... }    // only accepts ArrayList

// AFTER: depend on the supertype; pick the implementation at the edge
interface UserRepository { Optional<User> findById(String id); }
static final class InMemoryUserRepository implements UserRepository { /* HashMap-backed */ }

static final class GreetingService {
    private final UserRepository users;                 // any implementation
    GreetingService(UserRepository users) { this.users = users; }
    String greet(String id) {
        return users.findById(id).map(u -> "Hello, " + u.name()).orElse("Hello, stranger");
    }
}
static int countLongNames(Collection<String> names) {   // works for List, Set, Queue...
    return (int) names.stream().filter(s -> s.length() > 4).count();
}
```

Output:

```
== BEFORE ==
cannot even run it: no MySQL server on this laptop
long names in ArrayList: 2
== AFTER ==
Hello, Asha
Hello, stranger
long names in List:    2
long names in TreeSet: 2
```

The *before* cannot even run on a laptop, and `countLongNames(new TreeSet<>(...))` would not compile; the *after* runs with an in-memory repository and accepts any collection.

Trade-offs

Interface-typed	Concrete-typed
Swappable, testable (fakes/mocks)	Simple, one less type
Callers can't use implementation-only methods	Access to everything (e.g. `ArrayList.trimToSize`)
Extra interface to maintain	Changes ripple to callers

When not to use it

- **Value types / data classes** (`String`, `LocalDate`, records).
- A class with exactly one implementation and no test-seam need — `FooService` + `FooServiceImpl` everywhere is ceremony (YAGNI). Add the interface when a second implementation or a test double (fake, stub or mock) appears.
- When you need implementation-specific behaviour (e.g. `LinkedHashMap` access order for an LRU cache — a *cache* keeps recently used data at hand; LRU evicts the Least Recently Used entry) — then declare

that type.

Common mistakes

- Interfaces mirroring a class one-to-one with 30 methods (violates Interface Segregation, Chapter 4).
- Declaring `List` but casting to `ArrayList` inside.
- Still calling `new ConcreteThing()` inside the class that uses the interface — typed abstractly but still hard-wired.

Real-world examples

- **Java Collections Framework**: code uses `List`, `Map`, `Set`; `Collections.sort(List)` works on any list.
- **JDBC** (Java Database Connectivity, the standard SQL-database API): code uses the `Connection`, `PreparedStatement`, `ResultSet` interfaces; the driver (MySQL, PostgreSQL) implements them.
- **SLF4J** (a logging *facade* — one API in front of many logging libraries): code logs via `org.slf4j.Logger`; Logback or Log4j2 is plugged in at deployment.
- **Spring** injects beans (container-managed objects) by interface type (`@Autowired PaymentGateway gateway`).

In the interview

> **30-second answer:** "I type dependencies by roles — interfaces — and inject concrete implementations from outside. That lets me swap implementations, e.g. an in-memory repository for tests, and keeps callers stable. It's the practical form of the Dependency Inversion Principle."

Follow-ups:

- *Q: Relationship to DIP?* A: DIP says high-level modules depend on abstractions; programming to an interface is how you do it at each variable/field.
- *Q: Should every class have an interface?* A: No — only roles with multiple implementations or a need for test doubles.

Self-check

1. Q: Where should `new ConcreteClass()` appear? → A: At the edges: `main`, factories, DI configuration.

TL;DR

Depend on roles (`List`, `UserRepository`), not on concrete classes; choose the concrete class once, at the edge.

---

8. Encapsulate What Varies

What it is

Find the parts of your application that **change** (or differ between cases) and **separate them** from the parts that stay the same, behind their own abstraction. *Head First Design Patterns* calls this the first design principle; the GoF book says "encapsulate the concept that varies".

Analogy: a camera body stays the same while you swap lenses; the lens mount is the stable interface.

The problem it solves

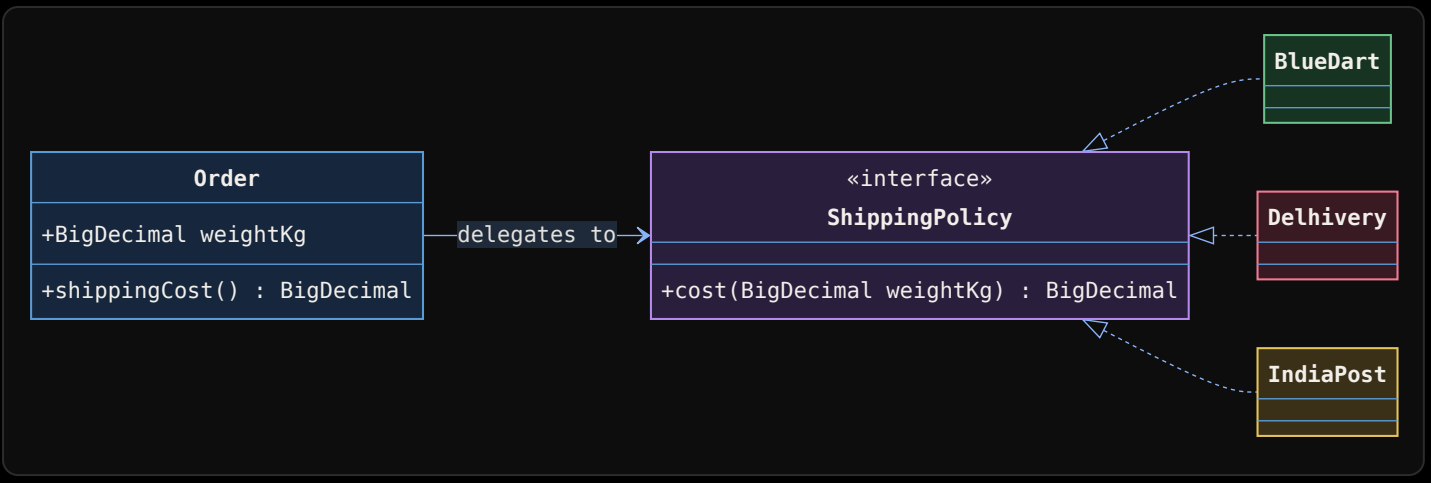
A volatile rule (shipping prices, tax, discounts, file formats) written inline in a stable class means every rule change edits and re-tests that class, risking unrelated behaviour. `if/else` or `switch` chains on a "type" string grow forever (violating OCP).

How it works

1. Ask: "Which parts changed in the last few releases, or differ per customer/region/type?"

2. Extract that part behind an interface (or a functional interface — exactly one abstract method, so a lambda can implement it).
3. Give the stable class a field of that type and delegate to it; add new variants as new implementations.

Almost every design pattern (a named, reusable solution to a recurring design problem; Chapters 8–11) applies this principle: Strategy (varying algorithm), State (state-dependent behaviour), Factory (creation), Observer (set of listeners), Template Method (steps).



Java example — before/after

`linear(base, perKg, weight)` returns base + perKg × weight as a `BigDecimal`. Full program: `code/lld/ch05/encapsulatevaries/Main.java`.

```
// BEFORE: the changing part is tangled into a stable class
BigDecimal shippingCost() {
    // every new carrier or price change edits Order -- a class that should be stable
    if (carrier.equals("BLUEDART")) return linear("50", "20", weightKg);
    else if (carrier.equals("DELHIVERY")) return linear("40", "25", weightKg);
    else if (carrier.equals("INDIA_POST")) return linear("30", "10", weightKg);
    throw new IllegalArgumentException("unknown carrier " + carrier);
}

// AFTER: the varying part is a separate, swappable object
@FunctionalInterface
interface ShippingPolicy { BigDecimal cost(BigDecimal weightKg); }

static final Map<String, ShippingPolicy> POLICIES = Map.of(
        "BLUEDART", w -> linear("50", "20", w),
        "DELHIVERY", w -> linear("40", "25", w),
        "INDIA_POST", w -> linear("30", "10", w));

record Order(BigDecimal weightKg, ShippingPolicy shipping) {
    BigDecimal shippingCost() { return shipping.cost(weightKg); }
}
// a brand-new flat-rate promo, zero edits to Order: new Order(twoKg, w -> new BigDecimal("0.00"))
```

Output:

```
== BEFORE ==
BLUEDART   90.00
DELHIVERY  90.00
INDIA_POST 50.00
== AFTER ==
BLUEDART   90.00
DELHIVERY  90.00
INDIA_POST 50.00
PROMO      0.00
```

The diagram draws named classes for clarity; the code uses lambdas implementing the same interface.

Trade-offs

Encapsulated	Inline
New variants without touching stable code	Everything in one place, easy to see
Each variant testable alone	Simple for 2 variants that never change
More types / indirection	Grows into long conditionals

When not to use it

When the "varying" thing has never varied and shows no sign it will (YAGNI). Two fixed branches are fine inline; extract when the third arrives or change becomes frequent.

Common mistakes

- Guessing the wrong axis of variation (extracting something stable while the volatile part stays inline).
- Leaving the `switch` on type in client code after introducing the strategy (just moving the conditional).

Real-world examples

- `java.util.Comparator` — sorting is stable, comparison varies.
- `ThreadPoolExecutor` takes a `RejectedExecutionHandler` (what to do when full) and a `ThreadFactory` (how threads are made).
- **Payment gateways** (Razorpay, Stripe, PayPal) behind one `PaymentGateway` interface.

In the interview

> **30-second answer:** "I identify what is likely to change — pricing, payment method — and hide it behind an interface so core classes stay closed to modification. It's the principle behind Strategy, State and most other patterns."

Follow-ups:

- *Q: How do you know what varies?* A: From requirements ("multiple payment methods"), history (what changed often), and the interviewer's hints.
- *Q: Relation to OCP?* A: Encapsulating the variation is how a class becomes open for extension and closed for modification.

Self-check

1. Q: What smell signals missing encapsulation? → A: A growing `if/else` / `switch` on a type code.

TL;DR

Separate what changes from what stays the same; put the changing part behind an interface.

---

9. Strive for Loosely Coupled Designs

What it is

Loose coupling: objects that interact know as little as possible about each other — ideally only a small interface (*Head First Design Patterns*: "strive for loosely coupled designs between objects that interact").

The problem it solves

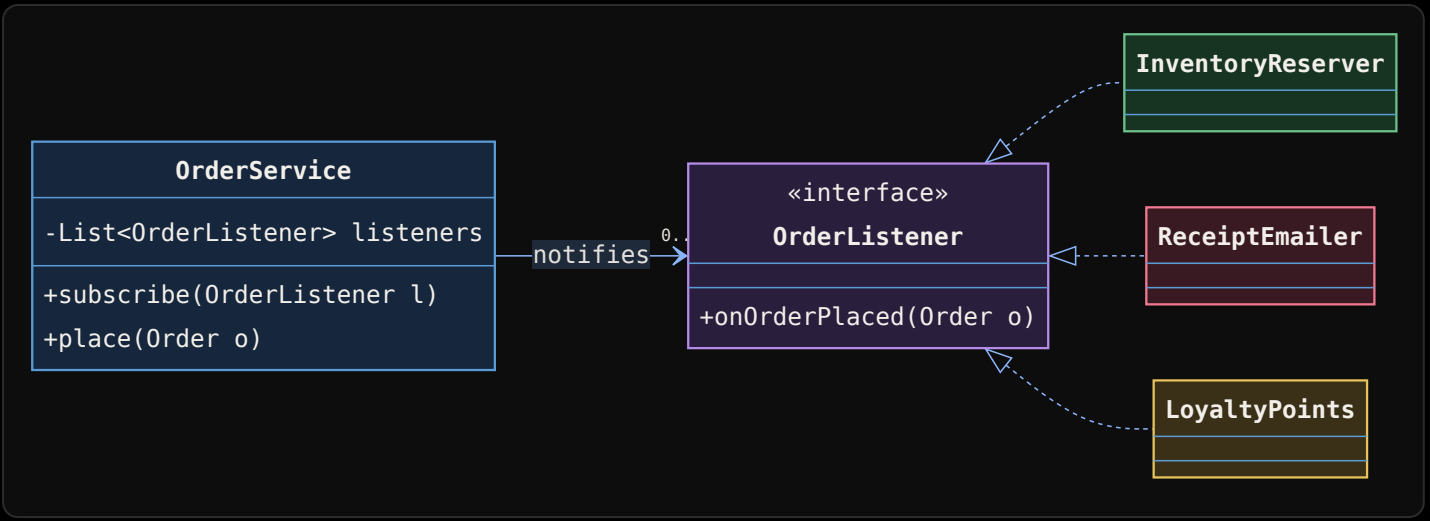
In a **tightly coupled** design a class creates and calls many concrete collaborators directly. Any change to a collaborator edits that class; you can't reuse it without dragging its dependencies along, or test

it in isolation.

How it works

1. Depend on **interfaces**, and **inject** dependencies instead of creating them inside.
2. Communicate through **events or messages** when the sender shouldn't know the receivers (Observer / publish-subscribe — senders publish, any number of subscribers receive, neither knows the other).
3. Keep interfaces **narrow** (one method is ideal for listeners).
4. Pass data as immutable values (records) rather than shared mutable objects.

The classic coupling types, tightest to loosest — content → common → external → control → stamp → data → message — are tabulated with examples in §4.4 Coupling.



Java example — before/after

Full program: `code/lld/ch05/loosecoupling/Main.java`.

```
// BEFORE: OrderService knows every class that cares about an order
static class OrderService {
    private final EmailSender email = new EmailSender();   // concrete, created inside
    private final Inventory inventory = new Inventory();
    private final Analytics analytics = new Analytics();
    void place(Order o) {
        System.out.println("placed " + o.id());
        inventory.reserve(o);
        email.sendReceipt(o);
        analytics.track(o);          // adding "loyalty points" means editing this class
    }
}

// AFTER: OrderService only knows a tiny listener interface
@FunctionalInterface
interface OrderListener { void onOrderPlaced(Order o); }

static final class OrderService {
    private final List<OrderListener> listeners = new CopyOnWriteArrayList<>();
    void subscribe(OrderListener l) { listeners.add(l); }
    void place(Order o) {
        System.out.println("placed " + o.id());
        listeners.forEach(l -> l.onOrderPlaced(o));   // doesn't know who listens or what they do
    }
}
// main subscribes reserve / email / track lambdas, then a NEW one with no edits:
// service.subscribe(o -> System.out.println("  +" + o.items() * 10 + " loyalty points"));
```

Output:

```
== BEFORE ==
placed o-1
  reserve 2 items
  email receipt to asha@example.com
  track order o-1
== AFTER ==
placed o-2
  reserve 3 items
  email receipt to ravi@example.com
  track order o-2
  +30 loyalty points
```

`CopyOnWriteArrayList` (a thread-safe list that copies its array on every write) lets listeners subscribe safely even while notifications run on other threads.

Trade-offs

Loose	Tight
Change and test parts independently	Easy to follow control flow
Add features by adding listeners	Direct calls are faster to trace in a debugger
Flow is less visible ("who handles this event?")	Every change edits the central class
Error handling/ordering between listeners must be designed	Order is explicit

When not to use it

- Inside a small, cohesive module whose classes always change together — decoupling adds ceremony.
- When order and atomicity (all steps or none) are critical (debit then credit in a transfer) — explicit orchestration is clearer and safer than fire-and-forget events.

Common mistakes

- A global event bus (one shared channel everyone publishes to and listens on) for everything, making the flow impossible to follow.
- Hidden coupling via shared mutable static state.
- Listeners that throw and stop the others (decide a policy: catch and log per listener).

Real-world examples

- **Swing / JavaFX / Android** listeners (`ActionListener`, `OnClickListener`).
- **Spring** `ApplicationEventPublisher` and `@EventListener`.
- **Apache Kafka** (a distributed *message broker*: producers append to named *topics*; *consumer groups* read at their own pace; see System Design Fundamentals) — an order service publishes `OrderPlaced`, and inventory, notifications and analytics consume independently.

In the interview

> **30-second answer:** "Loose coupling means collaborators know only small interfaces about each other. I achieve it with interfaces, dependency injection and events — the order service just publishes 'order placed' and new features subscribe without touching it."

Follow-ups:

- *Q: Coupling vs cohesion?* A: Coupling is between modules (want low); cohesion is within a module (want high).
- *Q: Downside of event-driven decoupling?* A: Harder to trace flows, ordering and failure handling; needs good logging and clear contracts.

Self-check

1. Q: Two techniques for loose coupling? → A: Depend on interfaces; inject dependencies (also: events, narrow interfaces).

TL;DR

Let interacting objects share only small interfaces; inject them and use events so parts change independently.

---

10. The Hollywood Principle (Inversion of Control)

What it is

"Don't call us, we'll call you." High-level components (a framework or base class) control the flow and **call** low-level components at the right moments; low-level components **hook in** but never call the high-level ones directly. This reversal of who-calls-whom is **Inversion of Control (IoC)**. **Library vs framework:** you call a library; a framework calls you.

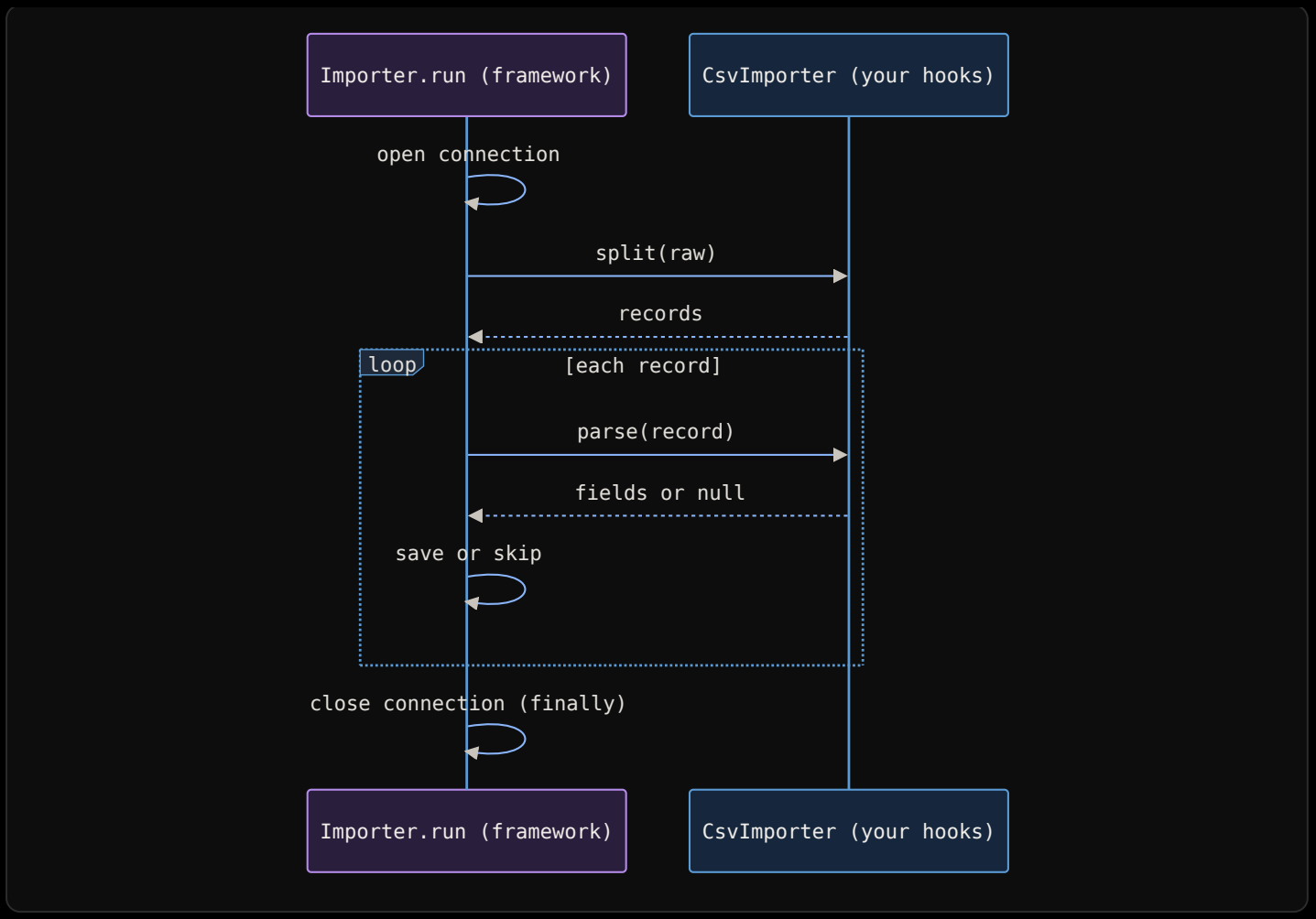
The problem it solves

Without it, every component drives the whole process itself: re-implements the algorithm's skeleton, decides the order of steps and calls shared services directly. Result: duplicated control flow, forgotten steps (like closing a database connection — the open session with a database) and **dependency rot** — high- and low-level components depending on each other in tangled ways.

How it works

Ways to invert control:

1. **Template Method** — a base class defines a `final` algorithm and calls abstract "hook" methods that subclasses fill in.
2. **Callbacks / listeners** — you register a function; the system calls it on an event (Observer).
3. **Dependency Injection** — a container creates your objects and passes them their dependencies; your class never looks them up.
4. **Lifecycle management** — a container starts/stops your components (`@PostConstruct`, servlet `init`/`destroy`; a *servlet* is a Java class handling web requests inside a server such as Tomcat).



Java example — before/after

Full program: `code/lld/ch05/hollywood/Main.java`.

```java
// BEFORE: each importer re-implements the whole flow and drives it itself
void run(String raw) {
    System.out.println("  open connection");              // step order copied into every importer
    for (String row : raw.split(";")) { /* parse, skip bad row, save */ }
    // forgot to close the connection -- nothing forced it
}

// AFTER: Template Method -- the framework calls you
abstract static class Importer {
    public final void run(String raw) {                   // final = subclasses cannot reorder or skip steps
        System.out.println("  open connection");
        try {
            for (String record : split(raw)) {
                String[] fields = parse(record);
                if (fields == null) { System.out.println("  skip bad record '" + record + "'"); continue; }
                System.out.println("  save " + fields[0] + "=" + fields[1]);
            }
        } finally {
            System.out.println("  close connection");       // guaranteed by the framework
        }
    }
    protected abstract List<String> split(String raw);       // hooks: WE call YOU
    protected abstract String[] parse(String record);
}
static final class CsvImporter extends Importer {
    @Override protected List<String> split(String raw) { return List.of(raw.split(";")); }
    @Override protected String[] parse(String r) { String[] f = r.split(","); return f.length == 2 ? f : null; }
}
// KeyValueImporter: split on "\n", parse on "="

// A tiny IoC "container": components register, the container decides when to start/stop them
interface Lifecycle { void start(); void stop(); }
static final class Container {
    private final List<Lifecycle> components = new ArrayList<>();
    void register(Lifecycle c) { components.add(c); }
    void boot() { components.forEach(Lifecycle::start); }
    void shutdown() { components.reversed().forEach(Lifecycle::stop); }  // reverse order, like Spring
}
```

Output:

```
== BEFORE ==
  open connection
  save a=1
  save b=2
  skip bad row 'oops'
== AFTER ==
  open connection
  save a=1
  save b=2
  skip bad record 'oops'
  close connection
  open connection
  save x=9
  save y=8
  close connection
  start database
  start cache
  start web
  stop web
  stop cache
  stop database
```

Subclasses supply only `split` and `parse`; the framework guarantees the close. The container applies IoC to lifecycles: components are started in order and stopped in reverse (as Spring destroys singletons — single-instance objects).

Trade-offs

IoC	Direct control
Framework guarantees flow, ordering, cleanup	You see the whole flow in your code
Less duplication across components	Duplicated skeletons
"Magic": harder to debug who calls what	Explicit, simple for tiny programs
Components tied to the framework's hooks	Independent

When not to use it

Small scripts or one-off logic with no repeated skeleton — a framework or template adds indirection with nothing to share.

Common mistakes

- Low-level hooks calling back up into the framework (e.g. a subclass calling `run()` recursively).
- Forgetting `final` on the template method, so subclasses override the skeleton.
- Confusing IoC (the general idea) with DI (one specific form).

Real-world examples

- **JUnit** (the standard Java unit-test framework) discovers and calls your `@Test` methods.
- **Servlets**: the container calls `doGet` / `doPost` (HTTP GET/POST handlers); **Spring MVC** calls your `@GetMapping` methods.
- **Spring IoC container** creates beans, injects dependencies, calls `@PostConstruct` / `@PreDestroy`.
- **Android** calls `Activity.onCreate`, `onResume`, `onDestroy`.
- `java.util.AbstractList` — you implement `get` and `size`; it implements iteration, `indexOf`, `equals` by calling them.
- **Spring** `JdbcTemplate` runs the connection/statement/cleanup skeleton and calls your `RowMapper`.

In the interview

> **30-second answer:** "The Hollywood Principle — 'don't call us, we'll call you' — means the high-level component controls the flow and calls low-level hooks. It's what separates a framework from a library. Template Method, callbacks and dependency injection all apply it, and it stops low-level parts from depending on high-level ones."

Follow-ups:

- *Q: IoC vs DI?* A: IoC is the general reversal of control; DI is IoC applied to obtaining dependencies (given to you, not looked up).
- *Q: Hollywood vs Dependency Inversion?* A: DIP is about depending on abstractions; Hollywood is about who calls whom. They often appear together.

Self-check

1. Q: Why make the template method `final`? → A: So subclasses cannot reorder or skip steps.

TL;DR

Let the framework own the flow and call your hooks; low-level parts plug in and never call up.

---

11. Separation of Concerns (SoC)

What it is

A **concern** is one aspect of what a program does: input, business rules, storage, presentation, security, logging. **Separation of Concerns** means each part of the program deals with one concern, so it can be understood, changed and tested alone. Dijkstra coined the phrase in "On the role of scientific thought" (1974, EWD 447). The basics — benefits, the "count the modules a change touches" test, SoC vs SRP — are in §4.5; here is what adds to it.

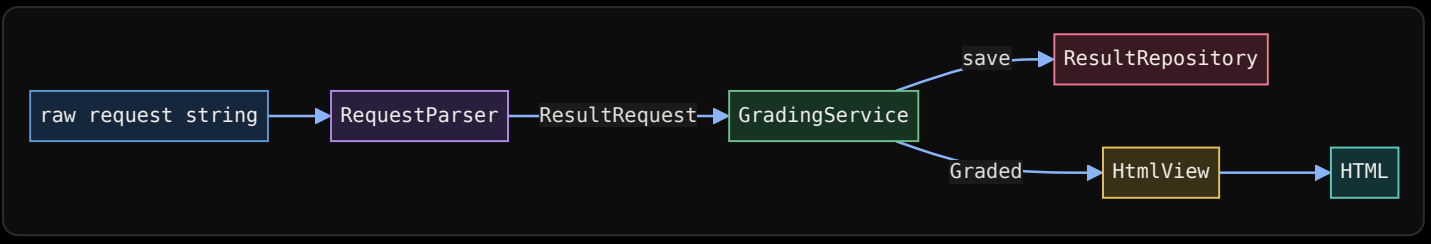
SoC is the **big-picture** version of SRP and applies at every level: functions, classes, layers (MVC — Model–View–Controller: data/rules, display, input handling; or controller/service/repository: web requests, business rules, database), services and even languages (HTML structure, CSS style, JS behaviour).

The problem it solves

Mixed concerns (*spaghetti code*): you can't change the HTML without risking the grading rule, test the rule without parsing strings, or switch storage without touching presentation.

How it works

1. List the concerns in the feature and give each its own unit (function → class → layer), with typed inputs/outputs, not raw strings.
2. Let data flow one way: parse → apply rules → store → present; lower layers don't know higher ones (the service doesn't know HTML).
3. Handle **cross-cutting concerns** (logging, security, transactions — groups of database changes that succeed or fail as one) with dedicated mechanisms — filters, decorators, proxies (stand-ins that intercept calls) or aspects (AOP, aspect-oriented programming: code written once and applied around many methods) — not sprinkled everywhere (Chapter 6, *Systems*).



Java example — before/after

Full program: `code/lld/ch05/soc/Main.java`.

```java
// BEFORE: one method does input parsing, rules, persistence AND HTML
static String handle(String request) {                    // "name=Asha&marks=87"
    // ...split on & and =, pick out name and marks...
    if (marks < 0 || marks > 100) return "<p style='color:red'>bad marks</p>";
    String grade = marks >= 90 ? "A" : marks >= 75 ? "B" : marks >= 50 ? "C" : "F";
    DB.put(name, marks);
    return "<p>" + name + " got " + grade + "</p>";
}

// AFTER: each concern is a separate class with one job
record ResultRequest(String name, int marks) {}
record Graded(String name, int marks, String grade) {}
static final class RequestParser { ResultRequest parse(String raw) { ... } }   // 1: raw input -> typed request
static final class GradingService {                                             // 2: rules only, no HTML/storage
    private final ResultRepository repo;
    Graded record(ResultRequest r) {
        if (r.marks() < 0 || r.marks() > 100) throw new IllegalArgumentException("marks must be 0..100");
        String grade = r.marks() >= 90 ? "A" : r.marks() >= 75 ? "B" : r.marks() >= 50 ? "C" : "F";
        repo.save(r.name(), r.marks());
        return new Graded(r.name(), r.marks(), grade);
    }
}
interface ResultRepository { void save(String name, int marks); int count(); }  // 3: storage
static final class HtmlView {                                                   // 4: presentation (swap for JSON
freely)
    String render(Graded g) { return "<p>" + g.name() + " got " + g.grade() + "</p>"; }
    String error(String msg) { return "<p style='color:red'>" + msg + "</p>"; }
}
```

Output:

```
== BEFORE ==
<p>Asha got B</p>
<p style='color:red'>bad marks</p>
== AFTER ==
<p>Asha got B</p>
<p style='color:red'>marks must be 0..100</p>
rows stored: 1
```

Trade-offs

Separated	Mixed
Change one concern safely	Quick to write for a one-off
Parallel team work (UI vs backend)	Everything visible in one place
More types and wiring	Tangled, hard to test

When not to use it

Tiny throwaway scripts. Also avoid over-layering (five layers that just pass data through — "lasagna code").

Common mistakes

- Business rules leaking into controllers or SQL.
- Separating by *technical type* only (all DTOs in one package) while one feature's logic is scattered — prefer cohesive feature packages.

Real-world examples

- **MVC** in Spring MVC, Rails, ASP.NET; **HTML / CSS / JavaScript** on the web.
- **TCP/IP network layers** (the internet's protocol stack: physical links, IP addressing, TCP's reliable delivery, HTTP): each layer handles one concern and talks only to its neighbours.
- **Spring AOP / servlet filters** separate security, transactions and logging from business code.

In the interview

> **30-second answer:** "Separation of Concerns means each module handles one aspect — parsing, rules, storage, presentation — so each can change and be tested independently. In LLD I show it as layers: controller, service, repository, with domain objects in the middle."

Follow-ups:

- *Q: SoC vs SRP?* A: SRP is SoC applied to a single class — one reason to change.
- *Q: What is a cross-cutting concern?* A: One that touches many modules (logging, auth, transactions); handled by AOP, proxies, decorators or filters.

Self-check

1. Q: Name four concerns in a typical web request. → A: Parsing input, business rules, persistence, presentation.

TL;DR

Give each concern its own home and let them communicate through clear, typed interfaces.

---

12. Principle of Least Astonishment (POLA)

What it is

Also the **Principle of Least Surprise**: a component should behave the way most users *expect* from its name, signature and context. If a behaviour would surprise a reasonable reader, change the behaviour or the name.

The problem it solves

Surprising code causes bugs even when it "works as designed", because callers act on expectations. Typical surprises: query-named methods that change state; methods that mutate and return `this` so aliases (other variables pointing to the same object) change; inconsistent naming (`size()` vs `length()` vs `count()` in one API); 0- vs 1-based confusion; silently swallowed errors.

How it works

1. Make names tell the truth: `find` / `get` / `is` must not modify state; if it creates, say so (`getOrCreate`).
2. Follow platform conventions (the `equals`/`hashCode` contract — equal objects must return the same *hash code*, the number hash collections like `HashMap` use to locate them; Java naming; `close()` via `AutoCloseable`), and be consistent within your API — same word for same concept.
3. Prefer immutable results and no hidden side effects.
4. Signal "maybe absent" in the type (`Optional`, a box holding a value or empty), not with a surprise `null` or magic value.
5. Document any unavoidable surprise prominently.

Java example — before/after

Full program: `code/lld/ch05/leastastonishment/Main.java`.

```
// BEFORE: surprising behaviour hidden behind innocent names
static class Money {
    long paise;
    Money add(Money other) { this.paise += other.paise; return this; }   // looks pure, actually mutates
}
static class UserDirectory {
    final Map<String, String> users = new HashMap<>();
    String findUser(String id) {                       // "find" silently CREATES a user
        return users.computeIfAbsent(id, k -> "new-user-" + k);
    }
}

// AFTER: names tell the truth; no hidden side effects
record Money(long paise) {
    Money plus(Money other) { return new Money(paise + other.paise); }   // returns a NEW value
}
static class UserDirectory {
    private final Map<String, String> users = new HashMap<>();
    Optional<String> findUser(String id) { return Optional.ofNullable(users.get(id)); }
    String getOrCreateUser(String id) { return users.computeIfAbsent(id, k -> "new-user-" + k); }
}
// main also probes two real JDK surprises: BigDecimal equals/compareTo, and Arrays.asList(...).add
```

Output:

```
== BEFORE ==
price=Rs 110.00 withTip=Rs 110.00   <-- price changed too!
users after a *find*: 1
== AFTER ==
price=Rs 100.00 withTip=Rs 110.00
find ghost -> Optional.empty, users=0
getOrCreate ghost -> new-user-ghost, users=1
== Real JDK surprises (good to know) ==
new BigDecimal("2.0").equals(new BigDecimal("2.00")) = false
new BigDecimal("2.0").compareTo(new BigDecimal("2.00")) = 0
Arrays.asList(...).add -> UnsupportedOperationException (fixed-size list)
```

The last block is real JDK behaviour and a classic gotcha: `BigDecimal.equals` compares scale too (`2.0` ≠ `2.00`) — use `compareTo` for numeric equality; `Arrays.asList` returns a fixed-size list backed by the array — use `new ArrayList<>(Arrays.asList(...))` if you need to grow it, `List.of(...)` if not.

Trade-offs

Following POLA sometimes means a longer name (`getOrCreateUser`) or an extra method — cheap compared to the bugs surprises cause. The only tension: "expected" depends on the audience, so follow *your* platform's and team's conventions.

When not to use it

Surprising users is never good. But don't twist a design to match a *wrong* expectation; fix the name so expectations become correct.

Common mistakes

- Getters with side effects (lazy creation, logging to DB, incrementing counters).
- Overloading `equals` (writing `equals(Money m)`) instead of overriding `equals(Object o)` — collections silently ignore it.
- Behaviour that changes based on a hidden global flag.

Real-world examples

- **java.time** was designed to fix the surprises of `java.util.Date`/`Calendar` (mutable dates, months numbered 0–11).
- `java.net.URL.equals` may perform DNS resolution (a network lookup turning a host name into an IP address) — documented, but a notorious surprise; `java.net.URI` is preferred as a map key.
- **Unix** `rm` has no undo; tools like `trash` reduce astonishment for new users.

- UI design: "Save" always in the same place; "Cancel" never saves.

In the interview

> **30-second answer:** "Least astonishment means an API does what its name and conventions suggest. Queries don't mutate, creation is named explicitly, absence is typed with Optional, value objects are immutable. Surprises like `BigDecimal.equals` considering scale are exactly what we try not to create."

Follow-ups:

- *Q: Example of violating it in Java?* A: A `get` method that lazily writes to the database, or `Money.add` mutating `this`.
- *Q: How does CQS help?* A: Command-Query Separation (Chapter 6) makes it obvious which methods change state.

Self-check

1. Q: What should `findUser` do if the user doesn't exist? → A: Return an empty `Optional` (or throw a documented exception), never silently create one.
2. Q: Why is `a.add(b)` returning a mutated `this` surprising? → A: Other references to `a` change unexpectedly (aliasing).

TL;DR

Do what the name promises and nothing more; follow conventions; make absence and side effects visible.

---

13. Fail Fast

What it is

A **fail-fast** system detects an invalid state or input **immediately**, where it arises, and stops with a clear error — rather than carrying on and failing mysteriously later (**fail slow / fail silently**).

Analogy: a smoke alarm that beeps at the first smoke, not after the house is on fire.

The problem it solves

Accept a `null` endpoint (a service URL) or negative retry count in a constructor and the crash comes hours later in unrelated code, with a stack trace (the active method calls at the error) pointing to the symptom, not the cause — possibly after bad data was saved. The longer the gap between cause and failure, the costlier the bug.

How it works

1. **Validate at boundaries**: constructors, public method entry, API/controller input, configuration load at startup (fail at **startup** for misconfiguration, not on the first request).
2. Use **guard clauses** (checks at the top of a method that throw or return immediately on bad input) — `Objects.requireNonNull`, `if (bad) throw new IllegalArgumentException(...)` — with a helpful message (what was wrong, what value, what was expected).
3. Make invalid objects **unconstructable** (validation in constructor/factory + immutability).
4. Don't swallow exceptions (`catch (Exception e) {}`).

Fail fast vs fail safe: *fail-safe* means a failure leaves the system in a safe state (a lift's brakes engage on power loss). Interview literature (not the JDK docs, which say "snapshot" or "weakly consistent") also calls **iterators** "fail-safe" when they never throw `ConcurrentModificationException` — `CopyOnWriteArrayList` iterates over a snapshot, `ConcurrentHashMap` iterators are weakly consistent — unlike *fail-fast* iterators (e.g. `ArrayList`) that throw it when the collection is structurally modified during iteration. The JDK documents fail-fast iterators as **best-effort** — never rely on them for correctness.

Java example — before/after

Full program: `code/lld/ch05/failfast/Main.java`.

```java
// BEFORE: garbage is accepted and blows up far away, much later
RetryConfig(String endpoint, int maxAttempts, Duration backoff) {
    this.endpoint = endpoint; this.maxAttempts = maxAttempts; this.backoff = backoff;  // no checks
}
// hours later, deep inside business code: c.endpoint.toUpperCase() ... c.backoff.toMillis() -> NPE

// AFTER: validate at the boundary -- the object can never exist in a bad state
record RetryConfig(String endpoint, int maxAttempts, Duration backoff) {
    RetryConfig {
        Objects.requireNonNull(endpoint, "endpoint");
        Objects.requireNonNull(backoff, "backoff");
        if (maxAttempts < 1) throw new IllegalArgumentException("maxAttempts must be >= 1, was " + maxAttempts);
        if (backoff.isNegative()) throw new IllegalArgumentException("backoff must not be negative");
    }
}

// JDK fail-fast iterator, and the safe alternative
List<String> list = new ArrayList<>(List.of("a", "b", "c"));
try {
    for (String s : list) if (s.equals("a")) list.remove(s);   // structural change during iteration
} catch (ConcurrentModificationException e) { /* print */ }
list.removeIf(s -> s.equals("b"));                              // the safe way
```

Output:

```
== BEFORE ==
config created, nothing complained...
much later: NullPointerException far from the real mistake
== AFTER ==
rejected at construction: endpoint
rejected at construction: maxAttempts must be >= 1, was 0
ok: RetryConfig[endpoint=https://pay, maxAttempts=3, backoff=PT0.1S]
== JDK fail-fast iterator ==
ArrayList iterator threw ConcurrentModificationException
after removeIf: [c]
```

`list.remove("a")` itself succeeded before the iterator noticed the change on its next step, which is why only `"b"` was left for `removeIf` to delete.

Trade-offs

Fail fast	Fail slow / silently
Bugs found early, near their cause	System "keeps running"
Clear diagnostics	Data corruption, confusing symptoms
Some upfront validation code	Less code, more debugging
May stop a whole process for a minor issue	—

When not to use it

Fail fast *inside* a component, but design **resilience at system boundaries**: a UI shouldn't crash because one widget failed; a batch job (processing many records in one run) may skip and log a bad record rather than abort millions. Detect early, decide deliberately (reject, retry, degrade), never ignore.

Common mistakes

- Catching and ignoring exceptions, or returning `null` / `-1` on error.
- Validating deep inside business logic instead of at entry.
- Using `assert` for argument validation in public methods — Java assertions are **disabled by default** (enabled with `-ea`), so they can't be the only guard.

Real-world examples

- `Objects.requireNonNull`; `List.of(null)` throws `NullPointerException` immediately.
- **Spring Boot** (Spring's ready-made application setup) fails at startup if a required bean or property is missing; `@Validated` configuration properties fail on bad values.
- **Circuit breakers** (a wrapper around remote calls that "trips" after repeated failures and rejects calls for a while; Resilience4j, earlier Netflix Hystrix, now in maintenance mode): when a downstream service (one you call) is unhealthy, calls fail immediately instead of waiting for timeouts.
- **Erlang/OTP "let it crash"**: processes fail fast and supervisors restart them.
- **CI pipelines** (Continuous Integration: automated build-and-test on every change) stop on the first failing stage.

In the interview

> **30-second answer:** "Fail fast means validating at the boundary and throwing a clear error immediately, so the failure is close to its cause. I put guard clauses in constructors and public methods so invalid objects can't exist. At system level, circuit breakers fail fast to avoid piling up on slow dependencies."

Follow-ups:

- *Q: Fail-fast vs fail-safe iterators?* A: Fail-fast (`ArrayList`, `HashMap`) throw `ConcurrentModificationException` on structural change during iteration (best-effort); snapshot/weakly consistent ones (`CopyOnWriteArrayList`, `ConcurrentHashMap`) never do.
- *Q: Isn't crashing bad for users?* A: Fail fast inside; handle gracefully at the edge (show an error, retry, degrade).

Self-check

1. Q: Where should validation go? → A: At boundaries: constructors, public method entry, startup config, API input.
2. Q: Why not rely on `assert` for public argument checks? → A: Assertions are disabled by default at runtime.

TL;DR

Check early, throw clearly, never swallow; make invalid states impossible to construct.

---

14. Immutability

What it is

An **immutable object** cannot change its observable state after construction; to "change" it you create a new object. `String`, `Integer`, `LocalDate`, `BigDecimal` and records with immutable components are examples.

The problem it solves

Mutable shared objects cause:

- **Aliasing bugs** — two references to one object; one changes it, the other is surprised.
- **Broken invariants** — a setter leaves the object half-valid.
- **Thread-safety problems** — concurrent reads and writes need locks (letting only one thread at a time run protected code).
- **Hash-collection bugs** — changing a field used by `hashCode` after putting the object in a `HashSet` makes it "disappear".

Immutable objects are **inherently thread-safe** (shared freely without locks), cacheable, safe as map keys and easy to reason about.

How it works

Effective Java's "Minimize mutability" (Joshua Bloch) gives five rules:

1. No methods that modify state (no setters, no mutators).
2. The class can't be extended (`final` class, or private constructors + static factories such as `of(...)`). Records are implicitly final.
3. All fields `final`.
4. All fields `private`.
5. Exclusive access to mutable components: **defensively copy** mutable inputs in the constructor (`List.copyOf`) and never hand out references to mutable internals (return copies or unmodifiable views).

A common extra: "wither" methods returning a new instance with one thing changed (`withStart`, `plusCity`).

Unmodifiable vs immutable: `Collections.unmodifiableList(list)` is a read-only *view* — changes to the original list show through. `List.copyOf` / `List.of` create an independent unmodifiable list. Records are **shallowly** immutable: a `List` component may still be mutable unless copied.

Java example — before/after

Full program: `code/lld/ch05/immutability/Main.java`.

```
// BEFORE: a mutable class that leaks its internals
static class Itinerary {
    private final List<String> cities;              // 'final' reference, but the LIST is mutable
    private LocalDate start;
    Itinerary(List<String> cities, LocalDate start) { this.cities = cities; this.start = start; }
    List<String> getCities() { return cities; }      // hands out the internal list
    void setStart(LocalDate d) { start = d; }
}

// AFTER: final fields, defensive copies, "wither" methods
record Itinerary(List<String> cities, LocalDate start) {   // shallowly immutable; List.copyOf fixes the list
    Itinerary {
        cities = List.copyOf(cities);   // copy in: caller's later edits cannot reach us
    }
    Itinerary withStart(LocalDate newStart) { return new Itinerary(cities, newStart); }
    Itinerary plusCity(String c) {
        List<String> next = new ArrayList<>(cities);
        next.add(c);
        return new Itinerary(next, start);
    }
}
// main: the caller edits its input list and clears getCities(); then t2 = t1.plusCity("Jaipur").withStart(...)
```

Output:

```
== BEFORE ==
shared trip is now: 2026-10-01 []   <-- changed by outsiders
== AFTER ==
cannot clear: unmodifiable
t1 = Itinerary[cities=[Delhi, Agra], start=2026-10-01]
t2 = Itinerary[cities=[Delhi, Agra, Jaipur], start=2026-11-05]
== JDK immutables ==
hello / HELLO ; 2026-01-31 plusMonths(1) = 2026-02-28 (d unchanged: 2026-01-31)
```

`final List` only stops the *reference* changing — outsiders both added to and cleared the list. In *after*, the accessor returns an unmodifiable list and "changes" produce new values while `t1` stays intact. `LocalDate.plusMonths` returns a new date (clamping Jan 31 + 1 month to Feb 28).

Trade-offs

Immutable	Mutable
Thread-safe, no locks	Needs synchronisation when shared
Safe map keys, cacheable, easy reasoning	In-place updates, no allocation
Allocation for every "change" (usually cheap on modern JVMs — Java Virtual Machines)	Aliasing and invariant bugs
Large objects: copying cost (use builders or persistent data structures, which share unchanged parts between versions)	—

When not to use it

- Hot loops building big results — use a local mutable builder (`StringBuilder`, then `toString()`) and expose the immutable result.
- Entities whose identity persists while state changes (a `BankAccount` balance) — keep them mutable but encapsulated; make their *values* (Money, Address) immutable.
- ORM entities (Object-Relational Mapping maps classes to tables; JPA is Java's standard ORM API) usually need no-arg constructors and setters.

Common mistakes

- Thinking `final` fields make an object immutable (not if a field refers to a mutable object).
- Returning the internal list from a getter.
- `Collections.unmodifiableList` over a list the caller still holds.
- Letting `this` escape during construction (e.g. registering a listener in the constructor) — other threads may see a half-built object.

Real-world examples

- `String`, wrapper types, `BigDecimal`, `java.time.*`, `List.of`, `Map.of`, records.
- **Git commits** are immutable snapshots identified by hash (an ID computed from the content).
- **Kafka log records** are immutable once appended.
- **Redux / React state** is treated as immutable, so change detection is a cheap reference check.

In the interview

> **30-second answer:** "Immutable objects never change after creation: final class, private final fields, no setters, defensive copies in and out. They're thread-safe without locks and safe as map keys. I make value objects like Money, Address and configuration immutable, and keep mutable state encapsulated inside entities."

Follow-ups:

- *Q: Are records immutable?* A: Shallowly — fields are final, but a mutable component (like an `ArrayList`) must be defensively copied.
- *Q: `unmodifiableList` vs `List.copyOf`?* A: The first is a live read-only view; the second an independent unmodifiable copy.
- *Q: Why must immutable classes be final?* A: A subclass could add mutable state or override methods to break the guarantees.

Self-check

1. Q: Does `private final List<String> items` make a class immutable? → A: No; the list itself can still be modified.

2. Q: Why are immutable objects thread-safe? → A: No thread can modify them, so there are no data races on their state (given safe construction).

TL;DR

No setters, final everything, copy mutable inputs and outputs; "changing" means creating a new value.

---

15. Design by Contract (DbC)

What it is

Design by Contract (Bertrand Meyer, Eiffel) treats a method as a contract between a **client** (caller) and a **supplier** (the method):

- **Precondition** — what must be true *before* the call; the caller's obligation (`pop` requires a non-empty stack).
- **Postcondition** — what the method guarantees *after* the call if the precondition held; the supplier's obligation (after `push(x)`, size grew by one and the top is `x`).
- **Class invariant** — what is always true about every object between public calls (`0 ≤ size ≤ capacity`).

Analogy: a courier contract — you pack and pay (precondition); delivery in 2 days (postcondition); the parcel is never opened in transit (invariant).

The problem it solves

Without explicit contracts each side guesses: the caller doesn't know what inputs are allowed; the method either checks everything defensively (duplicated checks everywhere) or nothing (silent corruption). Errors show up as ambiguous return values (`-1`) or lost data.

How it works

1. For each public method, write down the precondition, postcondition and class invariant (at least in Javadoc — the `/** ... */` comments that generate API docs).
2. Check preconditions at entry and throw a clear exception (`IllegalArgumentException`, `IllegalStateException`); check postconditions and invariants during development/testing (assertions or internal checks).
3. Give callers **query methods** to check preconditions themselves (`isEmpty()`, `isFull()`).
4. With inheritance, obey the **contract rules of subtyping** (what the Liskov Substitution Principle formalises): a subclass may **weaken** (accept more) preconditions and **strengthen** (promise more) postconditions, never the reverse, and must preserve the invariant.

Java example — before/after

Full program: `code/lld/ch05/contract/Main.java`.

```
// BEFORE: no stated contract; misuse silently corrupts state
void push(int x) { if (size < items.length) items[size++] = x; }   // silently drops when full
int pop() { return size > 0 ? items[--size] : -1; }               // -1: is that data or an error?

// AFTER: every public method states and checks its contract
/** pre: !isFull(). post: size() == old size + 1 and peek() == x. */
void push(int x) {
    require(!isFull(), "push requires !isFull()");
    int oldSize = size;
    items[size++] = x;
    ensure(size == oldSize + 1 && peek() == x, "push postcondition");
    checkInvariant();
}
/** pre: !isEmpty(). post: size() == old size - 1. */
int pop() {
    require(!isEmpty(), "pop requires !isEmpty()");
    int oldSize = size;
    int top = items[--size];
    ensure(size == oldSize - 1, "pop postcondition");
    checkInvariant();
    return top;
}
boolean isEmpty() { return size == 0; }
boolean isFull() { return size == items.length; }
// invariant: 0 <= size <= capacity (constructor also requires capacity > 0)
private void checkInvariant() { if (size < 0 || size > items.length) throw new AssertionError("invariant broken");
}
private static void require(boolean ok, String what) { if (!ok) throw new IllegalStateException("precondition
failed: " + what); }
private static void ensure(boolean ok, String what) { if (!ok) throw new AssertionError("postcondition failed: " +
what); }
```

Output:

```
== BEFORE ==
pop=2 pop=1 pop=-1   <-- 3 lost, -1 ambiguous
== AFTER ==
stack [1, 2] full=true
precondition failed: push requires !isFull()
pop=2 pop=1
precondition failed: pop requires !isEmpty()
```

Trade-offs

Explicit contracts	Implicit
Clear responsibilities, precise errors	Less code
Documentation that can be executed	Ambiguity, defensive checks everywhere
Runtime cost of checks (can be disabled for postconditions/invariants in production)	Silent corruption

When not to use it

Full postcondition/invariant checking in performance-critical inner loops — keep precondition checks on public APIs and move expensive postcondition checks into tests or `assert` statements.

Common mistakes

- Using `assert` for public preconditions (disabled by default).
- Subclasses strengthening preconditions (a `SavingsAccount.withdraw` rejecting amounts the base `Account.withdraw` contract allowed) — an LSP violation. (`Square extends Rectangle` is the mirror case: `Square.setWidth` breaks the *postcondition* that height stays unchanged.)
- Returning special values instead of stating a precondition.

Real-world examples

- **Java's API docs** are contracts: `List.get(int)` throws `IndexOutOfBoundsException` if out of range; `Comparable.compareTo` and `Object.equals`/`hashCode` specify contracts every implementation must honour (equal objects → equal hash codes).
- **Guava** `Preconditions.checkArgument/checkState` and Spring's `Assert` are precondition helpers; **Kotlin** `require()` / `check()` mirror preconditions and state checks.
- **Eiffel** builds in `require`, `ensure` and `invariant`; **Ada 2012** has `Pre`/`Post` aspects.

In the interview

> **30-second answer:** "Design by Contract defines preconditions the caller must satisfy, postconditions the method guarantees, and invariants always true for the object. I enforce preconditions with guard clauses, expose query methods like `isEmpty`, and remember subclasses can only weaken preconditions and strengthen postconditions — that's LSP."

Follow-ups:

- *Q: DbC vs defensive programming?* A: Defensive programming checks everything everywhere; DbC assigns each check to one party — preconditions to the caller (verified at entry), postconditions to the supplier.
- *Q: Which exceptions fit?* A: `IllegalArgumentException` for bad arguments, `IllegalStateException` when the object's state forbids the call, `NullPointerException` for nulls (via `requireNonNull`).

Self-check

1. Q: What can a subclass do to preconditions? → A: Weaken (accept more), never strengthen.

TL;DR

State and enforce who promises what: preconditions (caller), postconditions (method), invariants (always).

---

16. GRASP — General Responsibility Assignment Software Patterns

What it is

GRASP is Craig Larman's set of nine patterns (many say principles or guidelines), from *Applying UML and Patterns* (UML, the Unified Modeling Language, is the standard diagram notation), for the most frequent question in object design: **"Which class should be responsible for this?"** A *responsibility* is *knowing* something (data, related objects, derived values) or *doing* something (computing, creating, coordinating). GRASP is a thinking aid — a vocabulary for justifying design decisions — not code templates.

History note: in the 1st edition (1997) the ninth item was "Don't Talk to Strangers" (the Law of Demeter); the 2nd edition (2001) replaced it with the more general **Protected Variations**.

#	GRASP guideline	Question it answers	Short answer
1	Information Expert	Who does this job?	The class that has the information needed
2	Creator	Who creates an object A?	A class that contains, records, closely uses or has the data to initialise A
3	Controller	Who first handles a system event from the UI?	A non-UI object representing the use case (one user goal, e.g. checkout) or the whole system
4	Low Coupling	How do we reduce the impact of change?	Assign responsibilities to keep dependencies few
5	High Cohesion	How do we keep classes focused and understandable?	Assign responsibilities so each class does related work
6	Polymorphism	How do we handle behaviour that varies by type?	Give each type its own implementation of a shared operation
7	Pure Fabrication	What if no domain class fits without hurting cohesion/coupling?	Invent a helper class (e.g. a repository)
8	Indirection	How do we avoid direct coupling between two things?	Put an intermediate object between them
9	Protected Variations	How do we shield elements from changes in others?	Wrap points of instability behind a stable interface

Analogy: staffing a hospital. Whoever has the patient's chart writes the summary (Expert); the maternity ward registers newborns (Creator); the front desk routes visitors (Controller); nobody needs to know everyone (Low Coupling); each department has a focused speciality (High Cohesion).

The problem it solves

Beginners end up with one "god" class (knows and does far too much — often the UI screen or a `Manager`) plus anemic data classes, or logic spread randomly. GRASP gives repeatable reasons for each placement — and concrete interview explanations.

How it works

For each system event/use case (`scan item`, `pay`) choose a **Controller**; give each thing to be *known* or *computed* to its **Information Expert** and each object to be *created* to its **Creator**; use **Polymorphism** where behaviour varies by type and **Protected Variations** via **Indirection** (an interface or adapter — a class converting one interface into the one your code expects) where something external or unstable touches you; create a **Pure Fabrication** when no domain object is a good home (persistence, I/O — files, network). Judge every choice by **Low Coupling** and **High Cohesion** (the evaluative forces).

Java example — before/after (all nine in one checkout)

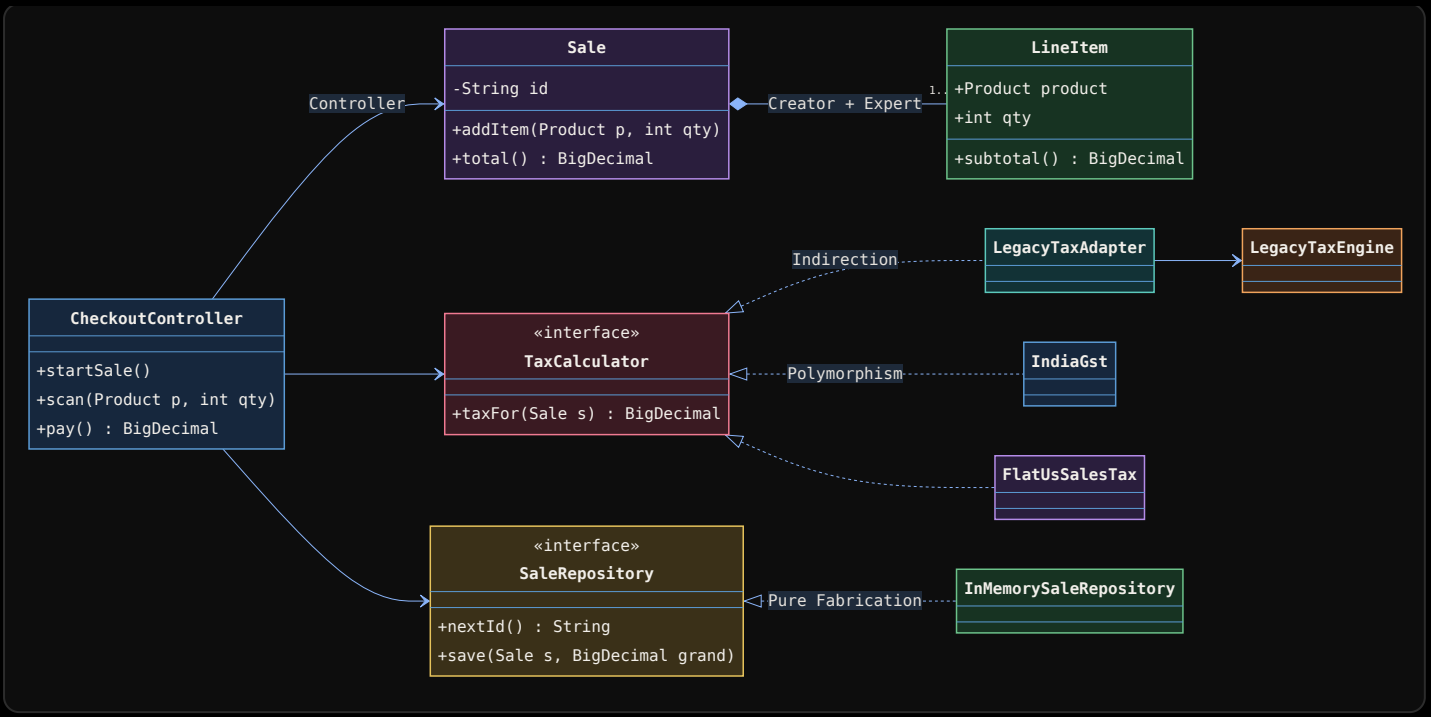
The *before* is a UI `CheckoutScreen` that creates line items, reaches into them to total the sale, hard-codes tax rules and builds SQL. The *after* assigns each responsibility by GRASP. Full program: `code/lld/ch05/grasp/Main.java`.

```
// BEFORE: a "god" screen class does everything
void onScan(Product p, int qty) {
    LineItem li = new LineItem();              // UI creates domain parts
    li.product = p; li.qty = qty;
    sale.items.add(li);
}
void onPayClicked(String region) {
    BigDecimal total = BigDecimal.ZERO;        // UI reaches into Sale -> LineItem -> Product
    for (LineItem li : sale.items) total = total.add(li.product.price().multiply(BigDecimal.valueOf(li.qty)));
    BigDecimal tax = region.equals("IN") ? total.multiply(new BigDecimal("0.18"))   // vendor rule inline
            : total.multiply(new BigDecimal("0.07"));
    sqlLog.add("INSERT INTO sales VALUES(" + total.add(tax) + ")");
}

// AFTER: responsibilities assigned with GRASP
record LineItem(Product product, int qty) {                       // INFORMATION EXPERT for its subtotal
    BigDecimal subtotal() { return product.price().multiply(BigDecimal.valueOf(qty)); }
}
static final class Sale {                                         // EXPERT for total + CREATOR of LineItems;
HIGH COHESION
    private final List<LineItem> items = new ArrayList<>();
    void addItem(Product p, int qty) { items.add(new LineItem(p, qty)); }
    BigDecimal total() { return items.stream().map(LineItem::subtotal).reduce(BigDecimal.ZERO, BigDecimal::add); }
}
interface TaxCalculator { BigDecimal taxFor(Sale sale); }         // PROTECTED VARIATIONS
static final class IndiaGst implements TaxCalculator {           // POLYMORPHISM (also FlatUsSalesTax at 0.07)
    public BigDecimal taxFor(Sale s) { return s.total().multiply(new BigDecimal("0.18")); }
}
static final class LegacyTaxAdapter implements TaxCalculator {   // INDIRECTION around a vendor engine we don't
control
    private final LegacyTaxEngine engine; private final String country;   // engine returns tax as a double, in
cents
    public BigDecimal taxFor(Sale s) {   // converts to/from the vendor's types at the boundary only
        long cents = s.total().movePointRight(2).longValueExact();
        return BigDecimal.valueOf(Math.round(engine.computeTaxCents(cents, country))).movePointLeft(2);
    }
}
interface SaleRepository { String nextId(); void save(Sale sale, BigDecimal grandTotal); int count(); } // PURE
FABRICATION
static final class CheckoutController {                           // CONTROLLER; LOW COUPLING: depends on
interfaces
    private final TaxCalculator tax; private final SaleRepository repo;
    private Sale current;
    void startSale() { current = new Sale(repo.nextId()); }
    void scan(Product p, int qty) { current.addItem(p, qty); }
    BigDecimal pay() {
        BigDecimal grand = current.total().add(tax.taxFor(current)).setScale(2, RoundingMode.HALF_UP);
        repo.save(current, grand);
        return grand;
    }
}
```

Output (tea ₹120 × 2 + cup ₹80; the legacy engine taxes "GB" at 20%):

```
== BEFORE ==
screen shows total 377.60
sql: [INSERT INTO sales VALUES(377.6000)]
== AFTER ==
IndiaGst -> grand total 377.60
FlatUsSalesTax -> grand total 342.40
LegacyTaxAdapter -> grand total 384.00
sales saved: 3
```



At runtime: the UI calls `scan` → controller → `Sale.addItem` (creates the `LineItem`); `pay` → `Sale.total()` (Expert) → `TaxCalculator.taxFor` (Polymorphism) → `SaleRepository.save` (Pure Fabrication). Each guideline in brief:

16.1 Information Expert

What it is

Give a responsibility to the class that **has the information** needed. `LineItem` knows price and quantity → `subtotal()`; `Sale` knows its line items → `total()`.

The problem it solves

Prevents "manager" classes that pull data out of others to compute things (the *Feature Envy* smell), duplicating logic and coupling to everyone's internals.

How it works

State the responsibility ("compute sale total") → what information is needed (line subtotals) → who holds it (Sale holds items; each LineItem holds price × qty). Partial experts cooperate.

Trade-offs

Keeps behaviour with data (Tell-Don't-Ask), but applied blindly it hurts cohesion: `Sale` has the data to save itself, yet SQL in `Sale` mixes concerns — hence Pure Fabrication.

When not to use it

When it would put technical concerns (persistence, UI, network) into domain objects.

Common mistakes

Computing everything in a service with getters; or the opposite, an entity that saves/prints/emails itself.

Real-world examples

`String.length()`, `List.size()`, `LocalDate.isLeapYear()`; in a Swiggy/Zomato-style cart, `Cart` computes its own item total.

In the interview

"`total()` lives on `Order` because it owns the line items — Information Expert." *Follow-up: when does Expert conflict with other principles?* When it would add persistence or UI to a domain class; then use Pure Fabrication.

Self-check

Q: Who computes a line item's subtotal? → A: `LineItem` — it has price and quantity.

TL;DR

The one with the data does the work.

16.2 Creator

What it is

Class **B** should create instances of **A** if B *contains* or aggregates A, *records* A, *closely uses* A, or *has the initialising data* for A. Here `Sale` creates `LineItem`s.

The problem it solves

Who calls `new`? Random placement spreads coupling to A's constructor; a class already related to A adds no new coupling.

How it works

Prefer the class that aggregates/contains A (strongest reason); otherwise one that records or closely uses it, or has its initial data. If creation is complex or varies (families, pooling, choosing a subclass), use a **Factory** or **Builder** (Chapter 8).

Trade-offs

Simple and low-coupling for plain objects.

When not to use it

Complex or conditional creation → Factory/Builder; framework-managed objects → DI container.

Common mistakes

UI or controller code doing `new LineItem(...)` and pushing it into the sale's list (the *before* code).

Real-world examples

`HashMap` creating its own node objects for its entries; a `Document` creating its `Paragraph`s; an `ExecutorService` creating `Thread`s through its `ThreadFactory` (a factory, because creation varies).

In the interview

"`Sale.addItem(product, qty)` creates the `LineItem` because the sale contains them — Creator." *Follow-up: Creator vs Factory?* Creator picks an existing class already related to A (no new coupling); a Factory is a Pure Fabrication added when creation is complex or must choose among subclasses.

Self-check

Q: Strongest reason for B to create A? → A: B contains/aggregates A.

TL;DR

The container (or the one with the initial data) creates.

16.3 Controller

What it is

The first object **beyond the UI layer** that receives and coordinates a **system operation** (an event from an actor — a person or external system — such as "scan item" or "pay"). Two kinds: a **facade controller** for the whole system/device (`Register`, `ParkingLotSystem`) or a **use-case controller** for one scenario (`CheckoutController`, `BookTicketHandler`).

The problem it solves

Keeps business logic out of UI classes, so the same logic works from a web page, a mobile app or a test.

How it works

List system events from use cases; assign each to a controller; the controller **delegates** — it coordinates, it doesn't do the domain work.

Trade-offs

Clean UI/domain separation, but a controller doing real work becomes **bloated** (low cohesion) — split into use-case controllers and push work to experts.

When not to use it

Very small programs with trivial UI and logic.

Common mistakes

Fat controllers with pricing, validation and SQL; controllers that are UI classes (a Swing `JFrame` computing totals).

Real-world examples

Spring `@RestController` → `@Service` (the service often plays GRASP Controller); `DispatcherServlet` is a front controller (one entry point routing every web request); in an Uber-like app, `TripService.requestRide()` handles "request ride".

In the interview

"`ParkingLotController.park(vehicle)` receives the system event and delegates to `ParkingFloor` and `SpotAllocator` — Controller, kept thin." *Follow-up: facade or use-case controller?* A facade controller is fine while there are few system events; when bloated, split into use-case controllers.

Self-check

Q: Should a controller calculate tax? → A: No — it delegates to the expert or a tax strategy.

TL;DR

A thin non-UI coordinator receives each system event and delegates.

16.4 Low Coupling

What it is

An **evaluative** guideline: among choices, prefer the one with **fewer dependencies between classes**. The controller depends only on interfaces (`TaxCalculator`, `SaleRepository`), not vendors or databases.

The problem it solves

High coupling: changes ripple, classes can't be reused alone, and understanding one requires understanding many.

How it works

Compare alternatives by who knows about whom; prefer interfaces, fewer collaborators, one-directional dependencies, no cycles; use DI.

Trade-offs

Striving for zero coupling produces bloated indirection.

When not to use it

Coupling to *stable* things (JDK, `String`, `List`) is harmless — decouple from unstable or external dependencies.

Common mistakes

Circular package dependencies; UI knowing the database; everything depending on a god class.

Real-world examples

Microservices communicating over well-defined APIs/events; JDBC drivers hidden behind interfaces.

In the interview

"My service depends on `PaymentGateway`, not on Stripe's SDK (Software Development Kit — the vendor's client library), so switching provider touches one adapter." *Follow-up: can coupling be zero?* No — objects must collaborate. Aim for few dependencies, on stable abstractions, without cycles.

Self-check

Q: Is depending on `java.util.List` bad coupling? → A: No — it is stable.

TL;DR

Choose the design with fewer, more stable dependencies.

16.5 High Cohesion

What it is

The other **evaluative** guideline: keep each class's responsibilities **focused and related**. `Sale` does sale maths; persistence is in the repository; tax in the calculator.

The problem it solves

Low-cohesion classes (`Utils`, `Manager`, a do-everything screen) are hard to understand, reuse and change, and change for many reasons (violating SRP).

How it works

When adding a responsibility, ask "is this related to what the class already does?" If not, place it elsewhere or create a class. Larman's four levels, worst to best: *very low* (one class responsible for many things in very different areas) → *low* (one class alone doing a big, complex job in one area) → *moderate* (a few light responsibilities in different areas, related to the class concept but not to each other) → *high* (a moderate number of related responsibilities in one area, collaborating with others for the rest — the goal).

Trade-offs

More, smaller files, each simple. Coupling and cohesion usually improve together.

When not to use it

Rarely; grouping for performance (a remote API returning coarse-grained data) may lower cohesion on purpose.

Common mistakes

"Helper"/"Utils" dumping grounds; entities that also send emails.

Real-world examples

`java.time.LocalDate` (dates only), `java.util.regex.Pattern` (regex only); Spring Data repositories (persistence only).

In the interview

"I split `BookingService` into seat locking, pricing and payment so each class has one focused job." *Follow-up: how do cohesion and coupling interact?* Moving unrelated work out of a class raises its cohesion and removes the dependencies that work dragged in.

Self-check

Q: A common low-cohesion smell? → A: A `Utils` / `Manager` class with unrelated methods.

TL;DR

Each class does one related set of things.

16.6 Polymorphism

What it is

When behaviour **varies by type**, give each type its own implementation of a shared operation instead of branching with `if/else` / `switch`. `IndiaGst`, `FlatUsSalesTax` and `LegacyTaxAdapter` each implement `taxFor`.

The problem it solves

Type-checking conditionals repeated across the code; adding a type means editing every branch (violates OCP).

How it works

Identify the varying operation (`taxFor`) → declare it on an interface/abstract class → implement per variant → callers invoke it without knowing the concrete type.

Trade-offs

Easy to add types; harder to add operations across all types (touch every class — Visitor, Chapter 11, flips this).

When not to use it

For closed sets of types where you often add *operations*, a `switch` over a `sealed` hierarchy (Java 21 pattern matching — a `switch` that tests an object's type and unpacks it in one step) can be clearer.

Common mistakes

Polymorphic classes for variation that doesn't exist (YAGNI); `instanceof` chains after introducing the interface.

Real-world examples

`Comparator` implementations, `InputStream` subclasses, `Runnable` / `Callable`, checkout payment methods (card, UPI, wallet).

In the interview

"Instead of `if (vehicle instanceof Car)`, each `Vehicle` answers `requiredSpotSize()` — Polymorphism." *Follow-up: a new operation across all types?* It touches every class; if frequent, use a `sealed` hierarchy with pattern-matching `switch`, or Visitor.

Self-check

Q: Which change does polymorphism make easy? → A: Adding a new type/variant.

TL;DR

Let each type implement the varying behaviour; don't switch on types.

16.7 Pure Fabrication

What it is

A class that **doesn't represent a domain concept**, invented to keep high cohesion and low coupling when Information Expert would give a bad result. The shop has no "repository", but SQL in `Sale` would ruin cohesion — so `SaleRepository`.

The problem it solves

Persistence, logging, ID generation and messaging don't fit any domain class without dragging technical concerns into it.

How it works

Notice the Expert choice would make a domain class do technical work → create a focused, reusable service class named for the job (`SaleRepository`, `NotificationSender`, `IdGenerator`).

Trade-offs

Great for cohesion and reuse; overuse produces an anemic domain.

When not to use it

When a real domain object is a natural home — prefer the Expert first.

Common mistakes

Moving domain logic (pricing, rules) into fabricated "Manager" services.

Real-world examples

DAOs (Data Access Objects — classes hiding database queries)/repositories, `ObjectMapper` (Jackson, the common Java JSON library), `Clock` providers, infrastructure `@Service` classes; most GoF helper objects (Strategy, Adapter, Factory objects).

In the interview

"`TicketRepository` isn't a domain thing — it's a pure fabrication that keeps persistence out of `Ticket`." *Follow-up: risk of too many?* An anemic domain, all behaviour in "services". Keep rules on domain objects; fabricate only for technical or cross-cutting work.

Self-check

Q: Is `Sale` a pure fabrication? → A: No — it's a domain concept. `SaleRepository` is.

TL;DR

Invent a helper class when no domain class is a clean home.

16.8 Indirection

What it is

Assign a responsibility to an **intermediate object** that mediates between two components so they aren't directly coupled. `LegacyTaxAdapter` sits between `CheckoutController` and the vendor's `LegacyTaxEngine`.

The problem it solves

Direct dependencies on external or volatile components (vendor SDKs, hardware, other subsystems).

How it works

Introduce an interface plus an adapter, facade, mediator (a hub objects talk through instead of to each other) or proxy; the two sides talk through it.

Trade-offs

Decoupling vs an extra hop (small performance cost, one more class). The saying usually attributed to David Wheeler — any problem in computer science can be solved by another level of indirection — has the retort "except the problem of too many levels of indirection".

When not to use it

Between stable, cohesive classes that always change together.

Common mistakes

Pass-through layers that add nothing.

Real-world examples

Adapter/Facade/Mediator/Proxy; JDBC's `DriverManager`; API gateways (a single front door routing client calls to many services); message brokers between producers and consumers.

In the interview

"I don't call the SMS vendor directly — an `SmsSender` interface with a `TwilioSmsSender` adapter provides indirection." *Follow-up: Indirection vs Protected Variations?* Indirection is the mechanism (an intermediate object); Protected Variations is the goal (shield the system from a point of change).

Self-check

Q: Which GoF patterns are forms of indirection? → A: Adapter, Facade, Mediator, Proxy (and Bridge).

TL;DR

Put a middleman between things that shouldn't know each other.

16.9 Protected Variations

What it is

Identify **points of predicted variation or instability** and wrap them with a **stable interface**, so variation doesn't ripple. `TaxCalculator` protects checkout from tax-rule and vendor changes. Larman notes it is essentially David Parnas's *information hiding* (keeping changeable design decisions hidden inside one module) and Meyer's Open/Closed Principle stated as a responsibility guideline (close to "encapsulate what varies"); the Law of Demeter — the 1st edition's "Don't Talk to Strangers" — is one mechanism for it.

The problem it solves

A change in one place (a vendor, a regulation, a format) forcing edits throughout the system.

How it works

Find variation points (external systems, changing rules, platforms) → define a stable interface around each → put variants behind it (Polymorphism, Indirection, data-driven config).

Trade-offs

Protection costs; protecting what never varies is speculative generality.

When not to use it

Stable, unlikely-to-change code (YAGNI) — protect only *predicted* variation.

Common mistakes

Leaky interfaces exposing vendor types (returning a `StripeCharge` from `PaymentGateway`).

Real-world examples

JDBC, JPA, SLF4J, Java's `ServiceLoader` for pluggable providers; the `java.nio.file.FileSystem` abstraction.

In the interview

"Payment providers are a variation point, so `PaymentGateway` is my protected variation — the booking flow never sees provider details." *Follow-up: which points to protect?* Predicted ones — vendors, regulations, formats, platforms — not every class.

Self-check

Q: SOLID principle closest to Protected Variations? → A: Open/Closed (also Dependency Inversion).

TL;DR

Wrap what's unstable behind a stable interface.

GRASP in the interview (whole set)

> **30-second answer:** "GRASP is Larman's nine guidelines for assigning responsibilities. Information Expert and Creator place most behaviour, a thin Controller takes system events, Polymorphism and Protected Variations handle what varies, Indirection and Pure Fabrication keep technical concerns out of the domain, and I judge every choice by Low Coupling and High Cohesion."

Follow-ups:

- *Q: GRASP vs GoF patterns?* A: GRASP is more fundamental: Strategy (Polymorphism + Protected Variations), Adapter (Indirection) and Repository (Pure Fabrication) are specific applications.
- *Q: Which two GRASP items are evaluative rather than assignment rules?* A: Low Coupling and High Cohesion — used to compare alternatives.
- *Q: Expert says `Sale` should save itself — why not?* A: It hurts cohesion and couples the domain to storage; use a Pure Fabrication (repository).

Self-check (GRASP)

1. Q: Who handles the "pay" button event after the UI? → A: A Controller (use-case or facade).
2. Q: Which guideline justifies an adapter around a vendor library? → A: Indirection (serving Protected Variations).
3. Q: Which guideline covers inventing `IdGenerator`? → A: Pure Fabrication.

TL;DR

Put work where the data is, let containers create, keep controllers thin, vary by polymorphism behind stable interfaces, invent helpers when needed — and judge it all by coupling and cohesion.

---

17. Domain-Driven Design (DDD) Essentials

What it is

Domain-Driven Design, named by Eric Evans in *Domain-Driven Design: Tackling Complexity in the Heart of Software* (2003), builds software whose code mirrors the business it serves. The **domain** is the subject area — lending, shipping, grocery retail. Two halves:

- **Strategic design** — splitting a large business into parts, each with its own model: **ubiquitous language**, **bounded contexts**, **context maps**, the **anti-corruption layer**.
- **Tactical design** — building blocks *inside* one part: **entities**, **value objects**, **aggregates**, **domain events**, **repositories**, plus domain services and factories.

DDD decides **which classes to have and where their boundaries are**, starting from the business rather than the database.

Analogy: in a hospital, "patient" means insurance and appointments at reception, blood type and allergies in theatre, amounts owed in billing. Each department (a bounded context) keeps its own file and vocabulary and exchanges only what it needs.

Key terms (defined once):

Term	Meaning
Ubiquitous language	The shared vocabulary domain experts and developers both use — in conversation, documents *and code*. If the business says "place an order", the method is `place()`, not `setStatus(2)`.
Bounded context	A boundary inside which one model and one language apply consistently. The same word may mean something else in another context (a Sales "order" vs a Shipping "consignment"). Often one team and one service.
Context map	A picture of the bounded contexts and how they relate (who depends on whom, who translates).
Anti-corruption layer (ACL)	A translation layer at a context's border that converts another context's (or a legacy — old, still-running — system's) model into yours, so theirs cannot leak in.
Entity	An object defined by its **identity**, which stays the same while attributes change (`Order` 1001 is the same order after items are added). Equality = same id.
Value object	An object defined only by its **attributes**, no identity; immutable; interchangeable when equal (`Money(₹350)`, an `Address`, a `DateRange`). Equality = same values.
Aggregate	A cluster of entities and value objects treated as **one unit for changes**, with one entity as its **aggregate root** — the only object outsiders may hold or call. The root enforces the cluster's invariants.
Domain event	A record of something the business cares about that happened, named in the past tense (`OrderPlaced`, `PaymentFailed`). Other parts react to it.
Repository	A collection-like interface for loading and saving **aggregates** — one per aggregate root (`OrderRepository`, never `OrderLineRepository`: outsiders reach an `OrderLine` only through its `Order`) — hiding the database (Chapter 11, §11.7.3).
Domain service	A domain operation that belongs to no single entity (e.g. "transfer money between two accounts"). Stateless.

The problem it solves

Business systems fail less from bad algorithms than from **muddled models**:

- **One model for everything** — a single `Customer` or `Product` class grows fields for every department until nobody can change it safely (a "big ball of mud").
- **Translation errors** — developers say `status=3`, the business says "on hold"; requirements get lost between vocabularies.
- **Anemic domain model** (Fowler's term) — data bags with getters/setters, every rule in services; rules duplicate and anyone can create invalid objects (Tell, Don't Ask violated at scale).

- **No consistency boundary** — without aggregates, any code updates any object, locking and transactions sprawl, and invariants ("an order's total never exceeds the credit limit") are checked on some paths only.
- **Tight coupling between teams** — if Shipping reads Sales' tables, Sales cannot change its schema.

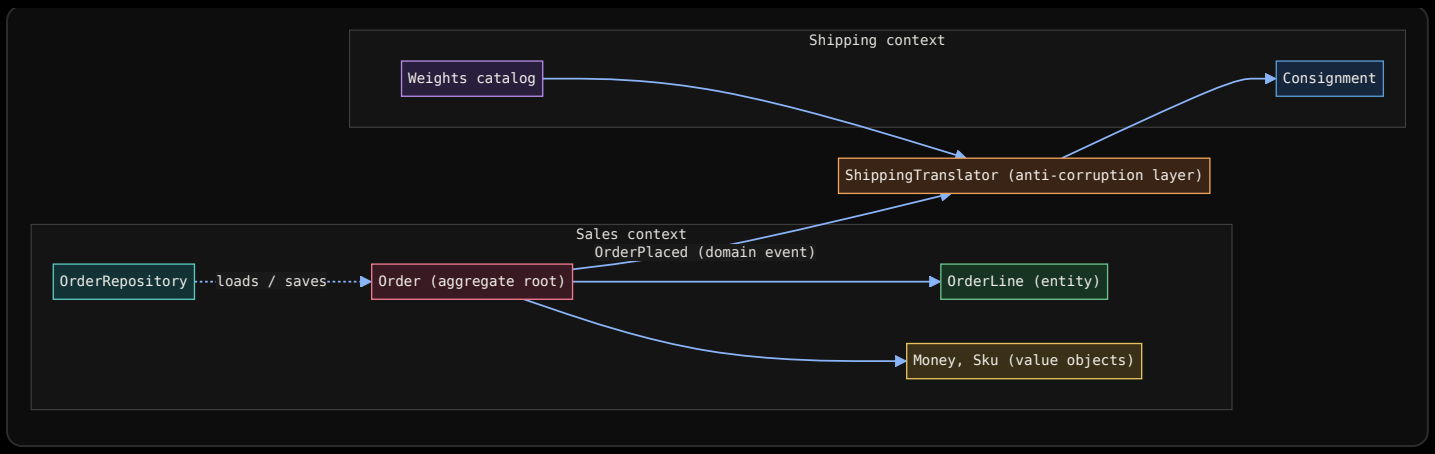
How it works

Strategic design — split by meaning, not by table

1. **Talk to domain experts and write down the language.** Nouns become candidate entities/value objects, verbs become methods, "when X happens" becomes a domain event.
2. **Draw the bounded contexts.** Where the same word starts meaning different things, there is a boundary. E-commerce: *Catalog* (descriptions, images), *Sales* (orders, prices, promotions), *Payments*, *Shipping* (parcels, weights, carriers), *Support*.
3. **Decide how contexts talk (the context map):** one side publishes events or an API and the other consumes it; a **shared kernel** (a small library both own — use sparingly); **conformist** (simply accept the other model); **anti-corruption layer** (translate — the safest choice against a legacy or third-party system).
4. **Map contexts to teams and services.** A bounded context is the most natural microservice boundary, which is why DDD is the usual starting point for splitting a monolith (one application built and deployed as a unit) — see the Strangler Fig notes.

Tactical design — building blocks inside one context

- **Prefer value objects.** Wrap primitives (raw built-in types like `double`, `String`) that carry rules: `Money` (amount + currency, no negatives), `Email`, `Quantity`, `DateRange`. They are immutable (§14), validate themselves (§13) and make signatures honest (`transfer(Money)`, not `transfer(double)`).
- **Entities** get an id value object (`OrderId`) and ubiquitous-language methods that protect their invariants.
- **Aggregate design rules** (Vaughn Vernon, *Implementing Domain-Driven Design*, 2013):
  1. Protect **true invariants** — what must be consistent *immediately* — inside the boundary.
  2. Keep aggregates **small** (often the root plus a few value objects or child entities).
  3. **Reference other aggregates by id** (`CustomerId`, not `Customer`).
  4. **Change one aggregate per transaction**; keep others in step via domain events (**eventual consistency** — they catch up shortly after; see System Design Fundamentals).
- **Domain events** are raised inside the aggregate on a state change and published **after** the change is saved — reliably via the **transactional outbox**: write the event to an outbox table in the same transaction as the aggregate; a relay publishes it (see the Saga and Event-Driven Architecture notes).
- **Application services** (a thin layer above the domain) load an aggregate, call one domain method, save it — they orchestrate, the aggregate decides (GRASP Controller, §16.3).
- **Reads can use a different model.** When screens need data joined across aggregates, a separate read model fed by events is often simpler — **CQRS** (Command Query Responsibility Segregation; see the CQRS notes); storing the events themselves as the source of truth is **Event Sourcing** (Chapter 11, §11.7.6).



Java example — before/after

Full program: `code/lld/ch05/ddd/Main.java`.

```java
// BEFORE: an anemic model -- data bags with public fields, rules in a service
static class OrderLine { public String sku; public int qty; public double price; }
static class Order { public List<OrderLine> lines = new ArrayList<>(); public String status = "NEW"; public double
total; }
// OrderService.addLine(...) is the only place that "knows" the rules... in theory;
// main sets status = "SHIPPED", then adds a line with qty -5 directly -- nothing stops it

// AFTER: the rules live inside the model, in the business's own words
record Money(long paise) {                                  // value object: immutable, equal by value, self-
validating
    Money { if (paise < 0) throw new IllegalArgumentException("Money cannot be negative"); }
    static Money rupees(long r) { return new Money(r * 100); }
    Money plus(Money o) { return new Money(paise + o.paise); }
    Money times(int n) { return new Money(paise * n); }
}
record OrderId(String value) { }
record CustomerId(String value) { }                         // another aggregate is referenced by ID, not by
object
sealed interface DomainEvent permits OrderPlaced, OrderCancelled { }
record OrderPlaced(OrderId orderId, CustomerId customer, Map<Sku, Integer> quantities, Money total) implements
DomainEvent { }
record OrderCancelled(OrderId orderId, String reason) implements DomainEvent { }

static final class Order {                                  // aggregate root: the only door into the cluster
    static final int MAX_UNITS_PER_SKU = 10;
    private final OrderId id;
    private final CustomerId customer;
    private final List<OrderLine> lines = new ArrayList<>(); // OrderLine: entity, lineNo unique within its Order
    private final List<DomainEvent> events = new ArrayList<>();
    private OrderStatus status = OrderStatus.DRAFT;

    void addItem(Sku sku, Money unitPrice, int quantity) {
        requireDraft("add items");
        if (quantity <= 0) throw new IllegalArgumentException("Quantity must be positive");
        int already = lines.stream().filter(l -> l.sku.equals(sku)).mapToInt(l -> l.quantity).sum();
        if (already + quantity > MAX_UNITS_PER_SKU)
            throw new IllegalStateException("At most " + MAX_UNITS_PER_SKU + " units of " + sku.code() + " per
order");
        lines.add(new OrderLine(lines.size() + 1, sku, unitPrice, quantity));
    }
    Money total() { return lines.stream().map(OrderLine::subtotal).reduce(new Money(0), Money::plus); }
    void place() {                                          // ubiquitous language, not setStatus("PLACED")
        requireDraft("place");
        if (lines.isEmpty()) throw new IllegalStateException("Cannot place an empty order");
        status = OrderStatus.PLACED;
        Map<Sku, Integer> q = new HashMap<>();
        lines.forEach(l -> q.merge(l.sku, l.quantity, Integer::sum));
        events.add(new OrderPlaced(id, customer, Map.copyOf(q), total()));
    }
    void cancel(String reason) {
        if (status == OrderStatus.CANCELLED) return;          // idempotent
        status = OrderStatus.CANCELLED;
        events.add(new OrderCancelled(id, reason));
    }
    private void requireDraft(String action) {
        if (status != OrderStatus.DRAFT) throw new IllegalStateException("Cannot " + action + ": order is " +
status);
    }
    List<DomainEvent> pullEvents() { List<DomainEvent> e = List.copyOf(events); events.clear(); return e; }
    @Override public boolean equals(Object o) { return o instanceof Order other && other.id.equals(id); } //
entity: by identity
}

interface OrderRepository { Optional<Order> byId(OrderId id); void save(Order order); }  // per AGGREGATE ROOT
// InMemoryOrderRepository.save(): store the order, THEN publish order.pullEvents() to subscribers
// (in production: an outbox table in the same transaction)

// A second bounded context: Shipping has its OWN model and words
record Consignment(String reference, int parcels, int weightGrams) { }
static final class ShippingTranslator implements Consumer<DomainEvent> {   // anti-corruption layer
```

```
    private final Map<String, Integer> gramsPerItem;          // Shipping's own catalog: weight, not price
    @Override public void accept(DomainEvent e) {
        if (e instanceof OrderPlaced p) {
            int grams = p.quantities().entrySet().stream()
                    .mapToInt(en -> gramsPerItem.getOrDefault(en.getKey().code(), 500) * en.getValue()).sum();
            Consignment c = new Consignment("SO-" + p.orderId().value(), (grams + 9_999) / 10_000, grams);
            // record and print "shipping context: created ..."
        } else if (e instanceof OrderCancelled c) { /* print "stop consignment ..." */ }
    }
}
```

Output:

```
== BEFORE: anemic model ==
total=0.30000000000000004 status=SHIPPED lines=3 (a line was added after shipping, with qty -5)
== AFTER: aggregate + value objects + events ==
rejected: At most 10 units of TEA-250 per order
total Rs 709.50 (money in paise: 0.1 + 0.2 problems cannot happen)
   shipping context: created Consignment[reference=SO-1001, parcels=1, weightGrams=920]
   event published: OrderPlaced
rejected: Cannot add items: order is PLACED
== identity vs value equality ==
Money(100) equals Money(100)? true   (value object)
two Order objects with id 1001 equal? true   (entity: identity decides)
loaded from repository: PLACED
   shipping context: stop consignment SO-1001 (customer request)
   event published: OrderCancelled
```

Before, `double` money prints `0.30000000000000004` and nothing stops a −5 line *after* "shipping". *After*, `Money` holds paise (1/100 rupee) in a `long`, the root refuses changes once placed or beyond 10 units per SKU (Stock Keeping Unit — a product's unique code), and Shipping never sees Sales' `Order` — its translator builds a `Consignment` from the event using Shipping's own concepts (weight, parcels).

Trade-offs

DDD brings	DDD costs
Code that reads like the business; rules in one place	Time with domain experts; learning the vocabulary
Clear boundaries for teams and services	More types (value objects, events, translators)
Aggregates give a clear transaction and locking unit	Eventual consistency between aggregates must be designed and explained
Models can change independently per context	Translation code at every context border
Rich model is easy to unit-test (one class in isolation) without a database	ORMs sometimes fight rich models (private setters, value object mapping)

When not to use it

- **CRUD-shaped apps** (Create, Read, Update, Delete — forms over tables, an admin panel) with few rules — a simple layered design is enough; tactical DDD would be ceremony (KISS, YAGNI).
- **Small teams with one small model** — strategic splitting pays off with several teams or sub-domains.
- **Technical problems** (a compiler, a video encoder) whose complexity is algorithmic, not business rules.
- Still worth borrowing: **value objects** and **ubiquitous naming** cost little and help everywhere.

Common mistakes

- **Anemic aggregates** — public setters and "services" holding the rules: the *before* code with DDD names.
- **Huge aggregates** ("Customer" containing all orders, addresses, payments) — every change locks everything.
- **One repository per table** instead of per aggregate root.

- **Publishing events before the transaction commits** — consumers react to changes that may be rolled back (undone rather than committed); use an outbox.
- **Sharing one domain model across services** "to avoid duplication" — couples the teams; DRY applies *within* a context, not across.
- **Treating DDD as folders** (`entities/`, `valueobjects/`) rather than modelling with domain experts.
- **Primitives for domain concepts** (`String email`, `double price`) — "primitive obsession" (Chapter 7).

Real-world examples

- **Microservice boundaries** along bounded contexts (orders, payments, shipping, catalog) — default advice in Sam Newman's *Building Microservices* and Microsoft's .NET microservices guide (reference app *eShopOnContainers* uses DDD aggregates and events).
- **Payments**: a `Payment` aggregate with an `Amount` value object and events `PaymentAuthorized`, `PaymentCaptured`, `PaymentRefunded`, consumed by ledger and notification contexts.
- **Java**: `java.time` types and `BigDecimal` are value objects in all but name; Axon (used in the CQRS notes) is built around aggregates and events.
- In this book: the grocery POS (point-of-sale, Chapter 19) has `Money`/tender value types and a `Sale` guarding its states; the shipping locker (Chapter 14, §14.7) separates `Compartment` (reusable door) from `Booking` (one parcel's lifecycle).

In the interview

> **30-second answer:** "DDD models software on the business. Strategically, I split the domain into bounded contexts — own model and ubiquitous language each, like Sales and Shipping — connected by events or APIs, with an anti-corruption layer at the border. Tactically: entities for identity, immutable value objects like Money, small aggregates whose root enforces invariants; one aggregate per transaction, others follow via domain events, one repository per aggregate root."

Follow-ups:

- *Q: Entity or value object?* A: Do you track it over time by identity? An `Order` yes (id 1001 stays 1001); an `Address` or `Money` no — you replace it.
- *Q: How big should an aggregate be?* A: As small as the invariants allow; reference others by id; use events for rules that can be eventually consistent.
- *Q: Bounded contexts vs microservices?* A: A context is a good service boundary (often one service per context), but can also be a module in a monolith.
- *Q: Keeping two aggregates consistent?* A: Change one per transaction, publish a domain event via an outbox, let a handler (or a saga — a chain of local transactions with compensating "undo" steps) update the other.

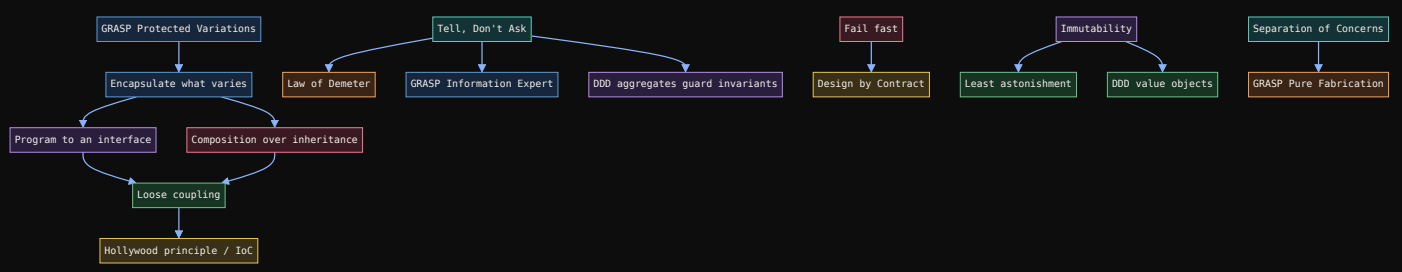
Self-check

1. Q: Two `Money(₹100)` objects — equal? Two customers named "Asha Rao"? → A: Money yes (value object); customers only with the same id (entities).
2. Q: Why must outsiders not hold an `OrderLine`? → A: They could change it without the `Order` root checking invariants.
3. Q: Why `CustomerId` instead of `Customer` inside `Order`? → A: Keeps aggregates small and independent; no loading or locking the customer to change an order.

TL;DR

Speak the business's language in code; split the domain into bounded contexts with their own models and translate at the borders; inside one context use entities (identity), immutable value objects (values), small aggregates whose root guards the invariants, domain events for what happens later, and one repository per aggregate root.

Chapter summary — how the principles fit together



Principle	Main goal	Typical smell it fixes	Key Java tool
DRY	One home per rule	Duplicated rules drifting	Value objects, constants, methods
KISS	Simplicity	Over-engineering	Plain code, `switch` expressions
YAGNI	Build only what's needed	Speculative generality	Delete unused code
Law of Demeter	Hide structure	Train wrecks	Intent-revealing delegate methods
Tell, Don't Ask	Behaviour with data	Anemic model, feature envy	Command methods on the owner
Composition over inheritance	Flexible reuse	Fragile base class	Forwarding/decorators
Program to an interface	Substitutability	Hard-wired concrete deps	Interfaces + DI
Encapsulate what varies	Isolate change	Growing type-switches	Strategy, lambdas
Loose coupling	Independent change	God orchestrators	Observer, events, DI
Hollywood / IoC	Framework owns flow	Duplicated skeletons	Template Method, callbacks, DI
Separation of Concerns	One concern per unit	Spaghetti code	Layers, MVC
Least Astonishment	Predictability	Hidden side effects	Honest names, `Optional`
Fail fast	Early, clear errors	Distant NPEs (NullPointerExceptions)	Guard clauses, `requireNonNull`
Immutability	Safe sharing	Aliasing, races	Records, `List.copyOf`
Design by Contract	Clear obligations	Magic error values	Pre/postconditions, invariants
GRASP	Place responsibilities	God classes	Expert, Creator, Controller, ...
DDD	Model the business; clear boundaries	Anemic model, big ball of mud, primitive obsession	Records as value objects, aggregate roots, domain events, repositories

Tensions to remember (and mention in interviews):

- DRY vs loose coupling — sharing code couples its users; sometimes duplicate across boundaries.
- YAGNI vs Encapsulate-what-varies/OCP — only abstract variation that is real or clearly predicted.
- Law of Demeter vs cohesion — too many delegate methods create a Middle Man; move behaviour instead.
- Information Expert vs Separation of Concerns — domain objects compute, fabrications persist.
- DRY vs bounded contexts — share nothing but events and ids across contexts; a little duplication beats a shared model that couples teams.

Chapter self-check

1. Q: Which principle is violated by `order.getCustomer().getAddress().getCity()` inside a service that then decides shipping? → A: Law of Demeter (and Tell-Don't-Ask); move the decision to where the data is or ask `order.shippingZone()`.
2. Q: A `find` method that inserts a row violates? → A: Least Astonishment (and Command-Query Separation).
3. Q: A `HashSet` subclass double-counts in `addAll` — which principle? → A: Composition over inheritance (fragile base class).
4. Q: What makes a record *truly* immutable when it holds a list? → A: Defensively copy it with `List.copyOf` in the compact constructor.
5. Q: In DDD, why is `Money` a value object but `Order` an entity? → A: Money is defined by its amount and currency and is replaced, not changed; an order keeps its identity while its lines and status change.

TL;DR (chapter)

One home per rule; simplest design for today's needs; behaviour with its data, talking only to friends; small interfaces, composition and IoC; separate concerns; no surprises; fail early; immutable values; explicit contracts; GRASP decides who does what, DDD decides which classes exist.

Chapter 6 — Clean Code

Design principles (Chapters 4–5) decide *which classes exist and how they relate*. **Clean code** is about what happens *inside* those classes: names, functions, comments, error handling, tests. In machine-coding rounds (build a small working program in about 60–120 minutes), interviewers read your code line by line — clean code often separates "hire" from "no hire" when two candidates reach similar designs.

This chapter covers every major topic of Robert C. Martin's *Clean Code* (2008) in the book's order (Chapters 1–17; the case studies, Chapters 14–16, are summarised in §13), in our own words with runnable Java 21 before/after programs. It ends with three practical topics: a **code review checklist**, an **error-handling strategy** for real services and **testing strategies** (unit vs integration tests, test doubles, mocks). Listings show the essential lines; the full program for each topic is in `code/lld/ch06/<topic>/`, and each "Output" block is that full program's real output.

Key terms (defined once here):

Term	Meaning
Clean code	Code that is easy to read, understand and change by someone other than its author. It does one thing well, has tests, and has no surprises.
Code smell	A surface symptom that *usually* signals a deeper design problem (e.g. a 200-line method). Not a bug by itself.
Refactoring	Changing the internal structure of code without changing its observable behaviour (Chapter 7).
Level of abstraction	How "high-level" a statement is. `processPayroll()` is high; `sb.append(',')` is low.
Side effect	Any change a function makes besides returning a value (fields, files, globals, output).
Test double	Any object that stands in for a real dependency during a test (dummy, stub, spy, mock, fake).
TDD	Test-Driven Development: write a failing test, make it pass, refactor — in very short cycles.

What is clean code, and why bother?

What it is

Well-known programmers describe clean code similarly: it reads like well-written prose, does one thing, is elegant and efficient, has tests, contains no duplication, expresses its intent, and looks written by someone who **cares**. Working definition: *code whose intent is obvious to the next reader, and which is easy to change safely.*

Analogy: a well-organised kitchen — anyone can walk in and cook. A messy kitchen "works" for its owner until someone else has to cook in it.

The problem it solves

Messy code slows teams more and more until productivity approaches zero and the team asks for a "grand redesign" that often reproduces the mess. We read code far more than we write it (often quoted as more than 10:1), so readable code makes writing faster too. "I'll clean it up later" rarely happens — *later equals never* (LeBlanc's law).

How it works

1. Treat readability as a first-class requirement.
2. Follow the **Boy Scout Rule**: *always leave the code a little cleaner than you found it* — rename a variable, split a long function, delete a dead comment each time you touch a file.
3. Write tests so you can clean without fear.

4. Clean in small steps (successive refinement): make it work, then make it right. Never ship "it works, but it's a mess".

Trade-offs

Clean code takes slightly longer the first time and saves time on every later read and change; throwaway scripts shift the balance.

When not to use it

Unreadable code is never a goal, but avoid **gold-plating** — endlessly polishing code that is already clear and not changing.

Common mistakes

- Thinking "working" means "done".
- Blaming deadlines: going fast by making a mess is an illusion; the mess slows you down immediately.
- Big-bang rewrites instead of continuous small improvements.

Real-world examples

- **Google's code review culture** requires reviewer approval for readability, with a per-language "readability" certification.
- **Linters and formatters** (tools that flag suspicious code or rewrite its layout) running in CI — Continuous Integration, a server that builds and tests every change — (Checkstyle, SpotBugs, SonarQube, google-java-format) enforce parts of clean code automatically.

In the interview

> **30-second answer:** "Clean code is code whose intent is obvious and which is cheap to change. In a machine-coding round I use intention-revealing names, small single-purpose functions, clear exceptions instead of error codes, immutable value objects (small objects like `Money` that never change after creation) and a few tests — and I leave time to refactor once it works."

Self-check

1. Q: What is the Boy Scout Rule? → A: Leave the code a little cleaner than you found it.
2. Q: Why does readability matter more than writing speed? → A: Code is read far more often than it is written.

TL;DR

Clean code = obvious intent + safe to change. Clean a little every time you touch code.

---

1. Meaningful Names

What it is

A good name answers three questions: **why does it exist, what does it do, how is it used**. If a name needs a comment to explain it, it isn't good enough.

Analogy: moving boxes labelled "Kitchen — glasses — fragile" vs "Box 7".

The problem it solves

Bad names force readers to keep a **mental map** ("`d` is elapsed days, `i1` is status, 4 means done"). Every reader pays that cost every time, and misunderstanding causes bugs.

How it works — the rules

Rule	Bad	Good
Reveal intention	`int d;`	`int elapsedTimeInDays;`
Avoid disinformation (names that suggest something false)	`accountList` for a `Set`; `l` / `O` that look like 1/0	`accounts`
Make meaningful distinctions — no number series or noise words	`a1, a2`; `ProductInfo` vs `ProductData`; `nameString`	`source, destination`; one `Product`
Use pronounceable names	`crtdTs`	`createdAt`
Use searchable names — avoid single letters and bare numbers except tiny local scopes	`7`, `e`	`MAX_LOGIN_ATTEMPTS`, `employee`
Avoid encodings — no Hungarian notation (type codes in names), no `m_` prefixes, no `I` prefix on interfaces	`strName`, `m_desc`, `IShapeFactory`	`name`, `description`, `ShapeFactory` (+ `ShapeFactoryImpl` or better, a specific name)
Avoid mental mapping	`r` for "lower-cased URL without host"	`normalizedPath`
Class names = nouns	`Manager`, `Processor`, `Data`, `Info` (vague)	`Customer`, `Invoice`, `AddressParser`
Method names = verbs; accessors `get` / `is` / `has`	`data()`	`postPayment()`, `isPosted()`
Prefer named static factories (static creation methods like `of...`) to overloaded constructors (several constructors with different parameter lists)	`new Money(23)` (rupees? paise, 1/100 of a rupee?)	`Money.ofRupees(23)`
Don't be cute	`yeetAll()`, `nuke()`	`deleteAll()`, `terminate()`
One word per concept	`fetch` / `retrieve` / `get` in sibling classes	pick one, e.g. `get`
Don't pun (same word, different meaning)	`add` for both "sum" and "append to collection"	`append`, `insert`
Use solution-domain names for technical concepts	—	`OrderRepository`, `RetryPolicy`, `EventQueue` (programmers know these)
Use problem-domain names for business concepts	—	`Premium`, `Policy`, `Claim`
Add meaningful context	lone `state`	`shippingState` or better, an `Address` class with `state`
Don't add gratuitous context	`AcmeShopCustomerAddress` in an app called "Acme Shop"	`Address`

Rule of thumb: the **longer the scope, the longer the name** — `i` is fine for a 3-line loop; a field used across a class deserves a full name.

Java example — before/after

Full program: `code/lld/ch06/names/Main.java`.

```java
// BEFORE
static class DtaRcrd102 { String strNm; int i1; int i2; boolean flg; }   // unpronounceable, encoded, meaningless
static class Mgr {                                  // vague noun
    List<DtaRcrd102> lst = new ArrayList<>();       // "lst" of what?
    List<DtaRcrd102> getThem() {                    // what is 4? what is 18000? why flg?
        List<DtaRcrd102> l1 = new ArrayList<>();
        for (DtaRcrd102 x : lst) if (x.i1 == 4 && x.i2 > 18000 && !x.flg) l1.add(x);
        return l1;
    }
    int d;                                          // elapsed time in days? seconds? since what?
}

// AFTER
enum TaskStatus { OPEN, IN_PROGRESS, BLOCKED, DONE }
record Task(String title, TaskStatus status, Duration timeSpent, boolean archived) {
    boolean isCompleted() { return status == TaskStatus.DONE; }          // boolean method reads as a question
    boolean tookLongerThan(Duration limit) { return timeSpent.compareTo(limit) > 0; }
}
static final class TaskBoard {
    private static final Duration LONG_RUNNING = Duration.ofHours(5);  // named constant, not a magic number
    private final List<Task> tasks = new ArrayList<>();
    void add(Task task) { tasks.add(task); }                            // method name = verb
    List<Task> findCompletedLongRunningTasks() {                        // says exactly what it returns
        List<Task> result = new ArrayList<>();
        for (Task task : tasks)
            if (task.isCompleted() && task.tookLongerThan(LONG_RUNNING) && !task.archived()) result.add(task);
        return result;
    }
}
record Temperature(double kelvin) {                 // named static factories beat overloaded constructors
    static Temperature ofCelsius(double c) { return new Temperature(c + 273.15); }
    static Temperature ofFahrenheit(double f) { return new Temperature((f - 32) * 5 / 9 + 273.15); }
}
```

Output:

```
BEFORE getThem(): 1 record(s) -- but what are they?
AFTER findCompletedLongRunningTasks(): [Task[title=Deploy, status=DONE, timeSpent=PT6H, archived=false]]
Temperature.ofCelsius(25) = 298.15 K, ofFahrenheit(212) = 373.15 K
```

After, the rule reads aloud: "completed tasks that took longer than five hours and aren't archived" — magic numbers (unexplained literals) became a named constant, a status code an enum, and `Temperature.ofCelsius(25)` is unambiguous where `new Temperature(25)` is not.

Trade-offs

Longer names cost a few keystrokes (IDEs — Integrated Development Environments such as IntelliJ IDEA — autocomplete them). Very long names can hint the function does too much — split it rather than abbreviate.

When not to use it

Conventional short names are fine where universally understood: `i` / `j` in tiny loops, `e` for a caught exception, `x` / `y` for coordinates, `T` / `K` / `V` for generics (type parameters, as in `List<T>`, letting one class work with many types).

Common mistakes

- Not renaming because "everyone knows what `tmp2` means" — renaming is cheap.
- Abbreviations only the author understands (`calcAmtPrc`).
- Inconsistent vocabulary (`customer`, `client`, `user` for one concept).

Real-world examples

- JDK naming: `isEmpty()`, `getOrDefault()`, `computeIfAbsent()`, `List.of()`, `Path.of()`, `Duration.ofMinutes()` — verbs for methods, named static factories.

- **Google Java Style Guide** and Oracle's conventions standardise casing (`UpperCamelCase` classes, `lowerCamelCase` methods, `UPPER_SNAKE_CASE` constants).

In the interview

> **30-second answer:** "I name things so the code reads like the requirement: nouns for classes, verbs for methods, booleans as questions, named constants instead of magic numbers, enums instead of status codes, and one consistent word per concept."

Follow-ups:

- *Q: Should interfaces start with `I`?* A: Not in Java convention; name the role (`PaymentGateway`) and give implementations specific names (`RazorpayGateway`).
- *Q: When is a short name okay?* A: When the scope is tiny and the meaning conventional (`i` in a short loop).

Self-check

1. Q: What's wrong with `ProductInfo` and `ProductData` in one codebase? → A: Noise words; no meaningful distinction.
2. Q: What's a pun in naming? → A: Using one word for two different meanings.

TL;DR

Names should reveal intent, be pronounceable and searchable, use one word per concept, and let the code read like the requirement.

---

2. Functions

What it is

Clean functions are **small**, **do one thing**, stay at **one level of abstraction**, take **few arguments**, have **no hidden side effects**, and either **do something or answer something** — never both.

The problem it solves

Long functions mix levels of detail, hide bugs, and can't be reused, named well or tested in pieces; flag/output arguments, hidden side effects and error codes each burden callers.

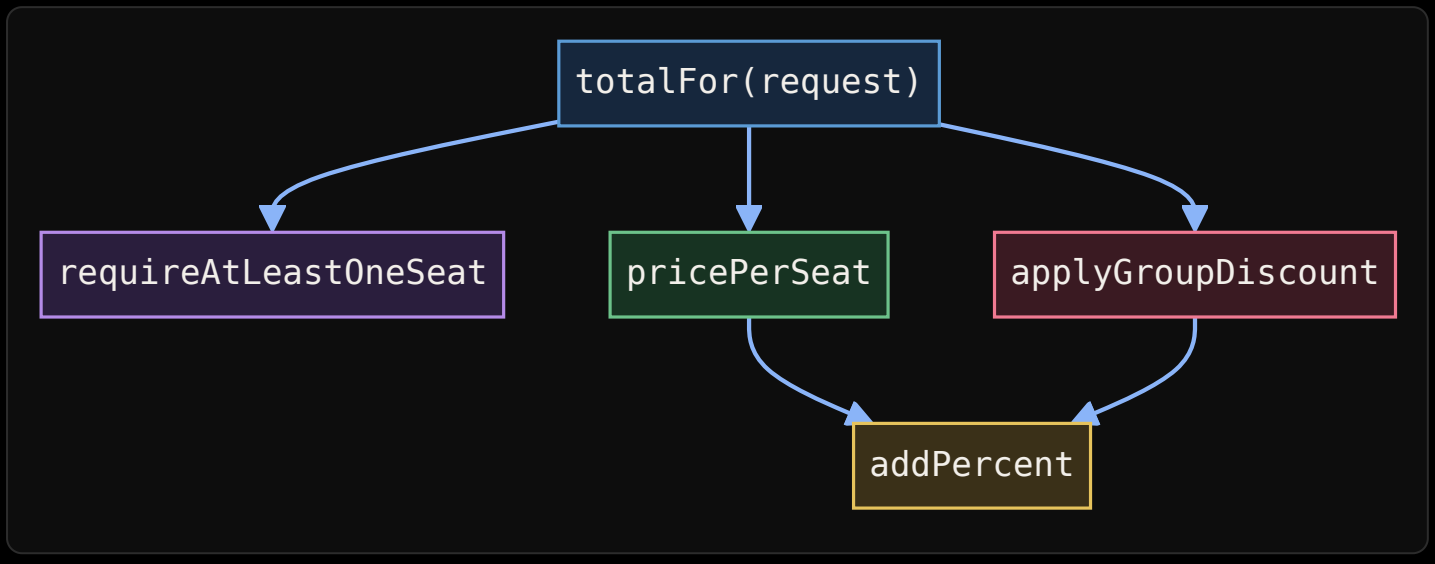
How it works — the rules

1. **Small, then smaller.** Functions should hardly ever reach 20 lines; the book praises two-to-four-line functions. Blocks inside `if` / `else` / `while` should ideally be one line — a call to a well-named function. Indentation should rarely exceed one or two levels.
2. **Do one thing.** A function does one thing if you can't extract another function from it with a name that isn't just a restatement of its body. "Sections" (declarations, initialisation, processing) mean several things.
3. **One level of abstraction per function.** Don't mix `getHtml()` (high) with `.append("\n")` (low).
4. **The Stepdown Rule.** Code reads top-down like a narrative: each function is followed by those it calls, one level lower. "To do X, we do A, then B. To do A, we …".
5. **Switch statements** on a type code tend to repeat and grow. Tolerate one only if it appears **once**, buried in a low-level class (e.g. a factory — a class/method whose job is creating objects — that creates polymorphic objects, i.e. objects of different classes used through one interface). Otherwise replace it with polymorphism (or enum data/behaviour).
6. **Use descriptive names.** A long descriptive name beats a short enigmatic one and a long comment; small functions are easier to name.
7. **Arguments: fewer is better.** Zero (niladic) is ideal, then one (monadic), two (dyadic); avoid three (triadic); more (polyadic) needs very special justification — and shouldn't be used anyway. Each

argument hurts readability and multiplies test cases.

- Common one-argument forms: a question (`isValid(email)`), a transformation (`Path resolve(name)`), an event (`onLoginFailed(attempts)`) — make the kind obvious.
- **Flag arguments** (`render(true)`) announce the function does two things. Split it or pass an enum.
- **Argument objects**: wrap arguments that travel together (`book(BookingRequest request)` instead of `book(show, seatType, count, day)`).
- **Argument lists** (varargs — `Object... args`, any number of values) count as one argument (`String.format(fmt, args...)`).
- **Verbs and keywords**: encode argument roles in the name (`copyFromTo(source, target)`, `renameFileTo(newName)`).

8. **Have no side effects.** `checkPassword` must not also initialise the session (the server-side record of a logged-in user). Hidden side effects create **temporal coupling** (works only if called at the right time). If necessary, put the side effect in the name — or better, separate it.
9. **Avoid output arguments.** `addSignature(letter)` — signature *to* the letter, or letter *to* something? Prefer `letter.addSignature()` (change the owning object's state).
10. **Command-Query Separation (CQS).** A function either *changes state* (command, returns `void`) or *returns information* (query, no state change), not both. `if (enable("darkMode"))` — asking, or enabling?
11. **Prefer exceptions to error codes.** Codes force `if` checks at every call site and deep nesting; exceptions separate the happy path (the normal, no-error flow) from error handling.
12. **Extract try/catch blocks** into their own functions — "error handling is one thing": `try` is the function's first word and nothing follows the `catch`/`finally`.
13. **Avoid an "error code magnet"** — a central `enum Error` everything imports forces recompiles and discourages new codes; new exception subclasses touch no existing code.
14. **Don't Repeat Yourself** — duplication is the root of much evil (Chapter 5).
15. **Structured programming** (Dijkstra's one-entry, one-exit: one `return`, no `break`/`continue`, never `goto`) matters mostly for large functions; small ones may use an occasional early `return`, `break` or `continue` for clarity (never `goto`).
16. **How to get there:** nobody writes clean functions first time. Write a working draft with tests, then refine — split, rename, remove duplication — keeping the tests green (all passing).



Java example — before/after

Full program: `code/lld/ch06/functions/Main.java`.

```java
// BEFORE: returns total in rupees, or -1 for bad input; validation, pricing, discounts, logging, session -- all in one
static int book(String seatType, int count, boolean weekend, List<String> log) {   // flag arg + output arg
    if (count <= 0) return -1;                                                      // error code
    int price;
    switch (seatType) {                                                             // switch on a type code
        case "SILVER": price = 150; break;
        case "GOLD": price = 250; break;
        case "RECLINER": price = 450; break;
        default: return -1;
    }
    if (weekend) price = price + price * 20 / 100;
    int total = price * count;
    if (count >= 4) total = total - total * 10 / 100;
    log.add("booked " + count + " " + seatType);                                    // mutates the caller's list
    SESSION.put("lastTotal", total);                                                // hidden side effect
    return total;
}

// AFTER
enum SeatType {                         // the type-code switch becomes data; parsing happens once, at the boundary
    SILVER(150), GOLD(250), RECLINER(450);
    final int basePrice;
    static SeatType parse(String raw) {
        try { return valueOf(raw); }
        catch (IllegalArgumentException e) { throw new InvalidBookingException("unknown seat type '" + raw + "'");
}
    }
}
enum DayType { WEEKDAY(0), WEEKEND(20); final int surchargePercent; }   // replaces the boolean flag
record BookingRequest(SeatType seat, int seats, DayType day) {}          // argument object

static final class TicketPricer {        // stepdown rule: each function one level more detailed than the one above
    private static final int GROUP_SIZE = 4;
    private static final int GROUP_DISCOUNT_PERCENT = 10;
    int totalFor(BookingRequest request) {                      // level 1: the story
        requireAtLeastOneSeat(request);
        int total = pricePerSeat(request) * request.seats();
        return applyGroupDiscount(total, request.seats());
    }
    private void requireAtLeastOneSeat(BookingRequest r) {        // level 2: the details
        if (r.seats() <= 0) throw new InvalidBookingException("seats must be positive, was " + r.seats());
    }
    private int pricePerSeat(BookingRequest r) { return addPercent(r.seat().basePrice, r.day().surchargePercent);
}
    private int applyGroupDiscount(int total, int seats) {
        return seats >= GROUP_SIZE ? addPercent(total, -GROUP_DISCOUNT_PERCENT) : total;
    }
    private static int addPercent(int amount, int percent) { return amount + amount * percent / 100; }  // level 3
}

static final class BookingService {      // CQS: book() changes state, returns nothing; history() answers, changes nothing
    private final TicketPricer pricer = new TicketPricer();
    private final List<String> history = new ArrayList<>();
    void book(BookingRequest request) {
        int total = pricer.totalFor(request);
        history.add(request.seats() + " x " + request.seat() + " on " + request.day() + " = Rs " + total);
    }
    List<String> history() { return List.copyOf(history); }
}

static String bookFromForm(BookingService service, String seat, int seats, DayType day) {  // error handling is "one thing"
    try { service.book(new BookingRequest(SeatType.parse(seat), seats, day)); return "OK"; }
    catch (InvalidBookingException e) { return "Sorry: " + e.getMessage(); }
}
```

Output:

```
== BEFORE ==
total=1080 log=[booked 4 GOLD] session={lastTotal=1080}
bad seat returns -1 -- caller must remember to check for -1
== AFTER ==
OK
OK
Sorry: unknown seat type 'PLATINUM'
Sorry: seats must be positive, was 0
  history: 4 x GOLD on WEEKEND = Rs 1080
  history: 2 x SILVER on WEEKDAY = Rs 300
```

What changed: the **boolean flag** became `DayType` (call sites read `DayType.WEEKEND`, not `true`); the **type-code switch** became data on `SeatType`; **error codes** (`-1`) became `InvalidBookingException` with a precise message; the **output argument** (`log`) and **hidden side effect** (`SESSION`) became explicit state owned by `BookingService`; `book`/`history` follow **CQS** (many real APIs pragmatically let a command return the created ID — a widely accepted exception).

Trade-offs

Many small functions	Few large functions
Each is nameable, testable, reusable	Everything visible in one place
Need good names to navigate	Hard to understand, test, change
Too-tiny splitting can scatter logic	Mixed abstraction levels

When not to use it

Don't split a genuinely simple 8-line function into 8 one-liners to hit a number; the goal is clarity. Size is almost never a real performance excuse in Java: the JIT (the Just-In-Time compiler inside the JVM — Java Virtual Machine — which turns hot bytecode into machine code at runtime) inlines small methods (copies their body into the caller) for you.

Common mistakes

- Boolean parameters in public APIs.
- Getters that change state (breaking CQS and least astonishment).
- Catching exceptions deep inside and returning `null`.
- Functions whose name says one thing and body does three.

Real-world examples

- `Collections.sort(list)`, `list.sort(comparator)` — command methods return `void`.
- `Iterator.next()` (an `Iterator` walks a collection one element at a time) is a well-known pragmatic CQS exception (advances *and* returns); `Queue.poll()` likewise. Know them; don't design new APIs that way without reason.
- `Files.readString`, `Files.writeString` — tiny, single-purpose APIs.

In the interview

> **30-second answer:** "I keep functions small and single-purpose, at one level of abstraction, top-down by the stepdown rule. I avoid flag arguments with enums or separate methods, keep argument counts low with parameter objects, separate commands from queries, and throw descriptive exceptions instead of returning error codes."

Follow-ups:

- *Q: How small is small?* A: One clear name and one level of abstraction — often under ~20 lines, frequently 3–10.

- *Q: What's wrong with `boolean` parameters?* A: The call site is unreadable and the function does two things.
- *Q: What's CQS?* A: Commands change state and return nothing; queries return data and change nothing.

Self-check

1. Q: How do you know a function does "one thing"? → A: You can't extract another meaningful function from it that isn't a mere restatement.
2. Q: When is a `switch` acceptable? → A: Once, buried in a factory/low level, never repeated.

TL;DR

Small, one thing, one level, top-down, ≤2–3 args, no flags, no hidden side effects, commands or queries, exceptions not codes.

---

3. Comments

What it is

Comments are at best a necessary evil, making up for our failure to express ourselves in code. Code changes; comments often don't, so they drift and **lie** — only the code is truly accurate. So **explain yourself in code first** and keep only comments that code *cannot* express.

The problem it solves

Comments as a crutch for unclear code, redundant comments, and outdated comments that mislead.

How it works

Replace a comment with a well-named function or variable whenever possible:

```
// free shipping if the order is big enough and domestic
if (order.total() >= 499 && !order.isInternational())
```

becomes `if (order.qualifiesForFreeShipping())` — no comment needed.

Good comments (keep these):

Kind	Example
Legal	Copyright/licence header
Informative	What a regex (text-matching pattern) matches, the format of a returned string
Explanation of intent	*Why* a decision was made ("we copy because callers may pass an unmodifiable list")
Clarification	Translating an obscure argument or return value from a library you can't change
Warning of consequences	"Not thread-safe" (unsafe from several threads at once), "takes 10 minutes to run", "billed per call"
TODO	Work that should be done but can't be now — with an owner; review them regularly
Amplification	Stressing the importance of something that looks trivial ("the trim is critical: …")
Javadoc in public APIs	Contracts callers depend on: params, return, exceptions

Bad comments (remove or replace):

Kind	What it looks like
Mumbling	A vague comment only the author understands
Redundant	`i++; // increment i`
Misleading	Subtly inaccurate description
Mandated	Javadoc on every function "because the rule says so" (`@param a the a`)
Journal	Change logs at the top of the file — version control (e.g. Git, which records every change) does this
Noise	`/** Default constructor. */`
Scary noise	Copy-pasted Javadoc noise with errors
Comment instead of a function/variable	Explaining an expression that could be named
Position markers	`// ********** Actions **********` banners
Closing-brace comments	`} // end while` — make the function shorter instead
Attributions and bylines	`/* Added by Rick */` — version control knows
Commented-out code	Others are afraid to delete it; it rots. Delete it — git remembers
HTML comments	Markup inside comments, unreadable in the editor
Nonlocal information	Commenting a system-wide default next to an unrelated function
Too much information	Historical discussions, RFC (internet standards document) excerpts
Inobvious connection	A comment that itself needs explaining
Function headers	Short, well-named functions don't need them
Javadoc in non-public code	Formal docs for internal code is usually noise

Java example — before/after

Full program: `code/lld/ch06/comments/Main.java` (it opens with a GOOD legal comment: a copyright/Apache-licence header).

```java
// BEFORE: comments used to excuse unclear code
// Changes:
//   2019-03-01 RK: created
//   2021-07-12 AB: fixed bug                                  <-- BAD: journal comment (git does this)
/**
 * The eligible method.
 * @param a the a                                              <-- BAD: noise / mandated Javadoc
 * @return the boolean
 */
static boolean eligible(Applicant a) {
    // check if applicant is eligible for a premium loan        <-- BAD: comment instead of a good name
    if (a.age() >= 21 && a.age() <= 60 && a.creditScore() > 750 && !a.hasDefaulted()
            && a.monthlyIncomePaise() >= 5_000_000) {
        return true;   // return true                           <-- BAD: redundant
    }
    // if (a.age() > 65) return false;                          <-- BAD: commented-out code
    return false;
} // end eligible                                              <-- BAD: closing-brace comment

// AFTER: the code explains itself; comments add what code can't
private static final int MIN_AGE = 21, MAX_AGE = 60;
private static final int GOOD_CREDIT_SCORE = 750;
private static final long MIN_MONTHLY_INCOME_PAISE = 5_000_000;   // Rs 50,000

static boolean isEligibleForPremiumLoan(Applicant a) {
    return isWorkingAge(a) && hasGoodCredit(a) && earnsEnough(a);
}
private static boolean isWorkingAge(Applicant a) { return a.age() >= MIN_AGE && a.age() <= MAX_AGE; }
private static boolean hasGoodCredit(Applicant a) { return a.creditScore() > GOOD_CREDIT_SCORE &&
!a.hasDefaulted(); }
private static boolean earnsEnough(Applicant a) { return a.monthlyIncomePaise() >= MIN_MONTHLY_INCOME_PAISE; }

// GOOD: informative comment -- clarifies what a regex matches (format: 10-character PAN, e.g. ABCDE1234F)
private static final Pattern PAN_FORMAT = Pattern.compile("[A-Z]{5}[0-9]{4}[A-Z]");

/**
 * GOOD: Javadoc on a public API -- states the contract callers rely on.
 * @param scores credit scores in the range 300..900
 * @return the median score; for an even count, the lower of the two middle values
 * @throws IllegalArgumentException if {@code scores} is empty
 */
static int medianScore(List<Integer> scores) {
    if (scores.isEmpty()) throw new IllegalArgumentException("no scores");
    // GOOD: explanation of intent -- we copy because callers may pass an unmodifiable list, and sorting in place
would fail
    List<Integer> sorted = scores.stream().sorted().toList();
    return sorted.get((sorted.size() - 1) / 2);
}

// GOOD: warning of consequences -- this call hits the credit bureau and is billed per request.
// TODO(loans-team): cache bureau responses for 24h once the bureau contract allows it.   <-- GOOD: TODO
static int fetchBureauScore(String pan) { return 780; }
```

Output:

```
BEFORE eligible(asha)=true, eligible(ravi)=false
AFTER  isEligibleForPremiumLoan(asha)=true, (ravi)=false
looksLikePan(ABCDE1234F)=true
medianScore([700, 820, 650, 900])=700
bureau score=780
```

Trade-offs

Comments are cheap to write and expensive to maintain: a few precise "why" comments are valuable; many "what" comments are a liability.

When not to use it

"No comments" is not the goal. Public libraries need Javadoc; complex algorithms (a published paper's method, a tricky concurrency protocol) deserve a comment explaining the approach and why.

Common mistakes

- Updating code but not the comment next to it.
- Keeping commented-out code "just in case".
- Comments as a to-do list no one reviews.

Real-world examples

- The **JDK's Javadoc** is model contract documentation (`HashMap`: iteration order not guaranteed; `ConcurrentHashMap`: weakly consistent iterators).
- **SonarQube** (a code-scanning server) flags commented-out code; many teams' linters reject a `TODO` without a ticket reference (an issue-tracker ID).

In the interview

> **30-second answer:** "I prefer expressive code over comments: named functions and constants replace most 'what' comments. I keep comments for why, warnings, legal headers, TODOs with owners and public API contracts, and I delete commented-out code because version control remembers it."

Self-check

1. Q: Name three good kinds of comment. → A: Legal, explanation of intent, warning of consequences (also informative, TODO, amplification, public Javadoc).
2. Q: Why do comments lie? → A: Code changes and comments aren't updated.

TL;DR

Say it in code; comment only the *why*, warnings, legal text and public contracts; delete dead comments and commented-out code.

---

4. Formatting

What it is

Formatting is **communication**: layout tells the reader what belongs together and what matters most. A team agrees one set of rules and applies them automatically everywhere.

The problem it solves

Random or cramped layout makes readers decode structure before logic and produces noisy diffs (line-by-line views of a change) in review (§15).

How it works

Vertical formatting (top-to-bottom):

1. **File size** — small files are easier to understand; many well-known Java projects keep most files around 200 lines, very few over 500.
2. **Newspaper metaphor** — class name and top of file give the high-level picture; detail increases downward.
3. **Vertical openness** — blank lines separate distinct concepts (package, imports, each method).
4. **Vertical density** — tightly related lines appear together, without blank lines or useless comments between them.
5. **Vertical distance** — keep related concepts close: local variables near first use; instance variables in one well-known place (Java: top of the class); **dependent functions** with the caller *above* the

callee, close by; **conceptual affinity** — similar functions (overloads, related operations) near each other.
6. **Vertical ordering** — high-level functions first, details later (the stepdown rule).

Horizontal formatting (left-to-right):

1. **Line width** — short enough to read without scrolling. Martin uses 120 characters; Google's Java style uses 100.
2. **Horizontal openness and density** — spaces around assignment operators and between arguments; none between a function name and its `(`; spacing can show precedence.
3. **No horizontal alignment** of declarations in columns — it draws the eye to the wrong thing (types vs names) and breaks with every rename.
4. **Indentation** shows scope hierarchy; don't collapse a short `if`/loop body onto one line if that hides structure.
5. **Dummy scopes** — make an empty `while`/`for` body visible (semicolon on its own line or `{}`); a trailing `;` is easy to miss.

Team rules — a codebase should look like one person wrote it. Encode the rules in an automatic formatter (IDE settings, `google-java-format`, Spotless, Checkstyle) and let CI enforce them.

Java example — before/after

Full program: `code/lld/ch06/formatting/Main.java`.

```java
// BEFORE: cramped, random order, aligned columns, no vertical openness
static class Playlist{private int cursor;
    private String helper(int i){return songs.get(i).toUpperCase();}
    public String next(){if(songs.isEmpty())return null;String s=helper(cursor);cursor=
(cursor+1)%songs.size();return s;}
    private final List<String>   songs   =   new ArrayList<>();
    private final String         name;
    public void add(String s){songs.add(s);}Playlist(String n){name=n;}
    public String name(){return name;}}

// AFTER: newspaper order -- headline first, details below
/** A looping playlist. (Headline: the class name and this one line tell you what it is.) */
static final class Playlist {
    // 1. Fields at the top, together (vertical density for closely related lines).
    private final String name;
    private final List<String> songs = new ArrayList<>();
    private int cursor;

    // 2. Constructor next.
    Playlist(String name) {
        this.name = name;
    }

    // 3. Public API -- each concept separated by a blank line (vertical openness).
    public String name() {
        return name;
    }

    public void add(String song) {
        songs.add(song);
    }

    public String next() {
        if (songs.isEmpty()) {
            return null;
        }
        String song = displayTitle(cursor);    // caller ABOVE callee (dependent functions close together)
        cursor = (cursor + 1) % songs.size();
        return song;
    }

    // 4. Private helpers just below the functions that use them.
    private String displayTitle(int index) {
        return songs.get(index).toUpperCase();
    }
}
```

Output:

```
BEFORE Road trip: KESARIYA YELLOW TUM HI HO KESARIYA
AFTER  Road trip: KESARIYA YELLOW TUM HI HO KESARIYA
Same behaviour -- formatting only changes how fast a human understands it.
```

Trade-offs

Almost none — formatting is cheap when automated; the only cost is agreeing on the rules once.

When not to use it

Generated code (e.g. from Protobuf, whose compiler writes message classes for you) needs no hand formatting; exclude it from formatters.

Common mistakes

- Mixing formatting and logic changes in one commit (a saved, described change in version control) — unreadable diffs.
- Personal styles inside a team codebase.
- Aligning declarations in columns.

Real-world examples

- **google-java-format**, **Spotless** (Gradle/Maven — Java build tools), **Prettier** (JS), **gofmt** (formats all Go code identically — style debates disappear), **Black** (Python).
- **EditorConfig** files share indentation settings across editors.

In the interview

> **30-second answer:** "Formatting communicates structure. I keep related lines together, separate concepts with blank lines, put callers above callees, fields at the top, and let an automatic formatter enforce one team style."

Self-check

1. Q: What is the newspaper metaphor? → A: High-level summary at the top, increasing detail below.
2. Q: Where should a private helper go? → A: Just below the function that calls it.

TL;DR

Format for the reader: headline first, related things close, concepts separated, one automated team style.

---

5. Objects and Data Structures

What it is

Two fundamentally different kinds of types:

- An **object** *hides* its data behind abstractions and *exposes behaviour*. Example: an `Order` with `isMetroDelivery()`.
- A **data structure** *exposes* its data and has *no meaningful behaviour*. Example: a record `Address(city, pincode)`, a DTO (Data Transfer Object — a plain data carrier).

Data abstraction is not just getters and setters on private fields: it means exposing abstract interfaces that manipulate the *essence* of the data without revealing its representation (`fuelPercentRemaining()` rather than `getFuelTankCapacityInLitres()` and `getLitresOfFuel()`).

Analogy: a vending machine (object) — press buttons, get results, never touch the coin box; a spreadsheet (data structure) — all values visible, formulas elsewhere.

The problem it solves

Confusing the two creates **hybrids** — public fields *and* significant behaviour — where both new functions and new types are hard. The difference tells you which design suits your likely changes.

How it works — the data/object anti-symmetry

	Procedural code (data structures + functions)	**OO code (objects)**
Add a new **function** (e.g. `perimeter`)	**Easy**: add one function; data types untouched	**Hard**: add a method to every class
Add a new **type** (e.g. `Triangle`)	**Hard**: edit every function	**Easy**: add one class; existing code untouched

So *everything is an object* is a myth: expect **new types** → objects and polymorphism; expect **new operations** over stable types → data structures with functions (or the Visitor pattern, Chapter 11, which adds operations without editing the classes). Java 21's `sealed` interfaces (listing exactly which classes may implement them) with records and pattern-matching `switch` (branching on an object's type) make the procedural style safe: the compiler reports a `switch` that misses a type.

The Law of Demeter (Chapter 5, §4) applies to objects: don't navigate an object's internals (`order.getCustomer().getAddress().city()` — a **train wreck**). It does *not* strictly apply to data structures, whose job is to expose data. For objects, **hide structure**: ask the object to do the job (`order.isMetroDelivery()`).

DTOs are the purest data structure — public (or record) fields, no behaviour — moving data across boundaries: database rows, JSON (plain-text key–value) bodies, messages. The *bean* form (private fields + getters/setters) adds no real encapsulation but some frameworks require it.

Active Record: a DTO that also has navigation methods like `save()` and `find()` and maps to one database row (Rails' ActiveRecord, some Java ORMs — Object-Relational Mappers, mapping classes to tables). Common mistake: putting business rules into active records, making hybrids; keep rules in separate objects that *use* the active record.

Java example — before/after

Full program: `code/lld/ch06/objects/Main.java`.

```
// STYLE 1: data structures + procedures
sealed interface Shape permits Square, Circle {}
record Square(double side) implements Shape {}
record Circle(double radius) implements Shape {}
static final class Geometry {            // all behaviour here: perimeter() was easy; a Triangle means editing
every method
    static double area(Shape s) {
        return switch (s) {
            case Square sq -> sq.side() * sq.side();
            case Circle c -> Math.PI * c.radius() * c.radius();
        };
    }
    static double perimeter(Shape s) {   // NEW FUNCTION: easy -- no shape class changes
        return switch (s) {
            case Square sq -> 4 * sq.side();
            case Circle c -> 2 * Math.PI * c.radius();
        };
    }
}

// STYLE 2: objects (polymorphic)
interface Shape { double area(); }
record Square(double side) implements Shape { public double area() { return side * side; } }
record Circle(double radius) implements Shape { public double area() { return Math.PI * radius * radius; } }
record Triangle(double base, double height) implements Shape {   // NEW TYPE: easy -- nothing else changes
    public double area() { return 0.5 * base * height; }
}   // ...but adding perimeter() would mean editing Shape and EVERY implementation

// Hiding structure (Law of Demeter)
static boolean isMetroDelivery(Order o) {   // BEFORE: train wreck
    return List.of("Mumbai", "Delhi", "Pune").contains(o.getCustomer().getAddress().city());
}
record Address(String city, String pincode) {                    // AFTER: a data structure -- exposing data is
its job
    boolean isMetro() { return List.of("Mumbai", "Delhi", "Pune").contains(city); }
}
static final class Order {                                        // an object: hides its structure
    private final Address shipTo;
    boolean isMetroDelivery() { return shipTo.isMetro(); }        // tell the order what you need
}

record OrderDto(String orderId, String city, long totalPaise) {}  // DTO: public data, no behaviour (e.g. a REST
response)
```

Output:

```
procedural Square[side=2.0]       area=4.00 perimeter=8.00
procedural Circle[radius=1.0]     area=3.14 perimeter=6.28
object     Square[side=2.0]       area=4.00
object     Circle[radius=1.0]     area=3.14
object     Triangle[base=3.0, height=4.0] area=6.00
BEFORE train wreck says metro? true
AFTER  order.isMetroDelivery()? true
DTO: OrderDto[orderId=o-9, city=Pune, totalPaise=45000]
```

Trade-offs

The anti-symmetry table above: choose by which change you expect more.

When not to use it

Don't force behaviour into DTOs that cross process boundaries (between separately running programs, e.g. over the network) — keep them dumb and version them carefully.

Common mistakes

- Getters and setters for every private field by reflex (that's not encapsulation).
- Hybrids: public mutable fields plus business methods.
- Business logic inside JPA (Java's standard ORM API) entities that also serve as API response bodies.

Real-world examples

- Records are Java's official data carriers; `java.awt.Point` has public fields (a data structure).
- `java.time.LocalDate` is an object: you call `plusDays`, `isLeapYear`, never see its fields.
- DTOs in Spring REST controllers (`@RequestBody CreateOrderRequest`); Rails ActiveRecord / Spring Data JPA entities.

In the interview

> **30-second answer:** "Objects hide data and expose behaviour; data structures expose data with no behaviour. OO makes adding types easy and functions hard; procedural does the opposite. I use records for DTOs and value data, rich objects for domain logic, and avoid hybrids and train wrecks."

Follow-ups:

- *Q: Is `a.getB().getC()` always wrong?* A: For objects, yes (Demeter). For pure data structures like records it's acceptable.
- *Q: Which pattern lets OO code add operations easily?* A: Visitor (Chapter 11), at the cost of making new types harder.

Self-check

1. Q: What is a hybrid? → A: A type with both exposed data and significant behaviour — to be avoided.
2. Q: What's an Active Record? → A: A DTO with persistence/navigation methods mapped to a DB row.

TL;DR

Objects hide data and expose behaviour; data structures expose data. Pick based on whether you'll add types or operations. No hybrids, no train wrecks through objects.

---

6. Error Handling

What it is

Error handling is important, but if it **obscures the logic** it's wrong. Clean error handling keeps the happy path readable and makes failures explicit, informative and hard to ignore.

The problem it solves

Error codes clutter every caller with `if` checks and are easy to forget; `null` returns cause `NullPointerException`s far away; checked exceptions (which the compiler forces every caller to catch or declare) from low-level libraries leak into every signature; vague messages slow debugging.

How it works — the rules

1. **Use exceptions rather than return codes** — caller logic stays clean; errors can't be silently ignored.
2. **Write your `try-catch-finally` first** when code can throw. The `try` block is like a transaction (all-or-nothing): whatever happens inside, the `catch` must leave the program consistent. In Java, prefer **try-with-resources** for anything `AutoCloseable` (anything with `close()`, such as a file or connection) — it is closed even if an exception is thrown.
3. **Use unchecked exceptions.** Checked exceptions violate the Open/Closed Principle: a new checked exception low down forces every method between it and the handler to change its signature. C#, Kotlin, Python and Scala have none. Use checked exceptions only when a caller can and must realistically recover (rare, deliberate).
4. **Provide context** — what failed, with which values, why (`account 'B' has 100 but 900 was requested`); keep the original exception as the **cause** when wrapping.
5. **Define exception classes by the caller's needs** — how will it *catch* them? Often one type per area is enough. **Wrap third-party APIs** so their exceptions become yours, minimising vendor dependency and easing library swaps.
6. **Define the normal flow — the Special Case pattern** (Martin Fowler): for an expected "absence", return an object that handles it (`NoMembership` with 0 points, `GuestCustomer`, an empty list) instead of throwing or returning `null`; callers need no special handling.
7. **Don't return `null`.** Return an empty collection, `Optional` (holds a value or is empty), a special-case object, or throw. Every `null` return is a future `NullPointerException`.
8. **Don't pass `null`.** Unless an API expects it, reject `null` arguments immediately (`Objects.requireNonNull`) — fail fast (Chapter 5).



Java example — before/after

Full program: `code/lld/ch06/errors/Main.java`.

```
// BEFORE
static final int OK = 0, E_NO_ACCOUNT = 1, E_NO_MONEY = 2;
static int transfer(String from, String to, long amount) {        // caller must check an int
    Long f = BALANCES.get(from), t = BALANCES.get(to);
    if (f == null || t == null) return E_NO_ACCOUNT;               // which one? no context
    if (f < amount) return E_NO_MONEY;                             // how much was missing?
    BALANCES.put(from, f - amount); BALANCES.put(to, t + amount);
    return OK;
}
static List<String> history(String id) { return null; }          // "no history" = null
static Integer loyaltyPoints(String id) { return POINTS.get(id); } // null for non-members

// AFTER: unchecked, domain-specific, with context -- defined by what the CALLER needs to handle
static class BankingException extends RuntimeException { /* (msg) and (msg, cause) constructors */ }
static final class AccountNotFoundException extends BankingException {
    AccountNotFoundException(String id) { super("account '" + id + "' does not exist"); }
}
static final class InsufficientFundsException extends BankingException {
    InsufficientFundsException(String id, long requested, long available) {
        super("account '" + id + "' has " + available + " but " + requested + " was requested");
    }
}
// SPECIAL CASE: "no loyalty membership" is an object -- callers need no null check
interface Loyalty { int points(); String tier(); }
record Member(int points) implements Loyalty { public String tier() { return points >= 100 ? "GOLD" : "SILVER"; }
}
enum NoMembership implements Loyalty { INSTANCE; public int points() { return 0; } public String tier() { return
"NONE"; } }

static final class SmsNotifier {        // wrapper around a vendor client that throws checked IOException
    private final VendorSmsClient client = new VendorSmsClient();
    void send(String phone, String text) {
        try { client.push(phone, text); }
        catch (IOException e) { throw new BankingException("could not send SMS to " + phone, e); }   // cause kept
    }
}

// in Bank:
void transfer(String from, String to, long amount) {
    Objects.requireNonNull(from, "from"); Objects.requireNonNull(to, "to");     // don't accept null
    long available = balanceOf(from);
    balanceOf(to);                                                               // validates 'to' exists
    if (available < amount) throw new InsufficientFundsException(from, amount, available);
    balances.merge(from, -amount, Long::sum);
    balances.merge(to, amount, Long::sum);
}
List<String> history(String id) { return List.of(); }                         // empty, never null
Optional<Long> findBalance(String id) { return Optional.ofNullable(balances.get(id)); }
Loyalty loyaltyOf(String id) {
    Integer p = points.get(id);
    return p == null ? NoMembership.INSTANCE : new Member(p);
}

// "Write the try first": main writes to an AutoCloseable AuditFile inside try-with-resources and throws --
// the output shows the file closed before the catch runs
```

Output:

```
== BEFORE ==
transfer returned 1 (which account? caller has to decode)
history(): NullPointerException -- someone forgot the null check
loyaltyPoints(B): NullPointerException on unboxing
== AFTER ==
transfer failed: account 'Z' does not exist
transfer failed: account 'B' has 100 but 900 was requested
transfer A->B of 200 ok; A=300 B=300
history(A) size = 0
findBalance(Z) = Optional.empty
loyalty A: 40 pts, tier SILVER
loyalty B: 0 pts, tier NONE
could not send SMS to 000123 (cause: socket timeout to sms-gw-3)
  audit file opened
  audit: transfer A->B 200
  audit file closed
  caught: simulated crash -- file was still closed first
```

The *before* `loyaltyPoints` crashes on auto-unboxing (Java's automatic `Integer` → `int` conversion, which throws on `null`).

Trade-offs

Choice	Pro	Con
Unchecked exceptions	Clean signatures, OCP-friendly	Compiler doesn't remind callers; document them
Checked exceptions	Compiler-enforced handling	Signature ripple; often caught and swallowed
`Optional` return	Absence visible in the type	Not for fields/parameters; small allocation
Special Case object	No checks at all in callers	Can hide a real problem if absence *should* be noticed
Result/Either type (holds either a success value or an error)	Expected failures as values, composable	More verbose in Java (see Error-handling strategy)

When not to use it

Exceptions are for errors, not normal control flow (don't throw to exit a loop). For *expected, frequent* outcomes (validating user input rows), returning a result value can be cleaner (§16).

Common mistakes

- `catch (Exception e) {}` — swallowing.
- Logging and rethrowing at every layer (the same error logged 5 times).
- Losing the cause when wrapping (`new MyException(e.getMessage())` instead of `new MyException(msg, e)`).
- Returning `null` from collection-returning methods.

Real-world examples

- **Spring's** `DataAccessException` hierarchy translates vendor-specific `SQLException`s into unchecked, meaningful exceptions (e.g. `DuplicateKeyException`) — "wrap third-party APIs, define exceptions by caller's needs".
- `Collections.emptyList()`, `Optional.empty()` — "don't return null"; `Objects.requireNonNull` — "don't pass null".
- `UncheckedIOException` (Java 8) wraps `IOException` for lambdas/streams, whose standard functional interfaces cannot throw checked exceptions.

In the interview

> **30-second answer:** "I throw unchecked, domain-specific exceptions with context instead of returning codes or null. I wrap third-party exceptions at the boundary, return empty collections or Optional for absence, use a special-case object when absence is normal, validate nulls at entry, and use try-with-resources for cleanup."

Follow-ups:

- *Q: Checked vs unchecked?* A: Checked forces handling but ripples through signatures and breaks OCP; I default to unchecked and reserve checked for recoverable conditions callers must handle.

Self-check

1. Q: Why are error codes bad? → A: They clutter callers, are easy to ignore, and lack context.
2. Q: Why keep the cause when wrapping? → A: So the original stack trace and message aren't lost.

TL;DR

Exceptions not codes; unchecked with context; wrap vendors; special cases, empty collections or Optional instead of null; never pass null; always clean up.

---

7. Boundaries

What it is

A **boundary** is where your code meets code you don't control: third-party libraries, frameworks, other teams' services, even very general JDK types like `Map`. Clean boundaries keep foreign code at the edge, behind interfaces shaped for *your* needs.

The problem it solves

Providers want broad APIs; users want focused ones. A raw `Map<String, Session>` passed around lets any code `clear()` it or insert wrong values, forces everyone to know the key format, and breaks every usage site when the API changes.

How it works

1. **Wrap boundary types.** Don't pass `Map`, vendor clients or framework types around; hide them in a class exposing only what your app needs (`SessionRegistry.open/activeSession/close`), keeping the boundary type **private** or at least out of public APIs — so a vendor change touches one place.
2. **Write learning tests.** When adopting a library, write small tests that call its API the way you intend to use it. They document your understanding, cost nothing extra (you had to learn the API anyway), and — re-run on each upgrade — reveal behaviour changes immediately.
3. **Use code that does not exist yet.** When another team's API isn't ready, define the interface *you wish you had* (`FraudCheck`), code and test against a fake, and later write an **Adapter** (Chapter 9) to the real API. This also gives a clean seam (a place where a test can plug in a substitute).

Java example — before/after

Full program: `code/lld/ch06/boundaries/Main.java` (the learning tests run on the chapter's `MiniTest` runner, §8).

```java
// BEFORE: a raw Map passed all over the codebase -- anyone can clear() it, put(null, ...), and must know the key format
static void loginHandler(Map<String, Session> sessions, String token, String user) {
    sessions.put(token, new Session(user, Instant.parse("2026-09-28T12:00:00Z")));
}
static void buggyAdminTool(Map<String, Session> sessions) { sessions.clear(); }   // oops: everyone logged out

// AFTER: the Map is hidden behind an interface shaped for OUR needs
static final class SessionRegistry {
    private final Map<String, Session> byToken = new ConcurrentHashMap<>();   // boundary type stays private
    void open(String token, Session session) { byToken.put(token, session); }
    Optional<Session> activeSession(String token, Instant now) {
        return Optional.ofNullable(byToken.get(token)).filter(s -> now.isBefore(s.expiresAt()));
    }
    void close(String token) { byToken.remove(token); }
}

// The fraud team's API isn't written yet: define the interface WE want, use a fake until an Adapter wraps the real one
interface FraudCheck { boolean isSuspicious(String userId, long amountPaise); }
static final class FakeFraudCheck implements FraudCheck {
    public boolean isSuspicious(String userId, long amountPaise) { return amountPaise > 10_000_000; }
}

// LEARNING TESTS: executable notes about how third-party (here: JDK) APIs really behave
@Test void hashmap_allows_one_null_key() { Map<String, Integer> m = new HashMap<>(); m.put(null, 1);
assertEquals(1, m.get(null)); }
@Test void concurrenthashmap_rejects_null_keys() {
    assertThrows(NullPointerException.class, () -> new ConcurrentHashMap<String, Integer>().put(null, 1));
}
@Test void map_of_rejects_duplicate_keys() { assertThrows(IllegalArgumentException.class, () -> Map.of("k", 1,
"k", 2)); }
@Test void string_split_drops_trailing_empty_strings() {
    assertEquals(2, "a,b,,".split(",").length);
    assertEquals(4, "a,b,,".split(",", -1).length);
}
```

Output:

```
== BEFORE ==
sessions after admin tool: 0  <-- nothing stopped the clear()
== AFTER ==
active at 11:00? true
active at 13:00? false
fraud(asha, Rs 2,00,000)? true
== Learning tests ==
JdkLearningTest
  PASS  concurrenthashmap rejects null keys
  PASS  hashmap allows one null key
  PASS  map of rejects duplicate keys
  PASS  string split drops trailing empty strings
4 passed, 0 failed
```

`SessionRegistry` also puts the "not expired" rule in one place. The learning tests pin down surprising but real API facts: `HashMap` allows one `null` key, `ConcurrentHashMap` rejects it, `Map.of` rejects duplicate keys, `String.split` drops trailing empty strings unless given a negative limit.

Trade-offs

Wrapping costs a class and some delegation; it pays off whenever the boundary type is used in more than one place or might change.

When not to use it

Stable, universal types used locally (a `Map` inside one method) are fine. The rule is about *passing* boundary types around your system.

Common mistakes

- Returning vendor types from your interfaces (`PaymentGateway.charge()` returning a `StripeCharge`) — leaks the vendor.
- Skipping learning tests and discovering library behaviour in production (the live system users use).

Real-world examples

- **Anti-Corruption Layer** (Domain-Driven Design, Chapter 5 §17): a translation layer between your model and a legacy/external system's model.
- **SLF4J** (Simple Logging Facade for Java) wraps the real logging library; **Spring's** `RestClient` / `WebClient` let you swap the HTTP engine.
- Contract tests (e.g. Pact) between microservices are a team-level version of learning tests.

In the interview

> **30-second answer:** "I keep third-party code at the edges: wrap it behind an interface shaped for my domain, don't pass raw library types around, write learning tests when adopting a library, and if a dependency doesn't exist yet I define the interface I need and use a fake until an adapter can bridge to the real API."

Self-check

1. Q: What is a learning test? → A: A test that exercises a third-party API to learn and pin down its behaviour.
2. Q: How do you handle an API that isn't built yet? → A: Define your own interface, use a fake, adapt later.

TL;DR

Keep foreign code at the edge behind your own interfaces; learn it with tests; adapt, don't leak.

---

8. Unit Tests

What it is

Unit tests are small, automated tests of one unit of behaviour (usually a class or a few collaborating objects) in isolation. *Clean Code* argues test code is **as important as production code**: it must be clean, because it's what lets you change production code without fear.

The problem it solves

Without tests, people stop cleaning and the design rots. *Dirty* tests break with every change, are abandoned, and then production code rots too.

How it works

The three laws of TDD (they lock you into a cycle perhaps thirty seconds long, tests just a few seconds ahead of production code):

1. You may not write production code until you have written a failing unit test.
2. You may not write more of a unit test than is sufficient to fail (and not compiling counts as failing).
3. You may not write more production code than is sufficient to pass the currently failing test.

A related, coarser description (Kent Beck's TDD, not these three laws) is **Red → Green → Refactor**: failing test (red), simplest pass (green), then clean up code and tests (refactor).

Keep tests clean — they enable the "-ilities": code stays flexible, maintainable and reusable because you can change anything while the tests pass.

Clean tests are readable tests:

- **Build–Operate–Check** (also **Arrange–Act–Assert** or **Given–When–Then**): build the test data, operate on it, check the results.
- A small **domain-specific testing language**: helpers like `cart("TEA", 3)` and `checkoutWith(coupons, audit)` hide setup noise.
- **Dual standard:** test code must be clean and readable but need not be as efficient as production code (building strings for assertions is fine).

One assert per test? Minimise asserts, but more importantly have **one concept per test**; several asserts checking one concept are fine.

F.I.R.S.T. — clean tests are:

Letter	Meaning	How our code does it
Fast	Run in milliseconds so you run them constantly	In-memory doubles, no network/DB
Independent	No test depends on another or on order	`MiniTest` creates a fresh instance per test
Repeatable	Same result in any environment, any time	Fixed `Clock` instead of `now()`
Self-validating	Pass/fail automatically, no reading logs	Assertions, not `println`
Timely	Written just before (or with) the production code	TDD cycle

Java example — a JUnit-free runner, clean tests and a dirty one

Instead of JUnit (the standard Java test framework), a tiny framework — a `@Test` annotation plus `MiniTest`, about 60 lines — uses reflection (inspecting classes and calling methods by name at runtime) to find `@Test` methods, runs each on a **fresh instance** of the test class (Independent) and reports PASS/FAIL/ERROR itself (Self-validating). The production class `Checkout` has four collaborators behind interfaces, so tests can replace them with doubles. All files: `code/lld/ch06/testing/` (`Checkout`, `MiniTest`, `Test`, `BeforeStyleTest`, `CheckoutTest`, `CheckoutIntegrationTest`, and `Main`, which runs all three test classes).

```java
// Checkout.java (production code under test)
public interface PriceCatalog { long pricePaise(String sku); }
public interface CouponRepository { Optional<Coupon> find(String code); }
public interface ReceiptSender { void send(Receipt receipt); }
public interface AuditLog { void record(String event); }
static final long HIGH_VALUE_PAISE = 1_000_000;   // Rs 10,000

/** Query: computes the total, changes nothing. couponCode may be null = "no coupon" -- documented in the
 *  contract ("don't pass null" allows null only where an API explicitly expects it). */
public long quote(List<Line> lines, String couponCode) {
    long subtotal = lines.stream().mapToLong(l -> catalog.pricePaise(l.sku()) * l.qty()).sum();
    return subtotal - discount(subtotal, couponCode);
}
/** Command: charges, sends the receipt, audits big orders. */
public Receipt checkout(List<Line> lines, String couponCode, String email) {
    if (lines.isEmpty()) throw new IllegalArgumentException("cart is empty");
    Receipt receipt = new Receipt(email, quote(lines, couponCode));
    sender.send(receipt);
    if (receipt.totalPaise() >= HIGH_VALUE_PAISE) audit.record("high-value order for " + email);
    return receipt;
}
private long discount(long subtotal, String couponCode) {
    if (couponCode == null) return 0;
    return coupons.find(couponCode)
            .filter(c -> !LocalDate.now(clock).isAfter(c.expiresOn()))   // injected Clock -> Repeatable
            .map(c -> subtotal * c.percentOff() / 100)
            .orElse(0L);
}
// plus simple real implementations for the integration test: InMemoryCatalog, InMemoryCoupons, OutboxSender

// BeforeStyleTest.java -- how NOT to write tests: test1() quotes twice and checks out once (several concepts),
// builds Checkout with Clock.systemDefaultZone() (not Repeatable), and only PRINTS a, b and r for a human
// to judge (not Self-validating) -- cryptic name, magic numbers

// CheckoutTest.java -- clean UNIT tests; the five kinds of test double:
static final class DummyAuditLog implements Checkout.AuditLog {       // DUMMY: fills a parameter, must never be used
    public void record(String event) { throw new AssertionError("dummy must not be called"); }
}
static final class StubCatalog implements Checkout.PriceCatalog { /* STUB: canned prices from a Map */ }
static final class FakeCoupons implements Checkout.CouponRepository { /* FAKE: working in-memory list */ }
static final class SpySender implements Checkout.ReceiptSender {       // SPY: records calls for later inspection
    final List<Checkout.Receipt> sent = new ArrayList<>();
    public void send(Checkout.Receipt r) { sent.add(r); }
}
static final class MockAuditLog implements Checkout.AuditLog {         // MOCK: pre-programmed expectations, verifies itself
    private final List<String> expected = new ArrayList<>(), actual = new ArrayList<>();
    MockAuditLog expect(String event) { expected.add(event); return this; }
    public void record(String event) { actual.add(event); }
    void verify() { assertEquals(expected, actual); }
}
// fixture helpers: FIXED_CLOCK = Clock.fixed(2026-09-28T10:00Z); checkoutWith(coupons, audit); cart("TEA", 3, ...)

@Test void quote_sums_price_times_quantity() {
    Checkout checkout = checkoutWith(new FakeCoupons(), new DummyAuditLog());   // Given
    long total = checkout.quote(cart("TEA", 3), null);                          // When
    assertEquals(36_000L, total);                                               // Then
}
@Test void expired_coupon_is_ignored() {
    var coupons = new FakeCoupons().add(new Checkout.Coupon("OLD", 50, TODAY.minusDays(1)));
    assertEquals(12_000L, checkoutWith(coupons, new DummyAuditLog()).quote(cart("TEA", 1), "OLD"));
}
@Test void checkout_sends_exactly_one_receipt() {
    checkoutWith(new FakeCoupons(), new DummyAuditLog()).checkout(cart("TEA", 1), null, "asha@example.com");
    assertEquals(List.of(new Checkout.Receipt("asha@example.com", 12_000L)), sender.sent);   // inspect the spy
}
@Test void high_value_order_is_audited() {
    var audit = new MockAuditLog().expect("high-value order for ravi@example.com");
```

```
    checkoutWith(new FakeCoupons(), audit).checkout(cart("TV", 1), null, "ravi@example.com");
    audit.verify();                                                    // the mock checks itself
}
// also: valid_coupon_takes_percentage_off, empty_cart_is_rejected (assertThrows + "nothing must be sent")

// CheckoutIntegrationTest.java -- real in-memory components wired together, no doubles for our own code:
// TEA 120 x 2 + CUP 80 with a 25% WELCOME coupon -> one Receipt of 24_000 paise lands in the OutboxSender
```

Output:

```
BeforeStyleTest
       a=36000 b=36000 r=Receipt[email=z, totalPaise=12000]  (is this right? someone must check by eye)
  PASS  test1
CheckoutTest
  PASS  checkout sends exactly one receipt
  PASS  empty cart is rejected
  PASS  expired coupon is ignored
  PASS  high value order is audited
  PASS  quote sums price times quantity
  PASS  valid coupon takes percentage off
CheckoutIntegrationTest
  PASS  full checkout flow with real components
TOTAL: 8 passed, 0 failed
```

`test1` "passes" but proves nothing. Each `CheckoutTest` method checks **one concept**, is named as a behaviour specification (`expired_coupon_is_ignored`), and uses Given/When/Then, a fixed clock and self-validating assertions (§17 covers unit vs integration testing and the five doubles).

Trade-offs

Well-tested code	Untested code
Fearless refactoring	Fear of change → rot
Tests are executable documentation	Knowledge in people's heads
Test code must be maintained too	Faster to write the first time

When not to use it

TDD is less useful for exploratory spikes (quick experiments — throw them away) and pure UI layout. Don't unit-test trivial getters or framework code you don't own (use learning tests).

Common mistakes

- Testing implementation details (private methods, exact call order) so tests break on every refactor.
- Shared mutable fixtures (data set up before tests) that make tests order-dependent.
- `Thread.sleep` or the real clock/network in unit tests.
- Test names like `test1`, `testCheckout`.

Real-world examples

- **JUnit 5** (`@Test`, `@BeforeEach`, `assertThrows`, `@DisplayName`), **AssertJ** fluent assertions, **Mockito** doubles.
- JUnit 5's default lifecycle creates a **new test-class instance per test method** — the same independence choice `MiniTest` makes.
- `java.time.Clock` exists partly so time-dependent code can be tested with `Clock.fixed(...)`.

In the interview

30-second answer: "I write small, fast, independent tests — one concept each, named as behaviour, Given/When/Then. I inject collaborators behind interfaces so I can use stubs, fakes or mocks, and

> inject a `Clock` for time. In a machine-coding round I at least write a `main` exercising key flows, plus a few unit tests for core logic if time allows."

Follow-ups:

- *Q: The three laws of TDD?* A: No production code without a failing test; only enough test to fail; only enough code to pass.
- *Q: One assert per test?* A: The real rule is one concept per test; several asserts on one concept are fine.

Self-check

1. Q: What's wrong with printing results in a test? → A: Not self-validating — a human has to check.
2. Q: How do you make a time-based test repeatable? → A: Inject a `Clock` and use `Clock.fixed` in tests.

TL;DR

Tests are first-class code: TDD cycle, one concept per test, readable Given/When/Then, and F.I.R.S.T.

---

9. Classes

What it is

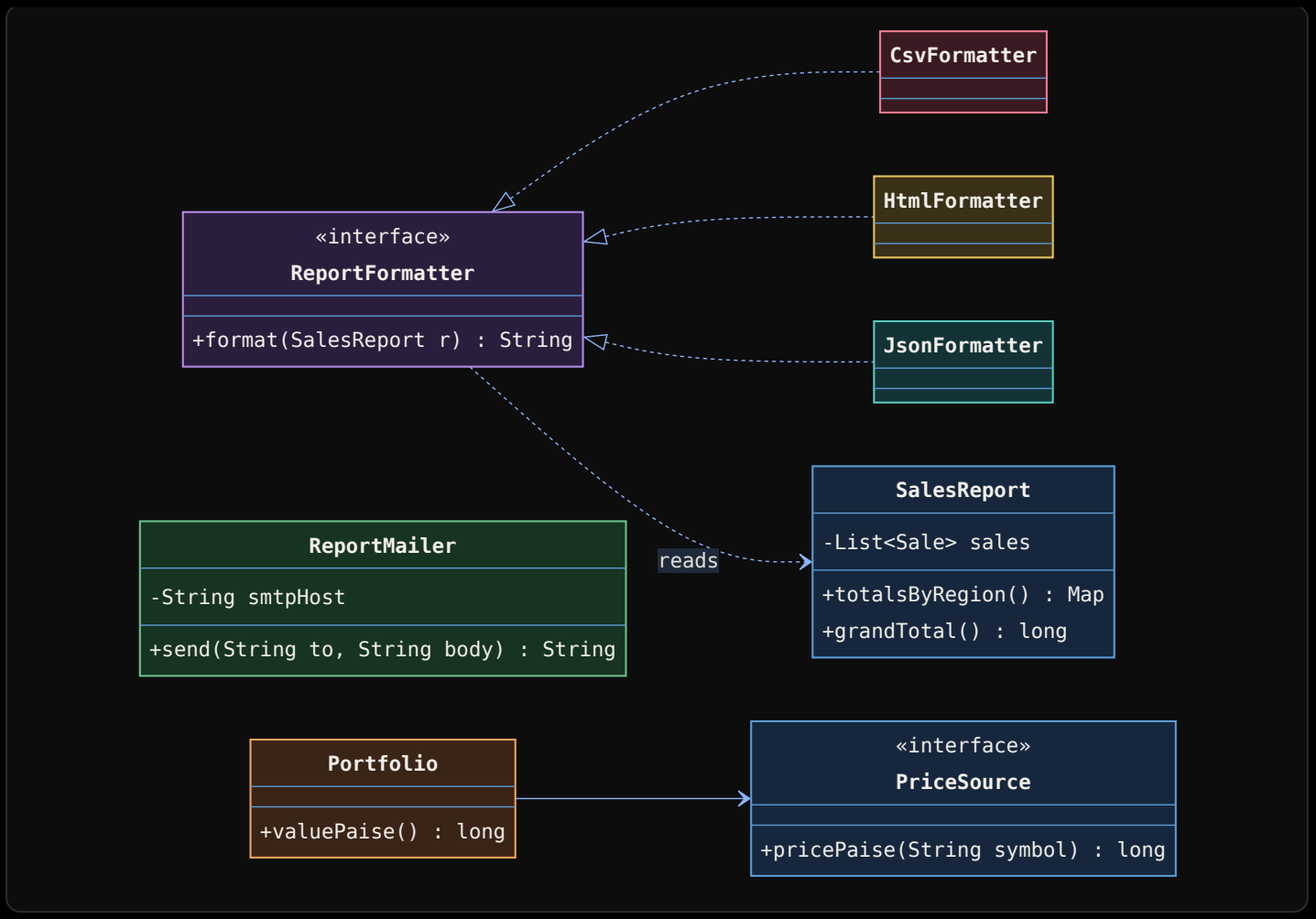
Clean classes follow the standard **organisation**, are **small** (measured in responsibilities, not lines), have **one reason to change** (SRP), are **cohesive**, and are **organised for change** so new features mean new classes rather than edits to old ones.

The problem it solves

"God" classes change for many reasons, and every change risks unrelated features; a growing `if/else` over formats reopens the same class repeatedly.

How it works

1. **Class organisation** (Java convention): public static constants → private static variables → private instance variables → public functions → private helpers right after the public function that uses them (stepdown rule). Public instance variables are rarely justified.
2. **Encapsulation**: fields and helpers private. Loosening to package-private/protected (visible within the package, or also to subclasses) *for tests* is a last resort.
3. **Small by responsibilities.** If you can't name it concisely, the name needs *Manager*, *Processor*, *Super*, or the description needs "and"/"or"/"but", it does too much; you should describe it in about 25 words.
4. **Single Responsibility Principle**: **one reason to change** (Chapter 4). Many small classes beat a few large ones — the same total code, better organised, like tools in labelled drawers rather than one big box.
5. **Cohesion**: maximal when every method uses every instance variable. When a subset of methods uses a subset of fields, that subset wants to be its own class — **maintaining cohesion results in many small classes**.
6. **Organise for change**: if a feature means *opening* a class (another branch in `render()`), split so new features are new classes (Open/Closed) — each report format became its own `ReportFormatter`.
7. **Isolate from change**: depend on abstractions, not volatile concrete details. `Portfolio` depends on `PriceSource`, not a live stock-exchange client, so it can be tested with fixed prices (Dependency Inversion).



Java example — before/after

Full program: `code/lld/ch06/classes/Main.java`.

```
// BEFORE: one class, many reasons to change
static class ReportManager {
    private final List<Sale> sales;
    private String smtpHost = "smtp.example.com";          // email concern
    private String outputFormat = "CSV";                   // formatting concern
    Map<String, Long> totalsByRegion() { ... }             // business concern
    String render() {                                      // grows a new branch for every format
        if (outputFormat.equals("CSV")) { /* build "region,total" lines */ }
        else if (outputFormat.equals("HTML")) { /* build <table> rows */ }
        throw new IllegalStateException("unknown format");
    }
    String email(String to) { return "SMTP " + smtpHost + " -> " + to + ": " + render(); }
}

// AFTER
static final class SalesReport {   // cohesive: every method uses `sales`; one reason to change (business rules)
    private final List<Sale> sales;
    SalesReport(List<Sale> sales) { this.sales = List.copyOf(sales); }
    Map<String, Long> totalsByRegion() {
        return sales.stream().collect(Collectors.groupingBy(Sale::region, java.util.TreeMap::new,
                Collectors.summingLong(Sale::amountPaise)));
    }
    long grandTotal() { return sales.stream().mapToLong(Sale::amountPaise).sum(); }
}
interface ReportFormatter { String format(SalesReport report); }   // a new format is a NEW class (OCP)
// CsvFormatter and HtmlFormatter implement it with the old branches' code
static final class JsonFormatter implements ReportFormatter {      // added later, nothing else edited
    public String format(SalesReport r) {
        return r.totalsByRegion().entrySet().stream()
                .map(e -> "\"" + e.getKey() + "\":" + e.getValue())
                .collect(Collectors.joining(",", "{", "}"));
    }
}
static final class ReportMailer {                                   // separate reason to change: delivery
    private final String smtpHost;
    String send(String to, String body) { return "SMTP " + smtpHost + " -> " + to + ": " + body; }
}
interface PriceSource { long pricePaise(String symbol); }          // isolating from change
static final class Portfolio {
    private final PriceSource prices;
    private final Map<String, Integer> holdings;
    long valuePaise() {
        return holdings.entrySet().stream().mapToLong(h -> prices.pricePaise(h.getKey()) * h.getValue()).sum();
    }
}
// main: PriceSource fixedPrices = symbol -> Map.of("INFY", 150_000L, "TCS", 400_000L).get(symbol);  // test-
friendly stub
```

Output:

```
== BEFORE ==
region,total North,6000 South,7000
SMTP smtp.example.com -> boss@example.com: <table><tr><td>North</td><td>6000</td></tr><tr><td>South</td>
<td>7000</td></tr></table>
== AFTER ==
CsvFormatter: region,total North,6000 South,7000
HtmlFormatter: <table><tr><td>North</td><td>6000</td></tr><tr><td>South</td><td>7000</td></tr></table>
JsonFormatter: {"North":6000,"South":7000}
SMTP smtp.example.com -> boss@example.com: {"North":6000,"South":7000}
portfolio value (paise) = 2300000
```

`ReportManager` had three reasons to change (business totals, output format, email delivery); after the split, `JsonFormatter` arrived without editing any class.

Trade-offs

Many small classes	Few big classes
Each is easy to understand and test	Fewer files to navigate
Changes are localised	Every change touches the same class
Need good names and packages to navigate	Merge conflicts (clashing edits to the same lines), fear of change

When not to use it

Don't split a small cohesive class into micro-classes that always change together — that's fragmentation, not SRP.

Common mistakes

- `*Manager`, `*Helper`, `*Utils` classes collecting unrelated methods.
- Splitting by *technical layer* only, leaving a god service.
- Making everything `public` "for tests".

Real-world examples

- `java.time` splits responsibilities across `LocalDate`, `LocalTime`, `ZonedDateTime`, `Duration`, `Period`, `DateTimeFormatter` — unlike the old `java.util.Date`/`Calendar`, which mixed many concerns.
- Spring's `JdbcTemplate` delegates row mapping to `RowMapper` and parameter setting to `PreparedStatementSetter`.

In the interview

> **30-second answer:** "I keep classes small by responsibility — one reason to change — and highly cohesive, so methods use most of the fields. When a feature would add another branch to an existing class, I split so the new feature is a new class, and I depend on interfaces to isolate volatile details."

Self-check

1. Q: How do you measure a class's size? → A: By its responsibilities, not lines.
2. Q: What is maximal cohesion? → A: Every method uses every instance variable.

TL;DR

Small (one responsibility), cohesive, open for extension, dependent on abstractions.

---

10. Systems

What it is

At system level, clean code means **separating the startup process (constructing and wiring objects) from the runtime logic that uses them**, using **Dependency Injection**, handling **cross-cutting concerns** in one place, and letting the architecture **grow incrementally**.

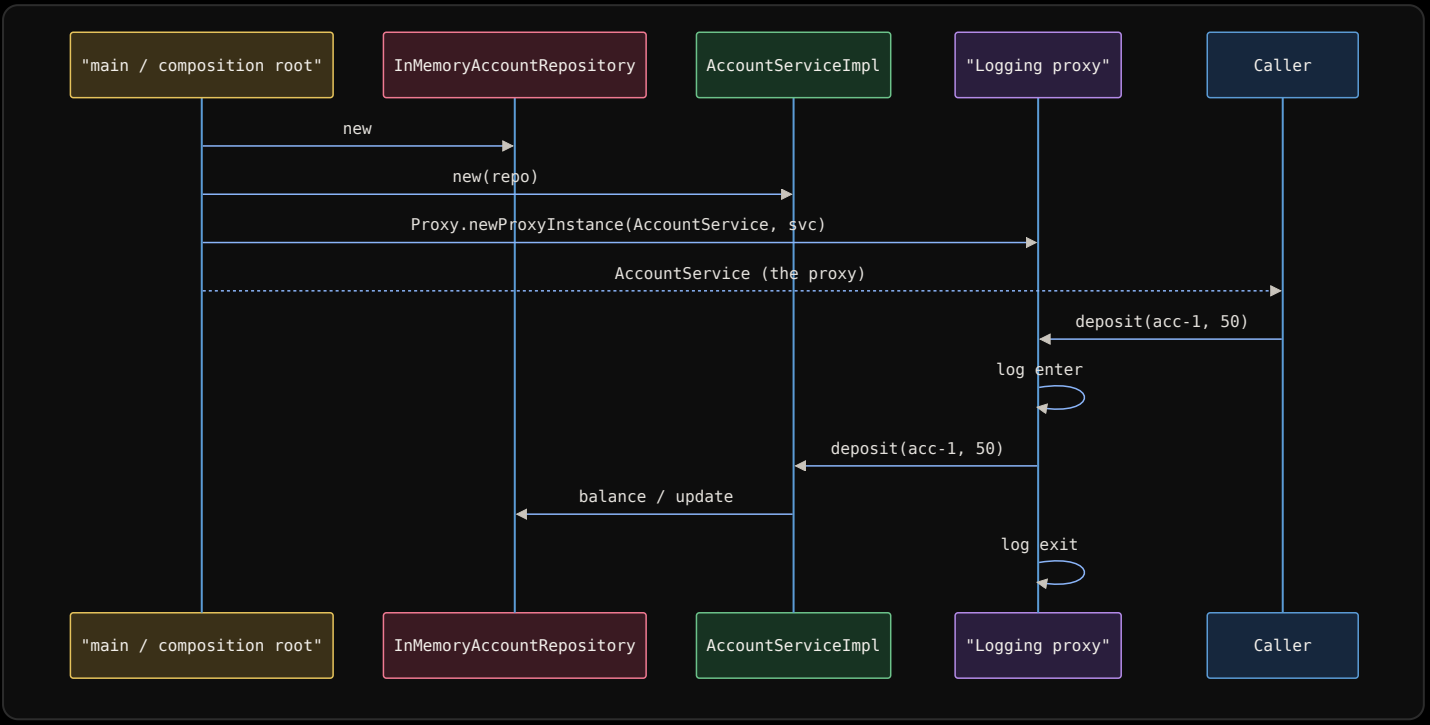
Analogy: a city — nobody designs every street up front; water, power and traffic are separate systems with clear interfaces that grow over time.

The problem it solves

Objects that create their own dependencies (`if (service == null) service = new MyServiceImpl(...)` — lazy initialisation, creating on first need) are hard-wired to concrete classes, untestable in isolation, and mix construction with use (violating SRP). Logging, security and transactions copied into every method make business code noisy and inconsistent.

How it works

1. **Separate construction from use.**
   - **Separation of `main`**: build all objects in `main` (or a startup module — the **composition root**) and pass them to the application, which knows nothing about how they were built; dependencies point from `main` toward the application, never back.
   - **Factories**: when the *application* must decide *when* to create something (a `LineItem` per scan), give it an **Abstract Factory** interface (whose methods create objects, so the caller never names concrete classes); the concrete factory stays on the `main` side.
   - **Dependency Injection (DI)**: an object receives its dependencies (constructor, setter or method parameter) instead of creating or looking them up — Inversion of Control (outside code, often a framework, creates objects and drives the flow) applied to dependencies. Prefer constructor injection: explicit, final, complete after construction.
2. **Scale up incrementally.** Well-separated concerns let the architecture grow (e.g. from a module of a monolith — one application deployed as a unit — to a separate service) without a rewrite.
3. **Cross-cutting concerns** (persistence, transactions, security, logging, caching — keeping recent results in fast storage; see System Design Fundamentals) are handled modularly:
   - **Java dynamic proxies** (`java.lang.reflect.Proxy`) wrap an interface and intercept every call — shown below.
   - **Pure-Java AOP frameworks** (AOP = aspect-oriented programming: a concern written once, as an *aspect*, applied to many methods; e.g. Spring AOP) generate proxies from annotations/config (`@Transactional`, `@Cacheable`, `@PreAuthorize`).
   - **AspectJ** — a full aspect-oriented language extension that weaves advice (the aspect's code) into bytecode (compiled Java classes the JVM runs).
   - **Decorators** and **servlet filters / interceptors** (code run before/after every web request) for simpler cases.
4. **Test-drive the architecture**: with logic in POJOs (Plain Old Java Objects, no framework dependencies) and concerns decoupled, start simple and evolve.
5. **Optimise decision making**: decide at the last responsible moment, when you know most — modularity allows it.
6. **Use standards wisely** — only with demonstrable value (heavyweight standards can bury simple logic).
7. **Systems need domain-specific languages**: a good DSL (a mini-language for one problem) or fluent API (methods that chain into readable sentences) lets code read like the domain (a test-fixture builder, a routing DSL).



Java example — before/after

Full program: `code/lld/ch06/systems/Main.java`.

```java
// BEFORE: the service builds its own dependency (lazy init) and mixes in logging
private AccountRepository repo() {
    if (repo == null) repo = new InMemoryAccountRepository();   // construction mixed with use; hard-coded impl
    return repo;
}
public void deposit(String id, long amount) {
    System.out.println("  LOG enter deposit " + id);          // cross-cutting code, copied into every method
    if (amount <= 0) throw new IllegalArgumentException("amount must be positive");
    repo().update(id, repo().balance(id) + amount);
    System.out.println("  LOG exit deposit");
}

// AFTER: pure business logic; dependency injected through the constructor; no logging code
static final class AccountServiceImpl implements AccountService {
    private final AccountRepository repo;
    AccountServiceImpl(AccountRepository repo) { this.repo = repo; }
    public long balance(String id) { return repo.balance(id); }
    public void deposit(String id, long amount) {
        if (amount <= 0) throw new IllegalArgumentException("amount must be positive");
        repo.update(id, repo.balance(id) + amount);
    }
}
// Cross-cutting concern written ONCE, applied to any interface via a JDK dynamic proxy (how Spring AOP works for interfaces)
static <T> T withLogging(Class<T> type, T target) {
    InvocationHandler handler = (proxy, method, args) -> {
        System.out.println("  LOG -> " + method.getName() + "(" + /* args joined */ ")");
        try {
            Object result = method.invoke(target, args);
            System.out.println("  LOG <- " + method.getName() + (result == null ? "" : " = " + result));
            return result;
        } catch (InvocationTargetException e) {
            System.out.println("  LOG !! " + method.getName() + " threw " + e.getCause().getMessage());
            throw e.getCause();                          // rethrow the real exception, not the reflection wrapper
        }
    };
    return (T) Proxy.newProxyInstance(type.getClassLoader(), new Class<?>[]{type}, handler);
}
// Composition root: the ONLY place that knows concrete classes and wires them together
static AccountService buildApplication() {
    AccountRepository repo = new InMemoryAccountRepository();
    AccountService core = new AccountServiceImpl(repo);
    return withLogging(AccountService.class, core);
}
// a unit test can wire the core differently: new AccountServiceImpl(new InMemoryAccountRepository())
```

Output:

```
== BEFORE ==
  LOG enter deposit acc-1
  LOG exit deposit
  LOG enter balance acc-1
  LOG exit balance = 150
balance = 150
== AFTER ==
  LOG -> deposit(acc-1, 50)
  LOG <- deposit
  LOG -> balance(acc-1)
  LOG <- balance = 150
  LOG -> deposit(acc-1, -5)
  LOG !! deposit threw amount must be positive
caller sees: amount must be positive
test double wiring works: 101
```

Trade-offs

DI + composition root + proxies	Self-constructing objects
Testable, swappable, one place for wiring	Simple for tiny programs
Cross-cutting code in one place	Duplicated, inconsistent concerns
"Magic" with frameworks; proxies only intercept calls *through* the proxy	Explicit and visible

A real Spring gotcha from the proxy mechanism: calling a `@Transactional` method *from another method of the same bean* (a Spring-managed object) — self-invocation — bypasses the proxy, so no transaction starts.

When not to use it

Small scripts and interview programs don't need a DI framework — manual constructor injection in `main` is enough (and what interviewers expect).

Common mistakes

- Service Locator calls (a global registry code asks for dependencies: `Registry.get(Foo.class)`) scattered in business code — hides dependencies.
- Field injection with `@Autowired` on private fields — hides dependencies, blocks immutability; prefer constructor injection.
- Business logic depending on framework annotations everywhere.

Real-world examples

- **Spring IoC container / Spring Boot**, **Google Guice**, **Dagger** (compile-time DI, common on Android), **Jakarta CDI**.
- **Spring AOP** implements `@Transactional`, `@Cacheable`, `@Async` via JDK dynamic proxies (interfaces) or CGLIB subclass proxies (generated subclasses wrapping the real class).
- Servlet filters and Spring `HandlerInterceptor`s for authentication and request logging.

In the interview

> **30-second answer:** "I separate construction from use: a composition root in `main` builds and wires everything, and classes receive dependencies through constructors. Cross-cutting concerns like logging or transactions go in one place — a decorator, proxy or aspect — not in every method. That keeps the domain in plain, testable objects."

Follow-ups:

- *Q: DI vs Service Locator?* A: DI pushes dependencies in (explicit in the constructor); a locator pulls them (hidden, harder to test).
- *Q: How does Spring implement `@Transactional`?* A: A proxy around the bean begins/commits/rolls back around the method call.

Self-check

1. Q: What is the composition root? → A: The one place (near `main`) where objects are constructed and wired.
2. Q: Three ways to handle cross-cutting concerns in Java? → A: Dynamic proxies, AOP frameworks/AspectJ, decorators (also filters/interceptors).

TL;DR

Build objects in one place and inject them; put cross-cutting concerns in proxies/aspects; grow the architecture incrementally.

---

11. Emergence — Kent Beck's Four Rules of Simple Design

What it is

Good design can **emerge** from four rules. In priority order, a design is simple if it:

1. **Runs all the tests** (passes them).
2. **Contains no duplication.**
3. **Expresses the intent of the programmer.**
4. **Minimises the number of classes and methods.**

(*Clean Code*'s Chapter 12, by Jeff Langr, uses this order. **Kent Beck's own usual wording and order** — summarised in Martin Fowler's "BeckDesignRules" — is **1. Passes the tests, 2. Reveals intention, 3. No duplication, 4. Fewest elements.** Fowler notes people swap the middle two and it rarely matters because they reinforce each other; rules 1 and 4 are always first and last. In an interview, quote Beck's order and mention *Clean Code* swaps 2 and 3.)

The problem it solves

They give an ordered checklist that consistently pushes code toward SRP, DIP and good structure.

How it works

1. **Runs all the tests** — an unverifiable system shouldn't ship. Testability pushes toward small, single-purpose, loosely coupled classes (DI and interfaces), so rule 1 alone drives much good design.
2. **Refactor, guided by rules 2–4**, every few lines:
   - **No duplication** — duplicate lines *and* duplicate implementation (`isEmpty()` could be `size() == 0`). Extract methods; use **Template Method** for higher-level duplication (a shared algorithm with varying steps).
   - **Expressive** — good names, small functions and classes, standard pattern names (`...Command`, `...Visitor`), tests as documentation. Most importantly: *care*.
   - **Minimal classes and methods** — no interface for every class, no dogmatic splitting of fields from behaviour. Lowest priority: SRP and no-duplication win conflicts.

Java example — before/after

Full program: `code/lld/ch06/emergence/Main.java`.

```java
// BEFORE: duplicated algorithm, magic numbers, needless layers
interface LeaveCalcStrategyProvider { LeaveCalc get(); }            // extra element with no purpose
static class LeaveCalc {
    int calcIN(Employee e) {                                         // unclear name
        if (e.onProbation()) return 0;
        int d = e.monthsWorked() * 18 / 12;                          // 18 per year? magic
        if (d > 45) d = 45;                                          // 45 cap? magic
        return d;
    }
    int calcUS(Employee e) { /* the same algorithm, copied, with 15 and 30 */ }
}

// AFTER: one algorithm (Template Method), named rules, only needed elements
abstract static class LeavePolicy {
    final int earnedLeaveDays(Employee e) {            // the shared algorithm, written once (rule 2)
        if (e.onProbation()) return 0;
        return Math.min(daysPerYear() * e.monthsWorked() / 12, maxCarryForwardDays());
    }
    protected abstract int daysPerYear();              // what varies, with intention-revealing names (rule 3)
    protected abstract int maxCarryForwardDays();
}
static final class IndiaLeavePolicy extends LeavePolicy {
    protected int daysPerYear() { return 18; }
    protected int maxCarryForwardDays() { return 45; }
}
// UsLeavePolicy: 15 and 30

// Rule 1 first: the refactoring is only safe because these tests pass before AND after
@Test void india_policy_matches_old_behaviour() {
    var old = new Before.LeaveCalc();
    var policy = new After.IndiaLeavePolicy();
    for (Employee e : new Employee[]{veteran, newHire, probationer})
        MiniTest.assertEquals(old.calcIN(e), policy.earnedLeaveDays(e));
}
// also: us_policy_matches_old_behaviour, carry_forward_is_capped, probationers_earn_nothing
```

Output:

```
LeavePolicyTest
  PASS  carry forward is capped
  PASS  india policy matches old behaviour
  PASS  probationers earn nothing
  PASS  us policy matches old behaviour
4 passed, 0 failed
Asha: India 45 days, US 30 days
Ravi: India 12 days, US 10 days
```

Tests (rule 1) prove behaviour is unchanged; duplicate algorithms became one Template Method (rule 2); magic numbers became `daysPerYear()` / `maxCarryForwardDays()` (rule 3); the purposeless `LeaveCalcStrategyProvider` was dropped (rule 4).

Trade-offs

Rules 2–4 can conflict (removing duplication can add a class; expressiveness can add a method). Resolve by priority, or find a design that satisfies both.

When not to use it

Always applicable — a lightweight mindset, not a process.

Common mistakes

- Refactoring without tests (breaking rule 1 to satisfy the others).
- Dogmatically minimising classes at the expense of SRP.

Real-world examples

Extreme Programming teams apply these rules continuously. The Template Method refactoring above mirrors how `java.util.AbstractList` removes duplication across list implementations.

In the interview

> **30-second answer:** "Kent Beck's rules of simple design, in priority order: passes the tests, reveals intention, no duplication, fewest elements (*Clean Code* lists no-duplication before intention; the two reinforce each other). I get the core working with a quick test or demo, then refactor out duplication and improve names, and don't add classes that don't earn their place."

Self-check

1. Q: Which rule comes first and why? → A: Runs all the tests — without them you can't refactor safely, and testability itself improves design.
2. Q: Which rule has the lowest priority? → A: Minimal classes and methods.

TL;DR

Tests pass → no duplication → clear intent → nothing extra.

---

12. Concurrency

What it is

Concurrency — several tasks in progress at once, usually on separate *threads* (independent paths of execution within one program) — is a **decoupling strategy**: it separates *what* gets done from *when*. It can improve throughput (work completed per second; see System Design Fundamentals) and structure, but is hard to get right. Clean concurrency keeps concurrency code small, separate and built on well-tested library tools. Chapters 12–13 cover concurrency in depth; this section is *Clean Code*'s view.

The problem it solves

Myths cause bugs: *concurrency always improves performance* (only with waiting or multiple cores — CPU units each running a thread at once — to exploit); *design doesn't change* (it does); *a web-server container means I needn't understand it* (requests still share state). Concurrency adds overhead, its bugs are often unrepeatable, and it needs a different design strategy.

How it works — defence principles

1. **Single Responsibility**: keep concurrency code separate — below, `countWords` is pure logic and the executor code (running tasks on a pool of reusable threads) is apart.
2. **Limit the scope of shared data**: encapsulate it, minimise the places touching it, guard critical sections (code touching shared data that one thread at a time must run, usually enforced with a *lock*).
3. **Use copies of data**: share immutable objects, or give each thread its own copy and merge at the end.
4. **Threads as independent as possible**: each processes its own data (like each request's local variables).
5. **Know your library**: thread-safe collections (`ConcurrentHashMap`), executors (`ExecutorService`), non-blocking atomics (`AtomicInteger`, `LongAdder` — single indivisible updates without locks), synchronisers (`CountDownLatch`, `Semaphore`, `CyclicBarrier`) — not hand-made `wait/notify`.
6. **Know your execution models** (Chapter 13): **producer–consumer** (bounded queue between producers and consumers), **readers–writers** (many readers, exclusive writers; balance throughput vs starvation), **dining philosophers** (competing for multiple resources → deadlock, livelock, starvation).
7. **Beware dependencies between synchronised methods** (`synchronized` — one thread at a time per object) on one shared object: if you must call several, use client-based locking, server-based locking (one method doing it all under its lock) or an adapted server.

8. **Keep synchronised sections small** — locks cause contention (threads queuing for one lock); lock as little as possible, correctness first.
9. **Correct shutdown is hard** — plan graceful shutdown (let running tasks finish) from the start (`shutdown()` + `awaitTermination()`, then `shutdownNow()`); beware threads waiting for signals that never come.
10. **Testing threaded code:** treat one-off failures as likely threading bugs (not "cosmic rays"); get non-threaded code working first; make it **pluggable** (1, 2, many threads) and **tunable** (configurable counts); run **more threads than processors**, on different platforms; **instrument** code to force failures ("jiggling" — inserted yields/sleeps that change interleavings, the order threads' steps mix; tools like jcstress).

Key vocabulary: **bound resources** (fixed-size resources like connection pools — reusable sets of open database connections), **mutual exclusion** (one thread at a time per resource), **starvation** (a thread never gets its turn), **deadlock** (threads wait for each other forever), **livelock** (threads keep reacting to each other without progress). *Clean Code*'s **Appendix A, "Concurrency II"**, goes deeper: client- vs server-based locking, deadlock's four conditions and how to break them, testing multithreaded code.

Java example — before/after

Full program: `code/lld/ch06/concurrency/Main.java`.

```
static Map<String, Integer> countWords(String page) {     // pure logic -- testable without threads
    Map<String, Integer> counts = new HashMap<>();
    for (String w : page.split(" ")) counts.merge(w, 1, Integer::sum);
    return counts;
}

// BEFORE: 4 threads share an unsynchronised counter
static int theCount = 0;                                  // shared mutable state, no synchronisation
// each thread: for every word w in PAGES (repeated): if (w.equals("the")) theCount++;   // read-modify-write race
// (sharing a plain HashMap this way is even worse: concurrent resizes can corrupt it)

// AFTER: each task works on its OWN data; results are combined once, at the end
static Map<String, Integer> run(int repeats, int threads) throws Exception {
    ExecutorService pool = Executors.newFixedThreadPool(threads);   // know your library
    try {
        List<Callable<Map<String, Integer>>> tasks = new java.util.ArrayList<>();
        for (int t = 0; t < threads; t++) {
            tasks.add(() -> {
                Map<String, Integer> local = new HashMap<>();       // thread-confined: no sharing, no locks
                for (int r = 0; r < repeats; r++)
                    for (String page : PAGES) countWords(page).forEach((w, c) -> local.merge(w, c, Integer::sum));
                return local;
            });
        }
        Map<String, Integer> total = new HashMap<>();              // combined by ONE thread
        for (Future<Map<String, Integer>> f : pool.invokeAll(tasks))
            f.get().forEach((w, c) -> total.merge(w, c, Integer::sum));   // Future.get() safely hands over each
result
        return Map.copyOf(total);                                            // publish an immutable copy
    } finally {
        pool.shutdown();                                                     // clean shutdown is part of the
design
        if (!pool.awaitTermination(5, TimeUnit.SECONDS)) pool.shutdownNow();
    }
}
// main runs the AFTER with 1, 4 and 16 threads -- more threads than cores is a good stress test
```

Output:

```
pure logic: {cat=1, mat=1, on=1, sat=1, the=2}
== BEFORE (racy; the number varies from run to run) ==
count(the) = 182526 expected 320000 -> lost 137474 updates
== AFTER (deterministic) ==
1 thread(s): count(the) = 80000 expected 80000
4 thread(s): count(the) = 320000 expected 320000
16 thread(s): count(the) = 1280000 expected 1280000
```

`theCount++` is a read-modify-write, so concurrent increments overwrite each other and updates are **lost** — the count varies per run, and a rare run may even be correct, which is why such bugs escape testing. In *after*, `Future.get()` (a `Future` is a handle to a task's later result) guarantees each result is safely visible to the merging thread.

Trade-offs

Confinement + merge (after)	Shared state + locks
No locks, no races, scales well	Simple mental model for small cases
Extra memory for per-thread copies	Contention, deadlock risk
Needs a merge step	Every access must be correctly synchronised

When not to use it

Don't add concurrency without a measured need; one thread is simpler and often fast enough. Prefer higher-level tools — parallel streams for pure CPU work; Java 21 virtual threads (lightweight JVM-managed threads, cheap enough for one per task) for many blocking I/O tasks (input/output where the thread just waits, e.g. a network call) — over manual thread management.

Common mistakes

- Sharing a `HashMap`/`ArrayList` between threads without synchronisation.
- Check-then-act races (`if (!map.containsKey(k)) map.put(k, v)`) — use `putIfAbsent`/`computeIfAbsent`.
- Forgetting to shut down executors (the JVM may not exit).
- Holding locks while calling unknown code (callbacks — functions passed in to be called later) → deadlock risk.

Real-world examples

- **`java.util.concurrent`** (Doug Lea): `ExecutorService`, `ConcurrentHashMap`, `CompletableFuture`, `ForkJoinPool` (work-stealing — idle threads take queued tasks from busy ones; used by parallel streams).
- **Map-reduce / Hadoop / Spark** (split big-data jobs across machines): independent workers on partitions + a merge — the same confinement idea at cluster scale.
- **Node.js and Redis** (a JavaScript server runtime, an in-memory data store) use single-threaded event loops (one thread handling ready events in turn) for main processing, avoiding shared-memory races by design.

In the interview

> **30-second answer:** "I keep concurrency separate from business logic, minimise shared mutable state — preferring thread confinement (each piece of data used by one thread), immutable data and merging results — and use `java.util.concurrent` rather than raw threads. I plan shutdown and test with more threads than cores."

Follow-ups:

- *Q: Why is `count++` not thread-safe?* A: It's read, add, write — two threads can read the same value and one update is lost. Use `AtomicInteger`/`LongAdder` or a lock.

Self-check

1. Q: What does concurrency decouple? → A: What gets done from when it gets done.
2. Q: How should you treat a one-off test failure in concurrent code? → A: As a likely real threading bug.

TL;DR

Separate concurrency code, share as little as possible (copies, immutability, confinement), use the library, keep locks small, plan shutdown, test hard.

---

13. Successive Refinement and the Case Studies (Chapters 14–16: how clean code is actually written)

Chapters 14–16 of *Clean Code* are case studies, not new rules: real code cleaned step by step. They show the *process*.

Chapter 14 — Successive Refinement refines a command-line argument parser (`Args`) — code reading options typed after a program's name, like `-l -p 8080`. Lessons:

1. **Write it dirty first, then clean it.** A working first draft is normal; *stopping* there is not.
2. **Stop when it starts to rot.** When each new feature (a new argument type) makes the code noticeably messier, stop adding features and restructure.
3. **Refactor in tiny steps with tests**, running them after every small change; "big-bang" restructurings tend to fail.
4. **Use design to absorb variation**: each argument type became its own class behind a common interface (a polymorphic "marshaler"), so new types didn't touch the parser — the same Open/Closed move as the report formatters (§9).
5. **Clean the error handling too** — collect error codes and messages into one exception class.

Chapter 15 — JUnit Internals improves `ComparisonCompactor` (by Kent Beck and Erich Gamma), already good and well tested. Lessons (codes refer to the §14 catalogue):

- Remove encoding prefixes on fields (`fExpected` → `expected`) — N6.
- Encapsulate conditionals and prefer positive ones (`if (shouldNotCompact())` → `if (canBeCompacted())`) — G28, G29.
- Rename a function whose name hides a side effect or second job (`compact` that also formats → `formatCompactedComparison`) — N7, G30.
- Make hidden temporal coupling explicit (finding the common suffix depends on first finding the prefix, so make the order visible) — G31.
- Tidy `+1`/`-1` boundary arithmetic into well-named variables — G33.
- Even good code can be left a little cleaner — the Boy Scout Rule.

Chapter 16 — Refactoring SerialDate cleans a date class from the open-source (free to use and modify) JCommon library:

1. **First, make it work** — more tests: coverage (the share of lines the tests execute) rose from about 50% to about 92%, exposing real bugs, which were fixed.
2. **Then make it right** — rename (`SerialDate` → `DayDate`), replace integer constants with enums (a `Month` enum; J3), move methods to the classes owning the data (G14/G17), introduce an **Abstract Factory** so the base class no longer knows its concrete subclass (G7), delete dead code, duplication and noise comments (G9, G5, C3).
3. Every step keeps the tests green.

TL;DR: make it work, then make it right, in small tested steps — and don't let a mess grow.

---

14. Smells and Heuristics (the checklist)

What it is

Clean Code ends with a catalogue of **smells and heuristics** — short rules with codes (C1, G5, N1, …), grouped by area, extending the smells from Fowler's *Refactoring* (Chapter 7). Use it as a review checklist: each item is a *hint* that something could be cleaner, not an absolute law.

The problem it solves

You must *recognise* problems quickly while reading; a named catalogue gives reviewers a shared vocabulary ("this is G28 — encapsulate that conditional").

How it works — the catalogue, grouped (names listed; meaning in our words)

Comments (C)

Code	Heuristic	Meaning
C1	Inappropriate information	Change history, authors, ticket metadata belong in version control / issue tracker
C2	Obsolete comment	Old, irrelevant or wrong comments — update or delete
C3	Redundant comment	Says what the code already says clearly
C4	Poorly written comment	If worth writing, write it well: brief, correct grammar, precise
C5	Commented-out code	Delete it; version control remembers

Environment (E)

Code	Heuristic	Meaning
E1	Build requires more than one step	Check out, run one command, get a build (compiled, packaged program)
E2	Tests require more than one step	One command (or one click) runs all tests

Functions (F)

Code	Heuristic	Meaning
F1	Too many arguments	Zero best, then one, two, three; more needs strong justification
F2	Output arguments	Surprising; change the owning object's state instead
F3	Flag arguments	A boolean argument means the function does two things
F4	Dead function	Never called? Delete it

General (G) — the largest group, clustered by theme:

Correctness and safety

Code	Heuristic	Meaning
G2	Obvious behaviour is unimplemented	Honour the principle of least surprise; implement what users would expect from the name
G3	Incorrect behaviour at the boundaries	Test edge and corner cases (inputs at the limits: empty, zero, maximum); don't rely on intuition
G4	Overridden safeties	Don't turn off compiler warnings, skip failing tests, or ignore errors
G21	Understand the algorithm	Don't fiddle with ifs until tests pass; know *why* it works
G26	Be precise	e.g. don't use floating point (`double`, inexact for decimals) for currency; handle concurrency, nulls and ties deliberately
G31	Hidden temporal couplings	If calls must happen in order, make the order explicit (each step returns what the next needs)
G33	Encapsulate boundary conditions	Put `+1`/`-1` edge logic in one named place (`int lastIndex = size - 1;`)

Duplication and dead weight

Code	Heuristic	Meaning
G5	Duplication	"One of the most important rules" in the book: every duplication is a missed opportunity for abstraction (methods, polymorphism, Template Method, Strategy)
G9	Dead code	Unreachable/unused code — delete
G12	Clutter	Unused variables, empty default constructors, meaningless comments

Structure and placement

Code	Heuristic	Meaning
G1	Multiple languages in one source file	Minimise mixing (e.g. Java + SQL + HTML + JS in one file)
G6	Code at wrong level of abstraction	Keep high-level concepts in base classes/interfaces and details in implementations
G7	Base classes depending on their derivatives	A base class shouldn't know its subclasses
G8	Too much information	Small, focused interfaces; hide data, utilities, constants
G10	Vertical separation	Define variables and private functions close to where they're used
G13	Artificial coupling	Don't put things together that don't depend on each other (e.g. a general enum inside a specific class)
G14	Feature envy	A method more interested in another class's data than its own should move there
G17	Misplaced responsibility	Put code where a reader would expect to find it
G18	Inappropriate static	Make a method static only if it will never need polymorphic behaviour
G22	Make logical dependencies physical	If a module depends on another's assumption, ask it explicitly instead of assuming
G35	Keep configurable data at high levels	Defaults/constants that are configuration belong near the top (e.g. in `main`/config), passed down
G36	Avoid transitive navigation	Law of Demeter — don't write `a.getB().getC()`

Readability and intent

Code	Heuristic	Meaning
G11	Inconsistency	Do similar things the same way throughout
G15	Selector arguments	Arguments (booleans, enums, ints) that select behaviour → split into functions
G16	Obscured intent	Dense, clever expressions and magic hex numbers hide meaning
G19	Use explanatory variables	Break calculations into named intermediate values
G20	Function names should say what they do	`invoice.adjust(5)` — five what? up or down? Name it `applyDiscountPercent(5)`
G23	Prefer polymorphism to if/else or switch/case	"One switch" rule: at most one switch per type selection, used to create polymorphic objects
G24	Follow standard conventions	Team/industry coding standard, applied consistently
G25	Replace magic numbers with named constants	Also applies to magic strings and tokens
G27	Structure over convention	Enforce design decisions with structure (e.g. abstract methods) rather than naming conventions
G28	Encapsulate conditionals	`if (isRefundable(order))` beats `if (order.isDelivered() && order.daysSinceDelivery() <= 7)`
G29	Avoid negative conditionals	`if (user.isActive())` beats `if (!user.isNotActive())`
G30	Functions should do one thing	Split multi-step functions
G32	Don't be arbitrary	Structure code for a reason and let the reason show
G34	Functions should descend only one level of abstraction	Statements in a function sit one level below the function's name

Java (J)

Code	Heuristic	Meaning
J1	Avoid long import lists by using wildcards (`import java.util.*;`)	The book's advice; note many modern style guides (e.g. Google Java Style) forbid wildcard imports — follow your team's standard
J2	Don't inherit constants	Don't implement an interface just to get its constants; use static imports
J3	Constants versus enums	Prefer `enum` (type-safe, can carry data and behaviour) over `public static final int` codes

Names (N)

Code	Heuristic	Meaning
N1	Choose descriptive names	Names make up most of readability
N2	Choose names at the appropriate level of abstraction	Name for the concept, not the implementation (`notify(recipient)`, not `sendSms(phone)` if it may later be email or push)
N3	Use standard nomenclature where possible	Pattern names, domain terms (`...Decorator`, `...Factory`)
N4	Unambiguous names	Name should make the function's work clear
N5	Use long names for long scopes	Short names only in tiny scopes
N6	Avoid encodings	No type/scope prefixes
N7	Names should describe side effects	`getOrCreateUser`, not `getUser`, if it creates

Tests (T)

Code	Heuristic	Meaning
T1	Insufficient tests	Test everything that could break
T2	Use a coverage tool	Find untested lines and branches (e.g. JaCoCo)
T3	Don't skip trivial tests	They're cheap and document behaviour
T4	An ignored test is a question about an ambiguity	`@Disabled` tests mark unclear requirements — resolve them
T5	Test boundary conditions	Off-by-one errors (one too many or too few) live at the edges
T6	Exhaustively test near bugs	Bugs cluster; where you find one, look for more
T7	Patterns of failure are revealing	Which tests fail together often points to the cause
T8	Test coverage patterns can be revealing	Code not executed by passing tests hints at why failures happen
T9	Tests should be fast	Slow tests don't get run

Java example — several heuristics on one function

Full program: `code/lld/ch06/smells/Main.java`. `Parcel(weightKg, distanceKm, membership, fragile)` is a record; `Membership` is `NONE` or `PRIME`.

```
// BEFORE
static double calc(Parcel p, boolean express) {                                  // double for money (G26)
    double c;
    if (!(p.membership() != Membership.PRIME)) c = 0; else c = 40;          // double negative
    if (p.weightKg() > 20 || p.distanceKm() > 1500 && !p.fragile() == false) c += 250;   // what does this mean?
    c += p.weightKg() * 12.5;
    if (express) c = c * 1.5;                                                    // flag argument
    if (express && p.membership() == Membership.PRIME) c = c * 0.9;
    return Math.round(c * 100) / 100.0;
}

// AFTER
enum Speed {                                                 // replaces the flag; each speed knows its multiplier
    STANDARD("1.0"), EXPRESS("1.5");
    final BigDecimal multiplier;
    Speed(String m) { multiplier = new BigDecimal(m); }
}
private static final BigDecimal BASE_FEE = new BigDecimal("40");          // rupees, exact decimal (G26)
private static final BigDecimal RATE_PER_KG = new BigDecimal("12.5");
private static final double HEAVY_KG = 20;
private static final int LONG_HAUL_KM = 1500;
private static final BigDecimal SPECIAL_HANDLING_FEE = new BigDecimal("250");
private static final BigDecimal PRIME_EXPRESS_DISCOUNT = new BigDecimal("0.9");

static BigDecimal shippingCost(Parcel p, Speed speed) {
    BigDecimal baseFee = isPrime(p) ? BigDecimal.ZERO : BASE_FEE;
    BigDecimal handlingFee = needsSpecialHandling(p) ? SPECIAL_HANDLING_FEE : BigDecimal.ZERO;
    BigDecimal weightCharge = BigDecimal.valueOf(p.weightKg()).multiply(RATE_PER_KG);   // explanatory variables
    BigDecimal cost = baseFee.add(handlingFee).add(weightCharge).multiply(speed.multiplier);
    if (isPrime(p) && speed == Speed.EXPRESS) cost = cost.multiply(PRIME_EXPRESS_DISCOUNT);
    return cost.setScale(2, RoundingMode.HALF_UP);                           // round once, explicitly, to paise
}
private static boolean isPrime(Parcel p) { return p.membership() == Membership.PRIME; }   // positive conditional
private static boolean needsSpecialHandling(Parcel p) {                                  // encapsulated
conditional
    boolean heavy = p.weightKg() > HEAVY_KG;
    boolean fragileLongHaul = p.distanceKm() > LONG_HAUL_KM && p.fragile();
    return heavy || fragileLongHaul;
}
// main compares before vs after for 4 parcels x {standard, express}
```

Output:

```
Parcel[weightKg=2.0, distanceKm=300, membership=NONE, fragile=false] express=false before=  65.00 after=  65.00
same
Parcel[weightKg=2.0, distanceKm=300, membership=NONE, fragile=false] express=true  before=  97.50 after=  97.50
same
Parcel[weightKg=25.0, distanceKm=100, membership=PRIME, fragile=false] express=false before= 562.50 after= 562.50
same
Parcel[weightKg=25.0, distanceKm=100, membership=PRIME, fragile=false] express=true  before= 759.38 after= 759.38
same
Parcel[weightKg=3.0, distanceKm=2000, membership=NONE, fragile=true] express=false before= 327.50 after= 327.50
same
Parcel[weightKg=3.0, distanceKm=2000, membership=NONE, fragile=true] express=true  before= 491.25 after= 491.25
same
Parcel[weightKg=3.0, distanceKm=2000, membership=PRIME, fragile=false] express=false before=  37.50 after=  37.50
same
Parcel[weightKg=3.0, distanceKm=2000, membership=PRIME, fragile=false] express=true  before=  50.63 after=  50.63
same
```

"same" proves behaviour didn't change. Fixed: double negative (G29), the unreadable `&&` / `||` / `== false` condition (G16, G28 → `needsSpecialHandling`), magic numbers (G25), flag argument (F3 → `Speed`), unnamed intermediates (G19), cryptic `calc` (N1/G20 → `shippingCost`), floating-point money (G26 → `BigDecimal` with one explicit `HALF_UP` rounding to paise). The *before* keeps `double` only to show the smell; never do that in real money code.

Trade-offs

A checklist can be applied dogmatically. Each item is a heuristic: when two conflict, favour readability and correctness.

When not to use it

Don't block a critical fix on style nits; log them as follow-ups.

Common mistakes

- Treating smell counts as a metric to game.
- Following the dated **Java** advice (J1) over your team's style guide.

Real-world examples

Static analysers (tools that inspect code without running it) encode many of these: **SonarQube** (cognitive complexity — a score for how hard code is to follow, duplicated blocks, commented-out code), **PMD** (unused code, god class), **Checkstyle** (naming, magic numbers), **SpotBugs** (bug patterns), **Error Prone** (Google).

In the interview

> **30-second answer:** "When reviewing I look for duplication, long functions, flag and output arguments, magic numbers, negative or complex conditionals, feature envy, train wrecks, misplaced responsibilities, dead code and weak tests. Most fixes are small refactorings: extract method, introduce constant, replace conditional with polymorphism."

Self-check

1. Q: Which heuristic does the book call one of its most important rules? → A: G5, Duplication.
2. Q: What does T4 say about a disabled test? → A: It signals an unresolved ambiguity in requirements.

TL;DR

Use the C/E/F/G/J/N/T catalogue as a shared review vocabulary; fix with small, tested refactorings.

---

15. Code Review Checklist (best practices)

What it is

A **code review** is a peer reading a change before it's merged (combined into the team's main branch), to catch defects, share knowledge and keep the codebase consistent. A checklist makes reviews thorough — and in interviews it's what you run over your own code before saying "done".

The problem it solves

Unreviewed changes bring bugs, security holes and inconsistent style; unfocused reviews chase nits and miss real issues.

How it works — the checklist (from most to least important)

1. **Correctness** — does it meet the requirement/ticket? Edge cases (empty, null, zero, max, duplicates, time zones, rounding)? Errors handled — nothing swallowed, meaningful messages, resources closed? Concurrency — shared mutable state guarded, no check-then-act races, thread-safe collections?
2. **Design** — fits the architecture? Right class/layer for each responsibility (SRP, GRASP)? Depends on abstractions (DIP)? Extensible without modification (OCP)? No over-engineering (YAGNI) or duplication (DRY)?
3. **Readability** — intention-revealing names, small functions, no magic numbers or flag arguments; comments explain *why*; no commented-out code.
4. **Tests** — new behaviour covered? Bug fixes include a regression test (one that stops the fixed bug returning)? Tests F.I.R.S.T., one concept each, readable names?
5. **Security** — input validation; no SQL/command injection (user input run as part of a query or shell command; prevented by parameterised queries, which send input separately as plain data); output encoding (escaping data put into HTML so it can't run as script); no secrets or credentials in code or logs; sensitive data (PII — personally identifiable information such as names, phones, emails) masked in logs; authorisation checks (may *this* user do *this*?) on every sensitive operation.
6. **Performance and resources** — N+1 queries (one query for a list, then one per item), repeated work in loops, unbounded collections/caches, missing pagination; timeouts (a maximum wait) on remote calls; streams closed.
7. **Operability** — useful logs at the right level (not too many); metrics (counters and timings for dashboards and alerts) for important events; feature flags (switches that turn a feature on/off without redeploying) for risky changes; backwards compatibility (old clients keep working) of APIs, messages and database migrations (scripted schema changes).
8. **Style and consistency** — ideally automated by formatter/linter so humans focus on the above.

How to review well (and be reviewed well):

- Keep changes **small** (a few hundred lines at most); large diffs get rubber-stamped (approved without real reading).
- Author: clear description (what, why, how tested); self-review first.
- Reviewer: be specific and kind; explain *why*; separate must-fix from optional ("nit:"); ask rather than accuse.
- Automate the mechanical (formatting, static analysis, tests in CI) so humans review design and logic.

Java example

Every before/after in this chapter is a review finding — e.g. `Before.book` in §2: flag argument (F3), error codes, output argument (F2), hidden side effect (`SESSION`), type-code switch (G23), magic numbers (G25).

Trade-offs

Reviews slow merging slightly but catch defects early, spread knowledge and keep style consistent. Too many reviewers or nit-heavy reviews slow teams without adding quality.

When not to use it

Emergency hotfixes (urgent fixes deployed straight to production) may be reviewed after merge, but should still be reviewed.

Common mistakes

- Reviewing only style while missing logic and security bugs.
- Huge pull requests (a pull request is a proposed change submitted for review and merging).
- Personal preferences presented as rules.

Real-world examples

- **GitHub pull requests / GitLab merge requests / Gerrit** (used by Android and Chromium).
- Google's published engineering practices guide reviewers and authors (design, functionality, complexity, tests, naming, comments, style).
- **CODEOWNERS** files route reviews to responsible teams.

In the interview

> **30-second answer:** "I review for correctness first — requirements, edge cases, errors, concurrency — then design, readability, tests, security and performance, and leave formatting to tools. I keep feedback specific, explain why, and mark nits as optional."

Self-check

1. Q: What should a reviewer check first? → A: Correctness (including edge cases and error handling).
2. Q: Why keep pull requests small? → A: Large diffs get shallow reviews.

TL;DR

Correctness → design → readability → tests → security → performance → operability; automate style; keep PRs small and feedback kind.

---

16. Error-Handling Strategy (best practices for real services)

What it is

§6 covered error-handling *rules* in code. An **error-handling strategy** is the *system-wide policy*: which exception types exist, where errors are translated, which are retried, how they're logged, and what callers (users/other services) see.

Analogy: hospital triage — every problem is classified, routed to the right place, and recorded once in the patient chart.

The problem it solves

Without a strategy everyone improvises: `null` returns, checked exceptions, log-and-rethrow at every layer (one error logged five times), stack traces shown to users, and retrying everything — including permanent failures, overloading struggling services.

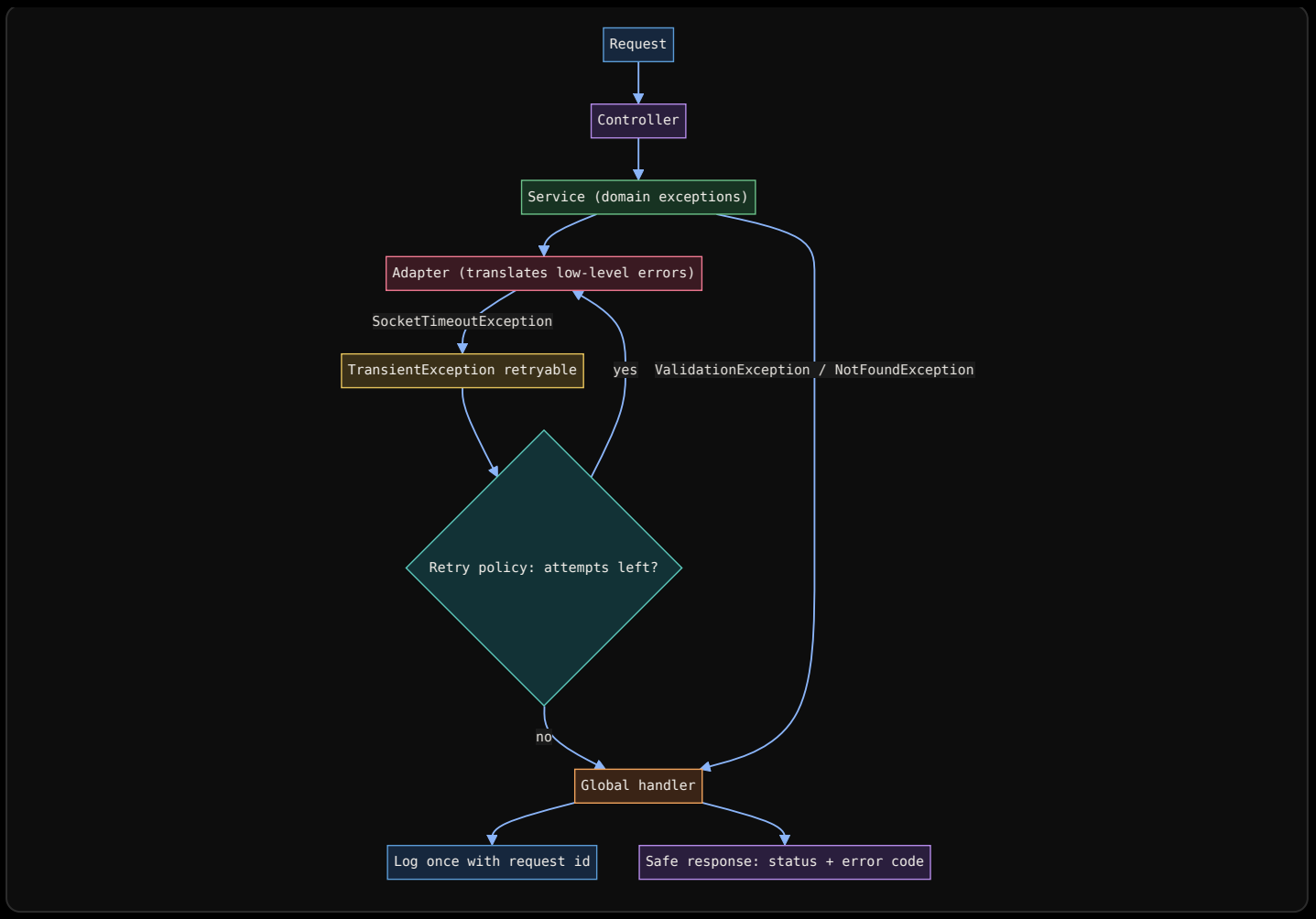
How it works

> **Key term — HTTP status codes:** every web-API reply carries a three-digit status. **2xx** = success (200 OK); **4xx** = the *client* sent something wrong (400 bad request, 404 not found, 409 conflict with current state); **5xx** = the *server* failed (500 internal error, 503 service unavailable — often temporary). Retrying usually makes sense only for some 5xx errors and timeouts. See System Design Fundamentals.

1. **Classify errors:**

Class	Examples	Handling
Validation / client errors	bad input, missing field	Reject immediately, 4xx, clear message, no retry
Business rule violations	insufficient funds, seat already booked	Domain exception or result value, 4xx (e.g. 409 conflict)
Not found	unknown id	404 / empty `Optional`
Transient infrastructure	timeouts, 503, connection reset, deadlock victim	Retry with backoff — waiting longer between attempts (only if the operation is **idempotent** — safe to repeat)
Permanent infrastructure	auth failure to DB, misconfiguration	Fail fast, alert
Programming bugs	NPE, index out of bounds	Don't catch locally; generic 500 at the edge, log with details, fix the bug

2. **One exception hierarchy** with an error code and a *retryable* flag, so handlers need no `instanceof` chains on vendor exceptions.
3. **Translate at boundaries**: adapters convert low-level exceptions (`SocketTimeoutException`, `SQLException`) into domain/infrastructure exceptions, keeping the cause.
4. **Retry only transient errors**, with a maximum attempt count, **exponential backoff** (each wait doubles: 10 ms, 20 ms, 40 ms…) and **jitter** (randomness so many clients don't retry in sync). Combine with **timeouts** and **circuit breakers** (a guard that stops calling a failing dependency for a while after repeated errors).
5. **Use result values for expected, per-item failures** (validating 10,000 imported rows — collect all errors instead of stopping at the first).
6. **Handle globally at the edge**: one handler maps exceptions to responses (status code, stable error code, safe message) and **logs once** with a correlation/request id (a unique ID stamped on all of one request's log lines). Never leak stack traces or internals to clients.
7. **Keep the system consistent**: transactions or compensating actions (follow-up steps that undo an earlier one, e.g. a refund) so partial failures leave no corrupt state; make handlers idempotent.



Java example

Full program: `code/lld/ch06/errorstrategy/Main.java`.

```
// 1. one exception hierarchy with error codes and a retryable flag
enum ErrorCode {
    VALIDATION_FAILED(400), NOT_FOUND(404), CONFLICT(409), DEPENDENCY_UNAVAILABLE(503);
    final int httpStatus;
}
static class AppException extends RuntimeException {
    final ErrorCode code;
    final boolean retryable;
    AppException(ErrorCode code, String message, boolean retryable, Throwable cause) { ... }
}
// ValidationException (VALIDATION_FAILED, not retryable), NotFoundException (NOT_FOUND, not retryable),
// TransientException (DEPENDENCY_UNAVAILABLE, retryable, keeps cause)

// 2. translate at the infrastructure boundary (InventoryClient)
int stockOf(String sku) {
    try { return rawHttpCall(sku); }
    catch (SocketTimeoutException e) { throw new TransientException("inventory service timed out", e); }
}

// 3. retry transient errors only
static <T> T withRetry(Supplier<T> action, int maxAttempts, long initialBackoffMs) {
    long backoff = initialBackoffMs;
    for (int attempt = 1; ; attempt++) {
        try {
            return action.get();
        } catch (AppException e) {
            if (!e.retryable || attempt == maxAttempts) throw e;
            System.out.println("    attempt " + attempt + " failed (" + e.getMessage() + "), retrying in " +
backoff + "ms");
            sleep(backoff);
            backoff *= 2;                                  // exponential backoff (add jitter in production)
        }
    }
}

// 4. Result type for expected, per-item failures
sealed interface Result<T> {
    record Ok<T>(T value) implements Result<T> {}
    record Err<T>(String error) implements Result<T> {}
}
// parseRow(line, csv) returns Err for "expected 2 fields", "qty must be positive", "qty '...' is not a number";
// the importer switches on Ok/Err (pattern matching) and collects ALL errors

// 5. global handler at the edge: logs once, hides internals
static <T> Response handle(String requestId, Supplier<T> endpoint, Function<T, String> render) {
    try {
        return new Response(200, render.apply(endpoint.get()));
    } catch (AppException e) {
        System.out.println("    [log once] " + requestId + " " + e.code + ": " + e.getMessage());
        return new Response(e.code.httpStatus, "{\"error\":\"" + e.code + "\",\"message\":\"" + e.getMessage() +
"\"}");
    } catch (RuntimeException e) {
        System.out.println("    [log once] " + requestId + " UNEXPECTED " + e);   // full details only in logs
        return new Response(500, "{\"error\":\"INTERNAL\",\"requestId\":\"" + requestId + "\"}");
    }
}
```

Output:

```
Retry a transient failure, then succeed:
    attempt 1 failed (inventory service timed out), retrying in 10ms
    attempt 2 failed (inventory service timed out), retrying in 20ms
  Response[status=200, body={"stock":7}]
Do NOT retry a permanent failure:
    [log once] req-2 NOT_FOUND: sku X-404 not found
  Response[status=404, body={"error":"NOT_FOUND","message":"sku X-404 not found"}]
Give up after max attempts:
    attempt 1 failed (inventory service timed out), retrying in 10ms
    attempt 2 failed (inventory service timed out), retrying in 20ms
    [log once] req-3 DEPENDENCY_UNAVAILABLE: inventory service timed out
  Response[status=503, body={"error":"DEPENDENCY_UNAVAILABLE","message":"inventory service timed out"}]
Unexpected bug -> generic 500, no internals leaked:
    [log once] req-4 UNEXPECTED java.lang.ArrayIndexOutOfBoundsException: Index 0 out of bounds for length 0
  Response[status=500, body={"error":"INTERNAL","requestId":"req-4"}]
Bulk import: collect ALL row errors instead of stopping at the first:
  imported [Row[line=1, sku=TEA, qty=2], Row[line=5, sku=MUG, qty=1]]
  rejected line 2: expected 2 fields
  rejected line 3: qty 'abc' is not a number
  rejected line 4: qty must be positive
    [log once] req-5 VALIDATION_FAILED: 3 invalid rows
  Response[status=400, body={"error":"VALIDATION_FAILED","message":"3 invalid rows"}]
```

req-2's `NotFoundException` is not retryable, so it fails at once; req-3 gives up after 3 attempts; req-4's bug becomes a generic 500 with only a request id.

Trade-offs

Choice	Benefit	Cost
Retries	Survive blips	Duplicate side effects unless idempotent; can amplify load
Global handler	Consistent responses, log once	Must avoid swallowing context
Result types	Explicit expected failures	Verbose in Java; mixing with exceptions needs a clear rule

When not to use it

Don't retry non-idempotent operations ("charge card") unless you use an **idempotency key** (a unique ID the client sends so the server can recognise and ignore repeats); never retry validation or authorisation failures.

Common mistakes

- Log-and-rethrow at every layer.
- Catching `Exception` in business code.
- Retrying immediately in a tight loop without backoff.
- Returning 200 with an error body.

Real-world examples

- **Spring** `@ControllerAdvice` **+** `@ExceptionHandler` — a global exception-to-response mapper; **RFC 9457 "Problem Details"** (successor of RFC 7807) standardises JSON error bodies, supported by Spring's `ProblemDetail`.
- **Resilience4j** (successor to Netflix Hystrix, now in maintenance mode): retry, circuit breaker, rate limiter (caps calls per second), bulkhead (isolates resources so one failing part can't exhaust them all).
- **AWS SDKs** (Amazon Web Services client libraries) retry throttling ("slow down" responses) and transient errors with exponential backoff and jitter by default.
- **Stripe's API** supports idempotency keys so clients can safely retry payment requests.

In the interview

> **30-second answer:** "I classify errors into validation, business, not-found, transient and bugs; use one exception hierarchy with codes and a retryable flag; translate low-level errors in adapters; retry only transient, idempotent operations with backoff and jitter; and map everything to safe responses in one global handler that logs once with a request id."

Follow-ups:

- *Q: Why jitter?* A: Without it, many clients retry at the same moments and overload the recovering service (thundering herd).
- *Q: How do you make payment retries safe?* A: Idempotency keys — the server stores the result per key and returns it for repeats.

Self-check

1. Q: Which errors should be retried? → A: Transient ones, and only for idempotent operations.
2. Q: Where should errors be logged? → A: Once, at the boundary/global handler (with a correlation id).

TL;DR

Classify, translate at boundaries, retry only transient + idempotent with backoff, result values for expected failures, one global handler that logs once and hides internals.

---

17. Testing Strategies (unit vs integration, test doubles, mocks)

What it is

A **testing strategy** decides *what kinds* of tests to write, *how many* of each, and *how to isolate* code under test. It builds on §8.

The problem it solves

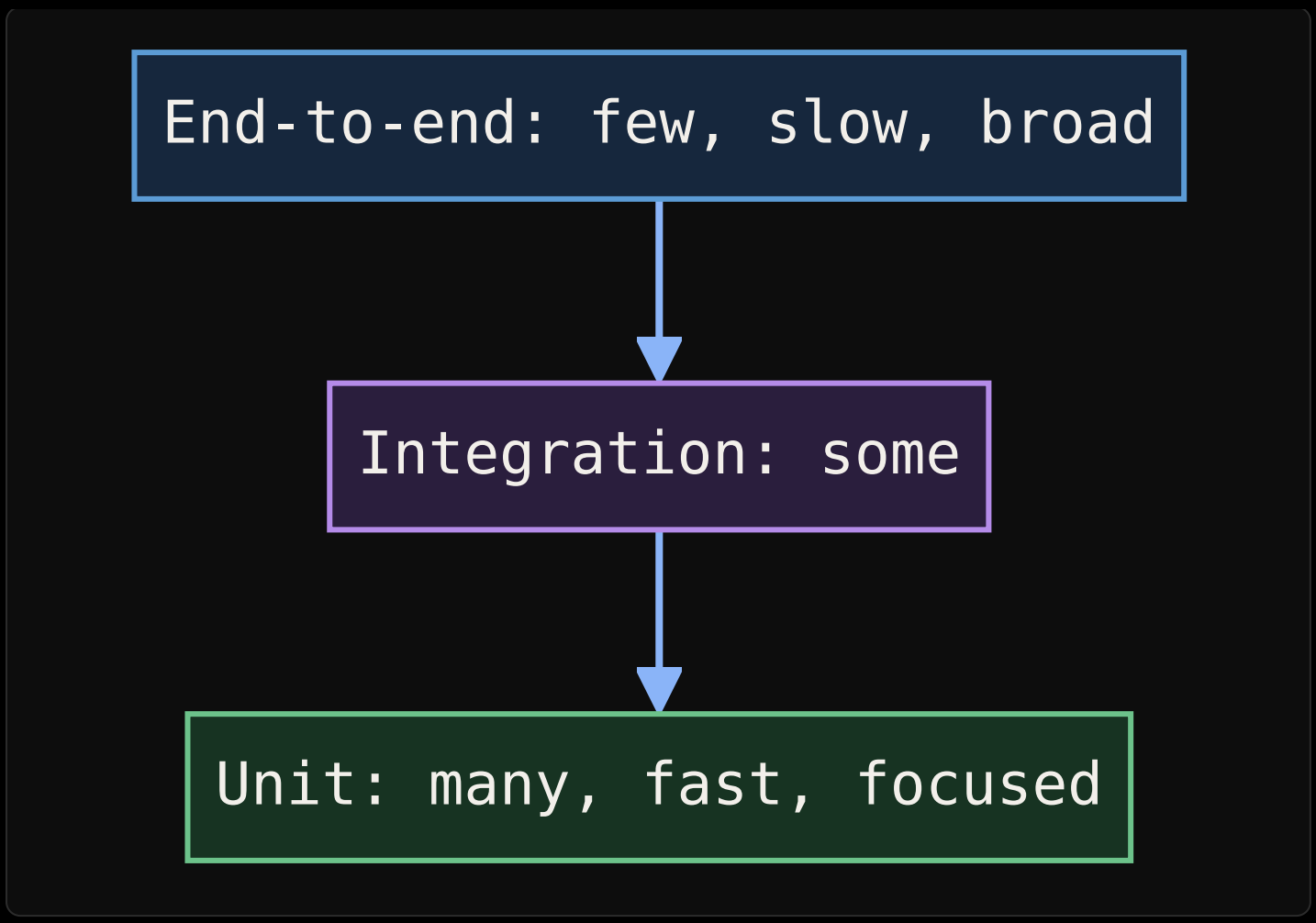
Too few tests → bugs reach production. Mostly slow end-to-end tests → slow, flaky (pass or fail with no code change) CI pipelines nobody trusts. Over-mocking → tests that pass while the real system is broken, and break on every refactor.

How it works

Kinds of tests:

Kind	Scope	Speed	Catches
Unit	One class/function, collaborators replaced by doubles	Milliseconds	Logic errors
Integration	Several real components together (DB, message broker — a server passing messages between services, your modules)	Seconds	Wiring, queries, serialisation (objects to bytes/text and back), config
Contract	Agreement between a consumer and a provider service	Fast	API mismatches between services
End-to-end (E2E)	The whole system through its UI/API	Slow	User-journey breakage
Others	Performance/load, security, exploratory, property-based, mutation	Varies	Non-functional issues, weak tests

The Test Pyramid (popularised by Mike Cohn): many fast unit tests at the base, fewer integration tests, very few E2E tests at the top. Avoid the "ice-cream cone" (mostly manual/E2E tests).



Test doubles (Gerard Meszaros' vocabulary, popularised by Martin Fowler) — all five appear in §8's `CheckoutTest`:

Double	What it does	In our code
Dummy	Passed to fill a parameter; never used	`DummyAuditLog` (throws if called)
Stub	Returns canned answers to calls	`StubCatalog`
Spy	A stub that records calls for later assertions	`SpySender.sent`
Mock	Pre-programmed with expectations; verifies the interaction itself	`MockAuditLog.expect(...).verify()`
Fake	A working, simplified implementation	`FakeCoupons`, in-memory repositories

State vs behaviour verification: stubs/fakes support *state verification* (check the result or resulting state); mocks support *behaviour verification* (check the right calls happened). Prefer state verification; use mocks for outgoing side effects you can't otherwise observe (emails sent, audit events).

What to test: public behaviour, business rules, edge cases (empty, null, boundaries, duplicates), error paths, and every bug fix (regression tests). Not private methods directly, trivial accessors, or the framework itself.

Mocking rules of thumb: don't mock types you don't own (wrap them and mock your wrapper — §7); don't mock value objects; one or two mocks per test at most — needing many means the class has too many dependencies.

Coverage: a coverage tool (e.g. JaCoCo) shows untested code; high coverage is *necessary but not sufficient* — mutation testing (e.g. PIT) checks whether tests actually detect changes.

Java example

See §8 (`lld.ch06.testing`): `CheckoutTest` uses all five doubles for fast unit tests, `CheckoutIntegrationTest` wires real in-memory implementations together, `BeforeStyleTest` shows what to avoid; all eight tests pass.

Trade-offs

More unit tests	More integration/E2E tests
Fast, precise failure location	Higher confidence the pieces fit
May miss wiring/config bugs	Slow, flakier, harder to debug
Over-mocking couples tests to implementation	Need environments (DB, containers — isolated runtime packages such as Docker)

When not to use it

Don't insist on unit tests for thin glue code that only wires frameworks — an integration test covers it better. Don't write E2E tests for every edge case — push those down to unit tests.

Common mistakes

- Mocking everything, including value objects and the class under test.
- Flaky tests (time, randomness, order dependence, shared DB state) left unfixed — they destroy trust.
- Testing implementation details (exact internal calls) instead of outcomes.
- No tests for error paths.

Real-world examples

- **JUnit 5**, **Mockito** (mocks/spies/stubs), **AssertJ**, **Testcontainers** (real PostgreSQL/Kafka in Docker for integration tests), **WireMock** (fake HTTP services), **Pact** (consumer-driven contract tests), **PIT** (mutation testing), **JaCoCo** (coverage).
- Spring Boot test slices (`@WebMvcTest`, `@DataJpaTest`) for focused integration tests.

In the interview

> **30-second answer:** "I follow the test pyramid: many fast unit tests with test doubles, fewer integration tests with real components like the database via Testcontainers, and a handful of end-to-end tests. I prefer state verification with stubs and fakes, use mocks only for outgoing side effects, and never mock types I don't own."

Follow-ups:

- *Q: Mock vs stub?* A: A stub supplies canned answers so the test can check state; a mock is set up with expectations and verifies the calls itself.
- *Q: Spy vs mock?* A: A spy records calls for the test to assert afterwards; a mock checks expectations itself.

Self-check

1. Q: Why a pyramid shape? → A: Lower-level tests are faster, cheaper and more precise; top-level tests are slow and brittle, so keep few.
2. Q: Is 100% coverage proof of good tests? → A: No — it shows code ran, not that assertions would catch bugs (mutation testing checks that).

TL;DR

Many unit tests, some integration, few E2E; pick the right double (dummy/stub/spy/mock/fake); verify state over interactions; don't mock what you don't own.

Chapter summary

Topic	Remember this
Names	Reveal intent, pronounceable, searchable, one word per concept
Functions	Small, one thing, one level, stepdown, ≤2–3 args, no flags, CQS, exceptions
Comments	Explain in code; comment only why/warnings/legal/public API
Formatting	Newspaper order, related things close, automated team style
Objects vs data	Objects hide data; data structures expose it; anti-symmetry decides
Error handling	Unchecked exceptions with context, wrap vendors, no null in or out
Boundaries	Wrap third-party code; learning tests; define interfaces you need
Unit tests	TDD laws, one concept per test, F.I.R.S.T.
Classes	Small by responsibility, cohesive, organised for change
Systems	Composition root + DI; cross-cutting via proxies/aspects
Emergence	Beck: tests pass → reveals intent → no duplication → fewest elements (*Clean Code* swaps the middle two)
Concurrency	Separate it, share little, use the library, test hard
Smells	C/E/F/G/J/N/T checklist
Code review	Correctness → design → readability → tests → security → performance
Error strategy	Classify, translate, retry transient, handle globally, log once
Testing strategy	Pyramid; right test double; state over behaviour verification

Chapter self-check

1. Q: A method `boolean saveAndValidate(Order o, boolean sendEmail)` breaks which rules? → A: Does two things, flag argument (F3), likely violates CQS (returns a boolean while changing state), name reveals multiple responsibilities.
2. Q: What should `findOrders(customerId)` return when there are none? → A: An empty list, never `null`.

TL;DR (chapter)

Write for the next reader: honest names, tiny single-purpose functions, few "why" comments, consistent layout, clear exceptions instead of codes or nulls, foreign code behind your own interfaces, clean fast tests, small cohesive classes wired from one place, simple design that emerges through refactoring, careful concurrency — and review against a shared checklist.

Chapter 7 — Refactoring

> **What you will learn.** What refactoring is (and is not), when to do it, how tests make it safe, the full catalogues of **code smells** (signals of *where* to refactor) and **refactorings** from Martin Fowler's *Refactoring* (2nd edition, 2018), a smell → refactoring lookup, a worked Java example from messy to clean in six compiling steps, and how refactoring leads toward design patterns (named, reusable solutions to recurring design problems; Chapters 8–11).
>
> **Why it matters for LLD interviews.** Machine-coding rounds (build a small working program in about 60–120 minutes; a core part of LLD — low-level design — interviews) often end with "now add feature X" or "what would you clean up?". Interviewers want you to name the smell ("this switch is repeated in three places — Repeated Switches") and the fix ("Replace Conditional with Polymorphism, giving a Strategy" — each interchangeable algorithm in its own class). This chapter gives you that vocabulary.

Chapter map

Part	Content
7.1	Refactoring fundamentals — definition, two hats, why, when, when not, tests, performance, rewrite vs refactor, IDE (Integrated Development Environment, e.g. IntelliJ IDEA) automation, legacy code
7.2	Code smells — the 24 smells of the 2nd edition, grouped the refactoring.guru way
7.3	The refactoring catalogue — all 61 named refactorings of the 2nd edition, by book chapter, plus the extra ones on refactoring.com
7.4	Smell → refactorings lookup table
7.5	Worked example — a messy invoice class refactored in six compiling steps
7.6	Refactoring toward patterns — Strategy, State, Special Case, Template Method, Factory
7.7	Trade-offs, mistakes, real-world examples, interview answers, self-check, TL;DR
7.8	Source layout, how to run, final program output

7.1 Refactoring fundamentals

What it is

Refactoring (noun) is a change made to the internal structure of software to make it easier to understand and cheaper to modify, *without changing its observable behaviour*.

Refactoring (verb) is restructuring software by applying a *series* of such small, behaviour-preserving changes.

Two words in that definition do all the work:

- **Behaviour-preserving.** Same outputs for the same inputs, same side effects (changes besides returning a value, such as writing a file), same errors. Only *internal* things change — names, where code lives, how it is split. (Performance may change a little; see 7.1.9.)
- **Small steps.** Each step is so small it is almost impossible to get wrong; the code compiles and the tests pass after *every* step, and many tiny steps chain into a big change. If you sit with broken code for an hour, you were not refactoring — you were rewriting.

Everyday analogy. Reorganising a restaurant kitchen while it stays open: one shelf at a time, and after each move the cooks can still make every dish.

> **Term — observable behaviour.** What a user (or calling code) can see: return values, printed output, data written, exceptions thrown. The *call stack* (the chain of method calls in progress), the *class structure* and the *names of private methods* are not observable behaviour.

What refactoring is NOT.

Activity	Is it refactoring?	Why
Renaming a method with the IDE	Yes	Behaviour unchanged, tiny step
Fixing a bug	No	Behaviour changes (on purpose)
Adding a feature	No	Behaviour changes
Performance tuning	No	Goal is speed, not clarity (though behaviour is preserved)
"I spent two days restructuring and nothing worked until the end"	No	That is restructuring/rewriting, not refactoring
Reformatting code	Borderline	Behaviour-preserving, but it does not improve design

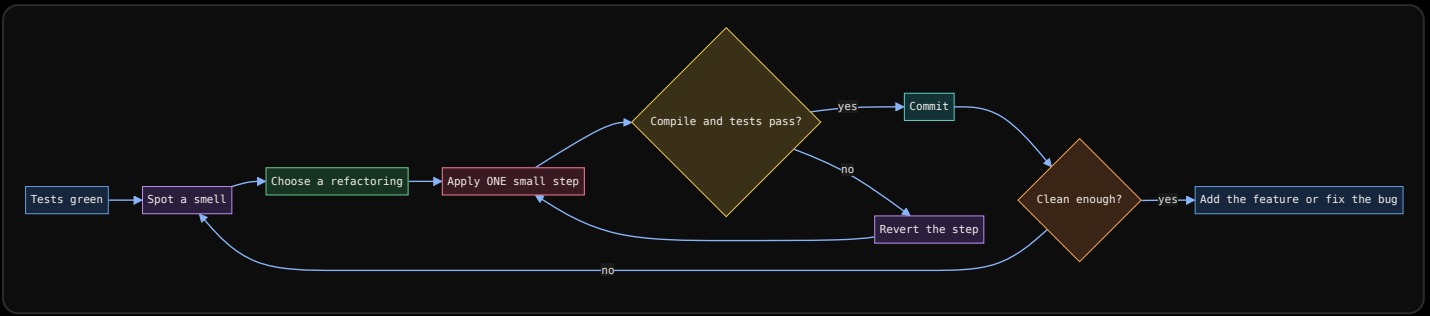
> **Term — restructuring.** The general word for reorganising code. Refactoring is a *specific kind* of restructuring: small, behaviour-preserving steps.

The problem it solves

Code decays: every quick fix, extra `if` and copy-paste makes the next change harder. This is **design decay** or **technical debt** — the extra cost paid on every future change because the code is harder to work with than it needs to be. Refactoring pays the debt down *continuously*, instead of hoping to get the design right up front (which never fully works, because requirements change).

How it works

1. **Make sure you have tests** that pin down current behaviour (7.1.7); they must be green (all passing).
2. **Spot a smell** — a surface sign the design could be better (7.2).
3. **Pick a named refactoring** that cures it (7.3, 7.4).
4. **Apply one small step** (ideally with the IDE's automated refactoring).
5. **Compile and run the tests.** Green? Commit locally (save a snapshot in version control such as Git). Red? Undo the step (don't debug) and take a smaller one.
6. **Repeat** until the code is clean enough for the job at hand.



7.1.1 The two hats

Kent Beck's metaphor: at any moment you wear **one of two hats**.

Hat	You are allowed to	You are NOT allowed to
Adding function	Add new behaviour, add new tests	Restructure existing code
Refactoring	Restructure code	Add behaviour or tests for new behaviour (you may add tests for *existing* behaviour you forgot to cover)

You swap hats often — maybe every few minutes — but always know which you wear. Mixing them ("I'll fix this bug while I extract this method") means a failing test can't tell you which change caused it.

7.1.2 Why refactor

- **It improves the design**, which otherwise decays under short-term changes — largely by **removing duplication** (each fact in one place).
- **It makes software easier to understand**: it moves your understanding *out of your head and into the code* for the next reader (including you in six months).
- **It helps you find bugs**: clearer structure exposes assumptions.
- **It helps you program faster.** Fowler's **Design Stamina Hypothesis**: a team that keeps internal quality high soon adds features *faster* than one that doesn't, because each feature builds on clean code instead of fighting a mess. Speed — not "clean code for its own sake" — is the economic argument.

7.1.3 When to refactor

Trigger	What it means	Example
Rule of Three (Don Roberts)	First time, just do it. Second time you do something similar, wince but duplicate. Third time, refactor.	Third copy of the same validation code → extract a validator
Preparatory refactoring	Refactor *just before* adding a feature, to make it easy to add. "Make the change easy, then make the easy change."	Before adding HTML output, split calculation from text formatting (7.5, step 3)
Comprehension refactoring	Rename or extract as you come to understand code you are reading, so the understanding moves into the code.	Rename `d` to `daysOverdue` after working out what it holds
Litter-pickup refactoring	Mess noticed in passing: fix it if quick, else note it. The **Boy Scout Rule** — leave the campground cleaner than you found it.	Remove an unused parameter while passing by
Planned vs opportunistic	Mostly *opportunistic* — woven into features and fixes; planned episodes (after neglect) should be rare.	Good: every feature PR (pull request — a change submitted for review) has a few small refactoring commits. Warning sign: a "refactoring sprint" (a fixed 1–4-week work period) every quarter
Long-term refactoring	Big changes (replace a library, split a module) done gradually over weeks with the system working — e.g. **Branch by Abstraction** (add an abstraction layer, switch callers to it, swap the implementation behind it, delete the old).	Moving from one ORM (library mapping objects to database tables) to another behind a repository interface
During code review	Suggest or pair on refactorings; Fowler prefers trying them *with the author* over only commenting.	"This method has four flag parameters — can we Remove Flag Argument?"

> **Term — Rule of Three.** Don't abstract on the first or second occurrence (you don't yet know what varies); abstract on the third, when the shape is clear.

Should you tell your manager? Fowler: don't ask permission to refactor any more than to write tests — it is how professionals deliver features quickly (large planned efforts should be discussed).

7.1.4 When NOT to refactor

- **Code you never need to touch or understand.** Messy code behind a clean API (the methods other code calls) that works and never changes can stay messy.
- **When rewriting is cheaper** (7.1.10) — a judgement call that usually needs experience.
- **Without a safety net** — if you can neither test it nor use safe automated refactorings, add tests first (7.1.11).
- **On a deadline that truly cannot move**, a big refactoring may wait — though the Design Stamina Hypothesis says skipping it usually makes you *slower* even short term; deadlines are rarely an excuse for small refactorings.
- **Published interfaces** whose callers you don't control (a public library API, a REST endpoint — a URL of a web API, see System Design Fundamentals — used by other teams). You can still refactor them, but must keep the old version working for a while and deprecate it (mark it outdated and due for removal) — see *Change Function Declaration*, migration mechanics.

7.1.5 Problems and costs of refactoring

Problem	What goes wrong	Mitigation
Slowing down new features	"Takes time away" from features	Refactor *because* it speeds features up; focus on code you're about to change
Code ownership	You cannot rename a method whose callers are in another team's repo (repository — a project's version-controlled code store)	Keep the old name as a delegating, `@Deprecated` method; migrate callers later; prefer weak (team-wide) code ownership
Long-lived feature branches (lines of work kept apart from the main code for a long time)	Refactoring renames and moves code, which makes merges painful	**Continuous Integration** (merge into the shared main line often, with automatic builds and tests) / trunk-based development (everyone works on one main branch): integrate at least daily, so refactorings conflict less
Missing tests	Refactoring without tests is risky	Add tests first; use only IDE-automated, provably-safe refactorings until tests exist
Legacy code	Big, untested, tangled	Feathers' techniques (7.1.11)
Databases	Schema (table structure) changes are hard to reverse	**Evolutionary database design**: small migration scripts (scripts that change the database structure), versioned with the code; use **parallel change** (add new column, copy, switch readers, remove old)

> **Term — parallel change (expand / contract).** A breaking change in three phases: *expand* (add the new thing alongside the old), *migrate* (move all users to it), *contract* (remove the old). Nothing is ever broken.

7.1.6 Refactoring, architecture and YAGNI

Because refactoring lets you reshape code safely later, you needn't get the architecture "right" up front: build **only what today's requirements need** and evolve the design — **YAGNI** ("You Aren't Gonna Need It"; no flexibility for imagined requirements). It doesn't mean ignoring architecture: trust refactoring to add flexibility when the need is real, and think hard up front only about things *truly* hard to change later, like a public API or a database schema.

7.1.7 Tests as the safety net

Refactoring is safe only if you detect a behaviour change immediately — that is what **self-testing code** gives you: an automated suite that checks itself (pass/fail, no human reading output) and runs in seconds.

- Run tests after **every** small step, so a failure points at the few lines just changed; if one fails, **revert** (don't debug a refactoring) and redo in smaller steps.
- Test **behaviour** (public API, outputs), not private structure; tests of private methods break on every refactoring and become a cost.
- **Characterisation tests** (Michael Feathers): for code without tests, record what it *currently does* — even if that looks wrong — so you notice if a refactoring changes it. 7.5 uses exactly this: a checker comparing every step's output with the original, byte for byte.
- **Mutation testing** (e.g. PIT for Java) checks that your tests catch changes: it introduces small bugs deliberately and reports any your tests miss.

> **Term — safety net.** Anything that tells you quickly and reliably that you broke something: unit tests (one class in isolation), integration tests (several real parts together), the compiler's type checks, and the IDE's automated refactorings (proven safe by the tool).

7.1.8 IDE automation

Modern IDEs perform many refactorings *automatically and safely* using the compiler's knowledge of the code — the **syntax tree** and **symbol resolution** (the parsed structure, and exactly which declaration each name refers to) — not text search. A text replace of `name` would also hit comments, strings and unrelated fields; the IDE's Rename changes only that symbol and its real references.

History in one line: Bill Opdyke's 1992 PhD thesis formalised refactorings; John Brant and Don Roberts built the Smalltalk (an early object-oriented language) *Refactoring Browser*, the first automated tool; Java IDEs (IntelliJ IDEA, Eclipse) popularised it.

Refactoring	IntelliJ IDEA (Windows/Linux · macOS)	Eclipse (Windows/Linux)
Rename	Shift+F6 · Shift+F6	Alt+Shift+R
Extract Method (Extract Function)	Ctrl+Alt+M · Cmd+Opt+M	Alt+Shift+M
Extract Variable	Ctrl+Alt+V · Cmd+Opt+V	Alt+Shift+L
Extract Field / Constant / Parameter	Ctrl+Alt+F / C / P · Cmd+Opt+F / C / P	(Refactor menu)
Inline	Ctrl+Alt+N · Cmd+Opt+N	Alt+Shift+I
Change Signature	Ctrl+F6 · Cmd+F6	Alt+Shift+C
Move	F6 · F6	Alt+Shift+V
"Refactor This" menu	Ctrl+Alt+Shift+T · Ctrl+T	Alt+Shift+T

(Default keymaps; check your IDE if you use a custom one.)

Large-scale automated refactoring tools such as OpenRewrite (recipes that, e.g., migrate a whole codebase between framework versions) and Google's Refaster templates in Error Prone (Google's bug-finding plugin for the Java compiler) apply the same refactoring across thousands of files.

> **Caution.** Automated refactorings are safe for statically known calls. They can miss **reflection** (`Class.forName("…")` — looking up classes or methods by name at runtime), names in **configuration files**, **serialised field names** (serialising = turning objects into text or bytes, e.g. JSON, a plain-text data format; JSON/Jackson, JPA — Java's standard ORM API — column names inferred from field names), and code in other repositories. After an IDE rename of a serialised field, the JSON your API returns changes — that *is* an observable behaviour change.

7.1.9 Performance and refactoring

"Refactoring adds method calls and small objects, so the program gets slower." Fowler's three-part answer:

1. **Most refactorings have negligible performance effect.** The JIT (Just-In-Time) compiler — the part of the JVM, Java's runtime, that compiles frequently run code to machine code while the program runs — inlines small methods (copies their body into the caller), so extra calls are nearly free.
2. **Programs spend most of their time in a small fraction of the code**, so optimising everywhere wastes effort. (The book retells Ron Jeffries' story of Chrysler's slow payroll system: nobody's guesses about the cause were right; profiling — measuring where a program actually spends its time — found one unexpected spot, and a small fix there made it fast.)
3. **Well-factored code is easier to tune**: profiling finds the hot spot (where most time goes), and in well-structured code it is small, isolated and easy to optimise.

Order: **make it work → make it clean (refactor) → make it fast (profile, then optimise only hot spots)**; a slightly slower refactoring is fine if it eases that later optimisation.

Example from this chapter. In 7.5 step 2 a summing loop becomes a stream (Java's pipeline for processing a sequence of elements). `DoubleStream.sum()` may use *compensated summation* (tracking rounding errors to make floating-point sums more accurate; OpenJDK, the reference Java implementation, does), so its result can differ from a plain loop in the last bit — a tiny *behaviour change*. To stay a true refactoring we used `reduce(0.0, Double::sum)`, which adds left to right exactly like the loop.

7.1.10 Refactoring vs rewriting

	Refactoring	Rewriting
Unit of change	Many tiny, safe steps	One big replacement
System works during the change?	Always	Usually not until the end
Risk	Low	High — known bugs fixed years ago get reintroduced; the old system keeps evolving while you rewrite
Delivers value	Continuously	Only at the end
Good when	Code mostly works, needs reshaping	Code is so broken it does not work at all, or the technology must change completely

Even a replacement is rarely done in one go. The **Strangler Fig** approach (Fowler, after a vine that slowly grows around a tree) builds the new system *around* the old one, routing one feature at a time to the new code until the old can be switched off — the rewrite equivalent of "small steps".

7.1.11 Legacy code

Michael Feathers (*Working Effectively with Legacy Code*, 2004) defines **legacy code as code without tests** — hard to refactor (no safety net) and hard to test (not designed for it). The way out is a loop:

1. **Identify change points** — where you need to change code.
2. **Find test points** — where you can observe behaviour.
3. **Break dependencies** using a **seam**: a place where behaviour can change without editing the code there, e.g. passing in an interface instead of calling `new Database()` inside a method (constructor injection), or overriding a method in a test subclass.
4. **Write characterisation tests** that pin down current behaviour.
5. **Make the change and refactor** under that safety net.

Other techniques from the book:

- **Sprout Method / Sprout Class** — the new feature as a *new*, fully tested method or class, called from the old code by a one-line change.
- **Wrap Method / Wrap Class** — rename the old method; a new one with the old name calls it plus the new behaviour (Decorator-like — a Decorator wraps an object to add behaviour).
- **Safe automated refactorings only** (IDE Rename, Extract Method) until tests exist.
- Refactor only what you touch; over time the most-changed code becomes the cleanest.

7.2 Code smells

What it is

A **code smell** (Kent Beck's term) is a *surface sign* that usually points to a deeper design problem. It is not a bug — the code works — but the code will be hard to understand or change. Like a smell in a fridge, it tells you *where to look*, not exactly what is wrong. Smells are **heuristics, not rules**: a long method or a switch is not always bad; look closer and use judgement.

The problem it solves

You must know *when* and *where* to refactor, not just how. Fowler and Beck deliberately give no precise metrics ("no method longer than 10 lines") but a catalogue of named smells with their usual cures. Names matter: in review, "Feature Envy" says in two words what would otherwise take a paragraph.

How it works

Notice discomfort ("I had to scroll", "I changed five files for one feature", "what is `x2`?") → match it to a named smell → look up its cures (each smell below, and 7.4) → apply them in small steps under tests.

Smell groups (refactoring.guru classification)

refactoring.guru (a popular website on refactoring and design patterns) groups smells into five families. It uses mostly the 1st-edition names, so the table maps them to the 2nd-edition names used in this chapter.

Group	Idea	Smells (2nd-ed name; refactoring.guru name in brackets if different)
Bloaters	Code, methods and classes that have grown so big they are hard to work with. They usually grow slowly.	Long Function *(Long Method)*, Large Class, Primitive Obsession, Long Parameter List, Data Clumps
Object-Orientation Abusers	Incomplete or wrong use of OO (object-oriented) principles.	Repeated Switches *(Switch Statements)*, Temporary Field, Refused Bequest, Alternative Classes with Different Interfaces
Change Preventers	One change forces many other changes.	Divergent Change, Shotgun Surgery *(also: Parallel Inheritance Hierarchies — a 1st-ed smell)*
Dispensables	Something pointless whose removal makes the code cleaner.	Comments, Duplicated Code *(Duplicate Code)*, Lazy Element *(Lazy Class)*, Data Class, Speculative Generality *(also: Dead Code — in the 2nd ed. this is the refactoring Remove Dead Code)*
Couplers	Too much coupling between classes, or too much delegation (handing work off to another object).	Feature Envy, Insider Trading *(Inappropriate Intimacy)*, Message Chains, Middle Man *(also: Incomplete Library Class — a 1st-ed smell)*
New in the 2nd edition (not in refactoring.guru's five groups)		Mysterious Name (a comprehension smell), Global Data and Mutable Data (both about uncontrolled shared state — closest to Couplers/Change Preventers), Loops (closest to Bloaters)

> **Term — coupling.** How much one piece of code depends on another. High coupling means a change in one place forces changes in others. **Cohesion** is the opposite idea inside one unit: how strongly the things inside a class or module belong together. Good design: low coupling, high cohesion.

Below are all 24 smells from the 2nd edition, in the book's order. Each has: **what it is**, **why it hurts**, a small **Java example** (smelly → cured), and **refactorings that fix it** (the ones Fowler names for that

smell; anything extra is marked). The names match the catalogue in 7.3.

> **Note on the snippets.** Several short snippets use `double` for prices or balances to stay tiny. Real money should use `BigDecimal` (Java's exact decimal type) or a `Money` value object (a small immutable object compared by its value) — see 7.2.11 and `code/smells/Money.java`.

7.2.1 Mysterious Name

What it is. A function, variable, field, class or module whose name does not tell you what it does or holds: `data`, `tmp`, `proc()`, `x2`, `Manager`, `handle()`.

Why it hurts. A bad name forces every reader to decode the implementation. If you *cannot* find a good name, the thing probably does too much or has no clear purpose.

```
// Smelly
double calc(List<Double> l, int d) { ... }

// Cured
double averageDailyRevenue(List<Double> dailyRevenues, int daysInPeriod) { ... }
```

Refactorings. Change Function Declaration (rename), Rename Variable, Rename Field.

7.2.2 Duplicated Code

What it is. The same (or nearly the same) code structure in more than one place.

Why it hurts. Every change must be made — and every copy found and its small differences checked — in every copy; sooner or later one is missed.

```
// Smelly: two methods repeat the same tax rule
double invoiceTotal(double net)  { return net + net * 0.18; }
double quoteTotal(double net)    { return net + net * 0.18; }

// Cured: one place owns the rule
private static final double GST_RATE = 0.18;
double withTax(double net)       { return net + net * GST_RATE; }
double invoiceTotal(double net)  { return withTax(net); }
double quoteTotal(double net)    { return withTax(net); }
```

Refactorings. Fowler lists: Extract Function; Slide Statements (to bring similar code together so it can be extracted); Pull Up Method (if the copies are in sibling subclasses). Also useful (not in Fowler's list for this smell): Replace Inline Code with Function Call (if a library already does it); Substitute Algorithm (if two different algorithms do the same thing).

7.2.3 Long Function

What it is. A function too long to understand at a glance. Fowler's heuristic: whenever you feel a block needs a **comment**, extract it into a function named after its *intent*.

Why it hurts. Hard to understand, reuse partially or test. Short, well-named functions are self-documenting: read the name, skip the body.

```java
// Smelly
void printOwing(Invoice inv) {
    System.out.println("*** Customer Owes ***");
    double outstanding = 0;
    for (Order o : inv.orders()) outstanding += o.amount();      // calculate outstanding
    inv.setDueDate(LocalDate.now().plusDays(30));               // record due date
    System.out.println("name: " + inv.customer());               // print details
    System.out.println("amount: " + outstanding);
}

// Cured: each block became a named function
void printOwing(Invoice inv) {
    printBanner();
    double outstanding = outstandingOf(inv);
    recordDueDate(inv);
    printDetails(inv, outstanding);
}
```

Refactorings. Extract Function (the main one, ~99% of the time), Replace Temp with Query, Introduce Parameter Object, Preserve Whole Object, Replace Function with Command, Decompose Conditional, Replace Conditional with Polymorphism, Split Loop.

7.2.4 Long Parameter List

What it is. A function that takes many parameters (often more than three or four), especially several of the same type, or boolean flags.

Why it hurts. Callers mix up the order (`move(10, 20, 5, 3)` — which is which?), every new parameter changes every caller, and long lists often hide a missing concept.

```java
// Smelly
Booking book(String hotelId, LocalDate from, LocalDate to, int adults, int children, boolean breakfast) { ... }

// Cured: parameter objects for the clumps, and an explicit enum instead of an unreadable boolean flag
record Stay(LocalDate from, LocalDate to) { }
record Guests(int adults, int children) { }
Booking book(Hotel hotel, Stay stay, Guests guests, MealPlan mealPlan) { ... }
```

Refactorings. Replace Parameter with Query, Preserve Whole Object, Introduce Parameter Object, Remove Flag Argument, Combine Functions into Class (several functions share the same parameters → make them fields of a class).

7.2.5 Global Data

What it is. Data readable and modifiable from anywhere: global variables, public static mutable fields (`static` fields belong to the class, shared program-wide), mutable singletons (classes with one shared instance).

Why it hurts. Any code anywhere can change it and you can't find *who* did; bugs appear far from their cause. Fowler calls it one of the most pungent smells, worsening with program size.

```java
// Smelly
public class Config { public static double discountRate = 0.1; }      // anyone can write it

// Cured: Encapsulate Variable - one controlled access point, easy to find every writer.
// (The range check is a NEW rule added afterwards, wearing the "adding function" hat - it is a behaviour change,
not part of the refactoring.)
public final class Config {
    private static volatile double discountRate = 0.1;
    public static double discountRate() { return discountRate; }
    public static void setDiscountRate(double rate) {
        if (rate < 0 || rate > 0.5) throw new IllegalArgumentException("rate out of range");
        discountRate = rate;
    }
}
// Even better: pass the value in (dependency injection) instead of reading a static at all.
```

Refactorings. Encapsulate Variable (first step, always), then narrow its scope — ideally move it into a class or module so only a few functions can see it.

7.2.6 Mutable Data

What it is. Data that is changed after creation, especially when it is shared, or when the same variable is updated for different purposes.

Why it hurts. A change in one place breaks code elsewhere that assumed the value stayed the same — hard to find, especially with threads (several paths of execution running at once). Functional programming forbids changing data; you needn't go that far, but limit where mutation can happen.

```java
// Smelly: callers can change the list inside the object, and total is a stored copy that can go stale
class Cart {
    public List<Item> items = new ArrayList<>();
    public double total;                          // must be updated by hand after every change
}

// Cured
final class Cart {
    private final List<Item> items = new ArrayList<>();
    void add(Item item) { items.add(item); }
    List<Item> items() { return List.copyOf(items); }                    // Encapsulate Collection
    double total() { return items.stream().mapToDouble(Item::price).sum(); } // Replace Derived Variable with
Query
}
```

Refactorings. Encapsulate Variable, Split Variable, Slide Statements, Extract Function, Separate Query from Modifier, Remove Setting Method, Replace Derived Variable with Query, Combine Functions into Class, Combine Functions into Transform, Change Reference to Value (use immutable value objects such as Java records — compact, immutable data classes).

7.2.7 Divergent Change

What it is. **One class is changed for many different reasons.** "I have to edit this class when we add a database, *and* when we add a financial instrument, *and* when we change the report format."

Why it hurts. It mixes responsibilities (breaking the **Single Responsibility Principle**, Chapter 4): each kind of change risks the others, and people edit the same file for unrelated reasons.

```
// Smelly: storage, pricing and formatting in one class
class ProductService {
    void saveToMySql(Product p) { ... }         // changes when the database changes
    double priceWithTax(Product p) { ... }      // changes when tax rules change
    String toCsv(Product p) { ... }             // changes when the export format changes
}

// Cured: one reason to change per class
class ProductRepository { void save(Product p) { ... } }
class PricingService    { double priceWithTax(Product p) { ... } }
class ProductCsvWriter  { String toCsv(Product p) { ... } }
```

Refactorings. Split Phase (when the changes fall into sequential phases, e.g. parse then compute), Move Function, Extract Function, Extract Class.

7.2.8 Shotgun Surgery

What it is. The opposite of Divergent Change: **one change forces many small edits in many classes.**

Why it hurts. You have to find all the places, and it is easy to miss one. The logic for one concept is scattered.

```
// Smelly: the "free-shipping threshold" rule appears in 4 classes
class CartView      { boolean showBanner(double t)  { return t >= 500; } }
class CheckoutCtrl  { double shipping(double t)     { return t >= 500 ? 0 : 40; } }
class EmailBuilder  { String line(double t)         { return t >= 500 ? "Free shipping!" : ""; } }
class ReportJob     { long freeOrders(List<Double> ts) { return ts.stream().filter(t -> t >= 500).count(); } }

// Cured: the concept lives in one class, and the four callers use it
final class ShippingPolicy {
    static final double FREE_SHIPPING_THRESHOLD = 500;
    static boolean isFree(double total)   { return total >= FREE_SHIPPING_THRESHOLD; }
    static double charge(double total)    { return isFree(total) ? 0 : 40; }
}
```

Refactorings. Move Function and Move Field (gather everything into one module), Combine Functions into Class, Combine Functions into Transform, Split Phase; sometimes Inline Function / Inline Class first to pull the scattered bits together, then extract a better-shaped unit.

	Divergent Change	Shotgun Surgery
Shape	One class, many reasons to change	One reason to change, many classes
Fix	Split the class	Gather the scattered code

7.2.9 Feature Envy

What it is. A method that is more interested in *another* class's data than in its own — it calls many getters (methods that just return a field) on another object to compute something.

Why it hurts. Behaviour away from its data means a data change needs changes elsewhere. The basic OO rule: **put behaviour next to the data it uses**. (Strategy and Visitor — an object carrying one operation across a group of classes — separate them deliberately, to allow swapping behaviour.)

```java
// Smelly: Order computes a line total using only Line's data
class Order {
    double lineTotal(Line line) {
        return line.getUnitPrice() * line.getQuantity() * (1 - line.getDiscount());
    }
}

// Cured: Move Function - Line computes its own total
record Line(double unitPrice, int quantity, double discount) {
    double total() { return unitPrice * quantity * (1 - discount); }
}
```

Refactorings. Move Function; Extract Function first if only *part* of the method is envious, then move that part.

7.2.10 Data Clumps

What it is. The same group of data items always appear together — as fields in several classes, or as parameters in many signatures: `(street, city, zip)`, `(startDate, endDate)`, `(x, y)`, `(amount, currency)`.

Why it hurts. The group is an unnamed concept; without a class, behaviour about it ("does this range contain a date?") is duplicated everywhere. **Test:** delete one item — if the others no longer make sense alone, they belong in an object.

```java
// Smelly
boolean isAvailable(LocalDate start, LocalDate end) { ... }
double price(LocalDate start, LocalDate end, double nightly) { ... }

// Cured (see code/smells/DateRange.java)
record DateRange(LocalDate start, LocalDate end) {
    boolean contains(LocalDate day) { return !day.isBefore(start) && !day.isAfter(end); }
}
boolean isAvailable(DateRange stay) { ... }
```

Refactorings. Extract Class (for clumps of fields), Introduce Parameter Object and Preserve Whole Object (for clumps of parameters). Then look for Feature Envy — behaviour will want to move onto the new class.

7.2.11 Primitive Obsession

What it is. Using primitive types (`int`, `double`, `String`, `boolean`) for domain concepts that deserve their own type: money as `double`, phone numbers as `String`, a status as `"ACTIVE"`, a temperature as a bare `int` without a unit.

Why it hurts. No validation (`"banana"` is a valid `String` phone number), no behaviour (formatting and comparing logic is duplicated), easy mix-ups (`transfer(double amount, double fee)` — swap them and nothing complains). Money as `double` also suffers from floating-point error (binary `double` numbers cannot store most decimal fractions exactly): `0.1 + 0.2` prints `0.30000000000000004` (see the smells output in 7.6).

```java
// Smelly
void charge(double amount, String currency) { ... }

// Cured (see code/smells/Money.java): validation and arithmetic live in one type
record Money(BigDecimal amount, Currency currency) {
    Money plus(Money other) { /* checks same currency, then adds */ }
}
void charge(Money amount) { ... }
```

Refactorings. Replace Primitive with Object, Replace Type Code with Subclasses, Replace Conditional with Polymorphism (for type codes that drive behaviour), Extract Class, Introduce Parameter Object.

7.2.12 Repeated Switches

What it is. The *same* `switch` (or `if/else if` chain) on the same type code (a field, e.g. a string or int, saying which kind of thing an object is) in several places. (1st edition: "Switch Statements" — *any* switch; the 2nd edition narrowed it — one switch is fine, *repetition* is the problem.)

Why it hurts. Adding a new case means finding and updating every switch — Shotgun Surgery. Miss one and you have a bug.

```
// Smelly: three switches on the same type
double speed(Bird b)    { switch (b.type) { case "EUROPEAN": ...; case "AFRICAN": ...; case "PARROT": ...; } }
String plumage(Bird b)  { switch (b.type) { case "EUROPEAN": ...; case "AFRICAN": ...; case "PARROT": ...; } }
int    lifespan(Bird b) { switch (b.type) { ... } }

// Cured: Replace Conditional with Polymorphism
sealed interface Bird permits EuropeanSwallow, AfricanSwallow, NorwegianBlueParrot {
    double speed();
    String plumage();
}
```

Refactorings. Replace Conditional with Polymorphism. (Java note: with a `sealed` type (one that lists all its permitted subtypes) or an `enum` (a fixed set of named constants), a *switch expression* is checked for exhaustiveness — covering every case — by the compiler, which removes much of the risk. A few such switches may be acceptable; polymorphism is still better when the behaviour per case is large.)

7.2.13 Loops

What it is. A `for` / `while` loop that filters, maps or aggregates where a pipeline would state the intent more clearly. (New in the 2nd edition, now that languages have first-class functions — storable in variables and passable; in Java, the Stream API since Java 8.)

Why it hurts. A loop makes the reader simulate the steps to see what it does. A pipeline names each step: *filter*, *map*, *collect*.

```
// Smelly
List<String> names = new ArrayList<>();
for (Employee e : employees) {
    if (e.department().equals("IT")) {
        names.add(e.name());
    }
}

// Cured: Replace Loop with Pipeline
List<String> names = employees.stream()
        .filter(e -> e.department().equals("IT"))
        .map(Employee::name)
        .toList();                    // Java 16+: returns an unmodifiable list
```

Refactorings. Replace Loop with Pipeline. (Note: `Stream.toList()` returns an *unmodifiable* list, while the loop built a mutable `ArrayList`. If a caller later adds to the list, that is a behaviour change — use `collect(Collectors.toCollection(ArrayList::new))` in that case.)

7.2.14 Lazy Element

What it is. An element that doesn't pull its weight — a class with one trivial method, a function whose body is as clear as its name, a subclass adding nothing. (1st edition: "Lazy Class".)

Why it hurts. Every element costs a name to learn, a file to open, an indirection to follow; adding nothing, it is pure cost. Often left behind by other refactorings, or created for growth that never came.

```
// Smelly: a wrapper that adds nothing
class NameFormatter { String format(String s) { return s.trim(); } }

// Cured: Inline Class / Inline Function
String cleanName = rawName.trim();
```

Refactorings. Inline Function, Inline Class, Collapse Hierarchy.

7.2.15 Speculative Generality

What it is. Hooks, abstract classes, parameters and special cases added "because we might need them someday" and never used: an abstract class with one subclass, unused parameters, methods called only by tests, a plug-in system with one plug-in.

Why it hurts. Harder to understand and maintain for no benefit — it violates **YAGNI**. If the need comes, refactoring adds the flexibility then, usually better shaped because the real requirement is known.

```
// Smelly
interface PaymentProcessorFactoryProvider { PaymentProcessorFactory get(); }   // one implementation, ever
void pay(Money m, Map<String, Object> futureOptions) { ... }                   // futureOptions never used

// Cured
void pay(Money m) { processor.process(m); }
```

Refactorings. Collapse Hierarchy (useless abstract class), Inline Function and Inline Class (useless delegation), Change Function Declaration (remove unused parameters), Remove Dead Code (for code used only by its own tests — delete the tests too).

7.2.16 Temporary Field

What it is. A field that is set and used only in some situations (e.g. only during one complex calculation) and is empty/null the rest of the time.

Why it hurts. Readers expect an object to need all its fields. A field that is usually `null` is confusing — "why is this null? is that a bug?"

```
// Smelly: these fields are only meaningful while computeRoute() runs
class Planner {
    private Graph graph;
    private Map<Node, Integer> tmpDistances;   // null except inside computeRoute
    private Node tmpTarget;                    // null except inside computeRoute
}

// Cured: Extract Class - the temporary state gets its own short-lived object
class Planner {
    private Graph graph;
    Route computeRoute(Node from, Node to) { return new RouteSearch(graph, from, to).run(); }
}
final class RouteSearch {                      // lives only for one search
    private final Map<Node, Integer> distances = new HashMap<>();
    ...
}
```

Refactorings. Extract Class (put the fields and the code that uses them in their own class), Move Function, Introduce Special Case (for the conditional code that checks whether the field is set).

7.2.17 Message Chains

What it is. A client asks one object for another object, then asks that for another, and so on: `order.getCustomer().getAddress().getCity().getName()`. Also a long chain of temporary variables doing the same thing.

Why it hurts. The client is coupled to the *whole navigation structure*; any change in between breaks it. It violates the **Law of Demeter** ("talk only to your immediate friends", Chapter 5).

```
// Smelly
String city = order.getCustomer().getAddress().getCity();

// Cured: Hide Delegate (see code/smells/Order.java)
String city = order.shippingCity();
```

Caution: do not blindly hide every delegate — that creates Middle Men. Often better: look at what the final object is *used for*, and Extract Function + Move Function that usage down the chain.

Refactorings. Hide Delegate, Extract Function, Move Function.

7.2.18 Middle Man

What it is. A class whose methods mostly just forward calls to another class, adding nothing. (Some delegation is good — that is encapsulation. The smell is when *most* of the class is pass-through.)

Why it hurts. An extra layer to read and maintain, with no value. Every new feature of the delegate requires a new forwarding method.

```
// Smelly
class Person {
    private Department dept;
    Person getManager()           { return dept.getManager(); }
    String getDepartmentCode()    { return dept.getCode(); }
    Budget getDepartmentBudget()  { return dept.getBudget(); }
    // ... ten more forwarding methods
}

// Cured: Remove Middle Man - let clients talk to the Department directly
Department department() { return dept; }
```

Refactorings. Remove Middle Man, Inline Function (for a few forwarders), Replace Superclass with Delegate or Replace Subclass with Delegate (if the middle man is tied up in inheritance).

Message Chains and Middle Man pull in opposite directions — Hide Delegate cures one and can cause the other. Use judgement to find the balance.

7.2.19 Insider Trading

What it is. Modules (classes, packages) poking at each other's internals — commonly subclasses knowing too much about their parents'. (1st edition: "Inappropriate Intimacy".)

Why it hurts. The modules are tightly coupled: you cannot change one without changing the other. Some trading is necessary; the goal is to keep it small and visible.

```
// Smelly: Invoice reaches into Customer's internal map
double discount = customer.internalLoyaltyTable.get(customer.tierCode) * 0.01;

// Cured: Customer owns its own rule, Invoice asks
double discount = customer.loyaltyDiscount();
```

Refactorings. Move Function and Move Field (reduce the need to talk), Hide Delegate (make a third module the go-between), Replace Subclass with Delegate or Replace Superclass with Delegate (when inheritance creates the intimacy).

7.2.20 Large Class

What it is. A class trying to do too much — too many fields, too many methods, too much code. Often called a **God Class**.

Why it hurts. It becomes a magnet for duplication and a bottleneck for changes (everyone edits it). It usually violates the Single Responsibility Principle.

```
// Smelly
class Employee {
    String name; String email;                    // identity
    String street, city, zip;                     // address clump
    double baseSalary, bonus, taxRate;            // payroll
    List<Review> reviews; int rating;             // performance
    ...
}

// Cured: Extract Class for each cohesive subset (look for fields with common prefixes/suffixes,
// or fields used together by the same methods)
class Employee {
    String name; String email;
    Address address;
    Compensation compensation;
    PerformanceRecord performance;
}
```

Refactorings. Extract Class, Extract Superclass, Replace Type Code with Subclasses (Extract Subclass). Tip: look at how *clients* use the class — distinct groups of clients using distinct subsets of methods suggest how to split it.

7.2.21 Alternative Classes with Different Interfaces

What it is. Two classes that do the same kind of job but have different method names or signatures, so you cannot substitute one for the other: `EmailSender.sendMail(to, body)` and `SmsGateway.dispatch(msg, phone)`.

Why it hurts. You cannot treat them polymorphically (through one shared interface, without caring which class you have), so callers write `if email ... else if sms ...`. It also often hides duplicated code.

```
// Smelly
class EmailSender { void sendMail(String to, String body) { ... } }
class SmsGateway  { void dispatch(String body, String phone) { ... } }

// Cured: rename and align signatures, then extract a common interface
interface Notifier { void send(String recipient, String message); }
class EmailNotifier implements Notifier { ... }
class SmsNotifier   implements Notifier { ... }
```

Refactorings. Change Function Declaration (make signatures match), Move Function (until the protocols match), Extract Superclass (if the implementations then share code). If you cannot edit one of the classes (third-party), wrap it in an **Adapter** (Chapter 9) — a class that converts one interface into the one you expect.

7.2.22 Data Class

What it is. A class with only fields and getters/setters (setters are methods that just overwrite a field), and no behaviour. Other classes manipulate its data in detail.

Why it hurts. Its behaviour lives in clients (as Feature Envy), often duplicated; public setters let anyone break its invariants (rules that must always hold, e.g. "balance is never negative").

Important exceptions. Immutable data carriers are fine: a Java **record** that is a computation's *result* (e.g. Split Phase output), a DTO (Data Transfer Object) crossing a network boundary, a value object. The smell is *mutable* data classes with their logic scattered around them.

```
// Smelly
class Account { private double balance; public double getBalance() {...} public void setBalance(double b) {...} }
// ...and in a service:
if (acc.getBalance() >= amt) acc.setBalance(acc.getBalance() - amt); else throw ...;

// Cured: Remove Setting Method + Move Function - the rule lives with the data
class Account {
    private double balance;
    void withdraw(double amount) {
        if (amount > balance) throw new IllegalStateException("insufficient funds");
        balance -= amount;
    }
}
```

Refactorings. Encapsulate Record, Remove Setting Method (for fields that should not change), Move Function and Extract Function (bring behaviour in from the clients), Split Phase (when the data class is the result of a phase — then it is fine as an immutable record). Also useful (not in Fowler's list for this smell): Encapsulate Collection for public collection fields.

7.2.23 Refused Bequest

What it is. A subclass inherits methods and data from its parent but does not want or need them — or worse, refuses the parent's *interface* (overrides methods to throw `UnsupportedOperationException`).

Why it hurts. The inheritance is wrong. Refusing *implementation* is mild (Fowler: often not worth fixing); refusing the *interface* is serious — it breaks the **Liskov Substitution Principle** (Chapter 4: a subclass must work anywhere its parent is expected), so code using the parent type fails with this subclass.

```
// Smelly: Stack extends a list type, and so inherits add(index, e), remove(index) ... which break stack rules
class Stack<T> extends ArrayList<T> {
    void push(T t) { add(t); }
    T pop() { return remove(size() - 1); }
}
// (java.util.Stack extends Vector - the JDK's own famous example of this smell.)

// Cured: Replace Superclass with Delegate - use a list, don't be one
final class Stack<T> {
    private final Deque<T> items = new ArrayDeque<>();
    void push(T t) { items.push(t); }
    T pop()        { return items.pop(); }
}
// Behaviour check: pop() on an empty stack now throws NoSuchElementException instead of
// IndexOutOfBoundsException, and ArrayDeque rejects null elements. If callers rely on either,
// keep the old behaviour (e.g. an ArrayList delegate) - a refactoring must not change it.
```

Refactorings. Push Down Method and Push Down Field (move unused parent members into a sibling that does use them, leaving the parent with only what is shared), Replace Subclass with Delegate, Replace Superclass with Delegate.

7.2.24 Comments

What it is. Comments aren't bad in themselves — Fowler calls them a *sweet* smell. The smell is comments as **deodorant**: explaining *what* a confusing block does instead of making the block clear.

Why it hurts. Comments go stale, and a *what* comment signals code that doesn't explain itself. **Good comments** (keep them): *why* something is done, a warning, a reference to an algorithm or bug ticket, a note of uncertainty, public API docs (Javadoc). Chapter 6, §3 Comments, catalogues good and bad comments in full.

```java
// Smelly
// check if the customer is eligible for the senior discount
if (customer.getAge() >= 60 && customer.getMembershipYears() > 2 && !customer.isBlocked()) { ... }

// Cured: Extract Function - the name replaces the comment
if (isEligibleForSeniorDiscount(customer)) { ... }

// Smelly: comment states an assumption
// rate must never be negative here
// Cured: Introduce Assertion
assert rate >= 0 : "rate must not be negative";
```

Refactorings. Extract Function (comment explains a block), Change Function Declaration (comment explains what a function does — rename it), Introduce Assertion (comment states a required condition).

1st-edition smell names (renamed or dropped in the 2nd edition)

refactoring.guru still uses the 1st-edition (1999) names. You may hear them in interviews:

1st-edition smell	Meaning	Where it went / fix
Long Method	Method too long to understand	Renamed **Long Function**
Switch Statements	Any complex switch	Narrowed to **Repeated Switches**
Lazy Class	Class that does too little	Generalised to **Lazy Element**
Inappropriate Intimacy	Classes too involved in each other's privates	Renamed **Insider Trading**
Parallel Inheritance Hierarchies	Every time you add a subclass in one hierarchy you must add one in another	A special case of Shotgun Surgery; fix with Move Function / Move Field so one hierarchy disappears
Incomplete Library Class	A library lacks a method you need and you cannot change it	Add a wrapper or utility (Introduce Foreign Method / Introduce Local Extension in the 1st ed.), or an Adapter
Dead Code	Unused variables, parameters, methods, classes	The 2nd ed. treats it as the refactoring **Remove Dead Code**

7.3 The refactoring catalogue

What it is

A **named refactoring** is a small, well-defined transformation with a name, a *motivation* (when and why), *mechanics* (the safe steps) and examples. Fowler's 2nd edition catalogues 61 in seven chapters (book Chapters 6–12); the same catalogue plus a few extras is online at refactoring.com/catalog. Many 1st-edition (1999) names became language-neutral ("function" not "method") — the old names appear as *aliases*. Many refactorings are **inverse pairs** (Extract Function ↔ Inline Function); move whichever way makes the code clearer *for the change you are about to make*.

How to read each entry

Motivation (when and why), a short **Before → After** Java sketch (`// ...` marks irrelevant code; obvious ones are omitted; compiling examples are in `code/` — 7.5, 7.6, 7.8), and the **inverse** refactoring in the heading, if one exists.

Catalogue at a glance

Book chapter	Refactorings
6 — A First Set	Extract Function, Inline Function, Extract Variable, Inline Variable, Change Function Declaration, Encapsulate Variable, Rename Variable, Introduce Parameter Object, Combine Functions into Class, Combine Functions into Transform, Split Phase
7 — Encapsulation	Encapsulate Record, Encapsulate Collection, Replace Primitive with Object, Replace Temp with Query, Extract Class, Inline Class, Hide Delegate, Remove Middle Man, Substitute Algorithm
8 — Moving Features	Move Function, Move Field, Move Statements into Function, Move Statements to Callers, Replace Inline Code with Function Call, Slide Statements, Split Loop, Replace Loop with Pipeline, Remove Dead Code
9 — Organizing Data	Split Variable, Rename Field, Replace Derived Variable with Query, Change Reference to Value, Change Value to Reference
10 — Simplifying Conditional Logic	Decompose Conditional, Consolidate Conditional Expression, Replace Nested Conditional with Guard Clauses, Replace Conditional with Polymorphism, Introduce Special Case, Introduce Assertion
11 — Refactoring APIs	Separate Query from Modifier, Parameterize Function, Remove Flag Argument, Preserve Whole Object, Replace Parameter with Query, Replace Query with Parameter, Remove Setting Method, Replace Constructor with Factory Function, Replace Function with Command, Replace Command with Function
12 — Dealing with Inheritance	Pull Up Method, Pull Up Field, Pull Up Constructor Body, Push Down Method, Push Down Field, Replace Type Code with Subclasses, Remove Subclass, Extract Superclass, Collapse Hierarchy, Replace Subclass with Delegate, Replace Superclass with Delegate
refactoring.com extras (web edition)	Replace Control Flag with Break, Replace Error Code with Exception, Replace Exception with Precheck, Replace Magic Literal, Return Modified Value

11 + 9 + 9 + 5 + 6 + 10 + 11 = **61** book refactorings.

7.3.1 A First Set of Refactorings (book chapter 6)

These are the refactorings you will use most — every day.

Extract Function *(alias: Extract Method)* · inverse: Inline Function

Motivation. Separate *intention* from *implementation*: if working out *what* a fragment does takes effort, extract it into a function named after that "what" — even one line, if the name says more than the code. Short functions are cheap in Java (the JIT inlines them).

```
// Before
void printOwing(Invoice invoice) {
    double outstanding = invoice.outstanding();
    System.out.println("name: " + invoice.customer());
    System.out.println("amount: " + outstanding);
}
// After
void printOwing(Invoice invoice) {
    printDetails(invoice, invoice.outstanding());
}
private void printDetails(Invoice invoice, double outstanding) {
    System.out.println("name: " + invoice.customer());
    System.out.println("amount: " + outstanding);
}
```

Inline Function *(alias: Inline Method)* · inverse: Extract Function

Motivation. When a function's body is as clear as its name, or when you have a tangle of badly factored small functions, put them back together — then extract again in better places. Also useful to remove needless indirection (a function that only delegates).

```
// Before
int rating(Driver d) { return moreThanFiveLateDeliveries(d) ? 2 : 1; }
boolean moreThanFiveLateDeliveries(Driver d) { return d.lateDeliveries() > 5; }
// After
int rating(Driver d) { return d.lateDeliveries() > 5 ? 2 : 1; }
```

Extract Variable *(alias: Introduce Explaining Variable)* · inverse: Inline Variable

Motivation. Complex expressions are hard to read. Name the parts with local variables. If the name makes sense beyond this function, prefer Extract Function (or Replace Temp with Query) instead so others can reuse it.

```
// Before
return order.quantity() * order.itemPrice()
     - Math.max(0, order.quantity() - 500) * order.itemPrice() * 0.05
     + Math.min(order.quantity() * order.itemPrice() * 0.1, 100);
// After
double basePrice = order.quantity() * order.itemPrice();
double quantityDiscount = Math.max(0, order.quantity() - 500) * order.itemPrice() * 0.05;
double shipping = Math.min(basePrice * 0.1, 100);
return basePrice - quantityDiscount + shipping;
```

Inline Variable *(alias: Inline Temp)* · inverse: Extract Variable

Motivation. A variable whose name says no more than the expression, or which gets in the way of another refactoring (e.g. Extract Function), should be removed.

Change Function Declaration *(aliases: Rename Function, Rename Method, Add Parameter, Remove Parameter, Change Signature)*

Motivation. Function names and parameters are the joints of the system. Rename a function the moment you understand it better; add or remove parameters to change what it depends on. Two ways: the **simple mechanics** (change the declaration and all callers at once — fine when you can see all callers, e.g. with the IDE) and the **migration mechanics** (create the new function, make the old one delegate to it and mark it `@Deprecated`, migrate callers gradually, then delete the old one — needed for published APIs).

```
// Before
double circum(double radius) { return 2 * Math.PI * radius; }
// After (migration mechanics)
double circumference(double radius) { return 2 * Math.PI * radius; }
@Deprecated(forRemoval = true)
double circum(double radius) { return circumference(radius); }   // delete when no callers remain
```

Encapsulate Variable *(aliases: Encapsulate Field, Self-Encapsulate Field)*

Motivation. Data is harder to refactor than functions (every reference changes at once). Wrapping widely used data in accessors (getter/setter methods) turns "reorganise data" into "reorganise functions" and gives one place for validation, logging or change notifications. Always the first step for Global Data (see the `Config` sketch in 7.2.5).

Rename Variable

Motivation. Good names are the heart of clear code. The wider the scope of a variable, the more its name matters. A short name is fine in a three-line lambda; a field needs a descriptive name.

Introduce Parameter Object

Motivation. A group of parameters that always travel together (a Data Clump) becomes one object. Signatures shrink, the relationship between the values is made explicit, and behaviour about them now has a home — often the start of a useful domain class.

```
// Before
List<Reading> readingsOutsideRange(Station s, int min, int max) { ... }
// After
record NumberRange(int min, int max) {
    boolean contains(int v) { return v >= min && v <= max; }
}
List<Reading> readingsOutsideRange(Station s, NumberRange range) { ... }
```

Combine Functions into Class

Motivation. When several functions work on the same data (often the same argument), make a class holding the data as fields with the functions as methods; the shared environment becomes explicit and clients stop passing the data around.

```
// Before
double base(Reading r) { ... }
double taxableCharge(Reading r) { ... }
double calculateBaseCharge(Reading r) { ... }
// After
final class ReadingCalculator {
    private final Reading reading;
    ReadingCalculator(Reading reading) { this.reading = reading; }
    double base() { ... }
    double taxableCharge() { ... }
}
```

Combine Functions into Transform

Motivation. For read-only source data, an alternative to Combine Functions into Class: a *transform* takes the raw data and returns an enriched copy with all derived values, keeping derivation logic in one place. (If the source can change, use a class — the copy would go stale.)

```
// Before (derived values computed in several places)
double base = baseRate(r.month(), r.year()) * r.quantity();
double taxable = Math.max(0, base - taxThreshold(r.year()));
// After
record EnrichedReading(Reading raw, double baseCharge, double taxableCharge) { }
static EnrichedReading enrichReading(Reading r) {
    double base = baseRate(r.month(), r.year()) * r.quantity();
    return new EnrichedReading(r, base, Math.max(0, base - taxThreshold(r.year())));
}
```

Split Phase

Motivation. When code deals with two different things in sequence (e.g. parse input, then compute; or compute, then format), split it into two phases connected by an intermediate data structure. Each phase can be understood and changed separately — a classic cure for Divergent Change. Compilers are the textbook example (tokenise — split text into words and symbols → parse into a structure tree → generate code); see 7.5, step 3.

```
// Before
String priceOrder(String orderLine) {
    String[] parts = orderLine.split("\\s+");
    Product p = catalog.get(parts[0].split("-")[1]);
    int qty = Integer.parseInt(parts[1]);
    return "Total: " + qty * p.price();
}
// After
record OrderRequest(String productId, int quantity) { }           // the intermediate data structure
OrderRequest parse(String line) {                                  // phase 1
    String[] parts = line.split("\\s+");
    return new OrderRequest(parts[0].split("-")[1], Integer.parseInt(parts[1]));
}
String price(OrderRequest req) {                                   // phase 2
    return "Total: " + req.quantity() * catalog.get(req.productId()).price();
}
String priceOrder(String orderLine) { return price(parse(orderLine)); }
```

7.3.2 Encapsulation (book chapter 7)

The key to modular design is deciding what each module *hides*. These refactorings hide data structures, temporary variables, and relationships between classes.

Encapsulate Record *(alias: Replace Record with Data Class)*

Motivation. A raw record-like structure (a `Map<String,Object>`, an array, a struct with public fields) forces clients to know its layout and lets anyone change it. Replace it with a class (or a Java `record`) whose accessors hide the storage and can compute derived values.

```
// Before
Map<String, Object> organization = Map.of("name", "Acme", "country", "GB");
String name = (String) organization.get("name");
// After
record Organization(String name, String country) { }
Organization organization = new Organization("Acme", "GB");
String name = organization.name();
```

Encapsulate Collection

Motivation. If a getter returns the internal collection, clients can modify it without the owner knowing. Offer `add` / `remove` methods on the owner, and have the getter return a read-only view or a copy.

```
// Before
class Person {
    private List<Course> courses = new ArrayList<>();
    List<Course> getCourses() { return courses; }                  // caller can add/clear!
    void setCourses(List<Course> c) { courses = c; }
}
// After
class Person {
    private final List<Course> courses = new ArrayList<>();
    List<Course> courses()        { return Collections.unmodifiableList(courses); }
    void addCourse(Course c)      { courses.add(c); }
    void removeCourse(Course c)   { courses.remove(c); }
}
```

Replace Primitive with Object *(aliases: Replace Data Value with Object, Replace Type Code with Class)*

Motivation. A simple value (a phone number string, a priority `"high"`) soon needs behaviour: validation, formatting, comparison. Wrap it in its own type as soon as you see that. A cure for Primitive Obsession.

```
// Before
List<Order> urgent = orders.stream().filter(o -> o.priority().equals("high") ||
o.priority().equals("rush")).toList();
// After
enum Priority { LOW, NORMAL, HIGH, RUSH;
    boolean higherThan(Priority other) { return compareTo(other) > 0; } }
List<Order> urgent = orders.stream().filter(o -> o.priority().higherThan(Priority.NORMAL)).toList();
```

Replace Temp with Query

Motivation. A temp holding a computed value is usable only inside its function; as a method, every method of the class can use it, and Extract Function gets easier (fewer locals). Best when the temp is assigned once and its computation has no side effects.

```
// Before
double price() {
    double basePrice = quantity * itemPrice;
    return basePrice > 1000 ? basePrice * 0.95 : basePrice * 0.98;
}
// After
double price()     { return basePrice() > 1000 ? basePrice() * 0.95 : basePrice() * 0.98; }
double basePrice() { return quantity * itemPrice; }
```

Extract Class · inverse: Inline Class

Motivation. Classes grow. When a subset of fields and methods clearly go together (and change together), move them to a new class. A cure for Large Class and Divergent Change.

```
// Before
class Person {
    String name; String officeAreaCode; String officeNumber;
    String telephoneNumber() { return "(" + officeAreaCode + ") " + officeNumber; }
}
// After
class Person {
    String name; TelephoneNumber officeTelephone;
    String telephoneNumber() { return officeTelephone.toString(); }
}
record TelephoneNumber(String areaCode, String number) {
    @Override public String toString() { return "(" + areaCode + ") " + number; }
}
```

Inline Class · inverse: Extract Class

Motivation. A class that no longer does enough (often after its responsibilities moved away) should be folded into its main user. Also a useful first step when re-splitting two classes differently: merge them, then extract again.

```
// Before
class Shipment { TrackingInformation trackingInfo; }
class TrackingInformation { String shippingCompany; String trackingNumber; }
// After
class Shipment { String shippingCompany; String trackingNumber; }
```

Hide Delegate · inverse: Remove Middle Man

Motivation. If a client calls `a.getB().doSomething()`, it depends on `B`. Adding a delegating method `a.doSomething()` hides `B`; if `B` changes, only `A` changes. A cure for Message Chains and the Law of Demeter.

```
// Before
Person manager = person.getDepartment().getManager();
// After
class Person { Person manager() { return department.getManager(); } }
Person manager = person.manager();
```

Remove Middle Man · inverse: Hide Delegate

Motivation. When a class has so many delegating methods that it is just a pass-through, let clients call the delegate directly. The right amount of hiding changes as the system changes.

Substitute Algorithm

Motivation. If you find a clearer way to do something, replace the whole body with the clearer algorithm (after decomposing the old one as far as possible, and with tests proving they agree). Also used to replace hand-written code with a library call.

```
// Before
String foundPerson(List<String> people) {
    for (String p : people) {
        if (p.equals("Don")) return "Don";
        if (p.equals("John")) return "John";
        if (p.equals("Kent")) return "Kent";
    }
    return "";
}
// After
String foundPerson(List<String> people) {
    Set<String> candidates = Set.of("Don", "John", "Kent");
    return people.stream().filter(candidates::contains).findFirst().orElse("");
}
```

7.3.3 Moving Features (book chapter 8)

These refactorings move code between contexts — between classes, between functions, and around inside a function.

Move Function *(alias: Move Method)*

Motivation. Put a function in the context (class/module) whose data and functions it references most. A cure for Feature Envy and Shotgun Surgery. Good modules keep related things together.

```
// Before
class Account {
    double overdraftCharge() {
        if (type.isPremium()) { double base = 10; return daysOverdrawn > 7 ? base + (daysOverdrawn - 7) * 0.85 :
base; }
        return daysOverdrawn * 1.75;
    }
}
// After - the rule depends mostly on AccountType, so it moves there
class AccountType {
    double overdraftCharge(int daysOverdrawn) { /* same logic, using isPremium() */ }
}
class Account { double overdraftCharge() { return type.overdraftCharge(daysOverdrawn); } }
```

Move Field

Motivation. Data structures are the backbone of a program. If you always pass a field of A alongside B, or a change in B forces a change to A's field, the field probably belongs in B. Encapsulate it first (Encapsulate Variable) to make the move easy.

```
// Before
class Customer { Plan plan; double discountRate; }
class Plan { }
// After - the discount rate is a property of the plan, not of each customer
class Customer { Plan plan; double discountRate() { return plan.discountRate(); } }
class Plan { double discountRate; double discountRate() { return discountRate; } }
```

Move Statements into Function · inverse: Move Statements to Callers

Motivation. If the same statements run every time a function is called, move them into the function, removing duplication at every call site.

```
// Before
result.add("<p>title: " + photo.title() + "</p>");
result.addAll(photoData(photo));
List<String> photoData(Photo p) { return List.of("<p>location: " + p.location() + "</p>"); }
// After
result.addAll(photoData(photo));
List<String> photoData(Photo p) {
    return List.of("<p>title: " + p.title() + "</p>", "<p>location: " + p.location() + "</p>");
}
```

Move Statements to Callers · inverse: Move Statements into Function

Motivation. When a function that was shared starts needing *different* behaviour for different callers, move the varying statements out to the callers.

```
// Before
void emitPhotoData(Writer out, Photo p) { out.write(p.title()); out.write(p.location()); }
// After - the location line moves out to every caller; behaviour is still identical
void emitPhotoData(Writer out, Photo p) { out.write(p.title()); }
// caller A:
emitPhotoData(out, p); out.write(p.location());
// caller B:
emitPhotoData(out, p); out.write(p.location());
// Now caller B can change its own line later (e.g. hide the location) - as a separate, deliberate behaviour
change.
```

Replace Inline Code with Function Call

Motivation. If code does what an existing function (yours or a library's) already does, call the function. You gain a name and one place to fix bugs.

```
// Before
boolean appliesToMass = false;
for (String s : states) { if (s.equals("MA")) appliesToMass = true; }
// After
boolean appliesToMass = states.contains("MA");
```

Slide Statements *(alias: Consolidate Duplicate Conditional Fragments)*

Motivation. Related statements belong together — e.g. declare a variable right before its first use; sliding often prepares Extract Function. The 1st-edition alias is one case: a statement repeated in every branch of a conditional slides out of it. You cannot slide past code the statement interferes with (shared variables, side effects).

```
// Before
String pricingPlan = retrievePricingPlan();
Order order = retrieveOrder();
double baseCharge = pricingPlan.base();
// After - declare each value next to its use
Order order = retrieveOrder();
String pricingPlan = retrievePricingPlan();
double baseCharge = pricingPlan.base();
```

Split Loop

Motivation. A loop doing two things makes you understand both to change either. Split it so each loop does one thing and can be extracted or turned into a pipeline. Iterating twice is rarely a bottleneck — optimise later if the profiler says so.

```java
// Before
int youngest = Integer.MAX_VALUE; double totalSalary = 0;
for (Person p : people) {
    if (p.age() < youngest) youngest = p.age();
    totalSalary += p.salary();
}
// After
double totalSalary = 0;
for (Person p : people) totalSalary += p.salary();
int youngest = Integer.MAX_VALUE;
for (Person p : people) if (p.age() < youngest) youngest = p.age();
```

Replace Loop with Pipeline

Motivation. Collection pipelines (`filter`, `map`, `reduce`) describe the processing as a series of named operations, which is easier to read than a loop. See the Loops smell.

```java
// Before
List<String> result = new ArrayList<>();
for (String line : lines) {
    if (line.isBlank() || line.startsWith("#")) continue;
    result.add(line.split(",")[1].trim());
}
// After
List<String> result = lines.stream()
        .filter(l -> !l.isBlank() && !l.startsWith("#"))
        .map(l -> l.split(",")[1].trim())
        .toList();
```

Remove Dead Code

Motivation. Unused code still costs readers time ("is this called somewhere?"). Delete it — version control remembers it if you ever need it back. Do not comment it out.

7.3.4 Organizing Data (book chapter 9)

Data structures play an important role; these refactorings improve how data is named and shared.

Split Variable ***(aliases: Remove Assignments to Parameters, Split Temp)***

Motivation. A variable (other than a loop counter or an accumulator) assigned more than once usually has more than one responsibility. Give each responsibility its own variable (ideally `final`). The 1st-ed "Remove Assignments to Parameters" is a special case: do not reuse a parameter as a local.

```java
// Before
double temp = 2 * (height + width);
System.out.println(temp);
temp = height * width;
System.out.println(temp);
// After
final double perimeter = 2 * (height + width);
System.out.println(perimeter);
final double area = height * width;
System.out.println(area);
```

Rename Field

Motivation. Field names explain the data structure; they deserve as much care as function names. Encapsulate first if the field is widely used, then rename. (For records, the accessor name changes too.)

Replace Derived Variable with Query

Motivation. Mutable data that is *computed from* other data can drift out of sync. Compute it on demand instead. Exception: if the source data is immutable, a stored derived value is safe (e.g. a record computed once).

```
// Before
class ProductionPlan {
    private double production;
    private final List<Adjustment> adjustments = new ArrayList<>();
    void apply(Adjustment a) { adjustments.add(a); production += a.amount(); }  // two updates to keep in sync
    double production() { return production; }
}
// After
class ProductionPlan {
    private final List<Adjustment> adjustments = new ArrayList<>();
    void apply(Adjustment a) { adjustments.add(a); }
    double production() { return adjustments.stream().mapToDouble(Adjustment::amount).sum(); }
}
```

Change Reference to Value · inverse: Change Value to Reference

Motivation. If an inner object needs no identity (tracking as one specific object — two telephone numbers with the same digits are "the same"), make it an immutable **value object**: replace it wholesale rather than mutate it, compare by value (`equals` / `hashCode`). Values are safe to share across threads; Java `record`s are ideal.

```
// Before
class TelephoneNumber { String areaCode; String number; void setAreaCode(String a) { areaCode = a; } }
person.telephone().setAreaCode("020");
// After
record TelephoneNumber(String areaCode, String number) { }                // immutable, equals by value
person.setTelephone(new TelephoneNumber("020", person.telephone().number()));
```

Change Value to Reference · inverse: Change Reference to Value

Motivation. If many objects hold *copies* of one logical entity (each order its own `Customer`) and it gets updated, the other copies don't see the change. Share **one** reference instead, usually via a **repository** (an object that stores domain objects and looks them up, usually by ID) that hands out the single instance per ID.

```
// Before
Order(String customerId) { this.customer = new Customer(customerId); }      // each order its own copy
// After
Order(String customerId, CustomerRepository repo) { this.customer = repo.findOrCreate(customerId); }
```

7.3.5 Simplifying Conditional Logic (book chapter 10)

Conditionals are where much of a program's complexity hides.

Decompose Conditional

Motivation. A complex `if` / `else` mixes *why* (the condition) and *what* (each branch). Extract the condition and each branch into well-named functions. A special case of Extract Function.

```
// Before
if (!date.isBefore(plan.summerStart()) && !date.isAfter(plan.summerEnd()))
    charge = quantity * plan.summerRate();
else
    charge = quantity * plan.regularRate() + plan.regularServiceCharge();
// After
charge = isSummer(date) ? summerCharge() : regularCharge();
```

Consolidate Conditional Expression

Motivation. Several checks that all lead to the same result are really one condition. Combine them with `&&` / `||` and extract the result into a named function. (Only when the checks have no side effects and truly form one decision.)

```
// Before
if (employee.seniority() < 2) return 0;
if (employee.monthsDisabled() > 12) return 0;
if (employee.isPartTime()) return 0;
// After
if (isNotEligibleForDisability(employee)) return 0;
boolean isNotEligibleForDisability(Employee e) {
    return e.seniority() < 2 || e.monthsDisabled() > 12 || e.isPartTime();
}
```

Replace Nested Conditional with Guard Clauses

Motivation. When one branch is the normal case and the others are unusual, check the unusual cases first and return immediately (**guard clauses**). The normal path then reads straight down, without deep nesting. ("One exit point" is not a rule worth keeping if it hurts clarity.)

```
// Before
double payAmount(Employee e) {
    double result;
    if (e.isSeparated()) result = 0;
    else { if (e.isRetired()) result = 0; else { result = normalPay(e); } }
    return result;
}
// After
double payAmount(Employee e) {
    if (e.isSeparated()) return 0;
    if (e.isRetired()) return 0;
    return normalPay(e);
}
```

Replace Conditional with Polymorphism

Motivation. When a switch on a type appears in several places (Repeated Switches), or when there is a base behaviour with variants, create a class per case and let dynamic dispatch (the runtime choosing which class's method to run from the object's actual type) pick the behaviour. A new case means a new class, not edits to every switch. This is how you arrive at **Strategy** or **State** (7.6 — State gives each state of an object its own class supplying that state's behaviour); 7.5 step 5 does exactly this.

```
// Before
double speed(Bird b) {
    return switch (b.type()) {
        case "EUROPEAN" -> 35;
        case "AFRICAN"  -> 40 - 2 * b.numberOfCoconuts();
        case "PARROT"   -> b.isNailed() ? 0 : 10 + b.voltage() / 10.0;
        default -> throw new IllegalArgumentException(b.type());
    };
}
// After
sealed interface Bird permits EuropeanSwallow, AfricanSwallow, NorwegianBlueParrot { double speed(); }
record EuropeanSwallow() implements Bird { public double speed() { return 35; } }
record AfricanSwallow(int numberOfCoconuts) implements Bird { public double speed() { return 40 - 2 *
numberOfCoconuts; } }
record NorwegianBlueParrot(boolean isNailed, int voltage) implements Bird {
    public double speed() { return isNailed ? 0 : 10 + voltage / 10.0; }
}
```

Introduce Special Case ***(alias: Introduce Null Object)***

Motivation. Many clients check for the same special value (usually `null`, or an "unknown customer") and do the same thing. Create a **special-case object** that answers those questions itself. The **Null Object** pattern is the most common form. See 7.6 for a runnable example.

```java
// Before
String name = (customer == null) ? "occupant" : customer.name();
Plan plan  = (customer == null) ? Plan.BASIC : customer.plan();
// After
final class UnknownCustomer implements Customer {
    public String name() { return "occupant"; }
    public Plan plan()   { return Plan.BASIC; }
}
String name = customer.name();          // repository returns UnknownCustomer instead of null
```

(Java's `Optional<T>` is a related tool: it makes "may be absent" visible in the type, but callers still decide what to do. A special case object *centralises* that decision.)

Introduce Assertion

Motivation. Make an assumption the code relies on (a discount rate is never negative) explicit with an assertion: it documents the assumption and catches programmer errors early; the program must be correct *without* it. In Java, `assert` is disabled by default (enable with `java -ea`), so do not use it to validate external input — use exceptions for that.

```java
// Before
double applyDiscount(double amount) {
    // discountRate should never be negative here
    return amount - discountRate * amount;
}
// After
double applyDiscount(double amount) {
    assert discountRate >= 0 : "discountRate must not be negative";
    return amount - discountRate * amount;
}
```

7.3.6 Refactoring APIs (book chapter 11)

Modules and their functions are the building blocks; APIs are the joints. These make APIs easier to understand and use.

Separate Query from Modifier

Motivation. A function that returns a value should have no observable side effects — then you can call it anywhere, as often as you like, safely. This is **Command–Query Separation** (Bertrand Meyer). If a function both returns something and changes state, split it into a query and a modifier.

```java
// Before
String alertForMiscreant(List<String> people) {
    for (String p : people) {
        if (p.equals("Don") || p.equals("John")) { setOffAlarms(); return p; }
    }
    return "";
}
// After
String findMiscreant(List<String> people) {                    // query - no side effects
    return people.stream().filter(p -> p.equals("Don") || p.equals("John")).findFirst().orElse("");
}
void alertForMiscreant(List<String> people) {                   // modifier
    if (!findMiscreant(people).isEmpty()) setOffAlarms();
}
```

Parameterize Function *(alias: Parameterize Method)*

Motivation. Functions with the same logic differing only in literal values become one function with a parameter (`tenPercentRaise(p)` + `fivePercentRaise(p)` → `raise(p, factor)`).

Remove Flag Argument *(alias: Replace Parameter with Explicit Methods)*

Motivation. A **flag argument** (a boolean or enum literal that the caller passes to choose which code path runs) makes calls unreadable: what does `book(customer, true)` mean? Provide an explicit function per

path.

```
// Before
void setDimension(String name, double value) { if (name.equals("height")) height = value; else if
(name.equals("width")) width = value; }
LocalDate deliveryDate(Order o, boolean isRush) { ... }
deliveryDate(order, true);
// After
void setHeight(double value) { height = value; }
void setWidth(double value)  { width = value; }
LocalDate rushDeliveryDate(Order o)    { ... }
LocalDate regularDeliveryDate(Order o) { ... }
```

Preserve Whole Object

Motivation. If you pull several values out of one object only to pass them to a function, pass the whole object. The function can then take whatever it needs, and signatures stay stable when needs change. (Don't do it if you want to keep the function independent of that object's type.)

```
// Before
int low = room.daysTempRange().low();
int high = room.daysTempRange().high();
if (!plan.withinRange(low, high)) alerts.add("room temperature out of range");
// After
if (!plan.withinRange(room.daysTempRange())) alerts.add("room temperature out of range");
```

Replace Parameter with Query *(alias: Replace Parameter with Method)* · inverse: Replace Query with Parameter

Motivation. If the function can easily get a value itself (from its own object, or from another parameter), the caller should not have to pass it. Removing the parameter simplifies every call. (Don't do it if it would add an unwanted dependency.)

```
// Before
double finalPrice() { return discountedPrice(basePrice(), discountLevel()); }
double discountedPrice(double base, int discountLevel) { return discountLevel == 2 ? base * 0.9 : base * 0.95; }
// After
double finalPrice() { return discountedPrice(basePrice()); }
double discountedPrice(double base) { return discountLevel() == 2 ? base * 0.9 : base * 0.95; }
```

Replace Query with Parameter · inverse: Replace Parameter with Query

Motivation. If a function reaches out to a global, a singleton or a mutable field, it is harder to test and not **referentially transparent** (same inputs do not always give the same output). Pass the value in as a parameter instead — the caller takes on the dependency.

```
// Before
double targetTemperature(HeatingPlan plan) {
    double current = Thermostat.INSTANCE.currentTemperature();        // hidden global dependency
    return Math.clamp(current, plan.min(), plan.max());
}
// After
double targetTemperature(HeatingPlan plan, double currentTemperature) {
    return Math.clamp(currentTemperature, plan.min(), plan.max());      // Math.clamp: Java 21+
}
```

Remove Setting Method

Motivation. If a field should not change after construction, remove its setter and set it in the constructor (make it `final`). The class then clearly communicates immutability.

Replace Constructor with Factory Function *(alias: Replace Constructor with Factory Method)*

Motivation. A constructor must return a new instance of exactly its class, under a fixed name, via `new`. A static factory method (a `static` method that creates and returns an instance) can have an intention-revealing name, return a subclass or cached instance, and hide construction details. (Joshua

Bloch's *Effective Java*, Item 1, makes the same argument. JDK examples: `List.of`, `Integer.valueOf`, `LocalDate.of`.)

```
// Before
Employee leadEngineer = new Employee(document.leadEngineer(), "E");
// After
static Employee createEngineer(String name) { return new Employee(name, "E"); }
Employee leadEngineer = Employee.createEngineer(document.leadEngineer());
```

Replace Function with Command *(alias: Replace Method with Method Object)* · inverse: Replace Command with Function

Motivation. Wrap a function in its own object (a **command**, usually with one `execute()` method). It can hold state across steps, support undo and be queued — and, most often in refactoring, lets a big function with many locals split into small methods sharing those locals as fields.

```
// Before
int score(Candidate c, MedicalExam exam, ScoringGuide guide) {
    int result = 0; int healthLevel = 0; boolean highMedicalRisk = false;
    // ... 60 lines using result, healthLevel, highMedicalRisk
    return result;
}
// After
final class Scorer {
    private final Candidate candidate; private final MedicalExam exam; private final ScoringGuide guide;
    private int result; private int healthLevel; private boolean highMedicalRisk;
    Scorer(Candidate c, MedicalExam e, ScoringGuide g) { candidate = c; exam = e; guide = g; }
    int execute() { scoreSmoking(); scoreCertification(); return result - Math.max(healthLevel - 5, 0); }
    private void scoreSmoking() { /* uses fields, no parameter passing */ }
    private void scoreCertification() { /* ... */ }
}
int score(Candidate c, MedicalExam exam, ScoringGuide guide) { return new Scorer(c, exam, guide).execute(); }
```

Replace Command with Function · inverse: Replace Function with Command

Motivation. If a command object is simple and adds nothing a function couldn't do, turn it back into a function. Commands are powerful but cost extra complexity.

```
// Before
final class ChargeCalculator {
    private final Customer customer; private final int usage;
    ChargeCalculator(Customer c, int usage) { this.customer = c; this.usage = usage; }
    double charge() { return customer.rate() * usage; }
}
double monthCharge = new ChargeCalculator(customer, usage).charge();
// After
static double charge(Customer customer, int usage) { return customer.rate() * usage; }
double monthCharge = charge(customer, usage);
```

7.3.7 Dealing with Inheritance (book chapter 12)

Inheritance is powerful and easy to misuse. These refactorings move features up and down a hierarchy and replace inheritance with delegation (composition — one object holding another and forwarding work to it) where needed.

Pull Up Method · inverse: Push Down Method

Motivation. Duplicate methods in sibling subclasses belong in the superclass. If they are similar but not identical, first make them identical (Parameterize Function), or use a Template Method (7.6) — a base-class method that fixes an algorithm's steps and lets subclasses fill some in.

Pull Up Field · inverse: Push Down Field

Motivation. Subclasses developed independently often have the same field (maybe with different names). Rename to match, then move the field to the superclass.

Pull Up Constructor Body

Motivation. Constructors in subclasses that repeat the same initialisation should call a superclass constructor that does it. In Java 21 `super(...)` must be the first statement of a constructor (Java 25 relaxed this with flexible constructor bodies, JEP 513 — a JDK Enhancement Proposal), so the common code goes into the superclass constructor.

```
// Before
class Manager extends Employee {
    Manager(String name, String id, int grade) { this.name = name; this.id = id; this.grade = grade; }
}
// After
class Employee { Employee(String name, String id) { this.name = name; this.id = id; } }
class Manager extends Employee {
    Manager(String name, String id, int grade) { super(name, id); this.grade = grade; }
}
```

Push Down Method · inverse: Pull Up Method

Motivation. If a superclass method is only relevant to one (or a few) subclasses, move it down to them. Callers must then know they have that subclass — only do it when they do.

Push Down Field · inverse: Pull Up Field

Motivation. A field used by only one subclass should live in that subclass.

Replace Type Code with Subclasses *(aliases: Extract Subclass, Replace Type Code with State/Strategy)* · inverse: Remove Subclass

Motivation. A type code field (`"engineer"`, `"manager"`) that drives behaviour hints at subclasses, after which conditionals can become polymorphism. Two ways: **direct inheritance** (subclass the class itself) or **indirect** (the class holds a separate type object, e.g. `EmployeeType`, which is subclassed — needed if the type can change at runtime or the class is already subclassed for another reason; this is the old "Replace Type Code with State/Strategy").

```
// Before
class Employee { String type; Employee(String name, String type) { ... } }
// After (direct)
abstract sealed class Employee permits Engineer, Manager, Salesman { ... }
static Employee create(String name, String type) {
    return switch (type) {
        case "engineer" -> new Engineer(name);
        case "manager"  -> new Manager(name);
        case "salesman" -> new Salesman(name);
        default -> throw new IllegalArgumentException("Employee cannot be of type " + type);
    };
}
```

Remove Subclass *(alias: Replace Subclass with Fields)* · inverse: Replace Type Code with Subclasses

Motivation. Subclasses that no longer differ in behaviour (e.g. they differ only in a constant) are not worth their cost. Replace them with a field in the superclass.

```
// Before
abstract class Person { abstract String genderCode(); }
class Male extends Person   { String genderCode() { return "M"; } }
class Female extends Person { String genderCode() { return "F"; } }
// After
class Person {
    private final String genderCode;
    Person(String name, String genderCode) { ...; this.genderCode = genderCode; }
    String genderCode() { return genderCode; }
}
```

Extract Superclass

Motivation. Two classes doing similar things have duplicated fields and methods. Create a common superclass and pull the shared parts up. (Alternative: Extract Class and use delegation. Fowler notes you can start with inheritance and switch to delegation later with Replace Superclass with Delegate if it becomes a problem.)

```
// Before
class Department { String name; double totalAnnualCost() { ... } int headCount() { ... } }
class Employee   { String name; double annualCost() { ... } String id() { ... } }
// After
abstract class Party { protected String name; abstract double annualCost(); }
class Department extends Party { ... }
class Employee extends Party { ... }
```

Collapse Hierarchy

Motivation. When a subclass and its superclass are no longer different enough to be worth keeping separate, merge them. A cure for Lazy Element and Speculative Generality.

Replace Subclass with Delegate

Motivation. Inheritance has two limits: you can vary only **one** axis (a person cannot be subclassed by both age-group and income-level), and subclasses are tightly coupled to their parent. Delegation (composition) handles both: the varying behaviour moves to a delegate object that can change at runtime. This is "favour object composition over class inheritance" (GoF — the "Gang of Four" authors of the 1994 *Design Patterns* book), and the core of the **State** and **Strategy** patterns.

```
// Before
class Booking { double basePrice() { return show.price(); } }
class PremiumBooking extends Booking { @Override double basePrice() { return super.basePrice() +
extras.premiumFee(); } }
// After
class Booking {
    private PremiumDelegate premiumDelegate;           // null unless premium - can be set at runtime
    void bePremium(Extras extras) { premiumDelegate = new PremiumDelegate(this, extras); }
    double basePrice() {
        double result = show.price();
        return premiumDelegate == null ? result : premiumDelegate.extendBasePrice(result);
    }
}
```

Replace Superclass with Delegate *(alias: Replace Inheritance with Delegation)*

Motivation. If a subclass uses only part of its superclass, or the superclass's methods do not make sense for it (Refused Bequest, broken Liskov substitution), stop inheriting and hold an instance instead, forwarding only what you need. Classic example: a `Stack` that extends a list type (the JDK's `java.util.Stack extends Vector` is often cited).

```
// Before
class Scroll extends CatalogItem { ... }     // but a scroll is a physical copy, not a catalogue entry
// After
class Scroll {
    private final CatalogItem catalogItem;   // delegate
    String title() { return catalogItem.title(); }
    boolean hasTag(String tag) { return catalogItem.hasTag(tag); }
    LocalDate lastCleaned;                   // scroll's own data
}
```

7.3.8 Extra refactorings on refactoring.com

The online catalogue (web edition) adds five refactorings that are not among the book's 61:

Replace Control Flag with Break ***(alias: Remove Control Flag)***

Motivation. A boolean "found" flag that only exists to stop a loop is clumsy. Use `break`, `continue` or `return`.

```
// Before
boolean found = false;
for (String p : people) {
    if (!found && p.equals("Don")) { sendAlert(); found = true; }
}
// After
for (String p : people) {
    if (p.equals("Don")) { sendAlert(); break; }
}
```

Replace Error Code with Exception

Motivation. Returning special codes (`-1`, `null`) for errors makes every caller check them — and callers forget. Throw an exception so errors cannot be silently ignored and the normal path stays clean.

```
// Before
int withdraw(double amount) { if (amount > balance) return -1; balance -= amount; return 0; }
// After
void withdraw(double amount) {
    if (amount > balance) throw new InsufficientFundsException(amount, balance);
    balance -= amount;
}
```

Replace Exception with Precheck ***(alias: Replace Exception with Test)***

Motivation. Exceptions are for *exceptional* behaviour. If the caller can easily check the condition first, check it instead of catching an exception.

```
// Before
double valueAt(int i) { try { return values[i]; } catch (ArrayIndexOutOfBoundsException e) { return 0; } }
// After
double valueAt(int i) { return (i >= 0 && i < values.length) ? values[i] : 0; }
```

Replace Magic Literal ***(alias: Replace Magic Number with Symbolic Constant)***

Motivation. A literal like `9.81` or `"STD"` with special meaning should get a name, so readers know what it means and there is one place to change it. (Don't bother when the meaning is obvious, e.g. `0` as an initial sum.) The worked example's step 4 does this.

```
// Before
double potentialEnergy(double mass, double height) { return mass * 9.81 * height; }
// After
static final double STANDARD_GRAVITY = 9.81;
double potentialEnergy(double mass, double height) { return mass * STANDARD_GRAVITY * height; }
```

Return Modified Value

Motivation. When a function modifies a variable passed in or shared, make it *return* the new value instead and let the caller assign it. The data flow becomes visible.

```
// Before
double totalAscent = 0;
calculateAscent();                      // secretly updates the field totalAscent
// After
double totalAscent = calculateAscent();
double calculateAscent() {
    double result = 0;
    for (int i = 1; i < points.size(); i++) {
        double diff = points.get(i).elevation() - points.get(i - 1).elevation();
        result += Math.max(diff, 0);
    }
    return result;
}
```

1st-edition names you may still hear

The *alias* in each heading above is the 1st-edition (1999) name you may still hear (e.g. Extract Method, Inline Temp, Introduce Null Object, Replace Method with Method Object, Replace Inheritance with Delegation); the five in 7.3.8 exist only in the online (web) catalogue.

7.4 Smell → refactorings lookup table

Use this when you have named a smell and need a cure. The first move is listed here; each smell's full list of cures, with when to use each, is under its 7.2 entry.

Smell	Group	Usual first move	Then / also
Mysterious Name	2nd-ed extra	**Change Function Declaration**	Rename Variable, Rename Field
Duplicated Code	Dispensables	**Extract Function**	Slide Statements, Pull Up Method
Long Function	Bloaters	**Extract Function**	Replace Temp with Query, Decompose Conditional, Split Loop, …
Long Parameter List	Bloaters	**Preserve Whole Object**, **Introduce Parameter Object**	Remove Flag Argument, Replace Parameter with Query
Global Data	2nd-ed extra	**Encapsulate Variable**	then shrink its scope
Mutable Data	2nd-ed extra	**Encapsulate Variable**	Split Variable, Replace Derived Variable with Query, Change Reference to Value, …
Divergent Change	Change Preventers	**Split Phase**, **Extract Class**	Move Function, Extract Function
Shotgun Surgery	Change Preventers	**Move Function**, **Move Field**	Combine Functions into Class/Transform, Inline first
Feature Envy	Couplers	**Move Function**	Extract Function first
Data Clumps	Bloaters	**Extract Class**, **Introduce Parameter Object**	Preserve Whole Object
Primitive Obsession	Bloaters	**Replace Primitive with Object**	Replace Type Code with Subclasses, Replace Conditional with Polymorphism
Repeated Switches	OO Abusers	**Replace Conditional with Polymorphism**	—
Loops	2nd-ed extra	**Replace Loop with Pipeline**	—
Lazy Element	Dispensables	**Inline Function**, **Inline Class**	Collapse Hierarchy
Speculative Generality	Dispensables	Collapse Hierarchy, Inline Function/Class	Change Function Declaration, Remove Dead Code
Temporary Field	OO Abusers	**Extract Class**	Move Function, Introduce Special Case
Message Chains	Couplers	**Hide Delegate**	Extract Function, Move Function
Middle Man	Couplers	**Remove Middle Man**	Inline Function, Replace Super/Subclass with Delegate
Insider Trading	Couplers	**Move Function**, **Move Field**	Hide Delegate, Replace Super/Subclass with Delegate
Large Class	Bloaters	**Extract Class**	Extract Superclass, Replace Type Code with Subclasses
Alternative Classes with Different Interfaces	OO Abusers	**Change Function Declaration**	Move Function, Extract Superclass
Data Class	Dispensables	**Encapsulate Record**	Remove Setting Method, Move Function
Refused Bequest	OO Abusers	**Replace Subclass with Delegate**, **Replace Superclass with Delegate**	Push Down Method/Field
Comments	Dispensables	**Extract Function**	Change Function Declaration, Introduce Assertion

And the reverse view — which **design principle** each family of smells violates (links to Chapters 4–6):

Smell family	Principle it violates
Divergent Change, Large Class	Single Responsibility Principle
Repeated Switches, Shotgun Surgery	Open/Closed Principle
Refused Bequest	Liskov Substitution Principle
Long Parameter List with flags, fat interfaces	Interface Segregation / clean function design
Global Data, Mutable Data	Encapsulation / information hiding (control who can change what)
Hard-wired `new` of concrete dependencies	Dependency Inversion
Duplicated Code	DRY (Don't Repeat Yourself)
Speculative Generality	YAGNI
Message Chains, Insider Trading	Law of Demeter / encapsulation
Feature Envy, Data Class	"Tell, Don't Ask" — put behaviour with data

---

7.5 Worked example — from messy to clean in six compiling steps

The starting point

A courier company prints invoices. Each shipment has a **service type** (`STD` standard, `EXP` express, `FRG` fragile), a weight and a distance.

Tariffs:

Type	Charge	Loyalty points
STD	5 + 1.2 per kg, plus 0.05 per km beyond 100 km	1 per full 100 km
EXP	12 + 2.0 per kg, plus 0.10 per km beyond 50 km	1 per full 100 km, +2 bonus
FRG	8 + 1.5 per kg + 3 handling; ×1.2 if heavier than 10 kg	1 per full 100 km

New feature request: "an HTML invoice for the web portal; next quarter, an OVERNIGHT service."

The current code (step 0, abridged — full file `code/worked/step0/Main.java`):

```
static String print(Inv inv) {
    double tot = 0;
    int pts = 0;
    int unused = 0;                                  // dead code
    StringBuilder r = new StringBuilder("Invoice for " + inv.cust + "\n");
    for (Shp s : inv.list) {
        double amt = 0;
        switch (s.t) {
            case "STD": amt = 5 + s.kg * 1.2;       if (s.km > 100) amt += (s.km - 100) * 0.05; break;
            case "EXP": amt = 12 + s.kg * 2.0;      if (s.km > 50)  amt += (s.km - 50) * 0.10;  break;
            case "FRG": amt = 8 + s.kg * 1.5 + 3;   if (s.kg > 10)  amt = amt * 1.2;            break;
            default: throw new IllegalArgumentException("unknown type: " + s.t);
        }
        pts += s.km / 100;
        if ("EXP".equals(s.t)) pts += 2;
        r.append("  ").append(s.t).append(" ").append(s.kg).append("kg ")
         .append(s.km).append("km: ").append(String.format(Locale.ROOT, "%.2f", amt)).append("\n");
        tot += amt;
    }
    r.append("Amount owed: ").append(String.format(Locale.ROOT, "%.2f", tot)).append("\n");
    r.append("Points earned: ").append(pts).append("\n");
    return r.toString();
}
```

Smells spotted.

Smell	Where
Mysterious Name	`Shp`, `Inv`, `t`, `kg`, `km`, `amt`, `tot`, `pts`, `r`
Long Function	`print` calculates charges, calculates points, and formats — three jobs
Divergent Change	HTML output would force copying the whole function; a new tariff means editing the formatting function
Repeated Switches (waiting to happen)	The type code `"EXP"` is checked in two places already
Primitive Obsession	The service type is a `String`; any typo compiles
Magic literals	`5`, `1.2`, `100`, `0.05`, `12`, ...
Data Class / Mutable Data	Public mutable fields in `Shp` and `Inv`
Dead code	`int unused`

Adding HTML now would duplicate everything, so we do **preparatory refactoring** first.

The safety net

First, a **characterisation check** — `lld.ch07.worked.Main` — records step 0's "golden" output and compares every later step's output with it **byte for byte**. The fixture (fixed test input) includes boundary cases (exactly 100 km for STD, exactly 10 kg for FRG), where refactorings most often break behaviour. In a real project this would be a JUnit test (the standard Java test framework); here it is a plain `main` that runs with the JDK (Java Development Kit) alone. Each step has its own package (`step0` … `step5`) so you can diff them; after *every* step the checker prints `SAME output`.

Step 1 — Extract Function, Rename Variable/Field, Remove Dead Code

- **Rename Field**: `t → type`, `kg → weightKg`, `km → distanceKm`; classes `Shp → Shipment`, `Inv → Invoice`.
- **Rename Variable**: `tot → totalAmount`, `pts → loyaltyPoints`, `r → result`.
- **Extract Function** ×3: the switch becomes `amountFor(shipment)`, the points logic becomes `pointsFor(shipment)`, the formatting becomes `money(amount)`.
- **Remove Dead Code**: `unused` is gone.

```
for (Shipment shipment : invoice.shipments) {
    double thisAmount = amountFor(shipment);
    loyaltyPoints += pointsFor(shipment);
    result.append(...).append(money(thisAmount)).append("\n");
    totalAmount += thisAmount;
}
```

The switch was *moved*, not changed — move before changing: two small safe steps instead of one risky one.

Step 2 — Inline Variable, Split Loop, Replace Temp with Query, Replace Loop with Pipeline, Slide Statements

The loop still does three jobs (print a line, add to total, add points). We:

1. **Inline Variable** `thisAmount` — call `amountFor(shipment)` directly.
2. **Split Loop** — one loop for printing, one for the total, one for points.
3. **Replace Temp with Query** — the total and the points become `totalAmount(invoice)` and `totalPoints(invoice)`.
4. **Replace Loop with Pipeline** — those two queries become streams.
5. **Slide Statements** — the declarations disappear; each value is computed where it is used.

```
static double totalAmount(Invoice invoice) {
    return invoice.shipments.stream().map(Main::amountFor).reduce(0.0, Double::sum);
}
static int totalPoints(Invoice invoice) {
    return invoice.shipments.stream().mapToInt(Main::pointsFor).sum();
}
```

A subtle trap. The obvious `mapToDouble(...).sum()` may use compensated summation (7.1.9) and differ from the loop in the last bit — a behaviour change; `reduce(0.0, Double::sum)` on a sequential stream adds in encounter order, exactly like the loop. (`int` addition is exact, so `IntStream.sum()` is safe.) We now call `amountFor` twice per shipment — the kind of cost Fowler says to ignore until a profiler shows it matters.

Step 3 — Split Phase, Encapsulate Record, Encapsulate Collection, Combine Functions into Transform

Now the key preparatory move. Calculation and formatting are two **phases**. We split them with an intermediate data structure:

- `StatementData` (a record) — everything the renderer needs: customer, one `Line` per shipment with its computed amount, total, points.
- `StatementCalculator.calculate(Invoice)` — phase 1, a **transform** (Combine Functions into Transform): raw invoice in, enriched data out.
- `TextRenderer.render(StatementData)` — phase 2. It knows nothing about tariffs.
- `Shipment` and `Invoice` become **records** (Encapsulate Record — no more public mutable fields), and `Invoice` copies its list with `List.copyOf` (Encapsulate Collection).

```
return new TextRenderer().render(new StatementCalculator().calculate(invoice));
```

Step 4 — Replace Primitive with Object, Replace Magic Literal

- **Replace Primitive with Object**: the `String` type code becomes `enum ServiceType { STANDARD("STD"), EXPRESS("EXP"), FRAGILE("FRG") }` with `fromCode(String)`; `"STD"` strings survive only where data enters and where it is printed. An unknown code now fails *at the boundary* (shipment creation) instead of deep in the calculation — same exception type and message, and here shipments are always built and printed together, so behaviour is unchanged (if a caller built invoices without printing them, the earlier failure would be a behaviour change to check first).
- **Replace Magic Literal**: every tariff number gets a name (`STD_FREE_KM`, `FRG_HEAVY_FACTOR`, ...).

- The switch becomes a **switch expression** over the enum; with no `default` branch, the compiler checks it is exhaustive.

```
return switch (s.type()) {
    case STANDARD -> STD_BASE + s.weightKg() * STD_PER_KG
            + (s.distanceKm() > STD_FREE_KM ? (s.distanceKm() - STD_FREE_KM) * STD_PER_EXTRA_KM : 0);
    case EXPRESS  -> ...
    case FRAGILE  -> { double base = ...; yield s.weightKg() > FRG_HEAVY_KG ? base * FRG_HEAVY_FACTOR : base; }
};
```

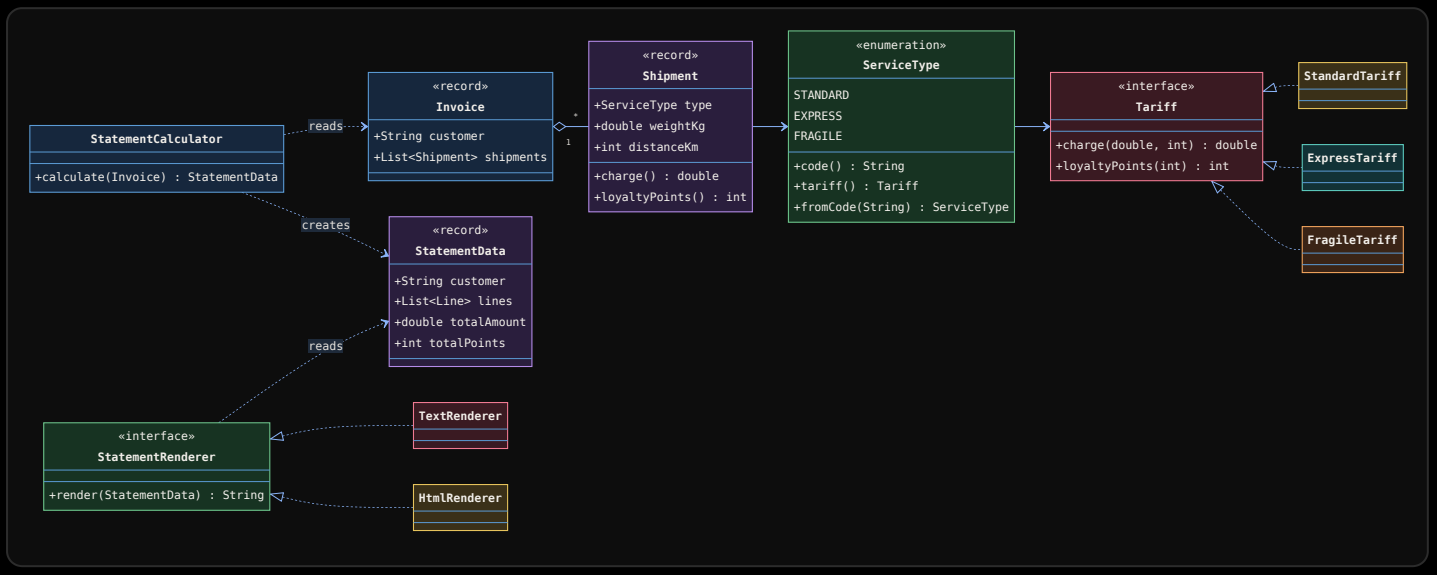
(Is `x + 0` the same as the old "`if` not taken"? For `double`, adding `+0.0` leaves any value unchanged except `-0.0`, which cannot occur here. The checker confirms it.)

Step 5 — Replace Conditional with Polymorphism, Move Function, then the new feature

- **Replace Conditional with Polymorphism**: each switch branch becomes a class implementing `sealed interface Tariff { double charge(double weightKg, int distanceKm); default int loyaltyPoints(int distanceKm) }`. `ExpressTariff` overrides `loyaltyPoints` to add its bonus — the second `"EXP"` check disappears too. Each enum constant holds its `Tariff` (a **Strategy**).
- **Move Function**: `Shipment.charge()` and `Shipment.loyaltyPoints()` — the calculator no longer reaches into shipment data (Feature Envy cured).
- `StatementCalculator` has **no conditionals left**.

Then — wearing the *adding function* hat now — the new feature is easy:

- Extract a `StatementRenderer` interface; `TextRenderer` implements it.
- Add `HtmlRenderer` — about 15 lines, reusing phase 1 unchanged.
- Adding OVERNIGHT next quarter = one new `Tariff` record, one new enum constant, and one more name in `Tariff`'s `permits` list (a sealed interface must list every implementation). The calculator and the renderers do not change (Open/Closed Principle for the calculation and rendering code).



At runtime `Main` calls `StatementCalculator.calculate(invoice)`, which asks each `Shipment` for `charge()` and `loyaltyPoints()` (each delegating to its `Tariff`) and returns `StatementData`; a `StatementRenderer` turns that into text or HTML.

What changed, step by step

Step	Refactorings	Smells removed
1	Extract Function ×3, Rename Variable, Rename Field, Remove Dead Code	Mysterious Name, part of Long Function, dead code
2	Inline Variable, Split Loop, Replace Temp with Query, Replace Loop with Pipeline, Slide Statements	Long Function (loop doing 3 jobs), Loops
3	Split Phase, Combine Functions into Transform, Encapsulate Record, Encapsulate Collection	Divergent Change, Data Class, Mutable Data
4	Replace Primitive with Object, Replace Magic Literal	Primitive Obsession, magic numbers
5	Replace Conditional with Polymorphism, Move Function, (then Extract interface for renderers)	Repeated Switches, Feature Envy

Checker output (`java lld.ch07.worked.Main`):

```
Golden output from step 0:
Invoice for Asha Traders
  STD 4.0kg 250km: 17.30
  EXP 2.5kg 80km: 20.00
  FRG 12.0kg 40km: 34.80
  STD 1.0kg 90km: 6.20
  STD 0.5kg 100km: 5.60
  FRG 10.0kg 150km: 26.00
Amount owed: 109.90
Points earned: 6
---
step0 (messy original)         SAME output
step1 extract + rename         SAME output
step2 split loop + pipeline    SAME output
step3 split phase              SAME output
step4 primitive -> object      SAME output
step5 polymorphism (final)     SAME output
All steps preserve behaviour.
```

What we deliberately did NOT do. Money is still a `double`: switching to `BigDecimal` (or `code/smells`' `Money`) would change some rounding — a *behaviour change* for the "adding function" hat, with its own tests and a product decision on rounding rules.

7.6 Refactoring toward patterns

What it is

Design patterns (Chapters 8–11) are not only designed up front: you often **arrive at a pattern by refactoring** when the code asks for it. Joshua Kerievsky's *Refactoring to Patterns* (2004) catalogues these moves, mostly compositions of Fowler's small refactorings. Don't force patterns in early (Speculative Generality); refactor *toward* one when a smell appears, and *away* from one that no longer pulls its weight.

How it works

> **Key term — patterns in this table not met earlier in this chapter** (all are covered in Chapters 8–11). **Factory**: an object or method whose only job is creating other objects; a **Builder** assembles one complex object step by step. **Observer**: subscribers register with an object and are notified automatically when it changes. **Bridge**: separates an abstraction from its implementation so the two can vary independently. **Composite**: a tree structure in which single items and groups of items are treated the same way.

Smell you see	Refactor toward	Via (Fowler's steps)
A switch/if-chain choosing an algorithm (e.g. discount by customer tier)	**Strategy**	Extract Function per branch → Replace Conditional with Polymorphism (Kerievsky: *Replace Conditional Logic with Strategy*)
Methods that check a status field and change it (`if status == PAID then ...`)	**State**	Replace Type Code with Subclasses (indirect, via a state object) → Replace Conditional with Polymorphism (Kerievsky: *Replace State-Altering Conditionals with State*)
`if (x == null)` repeated at many call sites	**Null Object / Special Case**	Introduce Special Case (Kerievsky: *Introduce Null Object*)
Sibling subclasses with the same algorithm skeleton but different steps	**Template Method**	Extract Function in each → Pull Up Method (Kerievsky: *Form Template Method*)
Many constructors with confusing boolean/int parameters	**Factory Method / Creation Methods**	Replace Constructor with Factory Function (Kerievsky: *Replace Constructors with Creation Methods*)
Many `new ConcreteX()` calls scattered; clients should not know which class	**Factory**	Replace Constructor with Factory Function + Move Function to a factory class (Kerievsky: *Encapsulate Classes with Factory*)
Complex object (in Kerievsky's refactoring, a Composite tree) built step by step with many calls in the right order	**Builder**	Kerievsky: *Encapsulate Composite with Builder*
A function too big to decompose because of many shared locals; or you need undo/queueing	**Command**	Replace Function with Command
Embellishment code (logging, caching) mixed into a core class	**Decorator**	Kerievsky: *Move Embellishment to Decorator*
Many objects polling one object for changes; hard-coded notification	**Observer**	Kerievsky: *Replace Hard-Coded Notifications with Observer*
Two classes with different interfaces doing the same job	**Adapter**	Change Function Declaration / Extract Superclass; wrap third-party code (Kerievsky: *Extract Adapter*, *Unify Interfaces with Adapter*)
A switch on a type code to create objects	**Factory + polymorphism**	Replace Type Code with Subclasses + Replace Constructor with Factory Function
Subclass explosion for combinations of features	**Strategy / Decorator / Bridge**	Replace Subclass with Delegate
A pattern that no longer earns its keep (one strategy, one decorator)	*Remove the pattern*	Inline Class, Collapse Hierarchy (Kerievsky: *Inline Singleton* and similar "away from" moves)

The runnable package `lld.ch07.towardpatterns` shows five of these, **before** and **after** side by side, and checks both give the same results (prices and loan amounts are `double` only for brevity; use `BigDecimal` or `Money` in production).

Replace Conditional Logic with Strategy

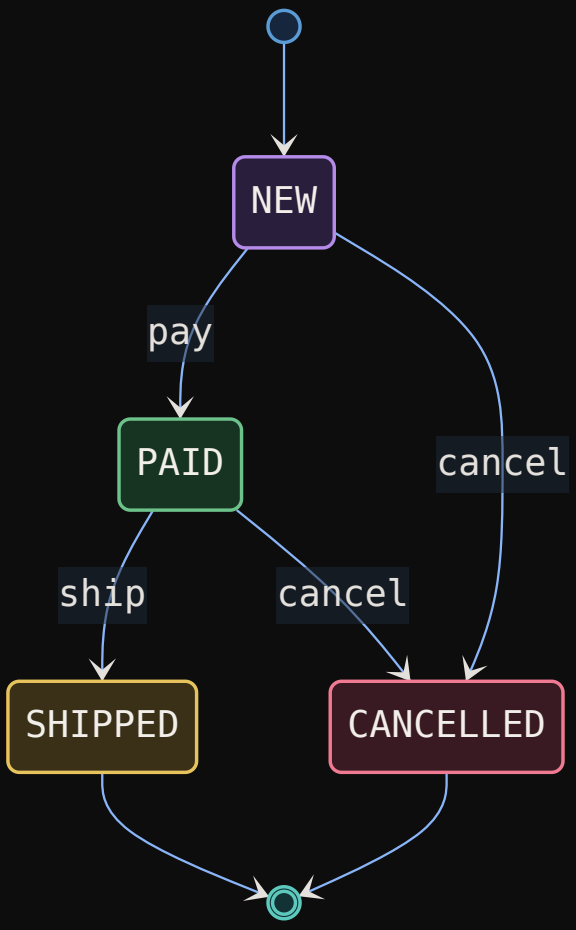
Before: a `switch (tier)` that every new tier must edit. **After:** a `DiscountPolicy` functional interface (one abstract method, so a lambda — a short inline function — can implement it), one policy per tier (classes if a policy grows) in an `EnumMap` (a map keyed by enum constants). New tiers are *registered*, not coded into a switch.

```
@FunctionalInterface
interface DiscountPolicy { double apply(double amount); }

double price(Tier tier, double amount) {
    return policies.getOrDefault(tier, fallback).apply(amount);
}
```

Replace State-Altering Conditionals with State

Before: every method checks and changes `status` strings — the lifecycle rules are scattered. **After:** each state is an enum constant implementing `OrderState`, overriding only the moves it allows; default methods (interface methods with a body) reject the rest. `Order` just delegates: `state = state.pay(this)`.




```
enum States implements OrderState {
    NEW {
        @Override public OrderState pay(Order o)    { o.log("paid"); return PAID; }
        @Override public OrderState cancel(Order o) { o.log("cancelled"); return CANCELLED; }
    },
    PAID {
        @Override public OrderState ship(Order o)   { o.log("shipped"); return SHIPPED; }
        @Override public OrderState cancel(Order o) { o.log("cancelled"); return CANCELLED; }
    },
    SHIPPED, CANCELLED
}
```

Strategy vs State: they have the same class structure. The difference is intent — a **strategy** is usually chosen by the client and stays fixed; a **state** changes *itself* as the object moves through its lifecycle.

Introduce Special Case (Null Object)

Before: `(c == null) ? "occupant" : c.name()` in every client. **After:** `find(id)` returns an `UnknownCustomer` that answers `"occupant"` and `"basic"` itself. Clients have no null checks.

Form Template Method

Before: `legacyCsv` and `legacyMarkdown` repeat the same skeleton (header, one row per item, footer). **After:** an abstract `Exporter.export()` (declared `final`) holds the skeleton once and calls abstract `header()`, `row()`, `footer()` steps; `CsvExporter` and `MarkdownExporter` fill them in.

Replace Constructors with Creation Methods

Before: `new LegacyLoan(5000, false, 12)` — what does `false` mean? **After:** a private constructor plus `Loan.termLoan(5000, 12)` and `Loan.revolver(2000)`.

Output (`java lld.ch07.towardpatterns.Main`):

```
Strategy:        GOLD 250 -> 200.0, NEW 250 -> 240.0 | same as switch: true
State:           cannot ship when NEW; paid; cannot pay when PAID; shipped; cannot cancel when SHIPPED; | same as
if-chains: true
Special Case:    Hello Ravi on premium / Hello occupant on basic | same: true
Template Method: CSV and Markdown exporters | same: true
Factory:         term 5000.0/12, revolver 2000.0/0 | same: true
All pattern refactorings preserve behaviour.
```

Small smell cures (package `lld.ch07.smells`)

Cures from 7.2 as real code: `Money` (Primitive Obsession), `DateRange` (Data Clumps), and `Order` with `shippingCity()` (Message Chains → Hide Delegate), `Line.total()` (Feature Envy → Move Function) and a read-only `lines()` view (Mutable Data → Encapsulate Collection).

Output (`java lld.ch07.smells.Main`):

```
double 0.1+0.2       = 0.30000000000000004
Money  0.10+0.20     = USD 0.30
mixing currencies    -> currency mismatch: USD vs EUR
booking contains 3rd = true, overlaps other = true
ship to Bengaluru, total INR 301.00
outsider tried lines().clear() -> UnsupportedOperationException (collection is encapsulated)
```

7.7 Putting it together

Trade-offs

Benefit	Cost / risk
Easier to read and change; features get faster (Design Stamina)	Takes time now; pay-off later and hard to measure
Bugs become visible	Without tests, you can introduce bugs
Design can evolve with requirements (less up-front guessing)	Renames and moves cause merge conflicts on long-lived branches
Small steps are safe and can stop at any point	Feel slow to beginners (fast with practice and IDE support)
More small classes and methods, each easy to understand	More indirection — too much creates Middle Men and Lazy Elements
Removes duplication	Premature abstraction (before the Rule of Three) risks the *wrong* abstraction, costlier than duplication

When not to use it

See 7.1.4. In short: not code you never touch, not without a safety net, not instead of a rewrite when the code doesn't work at all, not a published interface without a migration path — and never mixed with behaviour changes in one step.

Common mistakes

Mistake	Why it is a problem	Do this instead
Calling any code change "refactoring"	Behaviour changes hide inside "refactoring" commits; reviewers relax	Keep refactoring commits separate from feature/bug-fix commits
Big-bang refactoring (days with broken code)	High risk, merge pain, no way to stop halfway	Tiny steps, compile and test after each, commit often
Refactoring without tests, or debugging a failed step	Silent behaviour changes; wasted time	Characterisation tests first; revert and redo smaller (7.1.7)
Tests coupled to private implementation	Every refactoring breaks tests	Test behaviour through public APIs
Text search-and-replace for renames	Changes strings, comments, unrelated symbols	IDE Rename — and check reflection/config/serialisation (7.1.8)
Abstracting on the first duplication	Wrong abstraction; many flags later	Rule of Three
Changing rounding/ordering "while you're there" (e.g. `DoubleStream.sum()` vs loop, `toList()` vs mutable `ArrayList`)	Subtle behaviour changes	Know the exact semantics of the replacement API
Adding patterns everywhere "for flexibility"	Speculative Generality	Refactor toward a pattern when a smell demands it
Refactoring sprints as the main approach	Business sees refactoring as a separate, optional cost	Opportunistic refactoring inside every feature

Real-world examples

- **IntelliJ IDEA, Eclipse, VS Code (Java extension)** — automated Rename, Extract Method, Inline, Change Signature, Move, Pull Up / Push Down implement catalogue entries directly.
- **OpenRewrite** — open-source engine for large-scale automated refactoring with "recipes", e.g. migrating between Spring Boot (a popular Java framework) versions or from JUnit 4 to JUnit 5.
- **Google** — Error Prone with Refaster templates, used for large automated changes across its monorepo (one giant repository holding most of the company's code).
- **JDK history.** `java.util.Stack extends Vector` and `java.util.Properties extends Hashtable` are classic Refused Bequest cases; the JDK docs recommend `ArrayDeque` (via `Deque`) over `Stack` for new code. `Date` → `java.time` (Java 8) replaced Primitive Obsession/Mutable Data with immutable value types (`LocalDate`, `Instant`, `Duration`) — a new API alongside the old rather than an in-place refactor, because the old API is published.
- **Deprecation as migration mechanics.** The JDK and Spring keep old methods `@Deprecated` (Java 9 added `forRemoval`) and delegating for several releases — *Change Function Declaration*'s migration mechanics.
- **Java language features that support refactoring**: records (Encapsulate Record, value objects), `sealed` interfaces + switch pattern matching (safe type hierarchies), streams (Replace Loop with Pipeline), `Optional` (making absence explicit), `List.copyOf` / `Collections.unmodifiableList` (Encapsulate Collection).
- **Strangler Fig in industry.** Companies splitting a monolith (an application deployed as one unit) into services route one feature at a time to the new system (as Fowler's write-ups describe) instead of a big-bang rewrite.

In the interview

> **30-second answer — "What is refactoring?"** "Changing code's internal structure to make it easier to understand and modify, without changing its observable behaviour — in very small steps that each compile and pass the tests, so the system is never broken. Smells like Long Function, Duplicated Code or Repeated Switches show where; named refactorings like Extract Function, Move Function or Replace Conditional with Polymorphism fix them; tests are the safety net. The goal is economic: clean code lets you add features faster."

Follow-ups (with answers)

- **Q: How do you refactor code that has no tests?** A: As legacy code: characterisation tests first; if it's hard to test, break dependencies at a seam (pass an interface through the constructor instead of `new` inside); until tests exist, only IDE-automated, symbol-aware refactorings; Sprout Method/Class adds new, tested code without cleaning everything first.
- **Q: Refactor or rewrite?** A: Default to refactoring — small safe steps, system always working. Rewrite only when the code really doesn't work or the platform must change, and then piece by piece (Strangler Fig).
- **Q: Doesn't refactoring hurt performance?** A: Rarely: the JIT inlines small methods and most time is spent in a small part of the code. Refactor for clarity, then profile and optimise the real hot spots — easier in well-factored code.
- **Q: What would you refactor in this code?** (In a machine-coding round.) A: Name smells and cures: "`processOrder` is a Long Function — I'd Extract Function for validation, pricing and persistence. The switch on `vehicleType` appears in three places — Repeated Switches — I'd Replace Conditional with Polymorphism, giving a Strategy per vehicle type. `double price` is Primitive Obsession — a `Money` value object. And `getParkingLot().getFloor(1).getSpot(3)` is a Message Chain — Hide Delegate."
- **Q: When do you remove duplication?** A: Rule of Three — by the third occurrence what varies is clear; earlier abstraction risks the wrong one.
- **Q: Difference between Divergent Change and Shotgun Surgery?** A: Divergent Change: one class, many reasons to change → split it. Shotgun Surgery: one reason, many classes → gather the code.
- **Q: How do you get from a switch statement to the Strategy or State pattern?** A: Extract each branch into a function, then Replace Conditional with Polymorphism: an interface with one implementation per case, held by the object. Fixed by the client → Strategy; the object moves between cases itself → State.
- **Q: Is a data class always bad?** A: No — immutable records as values, DTOs or Split Phase output are fine; the smell is a *mutable* getter/setter class whose behaviour lives elsewhere.
- **Q: How do you rename a public API method used by other teams?** A: Change Function Declaration's migration mechanics: new method, old one delegating and `@Deprecated`, migrate callers, then remove it. For REST APIs, versioning or expand/contract.

Self-check

1. **Q:** Is fixing a bug while tidying the method one refactoring? **A:** No. The bug fix changes behaviour. Do the refactoring and the fix as separate steps (two hats), ideally separate commits.
2. **Q:** A method prints a banner, sums orders, sets a due date and prints details. Which smell and which refactoring? **A:** Long Function; Extract Function for each block (and maybe Split Loop / Replace Temp with Query).
3. **Q:** `book(hotel, from, to, 2, 1, true)` — which smells? **A:** Long Parameter List, Data Clumps (`from`, `to` and the two guest counts), a flag argument (`true`). Cures: Introduce Parameter Object (`Stay`, `Guests`), Remove Flag Argument (or replace the boolean with an explicit enum such as `MealPlan`).
4. **Q:** Why did the worked example use `reduce(0.0, Double::sum)` instead of `DoubleStream.sum()`? **A:** `DoubleStream.sum()` may use compensated summation and can differ from a plain loop in the last bit; `reduce` on a sequential stream adds left to right like the loop, so behaviour is preserved.
5. **Q:** Which smell does `class Stack extends ArrayList` have, and what is the cure? **A:** Refused Bequest (it inherits list operations that break stack rules, violating Liskov). Replace Superclass with

Delegate — hold a `Deque` instead.

6. **Q:** What is preparatory refactoring? **A:** Refactoring just before a feature so that the feature becomes easy to add — "make the change easy, then make the easy change". Step 3 (Split Phase) of the worked example did this for HTML output.
7. **Q:** When is a comment not a smell? **A:** When it explains *why* (a decision, a trade-off, a reference, a warning), not *what* the code does.
8. **Q:** A Java IDE rename of a field broke the JSON response. Was it a refactoring? **A:** It was meant to be, but the JSON field name (observable behaviour) changed, so behaviour was not preserved. Check serialisation, reflection and config when renaming.

TL;DR

- **Refactoring = small, behaviour-preserving changes** to internal structure, continuously, tests after every step; never refactor and change behaviour at once (two hats).
- **When:** Rule of Three, preparatory ("make the change easy, then make the easy change"), comprehension, litter-pickup, code review — mostly opportunistic. **Not** for code you never touch, nor without a safety net.
- **Smells** tell you *where*: 24 in the 2nd edition — Mysterious Name, Duplicated Code, Long Function, Long Parameter List, Global Data, Mutable Data, Divergent Change, Shotgun Surgery, Feature Envy, Data Clumps, Primitive Obsession, Repeated Switches, Loops, Lazy Element, Speculative Generality, Temporary Field, Message Chains, Middle Man, Insider Trading, Large Class, Alternative Classes with Different Interfaces, Data Class, Refused Bequest, Comments. Groups: Bloaters, OO Abusers, Change Preventers, Dispensables, Couplers.
- **Refactorings** tell you *how*: 61 named ones in seven families; the everyday core is Extract/Inline Function, Rename, Move Function, Extract Class, Split Phase, Replace Conditional with Polymorphism.
- **Performance:** refactor first, then profile and optimise hot spots. **Rewrite** only rarely, and then with a Strangler Fig. **Legacy code** = code without tests → characterisation tests, seams, sprout/wrap.
- **Patterns** are often reached by refactoring: switch → Strategy/State, null checks → Null Object, duplicated skeleton → Template Method, confusing constructors → Factory methods.

7.8 Full source code and output

The complete, compiling source for this chapter lives in `chapters/07-refactoring/code/` (the listings in 7.2–7.6 are the essential excerpts). **How to compile and run** (Java 21, standard library only):

```
cd chapters/07-refactoring/code
javac --release 21 -d /tmp/lld_07 $(find . -name '*.java')
java -cp /tmp/lld_07 lld.ch07.worked.Main           # characterisation check over all six steps
java -cp /tmp/lld_07 lld.ch07.worked.step5.Main     # final design + new HTML feature
java -cp /tmp/lld_07 lld.ch07.towardpatterns.Main   # refactoring toward patterns
java -cp /tmp/lld_07 lld.ch07.smells.Main           # small smell cures
```

Packages: `lld.ch07.worked` (checker), `lld.ch07.worked.step0` … `step5` (each has its own `Main`; steps 0–4 print the text invoice only), `lld.ch07.towardpatterns`, `lld.ch07.smells`. All 9 `Main` classes compile and run.

Folder	Files	What they show
`worked/`	`Main.java`	The characterisation check: records step 0's output as the "golden" text, runs every step, prints `SAME output` or `DIFFERENT output!` per step, and exits with an error if any step changed behaviour
`worked/step0` – `step2`	`Main.java`	The messy original (`Shp` , `Inv` , one `print` with a switch), then Extract Function/Rename (step 1) and Split Loop/pipelines (step 2)
`worked/step3` – `step4`	`Shipment` , `Invoice` , `StatementData` , `StatementCalculator` , `TextRenderer` (+ `ServiceType` in step 4), `Main`	Split Phase into calculate → render; then the `ServiceType` enum and named tariff constants
`worked/step5`	adds `Tariff` , `StandardTariff` , `ExpressTariff` , `FragileTariff` , `StatementRenderer` , `HtmlRenderer`	Polymorphic tariffs (no conditionals left in the calculator) and the new HTML feature
`towardpatterns/`	`StrategyRefactoring` , `StateRefactoring` , `SpecialCaseRefactoring` , `TemplateMethodRefactoring` , `FactoryRefactoring` , `Main`	Each has the *before* and *after* code and a `demo()` that checks both give the same result
`smells/`	`Money` , `DateRange` , `Order` , `Main`	Primitive Obsession, Data Clumps, Hide Delegate / Move Function / Encapsulate Collection cures

Output of `lld.ch07.worked.step5.Main` (steps 0–4 each print the same text invoice as its first nine lines; the checker's output is in 7.5):

```
Invoice for Asha Traders
  STD 4.0kg 250km: 17.30
  EXP 2.5kg 80km: 20.00
  FRG 12.0kg 40km: 34.80
  STD 1.0kg 90km: 6.20
  STD 0.5kg 100km: 5.60
  FRG 10.0kg 150km: 26.00
Amount owed: 109.90
Points earned: 6
--- new feature: HTML statement ---
<h1>Invoice for Asha Traders</h1>
<table>
  <tr><td>STD</td><td>4.0kg</td><td>250km</td><td>17.30</td></tr>
  <tr><td>EXP</td><td>2.5kg</td><td>80km</td><td>20.00</td></tr>
  <tr><td>FRG</td><td>12.0kg</td><td>40km</td><td>34.80</td></tr>
  <tr><td>STD</td><td>1.0kg</td><td>90km</td><td>6.20</td></tr>
  <tr><td>STD</td><td>0.5kg</td><td>100km</td><td>5.60</td></tr>
  <tr><td>FRG</td><td>10.0kg</td><td>150km</td><td>26.00</td></tr>
</table>
<p>Amount owed: <b>109.90</b></p>
<p>Points earned: <b>6</b></p>
```

The outputs of `lld.ch07.towardpatterns.Main` and `lld.ch07.smells.Main` are shown in 7.6.

Sources combined in this chapter

Martin Fowler, *Refactoring: Improving the Design of Existing Code*, 2nd ed. (2018) and the online catalogue at refactoring.com; refactoring.guru's smell groups; Michael Feathers, *Working Effectively with Legacy Code* (2004); Joshua Kerievsky, *Refactoring to Patterns* (2004). All explanations and code are original.